국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

보안 과제( ), 일반 과제(√) / 공개(√), 비공개( ), 발간등록번호( )

과제번호 2015-0-00280

SW컴퓨팅산업원천기술개발
최종보고서

2023. 02. 28.

주관연구개발기관 / 고려대학교 산학협력단

과학기술정보통신부
정보통신기획평가원

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

최종보고서

보안등급: 일반[√], 보안[ ]

중앙행정기관명	과학기술정보통신부	사업명	SW컴퓨팅산업원천기술개발
전문기관명	정보통신기획평가원	내역사업명	SW스타랩
공고번호	2015-0022호	연구개발과제번호	2015-0-00280

기술분류	국가과학기술표준분류	달리 분류되지 않는 SW (EE0299)	100% /100%					
	ICT연구개발기술분류	차세대 클라우드 컴퓨팅 (SWAI0502)	100% /100%					

연구개발과제명	국문	(SW 스타랩) 성능 및 보안 SLA 보장이 가능한 차세대 클라우드 인프라 SW 개발
	영문	(SW starlab) Next Generation cloud system software toward the guarantee of performance and security SLA

주관연구개발기관	기관명	고려대학교산학협력단	사업자등록번호	209-82-08298
	주소	(우:02841)서울 성북구 안암동5가 고려대학교 내	법인등록번호	

연구책임자	성명		유혁	직위	정교수
	연락처	직장전화	02-3290-3639	휴대전화	*******
		전자우편	hxy@kroea.ac.kr	국가연구자번호	*******

연구개발기간	전체	2015. 03. 01 - 2022.12. 31 (7 년 10 개월)
	단계 (해당 시 작성) 1단계	2015. 03. 01 - 2018. 12. 31 (3년 10개월)
	2단계	2019. 01. 01 - 2020. 12. 31 (2년)
	3단계	2020. 01. 01 - 2022. 12. 31 (2년)

연구개발비 (단위: 천원)		정부지원 연구개발비	기관부담 연구개발비		그 외 기관 등의 지원금				합계			연구개발비 외 지원금
					지방자치단체		기타( )					
		현금	현금	현물	현금	현물	현금	현물	현금	현물	합계	
총계		2,310,000		480,000	-	-	-	-	2,310,000	480,000	2,790,000	
1단계	1년차	300,000		60,000	-	-	-	-	300,000	60,000	360,000	
	2년차	300,000		60,000	-	-	-	-	300,000	60,000	360,000	
	3년차	270,000		60,000	-	-	-	-	270,000	60,000	330,000	
	4년차	300,000		60,000	-	-	-	-	300,000	60,000	360,000	
2단계	1년차	285,000		60,000	-	-	-	-	285,000	60,000	345,000	
	2년차	285,000		60,000	-	-	-	-	285,000	60,000	345,000	
3단계	1년차	285,000		60,000	-	-	-	-	285,000	60,000	345,000	
	2년차	285,000		60,000	-	-	-	-	285,000	60,000	345,000	

연구개발담당자 실무담당자	성명		유연호	직위	박사과정
	연락처	직장전화	02-3290-3639	휴대전화	*******
		전자우편	yhyoo@os.korea.ac.kr	국가연구자번호	*******

이 최종보고서에 기재된 내용이 사실임을 확인하며, 만약 사실이 아닌 경우 관련 법령 및 규정에 따라 제재처분 등의 불이익도 감수하겠습니다.

2023년 2월 28일

연구책임자: 유 혁 (서명)

주관연구개발기관의 장: 고려대학교 산학협력단 (직인)

과학기술정보통신부장관 귀하

- 2 -

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

< 요 약 문 >

<table>
<tr><td colspan="2">사업명</td><td colspan="3">SW컴퓨팅산업원천기술개발</td><td colspan="2" rowspan="2">연구개발과제번호</td><td colspan="2" rowspan="2">2015-0-00280</td></tr>
<tr><td colspan="2">내역사업명</td><td colspan="3">SW스타랩</td></tr>
<tr><td rowspan="2">기술분류</td><td>국가과학기술 표준분류</td><td>차세대 클라우드 컴퓨팅</td><td>100%</td><td colspan="5"></td></tr>
<tr><td>ICT연구개발기술 분류</td><td>달리 분류되지 않는 SW</td><td>100%</td><td colspan="5"></td></tr>
<tr><td colspan="2">연구개발과제명</td><td colspan="8">(SW 스타랩) 성능 및 보안 SLA 보장이 가능한 차세대 클라우드 인프라 SW 개발</td></tr>
<tr><td colspan="2">전체 연구개발기간</td><td colspan="8">2015. 03. 01 - 2022.12. 31(7 년 10 개월)</td></tr>
<tr><td colspan="2">총 연구개발비</td><td colspan="8">총 2,790,000천원
(정부지원연구개발비: 2,310,000천원, 기관부담연구개발비 : 480,000천원)</td></tr>
<tr><td colspan="2">연구개발단계</td><td colspan="3">기초[ ] 응용[√] 개발[ ]
기타(위 3가지에 해당되지 않는 경우)[ ]</td><td colspan="2">기술성숙도
(해당 시 기재)</td><td colspan="2">착수시점 기준(3)
종료시점 목표(5)</td></tr>
<tr><td colspan="2">연구개발과제 유형
(해당 시 작성)</td><td colspan="8"></td></tr>
<tr><td colspan="2" rowspan="3">연구개발과제 특성
(해당사항 모두 체크)</td><td>기술료비징수</td><td>√</td><td>사회문제해결</td><td></td><td>혁신도약형</td><td>√</td><td>경쟁형과제</td><td></td><td>표준화연계</td><td></td></tr>
<tr><td>공개SW</td><td>√</td><td>SW자산뱅크</td><td></td><td>연구데이터공개</td><td></td><td>일자리연계</td><td></td><td>규제샌드박스</td><td></td></tr>
<tr><td>정책지정</td><td></td><td>국제공동</td><td></td><td>사업화연계</td><td></td><td>소재부품장비</td><td></td><td>수요기업</td><td></td></tr>
<tr><td rowspan="2">연구개발
목표 및 내용</td><td>최종 목표</td><td colspan="8">* 연구개발: 성능 및 보안에 대한 사용자 요구사항을 보장하는 차세대 클라우드 시스템 SW
① CPU, 네트워크, 스토리지 성능에 대한 사용자 요구사항을 보장하는 시스템 SW
② 클라우드 레벨 보안 서비스의 보안 품질에 대한 사용자 요구사항을 보장하는 시스템 SW
③ ①, ②의 시스템 SW를 통합한 클라우드 플랫폼 SW

* 공개SW 생태계 구축: 클라우드 테스트베드 공개를 통한 시스템SW 특화 오픈소스 커뮤니티
① 클라우드 분산 시스템 SW 테스트 베드 구축
② 테스트 베드를 기반으로 오픈소스 커뮤니티 기여</td></tr>
<tr><td>전체 내용</td><td colspan="8">* 성능 및 보안에 대한 사용자 요구사항을 보장하는 차세대 클라우드 시스템 SW
① 성능/보안 SLA 보장을 위한 CPU, 네트워크, 스토리지 자원 통합 스케줄링 기법
② 성능/보안 SLA 보장을 위한 클라우드 작업 할당, 관리 기법 및 보안 서비스 수준 관리

* 연도별 연구 내용
① 1차년도 - 차세대 고성능 I/O 장치를 위한 SLA 보장 기법
② 2차년도 - SLA 보장 CPU 스케줄러 코드 개발
③ 3차년도 - CPU-I/O 통합 스케줄링 구조 설계
④ 4차년도 - CPU, 네트워크, 스토리지 통합 스케줄러 개발 및 오픈스택 노드에 적용
⑤ 5차년도 - 클라우드 레벨에서 성능 SLA 보장을 위한 자원 관리 기법
- 보안 SLA를 위한 BaaS 분석 및 보안 파라미터 추출
⑥ 6차년도 - Intra BaaS SLO 성능 보장 연구
- 보안 SLA를 위한 BaaS 특성 characterization</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

			⑦ 7차년도 – 이종 멀티 테넌트 기반 성능 SLA 보장 연구 – 이종 멀티 테넌트 네트워크의 가상화를 위한 최적화 연구 ⑧ 8차년도 – BaaS 성능/보안 SLA 보장 효율성 향상을 위한 최적화 연구
	1단계	목표	노드 레벨 성능 SLA 보장을 위한 I/O 및 CPU 스케줄러 연구 개발
		내용	– 성능 SLA 보장 수준: 하나의 물리 머신에서 CPU, 네트워크, 스토리지 성능에 대한 SLA 만족도에 대해 70%에서 단계적으로 증가하여 최대 90%까지 도달 – 보안 SLA 보장 수준: 2단계 보안 SLA 선행 연구로 노드 레벨에서 동작하는 공개 SW인 fail2ban 침입탐지 기능의 모니터링으로 인한 SLA 만족도 저하율 10% 수준 도달
	2단계	목표	클라우드 레벨 BaaS 보안 SLA 보장을 위한 연구 개발
		내용	– 성능 SLA 보장 수준: 여러 노드로 구성된 클라우드 환경에서 CPU, 네트워크, 스토리지 성능에 대한 SLA 만족도에 대해 95%에서 단계적으로 증가하여 최대 97%까지 도달 – 보안 SLA 보장 수준: BaaS 적용으로 인한 SLA 만족도 저하율 40%에서 단계적으로 증가하여 최대 20% 수준 도달
	3단계	목표	이종 멀티테넌트 환경에서의 성능 및 보안 SLA 보장을 위한 연구 개발
		내용	– 성능 및 보안 SLA 보장 수준: 노드 수를 점진적으로 증가시킴과 동시에 성능 및 보안 SLA 보장 수준을 향상을 목표로 함. 성능 SLA 만족도 최대 99.9% 달성 및 보안 SLA 제공으로 인한 CLS 저하율 1% 미만 달성 목표.

연구개발성과	*** 2021년 과학기술정보통신부 선정 국가연구개발 우수성과 100선,** **그 중에서도 정보·전자 분야 중 IT 분야 과제로 유일하게 “최우수” 과제로 선정됨** *** SCIE 논문 19건 출판** – JCR 상위 10% 논문 총 8건 (42% 차지) – 상위 1%, 4% 최상급 논문 포함 *** SCIE인정 국제최우수학술대회 9편 발표** – 한국연구재단 BK사업 리스트 기준 9편 발표 – 한국정보과학회 리스트 기준 4편 발표 – 2년연속 Google Scholar 기준 분야 내 H5 index 1위 학술대회 발표 달성 – Acceptance rate 10.7% 등 최고수준 학술대회 논문 발표 포함 *** 국제학술대회/기타 저널/국내 학술대회 논문** – CoNEXT 등 최우수학술대회 워크샵 등 최상위 국제학술대회 논문 21편 발표 – ACM 및 IEEE 등 저명 단체로부터 Best paper award 등 2회 수상 – 학생연구원 논문발표를 통해 국내 학술대회 우수논문상 11회 이상 수상 *** 기술이전/산학협력** – LG전자 및 미국에 $80,000 수준 기술이전 달성 (최초 과제 목표치: 0건) *** 특허** – 국제특허 출원 9건, 등록 4건 달성 / 국내특허 출원 19건, 등록 16건 달성

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

홍보성과	*** 클라우드 가상화 및 성능 보장 기술 개발, 최우수 학술대회 발표 언론매체 홍보** - 전자신문, 대학저널, 베리타스알파 등 총 7건 이상 - 고대 유혁 교수팀, 넷플릭스 등 스트리밍 서비스 및 인공지능 연산을 구동시키는 사용자 서비스의 획기적인 성능 개선과 비용의 절감 기대 (INFOCOM 2020 발표) *** 국가연구개발 우수성과 100선에서 최우수 과제 선정, 선정 결과 매체 홍보** - 전자신문, 한국대학신문, 뉴시스, 스마트경제, 인공지능신문 등 11건 이상 - "고려대 유혁 교수, 과기부 '최우수 연구 100선' 선정"
공개SW성과	*** 스타수, 커미트수, Fork 수 모두 목표치 대비 초과 달성** - 연구목표 대비 Commit 459%, Fork 695%, Star 380%, Pull 74건(목표치 0건) * 네트워크 성능 보장/가상화 기술은 그 우수성을 입증받아 미국 유관 분야 오픈소스 소프트웨어 및 표준을 주도하는 Open Networking Foundation과 협의 하에, 국제적인 소프트웨어의 소유권과 커뮤니티 운영권한을 이양받아 원**천기술을 보유 및 운영**중 * 본 연구진의 오픈소스SW 커뮤니티는 **월 2,000회 이상 조회되는 수준으로 성장**하였음
인력양성 성과	* 최우수 연구인력 35명 배출 * 정년트랙 전임교수 3명 배출(성균관대, 경북대, 전북대 등) * 인력양성 프로그램을 통해 박사과정 1명-SCIE급 논문 주저 10편 이상 저술 * 우수한 연구실적을 통해 박사과정 1명 BK 4단계 BK21 미래인재 펠로우십 선정 * Microsoft Research Asia 인턴 2명 선발 및 한국인 최초 Student TechFest 수상 * 석사과정 전원 최선호 직장 취업 달성 * 석사과정 중 ETRI/국방과학연구소/KIST 등 정부출연연구소 연구원 6명 배출 * 석사과정 연구원 ACM/IEEE 학술대회 Top paper award 2회 및 국내학술대회 우수논문상 11회 수상
연구개발성과 활용계획 및 기대 효과	*** 클라우드 기술을 이루는 수평적 수직적 기술 요소에 적용 가능** ① 수직적 기술 요소는 인프라(IaaS), 플랫폼(PaaS), 서비스(SaaS)를 이루는 전 분야에 적용 ② 수평적 기술 요소로는 Public 클라우드와 Private 클라우드에 모두 적용 가능 *** 수요처로서 이미 클라우드를 사용하는 기업부터, 클라우드를 도입하려는 기업까지 가능** ① 클라우드 서비스를 제공하는 IT 서비스 기업 ② 다중 사용자에게 인프라, 플랫폼, 서비스를 제공하는 기업 ③ 높은 신뢰성을 요구하는 Private 클라우드 구축/운영 중·소·대기업 ④ Public 클라우드 기반 새로운 서비스를 구축/제공하는 IT 서비스 기업 *** 성능/보안 SLA의 지원은 IT 시장에서 큰 시장 형성 및 고성장 추세인 분산 클라우드 서비스/플랫폼 영역과 사물인터넷 영역의 안전한 연결고리 역할 수행** ① 퍼블릭 클라우드 시장 규모: 2021년까지 2,783억 달러에 달할 것 (2018년 가트너 예측). ② 큰 시장 형성원인: 웹기반 다양한 서비스 및 가상화된 컴퓨팅 자원의 유기적 구성에 의한 시너지 ③ 사물인터넷 서비스 및 AI 기술 확대의 영향: 클라우드 접근성 향상으로, 컴퓨팅 자원 및 서비스 이용률 급증 *** 국내 클라우드 규제 완화와 공개SW의 도입 요구는 잠재적 시장 증대의 가능성을 내포**

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<table>
<tr><td></td><td colspan="11">① 클라우드 도입 비용의 이슈: 공개SW 클라우드 플랫폼의 품질 향상 →
민간/공공분야의 도입/활용률 증대되나 낮은 수준. 도입의 난관은 비용이며
이는 정확한 SLA 정의의 필요성을 내포
② 시장 증대/활성화 잠재성: 체계화된 SLA 수립 시도와 더불어 정책/제도적
체계화가 관건</td></tr>
<tr><td>연구개발성과의
비공개여부 및
사유</td><td colspan="11">해당없음</td></tr>
<tr><td rowspan="2">연구개발성과의
등록·기탁 건수</td><td rowspan="2">논문</td><td rowspan="2">특허</td><td rowspan="2">보고서
원문</td><td rowspan="2">연구
시설·
장비</td><td rowspan="2">기술
요약
정보</td><td rowspan="2">소프트
웨어</td><td rowspan="2">표준</td><td colspan="2">생명자원</td><td rowspan="2">화합물</td><td colspan="2">신품종</td></tr>
<tr><td>생명
정보</td><td>생물
자원</td><td>정보</td><td>실물</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td rowspan="2">연구시설·장비
종합정보시스템
등록 현황</td><td>구입
기관</td><td>연구시설·
장비명</td><td>규격
(모델명)</td><td>수량</td><td>구입
연월일</td><td>구입가격
(천원)</td><td>구입처
(전화)</td><td colspan="2">비고
(설치장소)</td><td colspan="2">ZEUS
등록번호</td></tr>
<tr><td></td><td>해당없음</td><td></td><td></td><td></td><td></td><td></td><td colspan="2"></td><td colspan="2"></td></tr>
<tr><td>국문핵심어
(5개 이내)</td><td colspan="2">분산 컴퓨팅</td><td colspan="2">클라우드 컴퓨팅</td><td colspan="2">통합 스케줄링</td><td colspan="2">클라우드 관리</td><td colspan="2">SLA 보장</td></tr>
<tr><td>영문핵심어
(5개 이내)</td><td colspan="2">Distributed
computing</td><td colspan="2">Cloud
computing</td><td colspan="2">Integrated
scheduling</td><td colspan="2">Cloud
management</td><td colspan="2">Service level
agreement
guarantee</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<목 차>

1. 연구개발과제의 개요..... 8

2. 연구개발과제의 수행 과정 및 수행 내용..... 14

3. 연구개발과제의 수행 결과 및 목표 달성 정도..... 84

4. 목표 미달 또는 미흡한 사항에 대한 원인분석 132

5. 연구개발성과의 관련 분야에 대한 기여 정도..... 133

6. 연구개발성과의 관리 및 활용 계획..... 134

7. 연구개발성과에 대한 후속연구 및 추가 개발 계획 135

8. 연구개발비 사용실적 138

붙임1. 자체 보안관리 진단표..... 140

붙임2. 공개 SW 요약서..... 141

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1. 연구개발과제의 개요

o 4차 산업혁명 시대, IoT, 인공지능의 기반 플랫폼으로 클라우드 컴퓨팅의 중요성 증대

▶ 클라우드 기술은 사용자에게 컴퓨팅 자원을 필요한 만큼 유연하게 제공할 수 있는 새로운 컴퓨팅 패러다임을 제시함. 최근 4차 산업혁명 시대를 맞이하여, 인공지능, 빅데이터 처리, IoT 등의 복합적 컴퓨팅 요구가 증대되면서, 이를 지원하기 위한 효과적인 플랫폼으로 각광받고 있음.

▶ 세계적인 분석 기관인 가트너는 최근 아래 그림 1과 같이 2018년 10대 전략 기술을 발표하였음. 발표된 10대 전략 기술들은 모두 대규모 데이터 처리와 컴퓨팅 자원의 효율적 관리를 요구하는 클라우드 워크로드로 볼 수 있음. 따라서, 이를 뒷받침할 클라우드 시스템 기술은 더욱 중요하게 되었음. 한편, 클라우드 시스템의 제공 형태는 퍼블릭 클라우드, 프라이빗 클라우드, 하이브리드 클라우드, 멀티 클라우드 등 다양한 형태로 확장하며, 점차 세분화되어 발전되고 있는 추세임. 특히 가트너는 2017년에도 크게 성장한 퍼블릭 클라우드 시장이 2018년에도 꾸준히 지속되어 2020년까지 유지될 것으로 보고 있음. 2017년에는 가트너가 예상한 6가지의 성장 동력이 되는 클라우드 요소 중 시스템 인프라서비스 (IaaS)가 가장 큰 성장을 보일 것으로 예측함.

▶ 또 다른 세계적 분석 기관인 포레스터 연구소 (Forrester research) 역시 2018년 클라우드 컴퓨팅 10대 예상 자료에서 퍼블릭 클라우드의 강세와 아마존, 구글 등 클라우드 플랫폼 기업의 지배적인 점유율을 예측하고 있음.

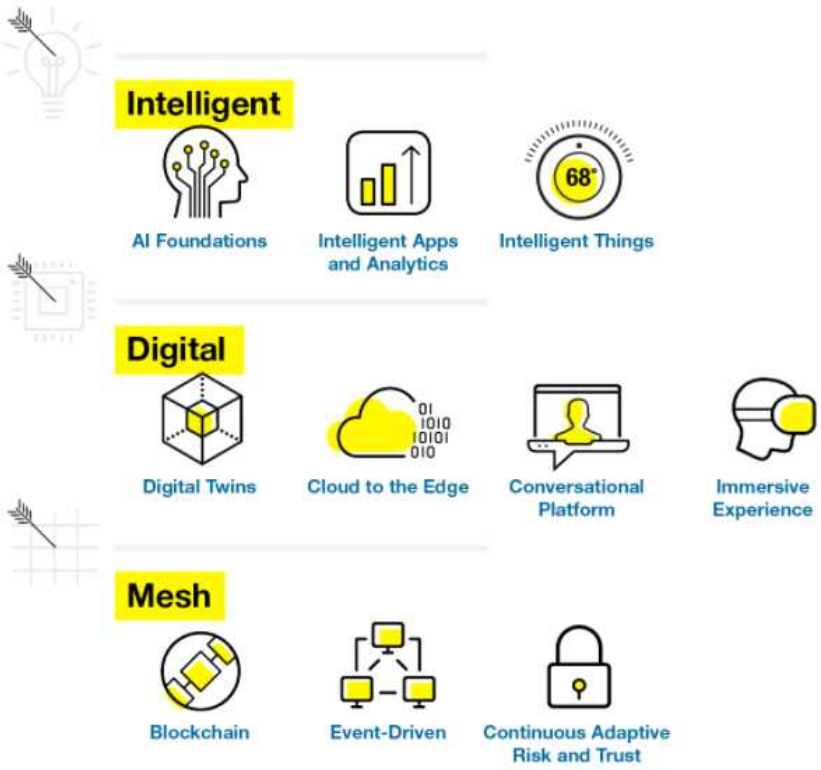


그림 1. 가트너 2018 상위 10 전략 기술 그림
(gartner.com/SmartWithGartner)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

▶ 이러한 흐름으로 인해 가트너는 전 세계 IT지출이 2020년까지 3,833억 달러에 달할 것으로 전망함. 그중 가장 두드러진 성장세는 클라우드 시스템 인프라 (IaaS)로 2020년까지 715억 달러에 이를 것으로 전망. 포레스터 연구소는 2018년 예측치에서 1460억 달러에서 1870억 달러로 연평균 성장률 22%를 예상하고 있음. 또한 2018년에는 클라우드 플랫폼 위에서 실제적으로 서비스 되는 SaaS 서비스 업체들의 경쟁이 매우 심화되어 클라우드 관련 시장은 더욱 치열해질 것이라 예상함.

o 클라우드 컴퓨팅 기술 후진국 : 해결책은 공개 SW

▶ 위와 같이 클라우드 기술은 계속해서 발전하고 있으나, 그러나 관련 국내 기술의 경쟁력은 타 국가에 비해 크게 떨어져 있는 상태임. 2012년 한국산업기술평가관리원에서 발간한 특허분석을 통한 산업기술 메가트렌드-정보통신산업에서는 2000년 이후의 특허를 분석하여 기술 경쟁력을 평가하였는데, 차세대 컴퓨팅 분야에서 클라우드 시스템 관련 기술을 포함하고 있음. 해당 보고서에서 우리나라는 기술력이 낮은 후발 그룹으로 분석하고 있으며, IP 시장 활동력도 낮은 것으로 판단하고 있음. 관련 기술을 선도하고 있는 미국의 경우, 클라우드 스토리지 시스템 분야에서 압도적인 성과를 기록, 해당 기술 분야의 중대성을 보여주고 있음.

▶ 이와 같이 뒤처진 클라우드 기술을 단기간 내에 따라잡기 위해서는 새로운 클라우드 플랫폼 기술을 제안하기보다, 공개 SW 기반의 클라우드 플랫폼을 기반으로 연구, 개발을 수행하는 것이 유리함. 이러한 공개 SW는 다음과 같은 장점을 가짐. 첫째, 비용이 낮고 수정이 용이하여 기업들이 언제나 쉽게 채택 가능함. 따라서 공개 SW의 발전에 따라 높은 파급력을 가질 수 있음. 둘째, 성공적인 공개 SW는 이미 전 세계적인 커뮤니티를 구축하고 있으며, 따라서 새로운 기술을 개발하면 해당 커뮤니티를 통해 빠르게 기술을 전파할 수 있음. 셋째, 코드 공개로 인해 폐쇄적인 SW에 비해 제공하려는 기능의 정확한 이해와 피드백이 가능함.

o 미래 클라우드 컴퓨팅 필요 기술: 사용자 요구 성능/보안 수준의 보장

▶ 클라우드 컴퓨팅은 물리적 자원을 여러 가상 머신 (VM: virtual machine)이 공유하도록 하는 가상화 (virtualization) 기술을 기반으로 함. 클라우드 컴퓨팅의 물리적 인프라인 클라우드 데이터 센터는 가상화 기술을 기반으로, 물리적 자원을 가상화하여 통합하는 서버 통합 (server consolidation) 기법을 이용함. 서버 통합 기술은 사용률이 낮은 물리적 서버들에 존재한 가상 머신들을 소수의 물리 서버에 집중시켜 사용률을 높이고 다른 서버들의 전력을 차단함으로써, 데이터 센터 전체의 유지비용을 절감함. 따라서 10~20% 정도의 사용률을 보이던 기존의 데이터 센터에 비해 클라우드 데이터 센터의 TCO (Total Cost of Ownership)는 현저히 낮음.

▶ 서버 통합 기술을 사용한 가상화 환경에서는 여러 가상머신들이 하나의 물리 머신에서 수행됨. 이때 각각의 VM은 개별적인 성능 요구 사항을 가지며, 이는 클라우드 컴퓨팅 사용자의 계약(SLA) 형태로 정형화됨. SLA는 사용자의 요구사항에 따라 다양한 형태를 취하는데, 예를 들어, VM에서 웹서버를 운용하고자 하는 사용자는 네트워크 응답 시간, 요청 처리 속도를 성능 기준으로 제시할 수 있고, 파일 서버를 운용하고자 하는 경우에는 초당 I/O 처리량 (IOPS, I/O operations per second), 혹은 대역폭 (bandwidth) 등을 기준으로 할 수 있음. 이러

한 I/O 중심의 작업 외에, 많은 CPU 작업을 요하는 하둡 (Hadoop) 등의 빅데이터 분석 도구를 수행하려는 경우에는 CPU 처리량, 혹은 사용률이 기준이 될 수 있음. 클라우드 서비스는 이런 SLA의 보장을 미래 목표로 함.

o 현재 클라우드 SLA 보장 현황 및 한계

▶ 현재 널리 사용되는 공용 클라우드 (public Cloud) 시스템에서는 사용자에게 이러한 SLA를 제시하고, 이를 보장하기 위해 노력하고 있음. 2017년 기준 조사한 바에 따르면, 아직까지도 성능/보안 SLA 보장은 제대로 이루어지지 않고 있으며, SLA 보장 역시 제한적인 수준에 이르고 있음.

표 1. 현재 클라우드 SLA 보장 현황 및 한계

클라우드	성능 SLA 보장	보안 SLA 보장	보장 수준	비고
구글 클라우드	×	×	Google Cloud Platform은 Storage, App Engine, Compute engine, BigQuery 등 다양한 서비스에 대해 SLO를 명시하고, 이를 위반시 재정적 보상을 명시하고 있음. 그러나, 그 SLO들은 uptime, downtime, error rate들 제한적인 서비스 가용성 메트릭만을 사용함. (e.g. 99.95% 또는 99.9%의 월 가동시간 SLA)	구글 클라우드 플랫폼 SLA 검색 (https://cloud.google.com/s/results/?q=SLA&p=%2F)
아마존 (EC2)	×	×	Amazon EC2 에서는 최소 99.95%의 월별 가동시간 비율에 대한 SLA 보장을 명시함. 이에 대한 위반시 고객은 사전에 명시된 방식으로 산정된 서비스 크레딧을 수령함. (월별 가동시간 비율이 [99% ~ 99.95%] 일시 서비스 크레딧 비율은 10% 제공되고 [99% 미만] 일 시, 서비스 크레딧 비율은 30%)	아마존 EC2 SLA 정책 정의에 대한 웹 페이지(https://aws.amazon.com/ko/ec2/sla/)
MS Azure	×	×	MS Azure 에서는 uptime 에 대한 보장과 downtime 보상 정책을 명시하고 있음. (월간 Uptime을 기준으로 일정한 가용성을 제공하지 못할 경우 사용자에게 서비스 크레딧으로 보상) 가상머신, 네트워크, 스토리지 등에 대해 제공	MS Azure SLA 정책에 대한 웹 페이지(https://azure.microsoft.com/en-us/support/legal/sla/)
OpenStack	△	×	단일 자원 성능 SLA 일부를 지원하고 있음. 네트워크의 경우 대역폭, 지연시간에 대해 제공하고 있고, 블록 스토리지를 제공하는 Cinder의 경우 초당 IOPS 및 대역폭 설정을 위한 API가 존재	네트워크- https://docs.openstack.org/mitaka/networking-guide/config-qos.html

▶ 표 1에 대표적인 4개 클라우드 서비스 업체 및 공개SW인 오픈스택의 SLA 보장 현황을 나타내었음. 마이크로소프트의 Azure, 구글, 아마존 S3 서비스와 같은 클라우드 스토리지 서비스는 월별 서비스 가능시간 (MUP: Monthly Uptime Percentage)로 데이터 가용성을 측정함.

SLA는 99.9%의 MUP를 목표로 하되, 만약 이를 보장하지 못하면 그 단계에 따라 요금의 25%, 50%를 환불해주는 형태로 제시됨. 그 외 개별적인 SLA를 설정할 수 있는 경우에도, 성능에 대한 정의는 어렵고, SLA 보장 실패 시에 대한 보상도 정의되어 있지 않은 수준임.

o 차세대 클라우드 플랫폼: 성능/보안 SLA 보장을 위하여

▶ 성능 및 보안에 대한 사용자 요구사항을 보장하는 클라우드 플랫폼 SW
① CPU, 네트워크, 스토리지 성능에 대한 사용자 요구사항을 보장하는 시스템 SW ② 보안에 대한 사용자 요구사항을 보장하는 시스템 SW ③ ①, ②의 시스템 SW를 내장한 클라우드 플랫폼 SW

▶ 사용자는 클라우드 분산 시스템 SW를 통해 클라우드 서비스 제공자가 제공하고자 하는 다양한 클라우드 서비스의 아웃소싱이 가능함. 이중 사용자 요구 기능의 품질 조건을 명세하는 SLA를 정의하여 서비스 충족도를 평가 가능함. 본 연구에서 SLA 보장을 통해서 클라우드 서비스의 아웃소싱 뿐 아니라, 더 나아가, 기존의 성능 및 보안 요구가 존재할 때 이에 대한 성능 저하가 없고 안전한 서비스를 추구할 수 있음. 이를 위해서 분산 클라우드 시스템 SW는 크게 두 가지의 요소가 정의되어야 함. 첫 번째는 성능 위주의 클라우드 품질 분석/평가를 위한 성능 SLA 요소이고 두 번째는 안정성에 초점을 두는 보안 SLA 요소임. 각각은 현존하는 잘 알려진 공개 SW 기반 클라우드 플랫폼인 오픈 스택 위에서 스태킹이 가능한 소프트웨어 컴포넌트로써 구축됨. 이를 통해서 하부의 클라우드 인프라는 일반적인 범용 공개 클라우드와 특수 목적의 기밀성이 요구되는 사설 클라우드 그리고, 이 두 가지 모두를 지원 가능한 하이브리드 형태의 클라우드 인프라를 관리하게 됨.

▶ 본 과제에서 개발하고자 하는 기술의 구조를 도식화하면 다음 그림과 같음

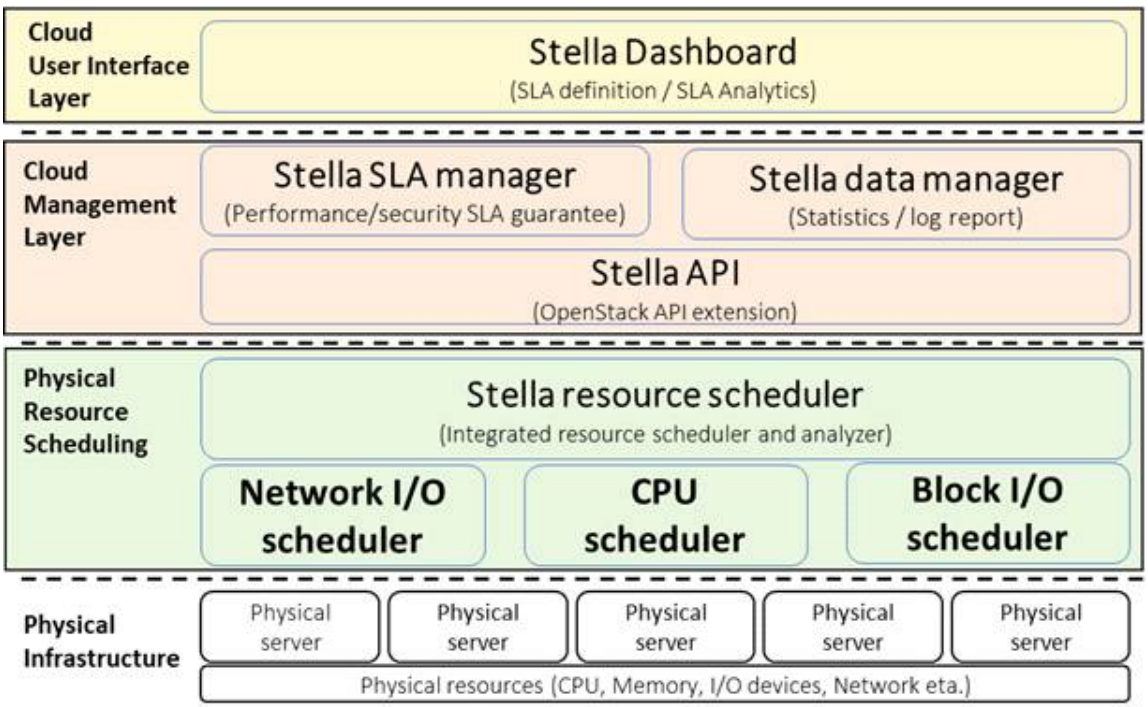


그림 2. 성능 및 보안 SLA 보장이 가능한 차세대 클라우드 인프라SW

▶ 위와 같은 차세대 클라우드 플랫폼은 아래와 같은 SLA 지원 내역을 가짐. 이를 위해 다음과 같은 핵심 기술이 필요함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

SLA 대상		제공 방식	필요 핵심 기술
성능	CPU	• **복합 작업에 대한 통합 보장** • 최대/최소 성능 보장 • 우선순위 기반 상대적 성능 보장 • 동적 SLA 변경 가능	① 다중 사용자의 성능 SLA 보장을 위한 CPU, 네트워크, 스토리지 자원의 통합 스케줄링 기법 ② SLA 보장을 위한 클라우드 서비스 용량 계산 및 작업 할당, 관리 기법
	네트워크		
	스토리지		
보안	보안기능 성능 보장		③ 보안 서비스의 성능 요구를 IaaS 레벨의 자원 성능으로 변환하는 기법 ④ 보안 SLA의 효율적인 모니터링 기술
	모니터링 오버헤드 최소화		

▶ 아울러 본 과제가 목표하는 차세대 클라우드 플랫폼이 보장하는 SLA는 다음과 같이 정의함. SLA는 서비스 제공자와 사용자 사이에서 그 대상이 되는 서비스의 품질과 보장 수준, 보장 범위, 보장 한계, 보상, 가격 등 에 대한 공식적인 규약임. 따라서 SLA의 정의를 위해서는 한 가지 요소가 아닌 다수의 SLA 구성요소 (SLA component)를 정의해야 함. 본 연구가 추구하는 SLA의 구성요소와 그 내용은 다음과 같음.

SLA 구성요소	설명	차세대 클라우드 플랫폼 SLA 내용 (성능/보안)
Purpose	본 SLA가 형성된 이유를 기술	• 성능 – 클라우드의 다양한 응용과 서비스들의 품질 보장을 위해서는 무엇보다 제공하는 서비스의 성능이 보장되어야 함. 그 성능 요구가 SaaS, PaaS 레벨의 상위에서 주어져도, 결국은 물리 자원을 제공하는 하위의 IaaS 레벨의 제공 성능이 보장될 필요가 있음. 따라서, IaaS의 핵심 자원인 CPU, 네트워크, 스토리지 등에 대한 제공 성능 보장이 없다면 상위 레벨의 요구도 달성이 불가능하므로 반드시 이를 보장할 필요가 있음. • 보안 – 클라우드에서 동작하는 보안 기능은 클라우드 사용자와 제공 서비스들의 보안성 제공을 위해 필수적인 요소임. 보안 기능의 품질 보장을 위해서는 기본적으로 보안 기능의 수행 성능과 신뢰성이 보장되어야 함. 이를 고려하면 IaaS 레벨에서는 기존의 성능 SLA 보장을 위해 요구되는 자원과 격리하여 제공할 필요가 있음. 때문에 이는 별도의 SLA로 명시하여 관리할 필요가 있음.
Validity period	SLA가 보장하는 시간 범위를 기술	• 성능 – 대상 서비스가 안정화되는 시간 (Settling time)부터 서비스 제공 시간 (service time)까지를 기준으로 함 • 보안 – 상시 구동을 위해 보안 기능이 가동되는 주기 시간을 기준으로 함
Restrictions	제공 서비스 충족을 위해 필요한 조건들	• 성능 – CPU, 네트워크, 스토리지 자원의 통합 스케줄링을 위한 최소 실현 조건은 충족해야 함 • 보안 – 보안 기능의 구동을 위해 필요한 최소 자원은 확보되어 있어야 함
Service-	승인된 서비스의	• 성능 – CPU 활용율 (%), 네트워크 초당 패킷처리 수

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

level objectives (SLO)	목표 품질 수준	(pps), 스토리지 대역폭 (MB/s) • 보안 – 초당 트랜잭션의 수 (TPS), 요구 지연시간 (s)
Penalties	SLA 위반시의 패널티	• 공통 – 상용 클라우드가 아니므로, 금전적 보상이 불가하므로, SLA 보장 평가를 위해 SLA 보장 점수 (SGS – SLA Guarantee Score)를 산정하고, 이 수치의 가감을 통해 패널티 적용함
Exclusions	보장 범위에서 벗어나는 조건들을 기술	• 공통 – 시스템 소프트웨어의 운용 수준으로 처리가 불가능한 그 외 모든 상황은 보장하지 못함. (e.g. 자연재해나, 외부 정전, 침입으로 인해 인프라의 물리적 손상 등)

▶ 위 표를 기반으로 본 연구는 SLA에서 품질 수준을 나타내는 SLO를 달성하기 위한 기술을 개발함. 그리고, SLA 평가를 위해서 CLS (customer-level satisfaction) 지표를 도입하였음. CLS는 주어진 SLO 대비 제공 성능 수준의 만족도 비율 (%)을 의미함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 연구개발과제의 수행 과정 및 수행 내용

연구 내용		연구개발 추진 방법 (이론적 · 실험적 접근방법)
차세대 고성능 I/O 장치를 위한 SLA 보장 기법	- 기존 플래시 스토리지를 위한 스케줄링 기법의 문제점 및 관련 연구 동향 분석	- 이론적 분석 결과를 토대로 기존 기법들의 단점 보완을 위한 설계목표를 수립하고, 기존 기법의 성능 및 공평성 달성 한계를 극복한 새로운 기법을 설계함 - SLA 보장을 위한 피드백 컨트롤 기법 및 스케줄링 구조 설계 기반 KVM 및 Xen 가상화 시스템에서 Linux 가상머신을 대상으로 평가함 (야후 클라우드 벤치마크 등 실제 워크로드 이용) - 이론적 분석 결과를 토대로 기존 기법들의 단점 보완을 위한 설계목표 수립 및 비례 공정 스케줄러를 기반으로 동적으로 네트워크 대역폭 할당함 - 사용자 작업에 따른 성능 요구사항 만족을 위해 다양한 성능 보장 정책 제공 및 유휴 자원 재분배로 자원 효율 향상, 실제 오픈소스 하이퍼바이저 기반 구현 및 실험망 구성과 성능 평가를 통해 개발된 기술 확인함
	- 기존 클라우드 환경에서의 네트워크 QoS 관리 기법 및 관련 연구 동향 분석	
	- 블록 I/O SLA 보장을 위한 새로운 스케줄러 구조 설계 및 개발	
	- 네트워크 자원 사용량에 따른 동적 네트워크 자원 할당 기법 개발	
SLA 보장 CPU 스케줄러 코드 개발	- 네트워크 I/O 성능 보장을 위한 스케줄러 구조와 메인 아이디어 도출	- 기존 KVM 가상화 환경에서 발생하는 CPU-네트워크 I/O 복합 작업의 성능 저하 문제를 해결하기 위한 피드백 제어 기반의 CPU 자원 할당량 조절 기법(FCCS)을 설계하고 구현 및 실험을 통해 기법의 성능 평가 - FCCS 알고리즘은 I/O 자원 사용 데이터 수집을 통해 네트워크 I/O 작업의 특성을 식별하고 식별된 작업을 부스팅 그룹으로 지정하여 CPU 자원을 우선적으로 할당함으로써 CPU 지연을 최소화함
	- 피드백 제어 기반의 스케줄링 모듈 개발	
	- First-order 모델로 인한 피드백 제어의 한계 극복을 위해 동적 피드백 제어 모델 설계	
	- 피드백 제어 모델 개선을 통한 성능 변동성 최소화	
CPU-I/O 통합 스케줄링 구조 설계	- 오픈스택을 포함한 기존의 단일 및 복합자원 처리 연구 현황 조사 및 분석	- 단일 자원 스케줄링의 한계 분석 및 한계점 극복을 위한 방안을 고안하여 I/O 성능을 고려하는 CPU 스케줄러 설계함 (기존 환경의 경우 CPU 자원: CPU-I/O 복합 작업의 블록 I/O 성능 보장 불가, I/O 자원: 가상머신 자원 의존성을 가진 I/O 성능 보장 불가) - CPU-I/O 자원을 차등적으로 재분배하는 통합 스케줄링 도입 및 실행 시간에 목표 성능 도달 확인하고, 미도달 시 차등적 재분배를 통해 의존 자원 충족
	- 실험을 통해 단일 및 복합 작업의 자원 사용량에 따른 성능 측정	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

		- Guarantee of SLA (GoS) 프레임워크의 제안 및 동작 검증: 가상화 환경에서 자원 의존성을 고려한 SLA 보장 시스템으로 단일 자원 블록 I/O 스케줄러(OIOS: 1차년도)와 CPU 스케줄러(FCCS: 2차년도)를 통합함 - 실제 가상화 환경에서 실험을 통해 목표 성능 달성 평가 결과, KVM 스케줄러와 단일 자원 스케줄러 대비 높은 수준의 SLA 성능 만족이 가능함을 보임
	- 작업특성 분석 결과를 기반으로 CPU-블록 I/O 성능 보장이 가능한 스케줄링 알고리즘 설계 및 개발	
CPU, 네트워크, 블록 I/O 통합 스케줄러 개발 및 오픈스택 노드에 적용	- CPU-I/O 통합 스케줄러 구조 확장 및 세부 동작 방식 정의	- 3차년도 결과물을 기반으로 시스템 효율성 향상을 위해 CPU 자원 할당을 이용하여 네트워크 및 블록 I/O 성능 제어가 가능한 통합 자원 스케줄러 설계 및 개발함 - 성능 모니터링, 자원 경쟁 탐지, 자원 스케줄링 조절로 구성된 통합 자원 스케줄러는 KVM 가상화 기반 커널 모듈로 구현하여 실제 서버에 적용 및 성능 평가 수행함 - 공인인증 시험기관을 통해 4차년도 연구 개발물의 정량적 목표 항목 시험을 진행함. 시험 결과 단일 CLS 보장항목에서 95% 이상, 복합 CLS 보장 90% 이상, 물리 자원 활용율 75% 이상 달성함
	- 통합 자원 스케줄링 구현	
	- 오픈스택 컴포넌트와 통합 자원 스케줄러의 연동	
클라우드 레벨에서 성능 SLA 보장을 위한 자원 관리 기법	- 관련 연구 조사 및 분석	- Multi-resource scheduling을 위한 DRF 기반 기존 연구 분석함 - 기존 DRF 기반 자원 스케줄링 기법의 한계를 보완하기 위한 SLO 기반 스케줄링 알고리즘 설계함 - 코드 레벨에서의 오픈스택 작업 분배 동작 방식 파악 및 관련 자료 조사함 - 오픈스택 작업 분배 콜체인을 기반으로 한 SLO 기반 스케줄링 세부 구조 도식화함
	- SLO를 고려한 클라우드 레벨 스케줄링 알고리즘 설계	
	- 기존 오픈스택 환경에서의 작업 분배 방식 분석	
	- 오픈스택 환경에 SLO 기반 작업 분배 알고리즘 구현	
보안 SLA를 위한 BaaS 분석 및 보안 파라미터 추출	- 공개SW BaaS 플랫폼 환경 구축 및 BaaS 예제 동작 방식 분석	- 실제 서버 환경에 공개SW BaaS 플랫폼인 하이퍼레저 패브릭 환경 구축 및 내부 구조 파악함 - 구축 한 서버 환경에 하이퍼레저 패브릭 예제를 수행하여 리눅스 기반 벤치마크를 이용한 자원 사용량 측정함 - 하이퍼레저 패브릭 예제 수행 시 보안 SLA (TPS, latency)에 영향을 미치는 파라미터 추출을 위한 성능 측정함 - 실험 결과를 기반으로 한 감사 도구 설계함
	- 공개SW BaaS 예제 성능 평가 및 자원 사용량 분석	
	- 공개SW BaaS 파라미터 추출을 위한 변수 설정 및 성능 평가	
	- 성능 측정 결과를 기반으로 한 보안 SLA 보장 BaaS 감사 도구 설계	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

Intra BaaS SLO 성능 보장 연구	- 실시간 성능 모니터링을 위한 기존 성능 계측 도구 분석	- BaaS SLO 성능 보장을 위해 필수적인 요소인 실시간 성능 모니터링 도구 분석 및 새로운 실시간 모니터링 기법 설계함 - 실시간으로 서비스 성능을 계측하여 실제 성능이 사용자의 SLO를 만족하는지 판단하는 과정을 제공하는 기존 도구인 BaaS의 하이퍼레저 캘리퍼를 상세 분석함 - 하이퍼레저 패브릭 기반 환경에서 보안 SLA에 큰 영향을 미치는 오더링 서비스 노드에 따른 차이를 연구함 - Kafka, Raft 등 합의 방식에 따라 자원 사용 및 성능 차이에 77% 이상의 큰 차이를 보임을 확인함
	- 오더링 서비스 노드에 따른 자원 사용량 및 성능 차이 분석	
보안 SLA를 위한 BaaS characterization	- BaaS 보안 SLA 파라미터 분석을 위한 보안 SLA 감사 도구 개발 및 테스트	- 실시간 성능 모니터링을 위한 기법 개발 및 보안 SLA 감사를 위한 기법 설계 및 실제 시스템에 구현을 통한 성능 평가 수행함 - 개발하는 도구는 실시간으로 BaaS 워크로드의 성능을 모니터링하여 일정 수준 이상 트랜잭션 처리 성능이 저하하는 경우 이를 CPU 자원 할당을 통해 개선하도록 동작함 - 개발한 기법(QiOi)을 실제 시스템에 적용하여 기존 기법의 성능 저하를 완전히 개선하였을 뿐만 아니라, CPU 사용량 또한 7% 개선함
멀티 테넌트 기반 BaaS 성능 SLA 보장 연구	-멀티 테넌트 환경에서 BaaS를 포함한 이종 워크로드간 성능 간섭 영향 분석/연구	- 이종 워크로드 성능 간섭 관련 연구 조사 및 분석함 - 실제 실험 환경 구축 및 다양한 워크로드를 활용한 성능 평가 수행함 - BaaS 워크로드 성능에 영향을 미치는 메트릭 파악을 통한 제어 기법 설계함 - 실제 시스템에 적용을 통한 성능 평가 수행함
	-이종 테넌트 간 고립을 지원하는 기법 설계 및 구현	
네트워크 가상화를 통한 테넌트 간 고립(isolation) 연구	- 이종 멀티 테넌트를 위한 프로그래머블 네트워크 가상화 연구	- 네트워크 가상화 관련 연구 조사 및 분석함 - 효율적인 네트워크 가상화를 위한 프로그래머블 네트워크 가상화 기법 설계함 - 기존 비가상화 네트워크 모니터링 기법 조사 및 가상화 네트워크 모니터링을 위한 요구사항 분석함 - 효율적인 가상 네트워크 모니터링을 위한 기법 설계함
	- 이종 멀티 테넌트를 위한 가상 네트워크 모니터링 기법 연구	
BaaS 성능/보안 SLA 보장 효율성 향상을	- 이종 멀티테넌트 네트워크 가상화 및 모니터링 비용 최소화 및 성능 최적화	- 이종 멀티테넌트 간의 네트워크 모니터링에 의해 발생하는 모니터링 정확도 및 가상화 병목을 제거하기 위한 기법 설계 및 개발함 - 이종 멀티테넌트 환경에서의 컨테이너

위한 최적화 연구		시스템 수준 성능 최적화를 위한 병목 분석, 새로운 구조 설계 및 검증함 - 개발한 기법은 이종 멀티테넌트 간 축약되는 모니터링 정보를 다시 풀어내어 정확도를 개선하고, 일종의 캐싱 기법을 적용하여 모니터링의 병목을 개선함
	- 통합 테스트 및 상호 운용 실험 수행 및 최적화	- 기계학습 기반 통합 제어 기술 최적화를 통해 2차년도 개발 기술을 복합 적용함 - 이종 멀티 테넌트 환경의 기계학습 기반 CPU-IO 통합 제어 기술을 기반으로, 통합 테스트 및 상호 간섭도 실험함 - 나아가, 공인인증을 통해 2단계 연구결과의 객관적 평가 추진함 - 공인인증을 통해 연구에서 목표로 하는 성과지표를 최종 달성함

1) 차세대 고성능 I/O 장치를 위한 SLA 보장 기법

■ 스토리지(플래시 장치)를 위한 SLA 보장 스케줄러 코드 개발

● 스토리지 장치에서 성능과 공평성의 이율배반성 (trade-off)

스토리지 장치를 위한 I/O 스케줄링 기법에서 성능과 공평성 간의 이율배반성이 존재한다는 것은 기존에 널리 알려진 내용임. 이율배반성의 근본적인 이유는 I/O 스케줄러 내부의 I/O 요청 큐(이하, 스케줄러 큐)와 스토리지 장치 내부의 I/O 요청 큐(이하, 장치 큐) 간의 관계에서 기인함. 본래 공평 큐잉 스케줄링 기법은 네트워크 장치에서 사용되는 방식으로, 스케줄러 큐 내부에 쌓여있는 I/O 요청들을 일정한 정책에 따라 정렬함으로써 장치에 전달되는 순서와 양을 조절하여 성능 요구 사항을 만족시킴. 예를 들어, 두 개의 VM이 동시에 I/O 요청들을 보내고 각기 똑같은 성능을 요구한다면, I/O 장치에 전달되는 I/O 요청들이 1:1의 비율이 되도록 스케줄러 큐 내부의 I/O 요청들을 정렬해서 전달하는 것임. 따라서 충분한 수준의 공평성을 제공하기 위해서는, 혹은 충분한 수준의 성능 조절이 가능하기 위해서는 스케줄러 큐에 많은 양의 I/O 요청이 적재되어 있어야 함.

네트워크 장치는 일반적으로 한 번에 하나의 요청을 받아들여 처리하며, 장치 큐가 존재는 하나 단순히 FIFO 형식으로 동작함. 그러나 스토리지 장치는 네트워크 장치와 다르게, 장치 큐가 더 복잡한 역할을 수행. HDD의 경우, 디스크 헤드(disk head)를 회전하는 플래터(platter) 위로 이동시켜 I/O 요청이 요구하는 위치에 접근함. 이 과정에서 걸리는 시간을 디스크 접근 시간 (disk access time)이라고 하며, 일반적으로 임의 접근(random access)가 많은 환경에서 HDD의 성능은 이 접근 시간에 크게 의존적임. 또한 이 과정이 물리적인 동작을 필요로 하기 때문에 최근 HDD가 보여주는 10ms 이내로 줄이는 것이 대단히 어려운 상황임. 이러한 접근 시간을 줄이기 위해 HDD는 디스크 헤드 움직임에 맞추어 장치 큐에 적재된 I/O 요청들을 재정렬함. 즉, 디스크 헤드 움직임을 최소화하기 위해 장치에 이미 전달된 요청의 순서를 다시 정렬하는 것임. 이는 결국 성능으로 이어지는 결정적 요인이기 때문에, 장치 큐에 많은 양의 I/O 요청이 적재되어 있을수록 HDD는 보다 좋은 성능을 제공할 수 있음.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

이와 동시에, 스케줄러 큐는 적재된 I/O 요청들을 이용해 공평성을 달성하려고 함. 따라서 스케줄러 큐와 장치 큐 모두가 공평성 혹은 성능 최대화를 달성하기에 충분한 양의 I/O 요청을 적재하고 있지 않다면, 어느 한 쪽을 포기하여야 하는 경우가 발생. 실제로 기업용 서버에서 측정한 I/O 요청의 길이를 보면, 평균적으로 약 4.87개가 전달되며, 90%의 경우 6.67개가 전달된다고 함[1]. 또 다른 연구에서는 1~3개가 평균적이고, 최대 8~16개 정도라고도 하였음[2]. 일반적으로 장치 큐의 크기가 32인 것을 감안하면, 스케줄러 큐와 장치 큐 모두가 충분한 양의 요청을 적재하고 있기는 어렵다고 할 수 있음.

이러한 성능-공평성 간의 이율배반성을 W. Jin 등이 SFQ(D) 스케줄러를 제안하며 자세하게 밝힌 바 있음[3]. SFQ(D)는 네트워크 장치에서 사용되던 SFQ (Start Time Fair-queueing) 방식을 스토리지에 적용한 스케줄러로, 괄호 안에 위치한 D는 스케줄러가 임의로 가정하는 장치 큐의 길이를 의미함. 예를 들어 SFQ(1)은 장치 큐의 길이를 1개로 가정하고, 스케줄러 큐에 보다 많은 I/O 요청을 적재하고자 함. SFQ(32)는 최대 32개의 요청이 동시에 장치에서 처리될 수 있다고 가정하여, 장치 큐에 보다 많은 I/O 요청을 전달함. 실험 결과, SFQ(1)이 SFQ(32)에 비해 보다 좋은 공평성, 혹은 성능 조절 능력을 보이며, 반면에 전체적인 HDD 성능은 떨어지는 결과를 보였음.

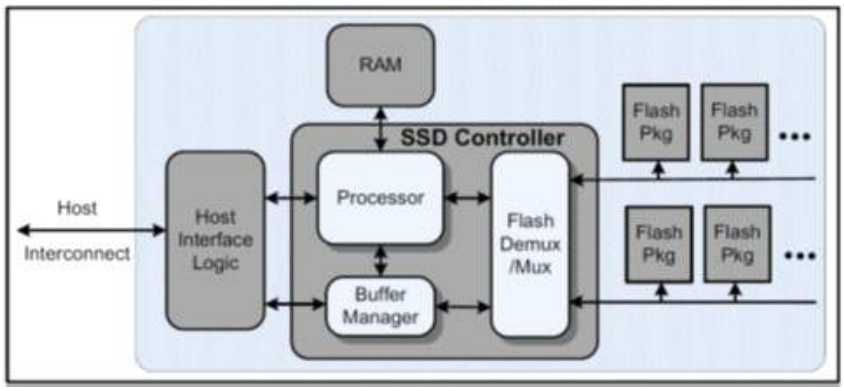


그림 3. SSD 컨트롤러와 여러 플래시 메모리 칩으로
구성된 SSD 내부 구조도

● SSD에서 이율배반성이 심화되는 이유

SSD에서는 앞서 밝힌 이율배반성이 더더욱 심화되어 나타남. 그 이유는 HDD 보다 장치 큐의 역할이 더욱 복잡해지고 중대해진 SSD의 내부 구조에서 기인함. 그림 3은 SSD의 내부 구조를 간략히 나타낸 것으로, 주요한 구성요소는 여러 채널로 연결된 다수의 플래시 메모리 칩과 SSD 내부의 I/O 컨트롤러임[4]. SSD가 기존 HDD보다 높은 성능을 보이는 것은 첫째, 응답시간이 일정하고 낮은 플래시 칩의 특성과 둘째, 여러 플래시 칩을 병렬로 연결하여 동시에 사용하기 때문임. 병렬성을 극대화하는 것은 SSD의 성능을 높이기 위한 핵심 기술이며, SSD 컨트

1) W. W. Hsu and A. J. Smith, “Characteristics of I/O traffic in personal computer and server workloads,” IBM Syst. J., vol. 42, no. 2, pp. 347-372, 2003.
2) A. Riska and E. Riedel, “Disk Drive Level Workload Characterization.,” USENIX Annu. Tech. Conf. Gen. pp. 97-102, 2006.
3) W. Jin, J. S. Chase, and J. Kaur, “Interposed proportional sharing for a storage service utility,” in ACM SIGMETRICS Performance Evaluation Review, vol. 32, no. 1, pp. 37-48, 2004.
4) N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, and R. Panigrahy, “Design tradeoffs for SSD performance,” in USENIX 2008 Annual Technical Conference on Annual Technical Conference, pp. 57-70, 2008.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

롤러가 SSD 내부의 CPU와 메모리 등을 이용하여 수행함. SSD 내부의 병렬성은 1) 플래시 칩들이 공유하는 각 채널, 2) 채널을 공유한 플래시 칩들 간의 inter-leaving 접근, 3) 플래시 칩 내부의 다이(die)들, 4) 다이 내부의 플레인(plane)들, 5) 칩 내부의 복사 연산(copy-back)과 동시에 수행 가능한 읽기/쓰기 연산 등으로, 대단히 복잡한 계층을 가지고 있음. 각 계층별로 병렬성을 극대화하기 위해 SSD 컨트롤러는 복잡한 알고리즘을 가지고 있으며, 많은 제조업체들이 각자의 노하우를 개발하고 있음.

가능한 다수의 채널, 다수의 플래시 칩이 동시에 작업을 수행하여야 성능이 극대화되므로 SSD 컨트롤러는 가능한 많은 수의 I/O 요청을 다루어야 함. 여러 I/O 요청이 장치 큐에 존재하여야 장치의 각 부분들이 유휴 시간 없이 동작하도록 할 수 있기 때문임. 따라서 큰 용량의 장치 큐와 이에 적재되는 요청의 수가 많은 것은 SSD에서 성능을 좌우하는 커다란 요인임. 이것은 물리적인 헤드의 움직임을 스케줄링하기 위한 HDD의 경우보다 더욱 큰 성능의 차이를 보임.

● SLA 보장을 위한 새로운 스케줄러 구조와 메인 아이디어

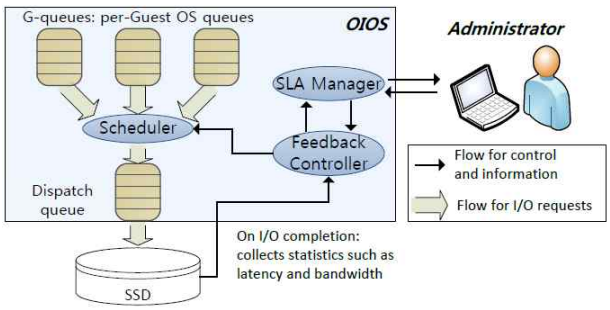


그림 4. 플래시 장치를 위한 I/O 스케줄러 구조

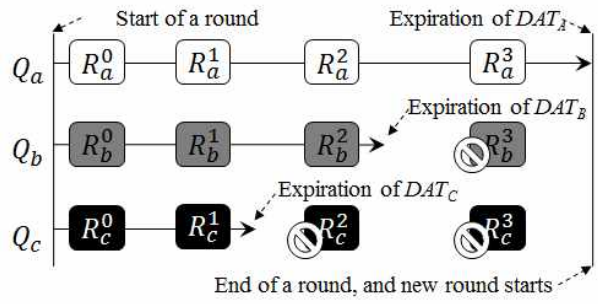


그림 5. I/O 스케줄러의 주요 동작 방식

우선 플래시 장치를 위한 새로운 스케줄러를 위한 구조를 작성하고 메인 아이디어를 개발하였음. 그림 4와 5는 각각 구조와 메인 아이디어를 나타내고 있음. 우선 새로운 스케줄러는 그림 4과 같이 실제 장치에서 측정된 성능을 스케줄러에 피드백하여 이를 스케줄링에 반영하는 구조로 이루어져 있음. 이때 SLA 매니저가 사용자에게 전달받은 SLA를 기준으로 스케줄링 파라미터를 변경함. 스케줄러는 각 가상머신에 대한 큐(g-queue)를 유지하며 스케줄링 파라미터에 따라 각 큐의 요청을 디스패치 큐(dispatch queue)를 통해 장치로 전달함.

● 동작 방식 1: 차등 시간 허용 기법

성능 SLA 보장을 위한 메인 아이디어는 장치를 사용하는 시간을 각 VM별로 세분화하여 차등적으로 배분하는 것인데, 기존 기법과 다르게 모든 VM이 스케줄러에 동시에 요청을 전달하도록 한 후에, 요청이 실제 장치에 전달되는 시간의 구간을 다르게 설정함. 본 연구팀은 B연구팀에서 제안한 병렬 I/O 처리 구조가 기본적으로 SSD의 성능을 극대화하는 역할을 성공적으로 수행한다고 판단하였음. 따라서 이 기술을 스케줄링의 기반으로 사용하고, 해당 구조에서 성능을 차등화하여 SLA를 보장할 수 있는 기법으로 발전시키고자 함. 기존의 병렬 I/O 처리 구조는 단순히 스케줄링 큐에 적재된 I/O 요청들을 SSD의 장치 큐로 동시에 전달시키는 방법

임. 이 점에 착안하여 본 연구팀은 기본적으로 모든 VM의 I/O 요청들이 동시에 SSD로 전달되는 구조를 취함. 그리고 하나의 라운드를 특정한 시간 단위(예. 10 ms)로 정의하고, 각 VM에 대해 라운드 내에서 I/O를 요청할 수 있는 시간을 서로 다르게 허용함. 본 연구팀은 이러한 스케줄링 기법의 이름을 차등 시간 허용 기법 (DAT: differently allowed time)이라 명명하였음.

그림 5는 차등 시간 허용 기법의 동작 방식을 VM_a, VM_b, VM_c 3개의 VM을 이용해 설명한 것임. Q는 각 VM에 대한 스케줄러 큐를 의미하며 각 Q에서 I/O 요청 R 을 FIFO 방식으로 I/O 장치에 전달함, 이때, 각 VM에 대해 서로 다르게 DAT를 할당하고, 가장 긴 DAT_a가 한 라운드의 길이가 됨. 3 VM이 동일한 패턴의 I/O 요청을 전달한다고 가정할 때, 우선 초반 각 2개의 요청은 동시에 장치로 전달됨. 이후 가장 짧은 DAT_c가 먼저 만료되어, R_c^2, R_c^3는 다음 라운드에 처리되게 됨. 그리고 R_a^2 R_b^2는 동시에 전달되고, 이후에 DAT_b가 만료됨. 마지막으로 R_a^3가 전달되고, DAT_a가 만료면서 해당 라운드가 종료됨. 본 연구팀은 이렇게 동시에 I/O 요청을 SSD에 전달하면서도 그 전달의 허용 시간을 다르게 함으로써 성능을 차등화하는 스케줄링 기법을 설계하고 구현하였음.

● 동작 방식 2: 피드백 컨트롤을 이용한 SLA 보장 기법

DAT 스케줄링 방식은 I/O 요청을 전달할 수 있는 허용 시간을 다르게 함으로써 각 VM의 성능을 조절하는데, 핵심적인 문제는 어떻게 절대적인 SLA 성능 수치를 제공하는가 하는 것임. 다시 말해, 서로 간의 차이를 얼마나 두어야 정확한 성능 차등화를 이룰 수 있는가 하는 점임. 이 문제를 해결하기 위해 극복해야 할 점은 각 VM의 I/O 워크로드는 항상 다르다는 점임. 읽기/쓰기 요청의 비율, 요청의 평균적인 크기, 요청 간의 평균 도달 시간 (inter-arrival time) 등 다양한 측면에서 대단히 다른 형태의 I/O 워크로드가 존재함. 가장 중요한 점은 이러한 I/O 워크로드의 특성이 시간에 따라 변경될 수 있다는 점임. 따라서 이런 다양한 VM에 대해 정확한 성능 차등화를 이루는 이상적인 DAT 값의 집합은 존재하지 않음.

본 연구팀은 이 동적으로 변화하는 워크로드에 맞추어 DAT를 변경할 수 있는 피드백 컨트롤 기법을 적용하여 SLA 보장을 달성하고자 하였음. 클라우드 시스템에서 각 VM은 일반적으로 웹서버, 스트리밍 서버 등의 특정한 기능을 수행하는 경우가 많음. 따라서 동적으로 I/O 특성이 변경되더라도, 일정 정도의 일관된 특성은 지니고 있다고 가정함. 그리고 적절한 크기의 피드백 윈도우를 설정할 수 있다면, 변경되는 I/O 특성에 맞추어 DAT를 변경하며 길지 않은 시간 내에 SLA로 요구되는 성능 수준을 달성할 수 있음. 본 연구에서 도출한 내용은 첫째, 적절한 윈도우의 크기를 결정하는 알고리즘 혹은 방법론과 둘째, 빠른 시간 내에 안정화할 수 있는 피드백 컨트롤 기법의 개발임.

위와 같은 목표로 피드백 컨트롤 기법을 구현하였으며, 피드백 컨트롤의 핵심은 다음 수식과 같음.

$$DAT_i^k = \frac{Goal_i^k - Perf_i^{k-1}}{\Delta Perf_i^{k-1}} \times \Delta DAT_i^{k-1} + DAT_i^{k-1}$$

수식에서 DAT_i^k는 I번째 VM에 대한 k번째 윈도우에서의 DAT 값을 의미함. $Goal_i^k$는 같은 윈도우에서의 목표 성능치이며, 사용자 요구 성능임. $Perf_i^{k-1}$는 이전 윈도우에서 얻은 성능값이며, DAT_i^{k-1}는 이전 윈도우에서의 DAT 값임. $\Delta Perf_i^{k-1}$는 이전 윈도우와 현재 윈도우의 성능

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

차이값이며, ΔDAT_i^{k-1}는 이전 윈도우와 현재 윈도우의 DAT 차이 값임.

이 수식의 목표는 이전 윈도우에서 설정된 DAT 값이 어느 정도의 성능 변화를 야기했는지를 바탕으로, 현재 성능과 목표 성능의 간극을 DAT값의 조정을 통해 메우고자 하는 것임. 본 연구팀은 최대한 단순한 구조의 피드백 컨트롤 수식을 세워, 다양한 메트릭에 동시에 적용될 수 있도록 하였음. 참고로 응답시간은 값이 낮을수록 성능이 좋은 것이며, 목표 응답시간이 낮아질수록 보다 많은 자원 할당을 필요로 하기 때문에 응답시간의 역수를 Perf로 설정함.

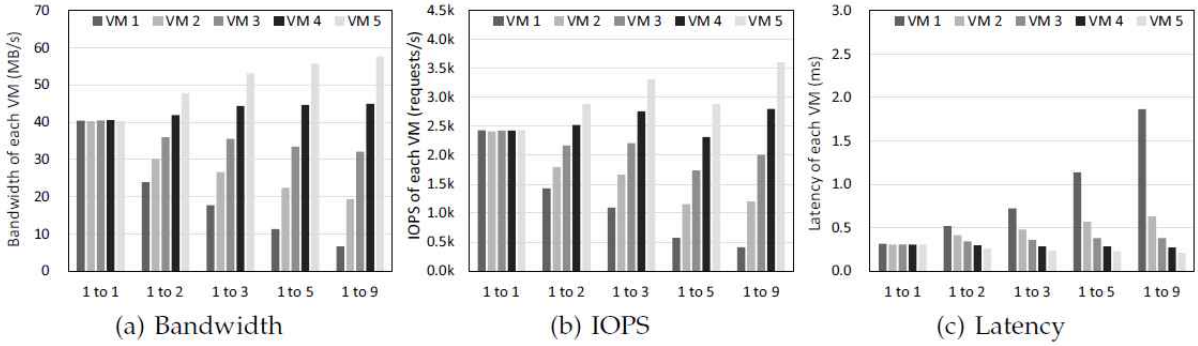


그림 6. 새로운 I/O 스케줄러의 성능 차별화 실험 결과. (a) 대역폭, (b) IOPS, (c) 응답 시간 항목에 대해 다섯 가지 차별화 시나리오를 실험. 모두 의도한 대로 성능을 분배하는 결과를 보임

● **성능 평가**

본 연구팀은 개발한 스케줄러의 성능을 평가하기 위해 총 10개 VM을 수행하는 클라우드 환경을 구축하여 성능 SLA 보장 정도를 실험하였고, 그 결과 다양한 사용자 SLA 시나리오와 VM이 새로이 추가되고 삭제되는 복잡한 환경에서 성공적으로 사용자 SLA를 보장함을 보였음.

첫 번째 결과로 위 그림 6과 같이, 다섯 개 VM에 대해 성능 차별화 (performance differentiation) 를 수행하는 기본적인 실험 결과를 제시하였음. 다섯 개 VM의 성능을 다섯 가지 시나리오에 맞게 분배하는 실험을 진행하였으며, 이때 시나리오는 모두 같은 성능을 제공하는 것부터 최대 1:2, 1:3, 1:5, 1:9의 비율로 비례적으로 할당하는 것임. 실제 각 VM의 성능이 원하는 비율만큼 잘 분배되었는지를 평가한다. 본 연구팀에서 개발한 스케줄러는 대역폭, IOPS, 응답 시간 측면에서 모두 거의 정확한 비율로 성능을 분배하였음을 확인하였음. 기존 스케줄러인 mClock, SFQ(D) 등을 이용해 동일한 실험을 진행하여 비교하였을 때, 기존 스케줄러들은 VM 간의 요구 성능 격차가 커질수록 의도한 대로 성능을 분배하지 못하였음을 확인하였음.

두 번째 실험에서는 다양한 형태로 주어진 성능 요구 사항을 얼마나 보장하는지 평가하였음. 우선 각각 다른 서비스를 수행하는 다섯 개 VM (VM1~VM5)에 대해 세 가지 요구 성능 시나리오를 부여하였음. 첫째, VM1에 대해 40MB/s 대역폭 예약을 제공함. 그 결과는 그림 7의 (a)에 보인 것과 같이, 아무런 스케줄링 동작이 없는 경우와 비교해 정확히 40 MB/s를 제공함을 볼 수 있음. 두 번째 시나리오는 VM2에 대해 300 req/s의 IOPS 제한을 수행한 결과임. 이에 따라 그림 7의 (b)에서 보듯 첫 번째 시나리오에서 약 700 정도의 IOPS를 보이던 VM2가 두 번째 시나리오에서는 300 req/s 이하의 성능을 보임을 알 수 있다. VM2의 성능을 제한함에 따라 다른 VM의 성능이 증가한 것도 확인할 수 있음. 세 번째 시나리오에서는 VM3에 대해 5ms의 응

답시간을 제공하고자 하였음. 응답시간은 짧을수록 더 성능이 높다고 할 수 있으므로, 그림 7의 (c)에서 보듯, 기존 시나리오에서는 35ms, 7ms 정도의 성능을 보이던 VM3가 세 번째 시나리오에서는 5ms 이하의 응답시간으로 성능이 높아졌음을 알 수 있음. 이에 따라 다른 VM들은 약간 성능이 저하됨을 확인할 수 있음.

위와 같이 실제와 같은 실험 시나리오에서 평가한 결과, 본 연구팀에서 개발한 새로운 I/O 스케줄러가 목표한 바와 같은 사용자 요구 성능 보장성을 갖추고 있음을 알 수 있었음. 보다 자세한 기술 설명과 추가 실험 결과는 기술 문서(“OIOS:Opportunistic I/O Scheduler for SLA Guarantee”) 로 작성하여 공개하였음.

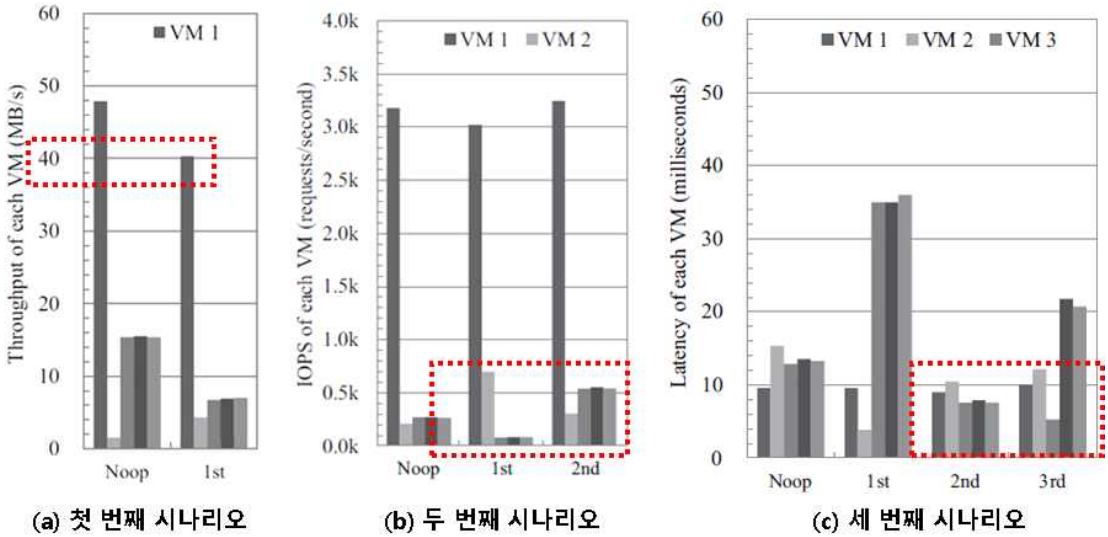


그림 7. 새로운 I/O 스케줄러의 요구 성능 달성 정도를 세 가지 시나리오로 실험한 결과 VM1에 대한 40 MB/s 대역폭 제공, VM2에 대한 300 req/s 제한, VM3에 대한 5ms 응답시간 제공을 요구하는 시나리오들을 만족시킴을 보임. Noop은 리눅스 FIFO 스케줄러를 비교 대상으로 제시한 결과임.

■ 네트워크장치 (10Gbps NIC)를 위한 SLA 보장 스케줄러 코드 개발

● 자원 사용량에 따른 동적 네트워크 자원 할당 기법 개발

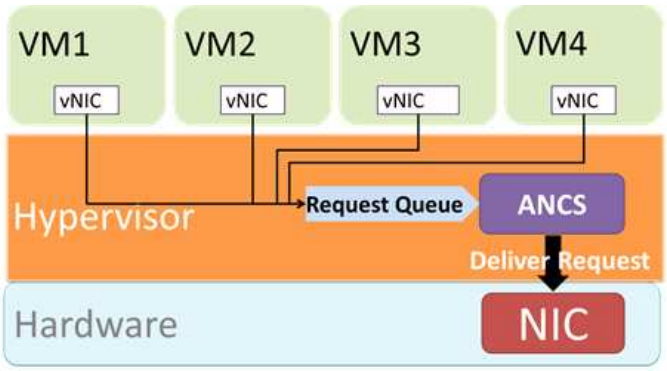


그림 8. ANCS 구조

본 연구에서는 클라우드 환경에서 성능 SLA 보장을 위한 네트워크 스케줄러인 Advanced Network Credit Scheduler (ANCS)를 개발하였음. ANCS는 비례 공정 스케줄러 (Proportional fair scheduler)를 기반으로 한 동적 네트워크 대역폭 할당 기법으로, 그림 8과 같이 Xen 기반

가상화 환경에서 네트워크 백엔드 드라이버의 vNIC (가상머신 네트워크 인터페이스) 스케줄링 방식을 개선한 네트워크 스케줄링 알고리즘임. 가상 머신들을 관리하는 드라이버 도메인의 네트워크 백엔드 드라이버에서 동작함. vNIC들의 작업 요청을 Round-Robin 방식으로 스케줄링하는 것은 기존의 방식과 동일하지만, 10ms마다 주기적으로 각 vNIC의 네트워크 자원인 크레딧을 계산하여 할당함으로써 동적으로 vNIC의 네트워크 성능 조절이 가능하다는 특징이 있음. 크레딧을 계산하여 할당하는 기준은 각 NIC에 따라 달라지는데, 가중치에 따라 비례적으로 할당하거나 최소 혹은 최대치 범위 내의 값을 할당함.

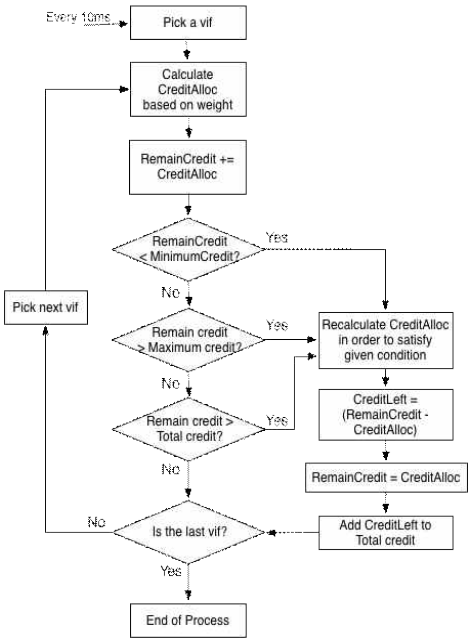


그림 9. 알고리즘 순서도

전체 동작 순서는 그림 9에 나타내었음. 본 기법에서는 전체 시스템 상에 존재하는 모든 vif들을 관리하며 각 vif의 크레딧을 순차적으로 할당함. 우선 각 vif의 가중치에 따른 크레딧 값인 CreditAlloc을 계산하여 현재 잔여 크레딧 값인 RemainCredit에 더함. 그리고 더해진 RemainCredit이 최소값 Minimum credit과, 최대값 Maximum credit, 전체 시스템의 크레딧 Total credit 범위를 만족하는지 확인하는데, 만약 범위를 벗어났다면 범위 내의 값으로 다시 CreditAlloc을 계산한다. 이전에 계산된 RemainCredit과 다시 계산 한 CreditAlloc의 차이는 CreditLeft로 전체 시스템의 크레딧을 조정하는데 쓰임. 새롭게 계산된 CreditAlloc은 RemainCredit에 더해지지 않고 그대로 할당되어 vif의 네트워크 자원으로 쓰임. 시스템 상의 모든 vif의 크레딧 계산이 이루어질 때까지 반복되며 10ms마다 전체 알고리즘이 수행됨.

● 다양한 성능 보장 정책 지원

ANCS는 클라우드 서비스 이용자의 다양한 요구사항을 만족시키기 위해 가중치 기반 공유 정책, 최소 대역폭 보장, 최대 대역폭 제한의 성능 제어 정책을 지원함. 우선, 가중치 기반 공유 정책은 전체 네트워크 자원을 각 가상머신의 가중치가 모든 가상머신의 가중치 합에서 차지하는 비율만큼 네트워크 자원을 할당하는 방식임. 가중치 기반 공유 정책에 의한 vif의 크레딧 계산은 수식(1)과 같다. vif(i)에 할당되는 네트워크 자원 변수인 *CreditAlloc*은 전체 시스템의 크레딧인 *TotalCredit*에서 vif(i)의 가중치인 Weighti가 전체 시스템에서 차지하는 비율로 계산됨. 따라서 각 가상머신의 네트워크 성능은 가중치에 비례하게 나타나며 가중치를 조절하여 네트워크 대역폭 성능을 제어할 수 있음.

$$\text{CreditAlloc} = \text{TotalCredit} \times \frac{Weight_i}{\sum Weight_i} \quad (1)$$

이처럼 가중치에 따라 비례적으로 네트워크 자원을 할당하는 방식은 상대적인 네트워크 성능을 보장할 수 있지만, 사용자가 정량적인 성능 수치를 요구하는 경우에는 이를 만족시키기 어려움. 그러므로 사용자의 요구에 따라 정량적 네트워크 성능을 보장하기 위해 최소 대역폭 보

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

장과 최대 대역폭 제한이 필요함. 최소 대역폭 보장은 vif의 최소 대역폭이 설정된 경우, 가중치에 따라 크레딧을 할당하였을 때 그 값이 최소 대역폭을 만족시키지 못한다면 크레딧을 추가로 할당함으로써 이루어짐. 최대 대역폭이 설정된 경우에는 vif가 할당받은 크레딧이 일정 값을 초과하지 않도록 함으로써 가상머신의 네트워크 성능을 항상 최대 대역폭 이하로 유지함. 최소 대역폭 보장과 최대 대역폭 제한 정책을 통해 클라우드 서비스 공급자는 시스템 환경의 변화에 관계없이 사용자에게 안정적인 성능을 제공할 수 있음.

● 작업 보존성 지원

클라우드 컴퓨팅 환경에서 중요시되는 자원 운영 효율성을 높이기 위해 ANCS에서는 모든 가상머신에서 네트워크 작업이 일어나지 않을 때에만 유휴상태가 되는 작업 보존성을 지원함. 작업 보존성 지원을 위해 ANCS는 주기적인 스케줄링 알고리즘을 이용하여 가상머신의 네트워크 이용량 변화에 따라 자원 할당량을 변경함. 이때 각 가상머신의 잔여 네트워크 자원 변수를 이용하여 가상머신의 네트워크 이용량을 추정함. 가상머신의 잔여 크레딧이 전체 시스템 크레딧 이상일 경우 해당 가상머신의 네트워크 사용량이 감소한 것으로 추정하고 할당되는 네트워크 자원을 다른 가상머신에 분배함으로써 네트워크 자원 이용률을 최대화함. 작업 보장성 지원을 위한 세부 알고리즘은 아래와 같음.

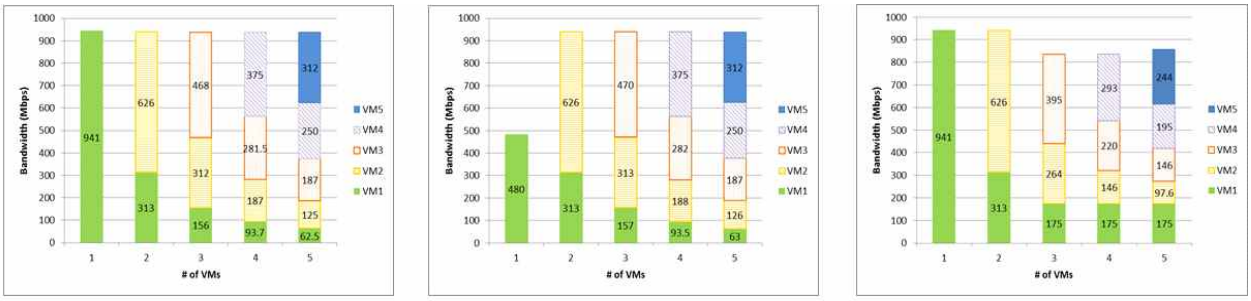
```
Remaining_credit : vif의 잔여 크레딧
credit_fair : 성능 제어 정책에 따라 할당받는 네트워크 자원
Credit_total : 전체 시스템의 네트워크 자원
Credit : 최종적으로 할당 받는 크레딧
remaining_credit += credit_fair
If remaining_credit > Credit_total Then
credit_left += (remaining_credit - credit_fair)
credit_total += credit_left
remaining_credit = credit_fair
```

ANCS에서는 주기적으로 가상머신의 vif에 각 vif가 요구하는 성능 제어 정책에 기반하여 크레딧을 계산하여 할당함. 이 때 vif의 크레딧 값이 일정 값 커지면 vif의 네트워크 사용량이 적어 할당 받은 크레딧을 모두 소모하지 않은 것이므로 해당 vif의 잉여 네트워크 자원을 다른 vif에 분배함. 이 같은 방식으로 네트워크 자원의 낭비 없이 각각의 가상머신에 따라 서로 다른 네트워크 성능 보장이 가능함.

● 성능 평가

2대의 물리 머신을 1:1로 1Gbps Ethernet 네트워크로 직접 연결하여 실험 환경을 구성하였음. 한 대의 머신에는 수정하지 않은 리눅스 3.11.10 버전을 설치하였고, 상대방 머신으로부터 패킷을 받아 성능을 측정하는 용도로 사용하였음. 다른 한 대의 머신은 Xen 4.2.1 버전을 가상화 플랫폼으로 적용하였음. Xen의 Dom0의 네트워크 백엔드 드라이버에 본 연구에서 개발한 동적 네트워크 자원 할당 기법을 구현하여 가중치 기반 공유 정책을 적용 하였을 때의 실험 결과를 그림 10.(a)에 나타내었음. 본 실험에서는 각 가상 라우터의 가중치를 각각 1, 2, 3, 4, 5로 설정하고, 각각의 가상 라우터에서 순차적으로 iperf를 실행하면서 네트워크 대역폭 변화

를 측정하였음. 실험 결과, 네트워크 작업을 수행하는 가상 라우터의 개수가 늘어남에 따라 각 가상 라우터의 네트워크 대역폭 성능이 서로 다르게 나타나는 것을 확인할 수 있었음. 이 때, 대역폭 값은 모두 동일한 값이 아닌 각 가상 라우터의 가중치에 따라 비례하게 나타났음. 또한 전체 대역폭의 합은 항상 940Mbps를 유지하여 네트워크 자원의 낭비 없이 네트워크 이용률이 100%를 달성함을 확인하였음.



(a) 가중치 기반 공유 정책 (VM1:VM2:VM3:VM4:VM5=1:2:3:4:5)

(b) 최대 대역폭 제한 (VM1=500Mbps를 제외하고 이전 실험과 동일)

(c) 최소 대역폭 보장 (VM1=200Mbps를 제외하고 이전 실험과 동일)

그림 10. ANCS 성능 평가 실험 결과

그림 10.(b)는 VM1에 대해 최대 대역폭을 500Mbps로 제한했을 때의 실험 결과임. 이 경우 VM1이 사용 가능한 네트워크 자원이 충분하더라도 실제 대역폭은 500Mbps를 초과하지 않도록 네트워크 자원 할당을 제한함. 따라서 T1의 경우 이전 실험 결과들과는 달리 VM1의 대역폭이 480Mbps로 나타남. T2~5의 경우는 VM1의 네트워크 대역폭이 500Mbps이기 때문에 이전과 같이 가중치 기반 공유 정책을 적용 받아 가중치에 따라 비례한 대역폭 성능이 측정됨.

그림 10.(c)은 최소대역폭 보장정책을 적용했을 때의 실험 결과임. 첫 번째로 실행하는 가상 라우터인 VM1에 대해 최소대역폭을 200Mbps로 설정하였고, 나머지 가상 라우터들에는 4.2.1에서의 실험과 동일하게 가중치를 설정하였음. 실험 결과, 가상 라우터의 개수가 점차 늘어나 가중치에 따른 VM1의 대역폭이 200Mbps 이하가 되는 경우(T3, 4, 5)에 VM1의 대역폭 성능은 175Mbps로 일정하게 나타났음. 단일 물리머신을 공유하는 가상 라우터의 개수가 늘어남에 관계없이 일정하게 가상 라우터의 네트워크 성능을 보장하였음. 다만 가상 라우터의 네트워크 성능이 설정한 최소 대역폭 값인 200Mbps보다 다소 낮은 175Mbps로 측정되는 이유는, 크레딧을 이용한 네트워크 자원 할당이 정확한 대역폭 성능으로 나타나지 않고 근사치의 네트워크 성능으로 도출되기 때문임. 또한 최소 대역폭 보장 정책을 적용하였을 때 전체 네트워크 성능이 90Mbps 정도 저하되었는데, 이는 최소 대역폭 보장을 위해 별도로 취급한 크레딧이 전체 시스템 관점에서는 자원 감소로 여겨졌기 때문임. 크레딧 계산 알고리즘을 부분적으로 수정함으로써 해결 가능하며 현재 최적화 작업을 진행 중.

2) SLA 보장 CPU 스케줄러 코드 개발

- **자원 의존성 문제**

가상화 환경은 하이퍼바이저라는 가상화 레이어를 통해 물리 자원을 가상화하여 가상 머신에게

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

분배함으로써 가상 머신에게 독립적인 실행 환경을 제공함. 하지만 물리 자원의 가상화로 인해 자원을 사용하기 위해서는 하이퍼바이저에서 에뮬레이션을 거쳐야 하는 오버헤드가 발생함. 예를 들어 가상 머신이 블록 I/O 요청을 I/O 장치에 보낸다면 하이퍼바이저에서 가상 머신의 I/O 요청을 실제 장치로 보내기 전에 가상 머신에게 제공한 가상 장치에 맞게 에뮬레이션함. 이러한 에뮬레이션으로 인해 물리 장치를 사용하는 환경과 차이가 발생함.

가상 머신의 I/O 처리를 위한 하이퍼바이저의 에뮬레이션은 물리 장치를 직접 사용할 때와 다르게 추가적인 자원을 필요로 함. 즉, 에뮬레이션은 하이퍼바이저가 가상 머신에게 제공한 가상 장치의 동작 방식을 소프트웨어적으로 수행하기 때문에 기존 환경과 다르게 CPU 자원을 필요로 함. 따라서 기존에는 I/O 처리를 위해 I/O 자원만 필요로 했지만, 가상화 환경에서는 I/O 처리를 위해 CPU 자원 역시 필요함. 반가상화(Para-virtualization)의 경우 이러한 에뮬레이션 때문에 발생하는 오버헤드를 줄이기 위해, 가상 머신의 I/O 장치 드라이버를 가상화에 맞게 수정함. 예를 들어, KVM의 경우 QEMU의 virtio를 사용하여 반가상화 형태로 가상 머신에게 I/O 장치를 제공하여 가상화 오버헤드를 줄임. 그러나 반가상화 환경이라도 가상 머신이 I/O 처리를 하는데 에뮬레이션 과정을 거치기 때문에 오버헤드가 존재하며, I/O 처리를 위해 CPU 자원이 필요함. 본 논문에선 이러한 가상화 환경에서 작업을 수행하는데 필요한 자원들의 관계를 자원 의존성이라고 정의함. 즉, 가상 머신이 블록 I/O 작업은 에뮬레이션을 위한 CPU 자원과 실제 I/O 처리를 위한 I/O 자원이 필요하므로 블록 I/O 작업은 CPU와 블록 I/O 자원에 의존성이 존재함.

가상화 환경에서 작업의 자원 의존성을 확인하기 위해 KVM 기반의 가상화된 서버를 구성하였음. 가상화된 서버는 SSD를 사용하며, 두 개의 가상 머신이 동작함. 가상 머신들은 각각 블록 I/O 작업 및 CPU 작업을 수행하며, 시나리오에 따라 요구 성능이 다르게 설정됨. 아래는 실험에 사용된 물리 머신 및 가상 머신의 사양과 실험 시나리오를 정리한 표임.

표 2. 실험에 사용된 머신 사양

		Version
Host server	CPU	Intel Xeon E5-2650 @ 2.30GHz
	Main memory	128 GB (DDR4 16G x 8)
	Storage	Samsung SSD 850 Pro 256GB
	Linux version	4.4.0 (Ubuntu 16.04)
	KVM version	QEMU 2.5.0
Virtual machine	vCPU	4
	Main memory	1GB
	Storage	SSD 10GB
	Linux version	4.4.0 (Ubuntu 16.04)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

표 3. 시나리오별 가상 머신에서 실행되는 작업 및 요구 성능

		VM 1	VM 2
Scenario 1	Job type	Block I/O (Random read 4K)	-
	SLA goal	Bandwidth 200 MB/s	
Scenario 2	Job type	Block I/O (Random read 4K)	CPU
	SLA goal	Bandwidth 200 MB/s	Guarantee 5 cores

본 실험에서 사용된 I/O 작업은 Iometer[5]를 활용하여 4KB 크기의 임의 읽기 작업을 수행했음. Iometer의 옵션으로 가상 머신의 vCPU 수와 동일한 작업 프로세스를 생성하고 각 작업 프로세스가 128의 큐 깊이를 가지도록 설정하여 최대 블록 I/O 성능을 보일 수 있도록 하였음. CPU 작업은 Sysbench[6]에서 제공하는 CPU 작업을 활용하여 가상 머신의 CPU 자원을 모두 사용하도록 하였음.

● 자원 의존성 실험 및 결과 분석

그림 11과 12는 시나리오 1의 실험 결과이다. 시나리오 1은 I/O 작업을 수행하는 가상 머신이 단독으로 존재하는 환경이기 때문에 가상 머신의 최대 성능을 확인할 수 있다. 그림 11을 보면 가상 머신의 I/O 대역폭이 340MB/s 임을 확인할 수 있다. 실험에서 설정한 요구 성능이 200MB/s이기 때문에 SLA 만족도는 170%를 유지하고 있다. 즉, 물리 머신에서 I/O 작업을 수행하는 가상 머신이 단독으로 존재할 경우 사용자의 요구 성능 달성이 가능하다.

아래 그림 13과 14는 시나리오 2의 실험 결과임. 시나리오 2에서는 I/O 작업을 수행하는 가상 머신 1번(VM1)과 CPU 작업을 수행하는 가상 머신 2번(VM2)이 존재하는 환경임. 시나리오 1과 다르게 시나리오 2에서의 I/O 작업을 수행하는 가상 머신이 낮은 성능을 가지고 있는 것을 확인할 수 있음.

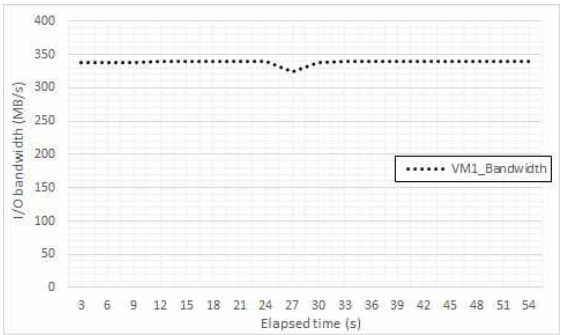


그림 11. 시나리오 1에 대한 가상 머신 성능

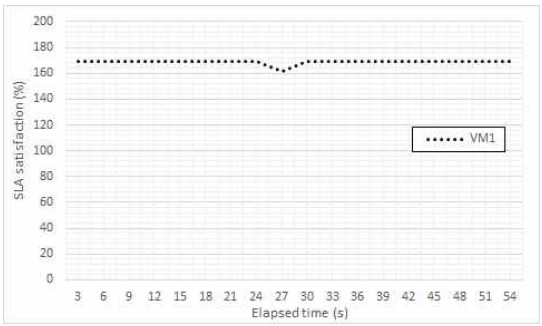


그림 12. 시나리오 1에 대한 SLA 만족도

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

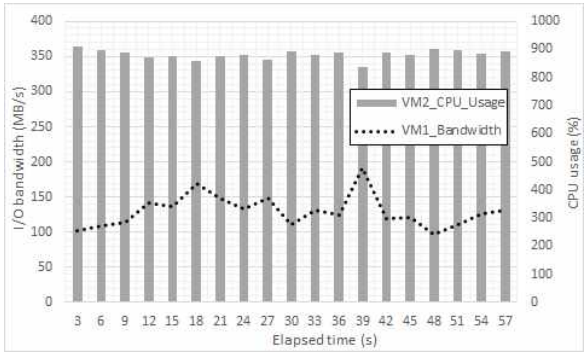


그림 13. 시나리오 2에 대한 가상 머신 성능

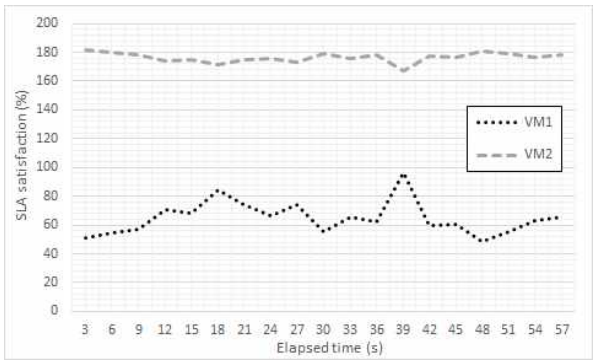


그림 14. 시나리오 2에 대한 SLA 만족도

그림 14는 시나리오 2에서 각 가상 머신의 요구 성능에 대한 SLA 만족도를 나타내고 있음. 시나리오 1에서는 170%의 만족도를 나타내던 VM1이 시나리오 2에서는 약 70%로 나타나며, CPU 작업을 수행하는 VM2가 180%의 SLA 만족도를 나타내고 있음. 시나리오 1과 2의 실험을 비교하면 CPU 작업에 의해 I/O 작업의 성능이 영향을 받는 것을 확인할 수 있음.

이러한 서로 다른 작업 간 성능 간섭이 발생하는 원인을 분석하기 위해 각 시나리오에 대해 가상 머신의 CPU 점유율과 I/O 성능을 확인하였음. 그림 15와 16은 각 시나리오별 가상 머신의 CPU 점유율 및 I/O 성능을 나타내는 그래프임. 그림에서 에뮬레이터는 하이퍼바이저에서 수행하는 장치 에뮬레이션이 사용하는 CPU 점유율을 나타냄. 그리고 그래프의 CPU 점유율의 최대값이 1000%인 이유는 물리 코어의 수가 10개이며 각 코어를 완전히 점유했을 때 100%이기 때문에 가상 머신이 최대 점유할 수 있는 CPU 점유율은 1000%가 됨.

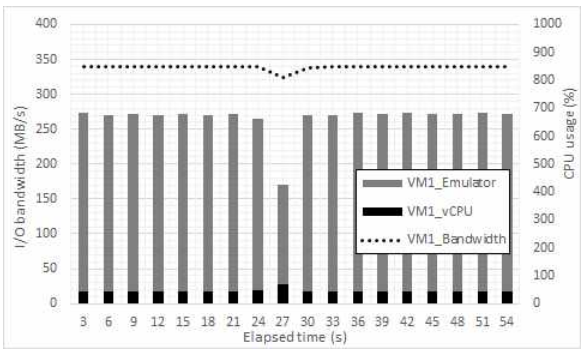


그림 15. 시나리오 1에 대한 CPU 점유율 및 I/O 성능

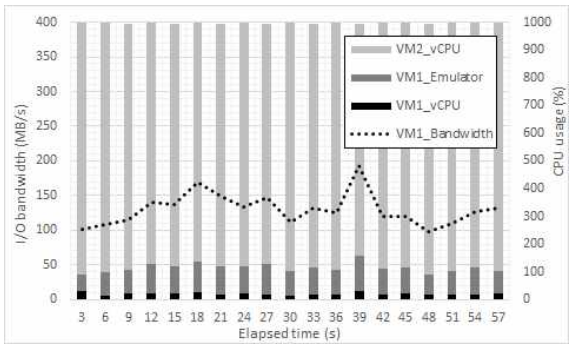


그림 16. 시나리오 2에 대한 CPU 점유율 및 I/O 성능

그림 15를 보면 I/O 작업을 수행하는 가상 머신이 단독으로 존재할 경우 CPU 자원을 약 650% 사용하는 것을 확인할 수 있음. 반면에 그림 16을 보면 I/O 작업을 수행하는 가상 머신(VM1)과 CPU 작업을 수행하는 가상 머신(VM2)이 함께 존재할 경우, VM1의 CPU 점유율이 100% 정도인 것을 확인할 수 있음. 그리고 나머지 900%를 VM2가 사용하는 것을 확인할 수 있음. 그리고 이러한 CPU 점유율 변화에 따라 I/O 성능 역시 변화하는 것을 볼 수 있음. 즉, 가상화 환경에서 I/O 작업의 자원 의존성이 I/O 자원뿐만 아니라 CPU 자원도 존재하기 때문에 위 실험과 같은 현상이 발생했음을 알 수 있음.

이처럼 가상화 환경에서 작업의 자원 의존성 문제로 인해 블록 I/O 작업이 CPU 작업에 의해 성

능 간섭을 받게 되고, 이는 성능 저하로 이어짐. 따라서 가상화 환경에서 성능 보장을 위해 단일 자원만 고려할 경우 작업의 자원 의존성으로 인해 성능을 보장하기 어려움. 물리 머신에서 동작하는 가상 머신들이 동일 작업을 수행할 경우, 작업의 자원 의존성이 동일하기 때문에 하나의 자원을 가지고 가상 머신 간 성능 제어가 가능함. 예를 들어, 모든 가상 머신이 블록 I/O 작업을 수행할 경우 I/O 자원을 제어하여 가상 머신의 성능을 차등화하거나 요구 성능 보장이 가능함. 하지만 실제 가상화가 사용되는 클라우드 환경에서는 물리 머신에 서로 다른 작업을 수행하는 가상 머신이 동작할 수 있으므로, 서로 다른 작업 간의 자원 의존성을 고려해야함. 즉, 가상화 환경에서 작업 자원 의존성을 고려하여 사용자 요구 성능 달성이 가능한 새로운 자원 할당 기법이 필요함.

- 설계기법 (GoS): Guarantee of SLA(이하 GoS) 프레임 워크는 가상화 환경에서 작업의 자원 의존성을 고려하여 가상 머신이 수행하는 CPU 및 블록 I/O 작업에 대한 요구 성능 보장을 위한 피드백 제어 스케줄링 기법임.

GoS 프레임 워크의 핵심 아이디어는 가상 머신들의 SLA 만족도를 기준으로 가상 머신에게 자원을 차등 분배하는 것임. 자원의 차등 분배는 기존의 공평하게 자원을 분배하는 방식으로는 가상 머신들이 요구하는 각기 다른 성능 조건을 만족할 수 없으므로, 가상 머신의 요구 성능 보장을 위해 필요함. 그리고 자원 차등 분배에서 중요한 요소가 바로 차등 분배 기준임. 자원 차등 분배 기준은 자원을 차등 분배할 때 해당 가상 머신이 할당받아야 할 자원을 결정하는 요소이기 때문에 기준에 따라 분배 정책이 달라진다고 할 수 있음. GoS 프레임 워크는 자원 차등 분배 기준을 SLA 만족도로 정하고, 가상 머신들의 SLA 만족도가 100%에 도달할 수 있도록 가상 머신 간 자원을 분배하는 자원 분배 기법임. GoS 프레임 워크의 구성 요소에는 SLA 매니저, 자원 차등 분배 스케줄러 및 통계 수집기가 있음.

■ 프레임 워크 구조 설계

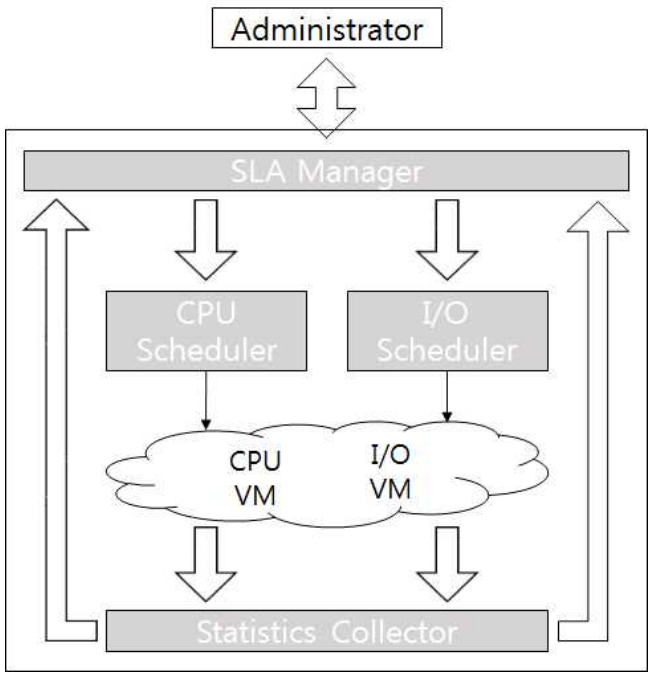


그림 17. GoS 프레임 워크 구조

그림 17은 본 논문에서 제안하는 GoS 프레임 워크의 구조임. GoS 프레임 워크는 I/O 스케줄러로 기존 기법인 OIOS를, CPU 스케줄러로는 리눅스의 기본 스케줄러인 CFS를 사용함. 그리고 GoS

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

프레임 워크를 위해 SLA 매니저와 통계 수집기 및 CPU 자원 차등 분배를 위한 컴포넌트를 구현하였음. 또한, GoS 프레임워크에서 OIOS가 동작할 수 있도록 수정하였음. 다음으로 각 컴포넌트에 대한 세부 사항을 설명함.

- **SLA 매니저**

SLA 매니저는 GoS 프레임 워크에서 사용되는 사용자의 성능 요구 사항인 SLA 값과 SLA 만족도, 가상 머신이 사용하는 자원 점유율과 가상 머신의 성능 등의 정보를 관리함. 또한, 관리자에게 SLA 값 설정과 SLA 만족도를 확인할 수 있는 API를 제공하여 관리자가 프레임 워크를 제어할 수 있도록 함.

SLA 매니저를 통해 관리자가 설정할 수 있는 SLA는 CPU와 블록 I/O와 관련된 값을 설정할 수 있음. CPU의 경우 가상 머신이 사용할 수 있는 vCPU의 수와 별개로 자신이 항상 점유하고 있길 원하는 CPU의 수를 SLA 값으로 설정할 수 있음. 예를 들어, 가상 머신의 vCPU 수가 10개일 때 사용자가 5개의 CPU를 항상 점유하고 사용하길 원하면 SLA 값으로 5코어를 설정하게 됨. 블록 I/O의 경우 가상 머신이 요구하는 대역폭을 SLA 값으로 설정할 수 있음.

SLA 매니저는 통계 수집기로부터 전달받은 각 가상 머신의 성능 정보를 통해 SLA 만족도를 계산하고 그 값을 각 자원 스케줄러에 전달함. SLA 매니저는 CPU에 대한 SLA를 설정한 가상 머신과 I/O에 대한 SLA를 설정한 가상 머신의 SLA 만족도를 구별하여 계산 및 관리하며 SLA 종류에 따라 자원 스케줄러가 해당 SLA 만족도를 사용할지가 결정됨.

$$CPU\,VM_i : SLA_satisfaction = \left(\frac{CPU_{occupancy_{time}}}{Period}\right) \div Target_core_{number} \times 100 \quad (1)$$

$$I/O\,VM_i : SLA_satisfaction = \frac{Bandwidth}{Target_bandwidth} \times 100 \quad (2)$$

위 식을 보면 CPU 작업을 수행하는 가상 머신의 SLA 만족도는 일정 주기에 해당 가상 머신이 점유한 CPU 시간을 가지고 CPU 점유율을 계산하고 이것을 사용자의 요구 성능인 목표 코어 수로 나누고 100을 곱하여 SLA 만족도를 백분율로 계산함. I/O 작업을 수행하는 가상 머신의 SLA 만족도는 실제 통계 수집기로부터 수집된 대역폭을 목표 대역폭으로 나누고 100을 곱하여 백분율로 계산함. 그리고 I/O 자원 스케줄러의 경우 CPU 가상 머신의 SLA 만족도를 사용할 필요가 없으므로 SLA 매니저는 I/O 가상 머신의 SLA 만족도만 I/O 자원 스케줄러에 전달함. 그리고 CPU 자원 스케줄러는 CPU 및 I/O 가상 머신 전체의 SLA 만족도를 가지고 자원 스케줄링을 진행함.

- **자원 차등 분배 스케줄러**

GoS 프레임 워크에서 자원 차등 분배 스케줄러는 SLA 매니저가 관리하는 SLA 만족도를 기반으로 실제 가상 머신에게 자원을 차등 분배하여 목표 성능을 달성하도록 하는 구성요소임. GoS 프레임 워크는 CPU 자원 차등 분배를 위해 CFS를, I/O 자원 차등 분배를 위해 OIOS를 사용함. 하지만 SLA 만족도를 기준으로 요구 성능 달성이 가능한 다른 방식의 스케줄러가 있다면 해당 스케줄러를 대용으로 활용할 수 있음.

GoS 프레임 워크에서 CPU 자원의 차등 분배를 위해서 CFS의 Quota를 활용함. Quota는 프로세

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

스가 일정 주기(period) 동안 CPU를 점유할 수 있는 시간을 결정할 수 있음. 즉, Quota의 크기에 따라 CPU 점유율을 변경할 수 있어 프로세스별로 CPU 자원을 차등 분배할 수 있음. 따라서 GoS 프레임 워크의 CPU 스케줄러는 CFS의 Quota 값을 조절하여 가상 머신이 요구하는 성능에 도달할 수 있도록 함. I/O 스케줄러의 경우, 기존에 연구가 되었던 가상화 환경에서 가상 머신의 I/O 성능 보장을 위한 I/O 스케줄러인 OIOS를 사용하여 I/O 자원을 차등 분배함. OIOS는 각 가상 머신에게 I/O 큐를 할당하고, DAT(Differentially Allowed Time)라는 값을 가상 머신에게 부여함. DAT는 가상 머신의 I/O 큐에서 실제 I/O 장치로 I/O 요청을 전송할 수 있는 시간을 나타내는 값으로 DAT 값에 따라 I/O 요청을 할 수 있는 시간이 정해지기 때문에 가상 머신 간 성능 차등화를 달성할 수 있음. 이를 통해 DAT 값을 조절하여 가상 머신이 요구하는 성능을 만족함

각 스케줄러는 피드백을 통해 변수를 조절하여 SLA 만족도를 100%에 도달하게 함. 즉, SLA 만족도를 기준으로 CFS의 Quota나 OIOS의 DAT 값을 피드백 제어하여 현재 SLA 만족도에서 100%의 만족도에 도달할 수 있는 Quota나 DAT 값을 구하고 그에 따라 각 가상 머신에게 자원을 분배함. OIOS의 경우 이러한 피드백 제어 기능을 제공하여 요구 성능 보장을 달성하게 함. 하지만 CFS의 경우 자원 차등 분배를 위한 Quota 기능을 제공할 뿐 요구 성능 달성을 위한 기능은 제공하지 않음. 따라서 CPU 자원 차등 분배를 위한 피드백 제어 컴포넌트를 추가로 구현하였음. 해당 내용은 4.2장에서 다룰 예정임.

- **통계 수집기**

GoS 프레임 워크의 통계 수집기는 가상 머신에 대한 정보를 수집하고 이를 정리하여 SLA 매니저에게 전달하는 역할을 함. 통계 수집기에서 수집하는 정보는 가상 머신의 자원 점유율과 I/O 성능이 있음. 자원 점유율은 CPU 자원에 대한 점유율을 수집하며, I/O 성능은 IOPS, 대역폭, 지연 시간을 수집함. 해당 정보는 커널에서 사용하는 구조체나 함수를 활용하여 수집함. 커널에서는 CPU 점유율이나 대역폭과 같이 일정 주기에 대한 정보가 아닌 누적된 통계를 저장하고 있으므로 통계 수집기에서는 누적된 정보를 가공하여 CPU 점유율과 대역폭의 형태로 계산한 결과를 SLA 매니저에게 전달함.

4.2 CPU 자원 차등 분배 컴포넌트

GoS 프레임 워크에서 CPU 자원 차등 분배로 사용하는 CFS는 앞서 언급한 대로 자원 차등 분배를 위한 Quota 기능을 제공할 뿐 실제 요구 성능 보장을 위한 기능은 제공하지 않음. 따라서 GoS 프레임 워크 구현을 위해 CFS의 Quota 값을 피드백 제어하여 가상 머신의 요구 성능을 만족할 수 있는 CPU 자원 차등 분배 컴포넌트를 추가로 구현하였음.

CPU 자원 차등 분배 컴포넌트가 사용하는 피드백 제어 식은 다음과 같음.

$$CPU_quota_i = \frac{\Delta CPU_quota_{i-1}}{\Delta SLA_{i-1}} \times (\mathrm{Goal} - SLA_{i-1}) + CPU_quota_{i-1} \quad (3)$$

위 식은 I-1 번째 피드백 주기에서 I 번째 Quota 값을 결정할 때 사용되는 피드백 제어 식임. I-1 번째 피드백 주기에서 Quota 값의 변화에 따른 SLA 만족도 변화의 비율을 가지고 목표 SLA 만족도인 100%에 도달하는 데 필요한 Quota 값을 계산하는 식임. 즉, 현재 피드백 주기는 I-1 번째이며 I-2 번째에서 I-1 번째로 변화하면서 관측된 정보를 기반으로 I 번째에 목표에 도달할 수 있는 Quota 값을 구하는 방식임. 해당 피드백 제어식의 모든 값은 각 피드백 주기에서 수집 가능

한 데이터들로 구성되어있음. 따라서 환경의 변화, 예를 들어 물리 머신의 성능이 달라지거나 가상 머신의 수, 할당받은 자원의 양, 요구 성능 등이 바뀌더라도 해당 환경에서 수집된 정보를 활용하여 모델의 값들을 설정하기 때문에 환경에 대한 의존도가 낮다는 장점이 있음. 하지만 비율만을 가지고 피드백 제어를 수행하기 때문에 불안정할 수 있다는 문제점이 있음. 하지만 단일 피드백 주기에 변경 가능한 Quota의 양을 제한하여 불안정성에 대한 문제를 해결할 수 있음. 그리고 피드백 제어는 OIOS와의 동기화를 위해 OIOS가 기준으로 사용하고 있는 3초를 동일하게 사용함.

SLA 만족도가 100±1%일 경우 CPU 자원 차등 분배 컴포넌트는 해당 가상 머신이 SLA를 만족하고 있다고 판단함. 그리고 SLA 만족도가 99% 미만인 가상 머신의 그룹을 SLA 불만족한 그룹이라고 정하며, 만족도가 101%를 초과하는 그룹을 SLA 초과 그룹이라고 정함. 이후 두 그룹에 대하여 Quota 값을 재분배하여 현재 SLA 목표를 달성한 가상 머신들의 성능에 영향을 주지 않고 나머지 가상 머신들의 SLA 목표를 달성할 수 있도록 함. 두 그룹에 대한 Quota 값을 재분배할 때, SLA 불만족 그룹의 가상 머신들이 피드백 제어를 통해 각 가상 머신이 필요한 Quota의 양을 계산함. 그리고 SLA 초과 그룹은 각 가상 머신의 SLA 만족도와 Quota의 비율에 따라 불만족 그룹에 제공 가능한 Quota 양을 계산함. 이후 두 값을 비교하여 작은 값을 실제 그룹 간 교환되는 Quota 값으로 설정하고 해당 값을 가상 머신들의 불만족 비율, 또는 초과 비율에 따라 각 그룹에서 분배함. 즉, 초과 그룹의 경우 교환되는 Quota 값을 초과한 비율에 따라 나눈 후 해당 Quota 값만큼을 현재 Quota에서 빼게 됨. 불만족 그룹의 경우 교환되는 Quota 값을 불만족 비율에 따라 나눈 후 해당 Quota 값만큼을 현재 Quota에 더하게 됨. 이 방식은 전체 Quota를 일정하게 유지하도록 하며, 그룹 간의 Quota 교환은 물리 머신 내에서 발생하기 때문에 오버헤드가 적음. 또한, 해당 방식은 초과 그룹이 피드백 제어 이후 불만족 그룹으로 넘어가는 것을 방지하는 효과를 가짐. 하지만 현재 초과한 양을 비율에 따라 계산하기 때문에 동일한 Quota 값에서 다른 SLA 만족도를 가지게 되면 해당 가상 머신이 불만족 그룹으로 옮겨갈 가능성이 존재함.

마지막으로 해당 CPU 자원 차등 분배 컴포넌트는 CFS의 Quota를 사용하기 때문에 작업 비보존(Non-work conserving)형태로 동작함. 따라서 물리 머신의 CPU 자원을 완전하게 활용할 수 없음. Quota를 사용하지 않는 CFS의 경우 I/O 작업으로 발생하는 CPU 유휴 시간에 CPU 작업이 CPU를 사용하도록 함으로써 CPU 자원의 활용을 극대화함. 하지만 Quota를 활용할 경우 CPU 작업이 사용할 수 있는 CPU 시간이 없다면 CPU 유휴 시간에 CPU를 점유할 수 없으므로 CPU 자원을 완전하게 활용하지 못함. 하지만 Quota는 작업이 CPU를 점유할 수 있는 시간을 엄격하게 제한하도록 하여 작업 간 CPU 자원 분배를 절대적으로 할 수 있도록 함.

3) CPU-I/O 통합 스케줄링 구조 설계

- **통합 자원 스케줄링 기법** - 가상화 환경에서 가상머신에서 활용하는 CPU와 범용적인 I/O 자원 관리의 통합을 목적으로 진행하였음. 구현의 측면에서 기존의 CPU와 블록 I/O의 통합을 중심으로 개발된 연구 결과물의 기능을 확장하여 네트워크 I/O를 지원하도록 개발함. 또한, 3차년도까지 이미 설계된 내용을 바탕으로 시스템의 효율성을 위해 CPU 자원에 대한 제어만으로 블록 I/O 및 네트워크 I/O 성능 제어가 가능하도록 설계하여 노드 레벨 자원 스케줄링의 효율성을 높임.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

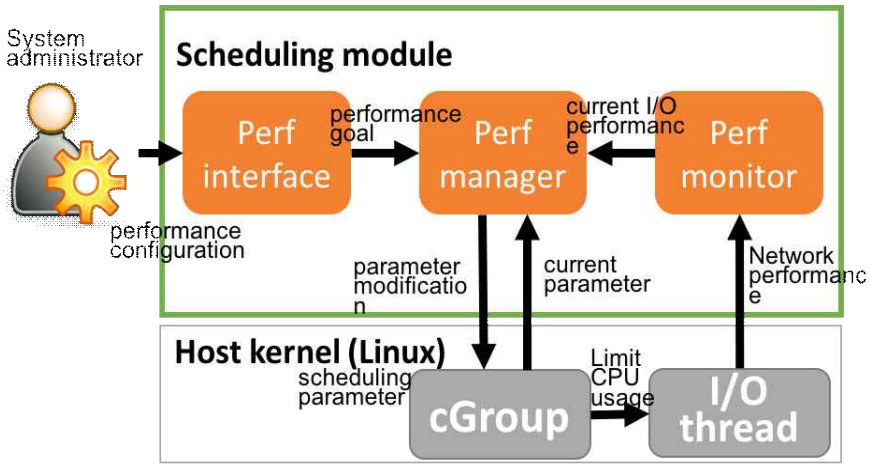


그림 18. 통합스케줄러 동작 방식

- **성능 모니터링 (Perf monitor)** - 주기적으로 가상머신의 자원 사용량 모니터링 및 요구 성능/실제 성능 비교함. 1초마다 시스템 상에 동작하는 가상머신의 CPU 및 블록 I/O, 네트워크 I/O 사용량 및 성능 정보 수집함. CPU와 블록 I/O의 경우 각각 리눅스에서 제공하는 cGroup[5] API와 장치 통계정보를 이용하여 실시간 자원 사용량 및 성능을 계산함. 네트워크 I/O의 경우 KVM 가상화 환경에서 가상머신의 네트워크 작업을 처리하는 vHost 드라이버에서 가상머신 별 송신하는 패킷의 개수와 대역폭을 계산함. 각 가상머신의 성능은 사용자로부터 입력받은 요구 성능과 비교하여 실제 성능이 요구 성능에 미치지 못하는 경우 2단계의 자원 경쟁 탐지 및 자원 스케줄링 조절을 수행함

- **가상머신에서 수행되는 작업의 특성 확인 (Perf interface)** - 각 가상머신은 사용자에 의해 특정 자원에 대한 요구 성능 설정이 가능함. 자원을 종류는 CPU, 블록 I/O, 네트워크 I/O 세 가지로 설정할 수 있고 CPU의 경우 가상머신의 CPU 활용율을 요구 성능으로 설정하고 블록 I/O의 경우 대역폭 (MB/s)이 설정 가능함. 네트워크 I/O의 경우 초당 처리되는 패킷의 수, 또는 대역폭 (Mbps)로 요구 성능을 설정할 수 있음.

- **자원 경쟁 탐지 및 자원 스케줄링 조절 (Perf manager)** - 가상머신의 작업 특성(CPU or I/O-intensive)에 따른 스케줄링 정책을 수행함. CPU 가상머신의 경우 가상머신의 CPU 활용율 달성을 목표로 하므로 가상머신의 가상 CPU 사용량을 조절함. 블록 I/O와 네트워크 I/O 가상머신의 경우 가상머신의 CPU 활용율 보다는 블록 I/O과 네트워크 성능 달성이 중요하므로 가상머신의 I/O 성능을 결정하는 I/O 스레드의 CPU 사용량을 조절함. 블록 I/O과 네트워크 작업 처리는 별도의 I/O 스레드에서 담당함. 본 연구 개발물에서는 가상머신의 작업특성에 따라 블록 I/O 또는 네트워크 스레드의 CPU 사용량을 독립적으로 제어함.

- **피드백 제어 기반의 CPU 자원 할당량 조절** - 가상머신이 여러 작업을 수행하는 과정에서 실제 성능이 사용자에 의해 설정된 요구 성능에 미치지 못하는 경우 통합 자원 스케줄러에서는 주기적으로 가상머신의 CPU 사용량 및 블록 I/O및 네트워크 스레드의 CPU 사용량을 조절하여 요구 성능을 달성하도록 함. CPU 사용량 제어 알고리즘은 피드백 제어를 기반으로 목표 성능 대비 실제 성능의 차이를 계산하여, 목표 성능을 달성하기 위한 CPU 사용량의 비율을 각 가상머신에 적용함. 작업특성 분석을 위한 사전 프로파일링이 필요한 기존 연구와는 달리 피드백

5) Rosen, Rami. “Resource management: Linux kernel namespaces and cgroups.“ *Haifux, May* 186 (2013).

기법을 통해 실시간으로 목표 달성을 위한 CPU 사용량 제어가 가능하므로 프로파일링을 위한 오버헤드를 최소화 할 수 있음. 가상머신 및 블록 I/O 및 네트워크 스레드의 CPU 사용량 제어는 리눅스 cGroup에서 제공하는 API를 활용.

4) CPU, 네트워크, 블록 I/O 통합 스케줄러 개발 및 오픈스택 노드에 적용

가. SLO 세팅을 위한 대시보드 확장: Stella-Horizon 프로토타입

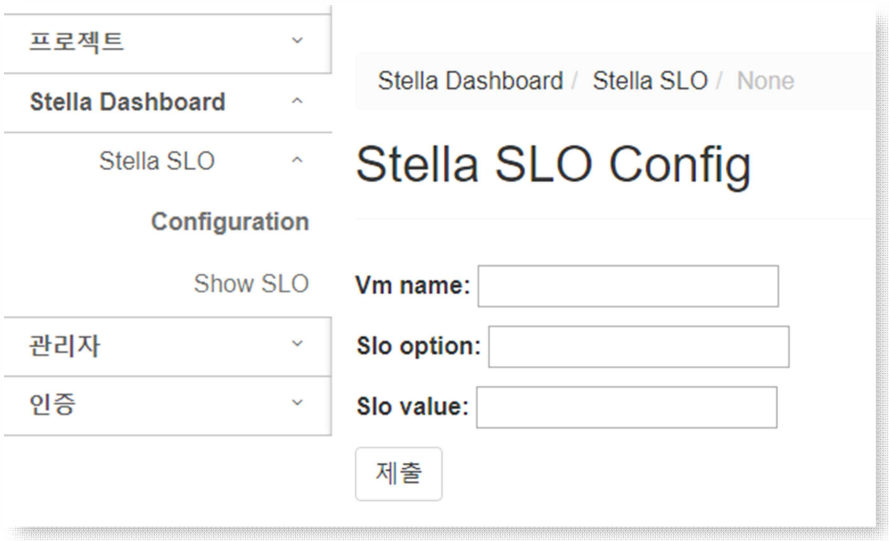


그림 19. Stella-Horizon 화면 구성

- 스텔라 클라우드를 통해 가상머신의 SLO를 설정하기 위해서는 기존 클라우드 환경에서 동작하는 대쉬보드를 수정하여 SLO configuration 기능을 추가 해야함.
- 본 연구팀은 SLO 구성 기능이 추가된 대시보드를 Stella Dashboard로 명명함. 그리고, Stella Dashboard의 OpenStack 버전을 Stella-Horizon으로 명명함. Stella-Horizon을 개발하기 위해 OpenStack의 레퍼런스 대시보드인 Horizon을 수정했음 (그림 19). OpenStack Horizon은 OpenStack의 하위 프로젝트 중 하나로 OpenStack 기반 클라우드에서 웹 기반 대시보드를 지원함. Horizon은 Python 웹 프레임워크인 Django를 기반으로 만들어짐. Horizon은 대시보드 레퍼런스로 쓰이기 때문에 다양한 업체에서 커스터마이즈하여 사용하고 있음.
- Stella-Horizon은 위 그림과 같이 SLO 설정(Stella SLO Config) 페이지를 제공함. SLO 설정을 위해 오픈스택 대시 보드 상의 가상머신 이름(VM name), SLO 적용할 항목(SLO option), SLO 값(SLO value)를 입력한 다음 제출 버튼을 누르면 가상머신의 SLO를 설정하는 명령을 Horizon을 통해 노드레벨 통합 자원 스케줄러에게 전달함.
- Stella-Horizon는 오픈스택 국내 커뮤니티에게 공개 시연(18년 12월)을 앞두고 있으며, 커뮤니티 구성원들로부터 피드백을 받아 더욱 고도화할 예정임.
- Stella-Horizon의 공개SW 코드는 GitHub 저장소를 통해 항상 최신버전을 공유하고 있음 (https://github.com/KUoslab/Stella_Horizon)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

나. 클라우드 수준 Stella API 서비스 컴포넌트: Stella-OpenStack 프로토타입

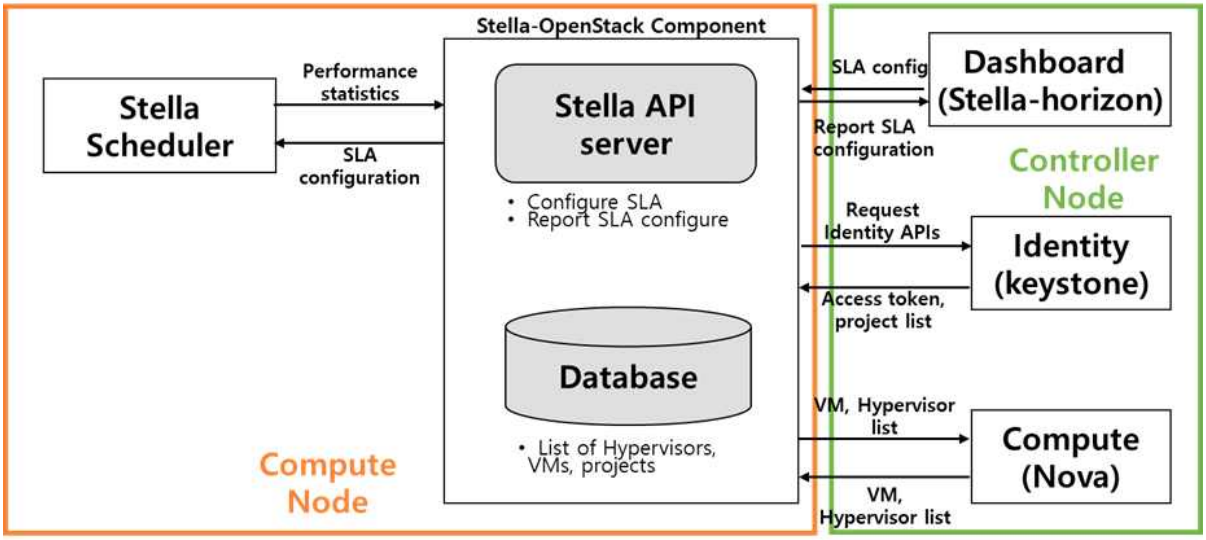


그림 20. Stella-OpenStack의 구조 및 다른 컴포넌트와의 관계

- Stella-Horizon에서 구성한 SLO는 노드레벨 스케줄러에게 전달되어야 하는데, 이때 클라우드 환경에서 전달 가능한 API의 형태여야 함. 노드레벨 스케줄러는 Stella Scheduler, 처리되는 API를 Stella API로 명명함.
- 또한 OpenStack 환경의 노드에서 Stella-Horizon로부터 구성된 사용자의 SLO를 Stella API를 호출해서 Stella Scheduler의 SLO로 설정하는 OpenStack 컴포넌트가 있어야함. 이는 Stella-OpenStack으로 명명함.
- 위 그림 20는 Stella-OpenStack 컴포넌트의 기능과 다른 OpenStack 컴포넌트와의 상호작용 및 Stella-scheduler와의 상호작용을 나타내는 그림임. Stella-OpenStack은 호출한 API를 처리하는 Stella API server 와 Database로 구성됨.
- Stella-OpenStack의 Database는 간단한 Key-Value 저장소로 Stella가 동작하는 OpenStack 클라우드에 하이퍼바이저 목록과 IP 주소, OpenStack 플랫폼에서 사용하는 가상머신 이름과 하이퍼바이저상의 실제 가상머신 이름과 소속된 테넌트 정보를 저장함.
- Stella API는 OpenStack 및 다른 컴포넌트와 호환을 위해 API는 HTTP 기반 Rest (Representational State Transfer) API와 호환되고, 데이터 전달은 JSON (JavaScript Object Notation) 데이터 형식을 사용함.
- 현재 프로토타입 수준에서 구현된 Stella API는 Stella-OpenStack의 상태를 확인하는 Status, 가상머신 정보과 세팅된 SLO를 확인할 수 있는 listVM, SLO를 세팅하는 SetSLO, 하이퍼바이저의 정보를 요청하는 listHypervisor가 정의되어 있음.
- Stella-OpenStack이 SLO를 세팅하기 위해 인증을 제공하는 Identity 컴포넌트와 Compute 컴포넌트와 통신해야함. Compute 컴포넌트인 Nova와 통신하는 이유는 SLO 세팅 명령을 처리하는 하이퍼바이저의 IP와 그 위에서 동작하는 가상머신의 이름을 얻기 위함이고, Identity 컴포넌트인 Keystone과 통신하는 이유는 Compute 컴포넌트에 접근하기 위해 필요한 인증을 수행하기 위함임.
- 인증을 위해 Stella-OpenStack은 Keystone으로부터 Compute 컴포넌트의 접근 토큰(Access

token)을 받음. 이를 위해서 Keystone에는 Stella-OpenStack의 인증을 위한 정보가 미리 등록 되어 있어야함. 사전에 등록한 정보를 기반으로 접근 토큰을 받은 후에 Stella-OpenStack은 Compute 컴포넌트에게 정보를 요청할 수 있음.

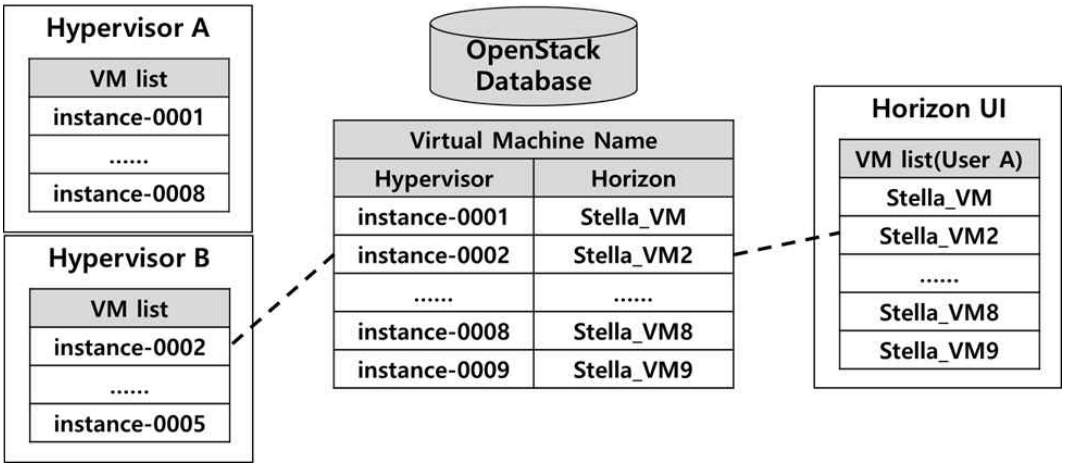


그림 21. Horizon에서 보는 가상머신 이름과 하이퍼바이저에서 가상머신 이름의 차이

- SLO를 세팅하기 위해 Stella-OpenStack이 Compute 컴포넌트에게 하이퍼바이저의 IP와 하이퍼바이저내에서 쓰이는 가상머신의 이름을 요청하는 이유는 OpenStack에서 쓰이는 가상머신의 이름과 하이퍼바이저 내부에서 쓰이는 가상머신의 이름이 다르기 때문에 변환을 위해 필요함. 위 그림 21은 OpenStack 환경에서 사용자가 보는 가상머신의 이름과 Hypervisor상에서 가상머신 이름의 차이를 보여줌. 우측의 Horizon UI의 이름은 사용자가 보는 가상머신의 이름임. 예를 들어 사용자가 사용하는 Stella_VM2는 Hypervior B라고 불리는 컴퓨트 노드에서 instance-0002라는 이름으로 식별됨. 사용자는 Horizon UI 상에서의 가상머신 이름만 알기 때문에 Stella-OpenStack은 이름을 변환하는 기능이 필요함.

- 또한, 사용자가 Stella_VM2에 SLO를 세팅하기 위해서는 Stella-OepnStack이 instance-0002가 동작하는 Hypervisor B의 주소를 알아낸 다음, Hypervisor B에서 동작하는 Stella-scheduler에게 instance-0002의 SLO 값을 전달함.

- 클라우드 환경에서 달성한 성능을 시각화하기 위해 클라우드 환경에서 사용되는 성능 모니터링 도구를 조사하고, 이를 스텔라 클라우드에 적용함.

- 결론적으로, 알려진 모니터링 도구가 SLO 적용을 모니터링하는 방법을 직접적으로 제공하고 있지 않음. 따라서, 오픈스택 커뮤니티에서 직접 관리하는 프로젝트 및 오픈스택과 관련 없지만, 클라우드 환경에서 널리 쓰이는 성능 모니터링 도구를 정리함. 아래 내용은 조사 대상에 대해 요약한 내용임.

- 오픈스택 프로젝트 중에서 조사 대상은 Blazar, Ceilometer, Congress임.

  (1) Blazar: 오픈스택에서 자원 예약을 제공하는 서비스임. 사전에 클라우드 관리자가 만든 정책을 기반으로 특정한 자원을 예약해서 서비스 사용자가 이용할 수 있는 기능을 제공

  (2) Ceilometer는 오픈스택에서 자원 사용량을 추적하고, 클라우드 관리자가 자원 사용에 대한 이벤트나 정책을 만들었을 때 경고 같은 알림을 제공

  (3) Congress는 오픈스택 서비스의 전반에 통제 및 규정 준수를 위해 클라우드 관리자가

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

정책을 만들고 이를 감사하는 기능을 제공

- 알려진 오픈소스 모니터링 도구는 Graphite, Telegraf, Influxdb, Grafana 등임.

(1) Graphite는 클라우드 같은 분산환경에서 발생하는 로그를 수집해서 시각화하는 기능을 제공함.

(2) Telegraf는 시스템에서 발생하는 데이터를 수집해서 시계열 데이터베이스에 전달하는 클라이언트임. 플러그인을 통해 수집할 수 있는 데이터를 추가 가능.

(3) Influxdb는 시계열 데이터를 저장하는 데이터베이스로 SQL 기반 문법을 통해 저장한 데이터를 조작할 수 있는 기능을 제공함.

(4) Grafana는 수집한 데이터를 시각화 할 수 있는 도구임. 다양한 데이터베이스 및 수집 방법을 제공하고 있음.

자세한 평가 내용은 아래 표에서 확인할 수 있음.

이름	통계 수집	SLO 제공	시각화	확장성
Blazar*	X	O (예약된 자원을 통해 제공)	X	O (관리자가 정책을 만들 수 있음)
Ceilomter	O (오픈스택 수준의 통계정보 수집 가능)	X	X	X
Congress*	X	O (Ceilometer와 연동되어서 사용 가능)	X	X
Graphite	O (통계를 얻는 collectd와 데이터를 저장하는 Whisper 컴포넌트로 구성)	X	O (Graphite-Web을 통해 시각화 가능)	X
Telegraf***	O (통계를 수집 기능에 특화됨)	X	X	O (수집하는 데이터 및 데이터 저장소를 플러그인 형태로 추가 가능)
Influxdb***	O (시계열 데이터베이스로 통계 저장에 특화됨)	X	X	O (SQL과 유사한 InfluxQL 제공 통해 확장 가능)
Grafana***	X	X	O (다양한 종류의 그래프 지원)	O (다양한 데이터베이스 및 데이터 형식 지원, 범용적인 시각화 플랫폼으로서 개발됨)

* Blazar는 SLO를 제공할 수 있지만, 통계 수집 및 스텔라 스케줄러와의 연동을 위한 확장이 불가능함

** Congress는 SLO 적용이 가능하지만, 스텔라 스케줄러와 연동을 위한 확장성이 미흡함.

*** 스텔라 클라우드에서 성능 모니터링을 위해 사용하는 오픈소스 도구

- 조사를 토대로 본 연구에서는 성능 모니터링을 위해 Grafana-Infuxdb-telegrf를 사용함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

다. 스텔라 클라우드 프로토타입

- 스텔라 스케줄러를 클라우드 수준에서 보이기 위해서는 SLO를 관리하고, 통계를 수집해야 하고, 수집데이터를 시각화 해 줄 필요가 있음. 따라서 스텔라에서 제공하는 기능을 클라우드 관리자에게 제공하기 위한 인터페이스인 Stella-Horizon을 만들고, 사용자의 요청을 내부적으로 처리하는 Stella-OpenStack, 성능 모니터링을 위한 Telegraf, Infulxdb, Grafana를 통해 프로토타입에서 필요한 기능을 제공할 수 있었음. 언급한 모든 컴포넌트 및 기능을 통합한 테스트 클라우드의 구조는 아래 그림과 같음.

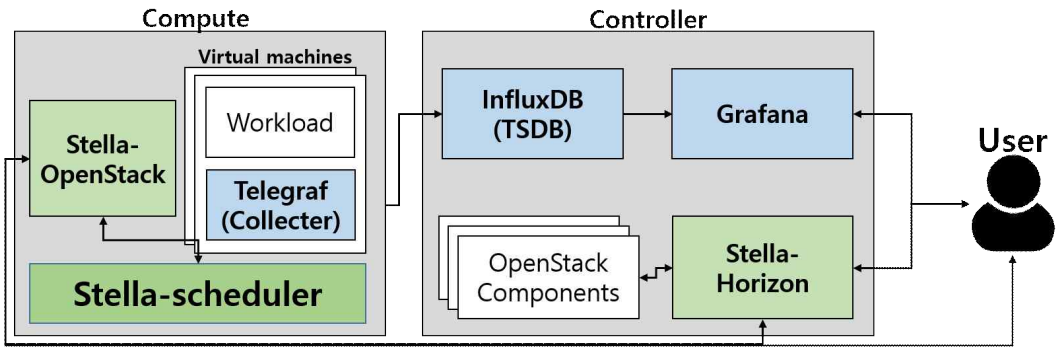


그림 22. 스텔라 오픈스택 연동 구조

- 오픈스택 클라우드 사용자는 스텔라 서비스를 이용하기 위해 웹을 통해 접근하거나 Stella-OpenStack 컴포넌트에서 제공하는 API를 통해 Stella 스케줄러의 기능을 사용할 수 있음. 사용자는 웹 기반으로 클라우드 서비스를 제공하는 Stella-Horizon을 통해 기존 오픈스택의 서비스와 Stella-OpenStack의 서비스에 접근할 수 있음. 예를 들어 가상머신을 생성한 다음 생성한 가상머신의 SLO를 세팅하기 위해서는 Stella-Horizon을 통해 오픈스택 서비스인 Nova에 접근해서 가상머신을 만들고, SLO를 세팅하기 위해 Stella-OpenStack에게 공개된 API를 통해 명령을 내릴 수 있음. 또한, 사용자는 Stella-Horizon 없이 Stella-OpenStack에서 제공하는 Stella API를 통해 SLO를 세팅할 수 있음. 또한, 사용자는 웹 기반 시각화 도구인 Grafana를 통해 성능 통계정보를 얻을 수 있음. 가상머신이 동작하는 동안 가상머신에 설치된 Telegraf는 성능 통계정보를 수집하고, 이를 InfluxDB에 저장한다. InfluxDB에 저장된 데이터는 Grafana를 통해 그래프로 시각화되어 사용자가 볼 수 있음.

■ 공인인증 시험기관을 통한 정량적 목표 항목 달성 확인

- 정량적 개발 목표의 4개 항목에 대해 한국정보통신기술협회 (TTA)의 확인 및 검증 시험 (verification & validation) 인증 서비스를 통해 4차년도 연구 개발물의 목표 달성을 확인함. 시험 결과, CPU-블록 I/O CLS, CPU-네트워크 CLS, CPU-블록 I/O, 네트워크 복합 CLS의 3가지 항목에 대해 모두 평균 100% 이상의 결과를 보여 4차년도 목표치인 95%를 달성하였음. CPU, 블록, 네트워크 사용율 항목은 평균 77%로 목표치인 75%를 달성함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



시험결과서

한국정보통신기술협회
소프트웨어시험인증연구소
주소: 경기도 성남시 분당구 분당로 47
전화: 031-780-9278, Fax: 031-724-0189

결과서번호: BT-A-18-0528

1. 의뢰자
- 회사(기관)명: 고려대학교 산학협력단
- 주소: (02841) 서울시 성북구 안암로 145
- 계약일자: 2018.9.14.

2. 제품명: 가상화 환경용 통합 자원 스케줄링 모듈

3. 시험기간: 2018.10.19. ~ 2018.10.25.

확 인	작성자 성 명 : 이 동 아	승인자 직 위 : 센터장 성 명 : 김 진 영

2018.10.25.

한국정보통신기술협회 회장

TTA-SE1-18-A(R)

그림 23. 1단계 연구 결과물 공인시험 결과서

5) 클라우드 레벨에서 성능 SLA 보장을 위한 자원 관리 기법

가. 관련 연구 조사 및 분석 - 멀티 리소스 스케줄링

- DRF 기반 자원 스케줄링: 클라우드 데이터센터에서는 다양한 워크로드가 수행되며 워크로드 특성에 따라 서로 다른 종류의 자원을 동시에 필요로 함. 예를 들어 분산 파일처리 프레임워크인 Hadoop의 경우 작업 처리를 위해 CPU와 메모리 자원을 동시에 필요로 하며, 이러한 이종 자원에 대한 요구사항을 만족하기 위해 dominant resource fairness (DRF) 개념이 제시되었으며 표 4와 같이 이후 이를 기반으로 한 여러 스케줄링 방식이 제안되었음.

- 성능 SLO 보장 측면에서의 DRF 방식의 한계점: DRF를 기반으로 한 클라우드 스케줄링 알고리즘은 스케줄링 목표에 따라 성능 고립을 제공하거나, 자원 활용율을 향상시킴. 그러나, DRF 기반의 스케줄링 알고리즘은 사용자가 작업의 성능 SLO 달성을 위한 1) 자원 요구량을 일일이 명시해야 하고, 2) 동적으로 자원 요구량이 변하는 경우 이를 지원하기 어려운 단점이 있음. 자원 요구량을 명시하기 위해서는 작업을 수행하기 전 오프라인 프로파일링을 수행

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

해야 하고, 동적으로 변하는 자원 요구량을 반영하기 위해서는 전체 시스템에 대한 자원 할당 알고리즘을 다시 수행해야하기 때문에 스케줄링 오버헤드 증가하는 문제가 있음.

표 4. 클라우드 환경에서 자원 할당을 위한 멀티 리소스 스케줄링 연구

	자원간 종속성 고려	성능 고립 지원	자원 활용율 최대화	낮은 오버헤드
Mesos [NSDI ‘11]	X	O	X	O
DRF [NSDI ‘11]	X	O	X	X
HUG [NSDI 16	X	O	O	X
Borg [Eurosys ‘15]	X	X	X	X
H-DRF [SoCC ‘13]	X	X	X	O
CARBYNE [OSDI ‘16]	X	O	O	N/A
DC-DRF [SoCC 18]	X	O	O	X
PS-DSF [TPDS ‘18]	X	X	O	O

- 자원 간 종속성 고려의 필요성: 특히, DRF 기반 방식은 성능 SLO 달성을 위한 자원 간 종속성을 고려하지 않는 단점이 있음. 자원 간 종속성은 특정 자원을 활용하여 작업을 수행할 때, 다른 자원의 사용량에 따라 성능이 영향받는 것으로, 네트워크 작업 수행 시 CPU 자원 할당에 따라 성능이 변화하는 것을 예로 들 수 있음. 웹서버가 원격 사용자의 요청을 처리하는 경우, 요청을 수신하고 다시 송신하기 위해 네트워크 자원이 필요함. 하지만, 네트워크 자원을 충분히 할당받더라도, 웹서버가 사용자 요청을 처리하는데 요구되는 CPU 자원을 충분히 할당받지 못하면 웹서버의 처리 성능이 저하될 수 있음. 본 연구에서는 클라우드 환경에서 SLO 달성을 위해 자원 간 종속성을 고려하는 작업 분배 알고리즘 개발을 목표로 함.

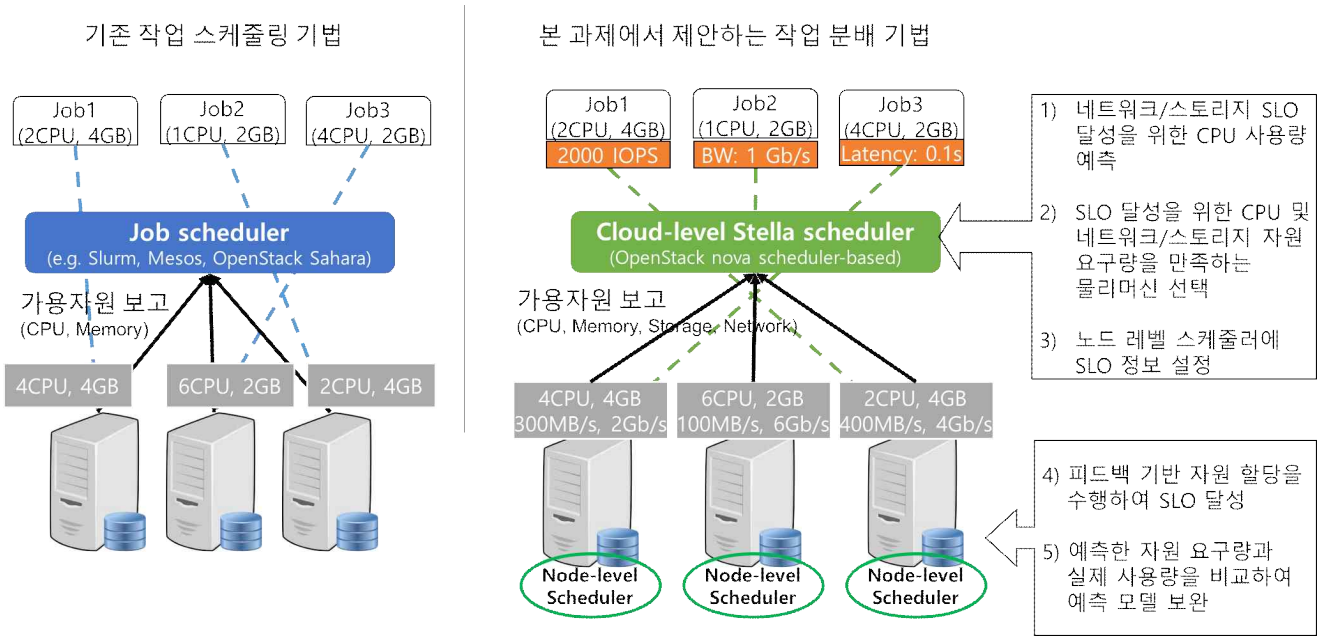


그림 24. 기존 스케줄링 기법과 IReS 방식 비교

나. SLO를 고려한 클라우드 레벨 스케줄링 알고리즘 설계

- 본 연구에서는 클라우드 환경에서 자원 간 종속성을 고려한 작업 분배 스케줄러를 제안함 (Inter-resource dependency-aware VM scheduler; IReS). IReS는 그림 24과 같이 사용자가 작업별로 필요한 자원을 명시하여 스케줄링에 반영하는 기존 방식과는 달리, 작업 단위 SLO를 만족할 수 있는 물리서버를 선택하여 작업을 수행하는 것을 목표로 함. 사용자로부터 작업 수행 요청을 받아 물리 서버를 선택하는 클라우드 레벨 스케줄러와 물리 서버 상에서 SLO 달성을 위해 동적으로 자원 할당을 수행하는 노드 레벨 스케줄러 2레벨로 구성되어 있음. 스케줄링 동작 순서는 크게 5단계로 구성되며 세부 내용은 다음과 같음.

  [1단계-클라우드 레벨 스케줄러] SLO 달성을 위한 CPU 사용량 예측: 클라우드 환경에서 수행되는 작업의 네트워크/스토리지 성능은 해당 작업에 할당되는 CPU 사용량에 비례함 (그림 25 참조). 따라서, 네트워크/스토리지 SLO 달성을 위해서는 네트워크/스토리지 자원뿐만 아니라, CPU 자원의 할당이 필요함. 본 연구에서는 SLO 달성에 요구되는 CPU 자원 할당을 목표로 하며, 이를 위해 선형 회귀 분석을 이용하여 네트워크/스토리지 SLO 달성에 필요한 CPU 자원 양을 예측하는 방식을 제안함.

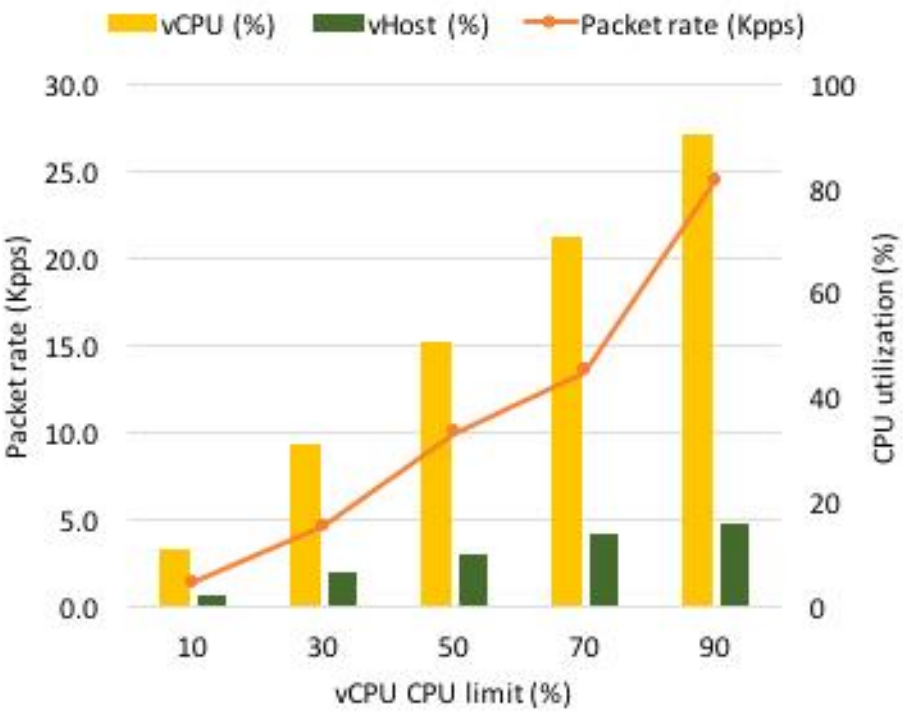


그림 25. 가상머신에 할당되는 CPU 자원 (vCPU CPU limit)에 따른 패킷 처리 성능

[2, 3단계-클라우드 레벨 스케줄러] CPU/네트워크/스토리지 자원 요구량을 만족하는 물리 서버 선택 및 정보 전달: 1단계에서 도출한 요구 자원을 할당 가능한 물리 서버를 선택하는 알고리즘은 다음과 같음. 물리 서버를 연결 리스트 형태로 관리하는 자료구조를 순회하며 각 물리서버의 CPU/네트워크/스토리지 가용 자원과 요구 자원을 비교하는데, 3개 자원에 대해 모두 가용 자원이 요구 자원보다 큰 값을 가지는 물리서버가 선택됨. 동시에 여러 물리 서버에서 가용 자원이 충분한 경우, 해당 작업이 수행되는 경우 자원 활용율이 가장 높아지는

물리 서버를 최종 선택하여 작업을 실행함. 작업 실행 시, 선택된 물리 서버에 해당 작업의 SLO 값과 CPU 사용량의 예측값이 전달됨.

[4, 5단계-노드 레벨 스케줄러] 피드백 기반 자원 할당을 통한 SLO 달성 및 자원 사용량 예측 모델 보완: 노드 레벨 스케줄러는 4차년도 연구 결과물인 xGoS를 활용하며, 클라우드 레벨 스케줄러로부터 SLO와 필요한 자원 양을 전달받아 주기적으로 해당 작업에 CPU/네트워크/스토리지 자원 할당을 수행함. xGoS는 피드백 기반 자원 할당을 수행하기 때문에 주기적으로 실제 성능과 SLO를 비교하여 자원 할당을 조정함으로써 SLO 달성을 위한 자원양을 도출할 수 있음. 이 때, xGoS에 의해 도출된 SLO 달성을 위한 자원 양이 클라우드 레벨 스케줄러로부터 전달받은 필요한 자원 양과 차이가 있는 경우, xGoS는 클라우드 레벨 스케줄러에 해당 정보를 전달하여 예측 모델을 보완하도록 함.

다. SLO 기반 작업 분배 알고리즘 오픈스택에 적용

● 기존 오픈스택 환경에서의 작업 분배 방식 분석

- 그림 26과 같이 기존 오픈스택 환경에서는 필터, 웨이팅 2단계로 가상머신을 수행할 물리서버가 선택됨. 필터에서는 시스템 상에 존재하는 물리서버 중, 가상머신에 할당되는 CPU 개수, 메모리 및 디스크 크기를 수용 가능한 물리서버를 선택함. 웨이팅 단계에서는 필터에서 선택된 여러 물리서버들의 가중치를 계산하여 가장 높은 가중치를 가지는 물리서버를 최종적으로 선택함. 가중치를 계산하는 방식은 시스템 설정에 따라 달라질 수 있으나, 기본적으로는 메모리 사용량이 가장 적은 물리서버를 선택하도록 되어있음.

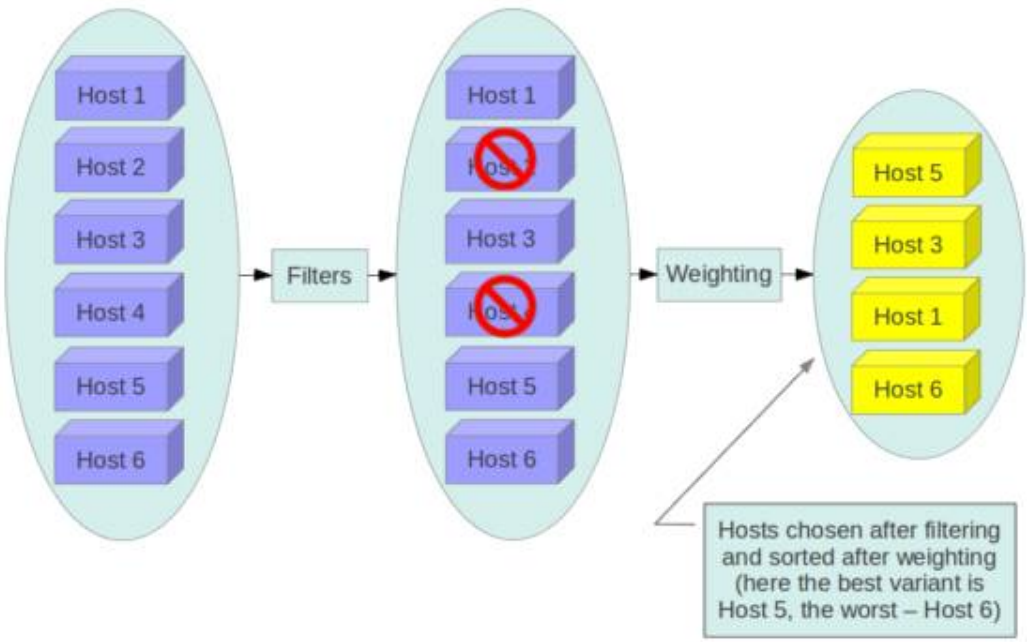


그림 26. 기존 오픈스택에서의 작업 분배 알고리즘 도식화

● 오픈스택 환경에 SLO 기반 작업 분배 알고리즘 구현: 오픈스택 상에서 가상머신 생성은 그림 27와 같이 사용자가 오픈스택 대시보드에 가상머신 (VM)생성을 요청하면 노바 스케줄러에서 물리서버를 선택하여 노바 에이전트에 전달하는 순서로 동작함. 본 연구에서 제안하는

IReS 적용을 위해 오픈스택 대시보드를 수정하여 사용자 요청을 노바 스케줄러가 아닌 IReS로 전달하도록 함. IReS는 파이썬 어플리케이션으로 구현되며 사용자 요청을 전달 받아 스케줄링 알고리즘을 수행하여 노바 에이전트에 최종 선택한 물리서버에 VM 생성을 요청함.

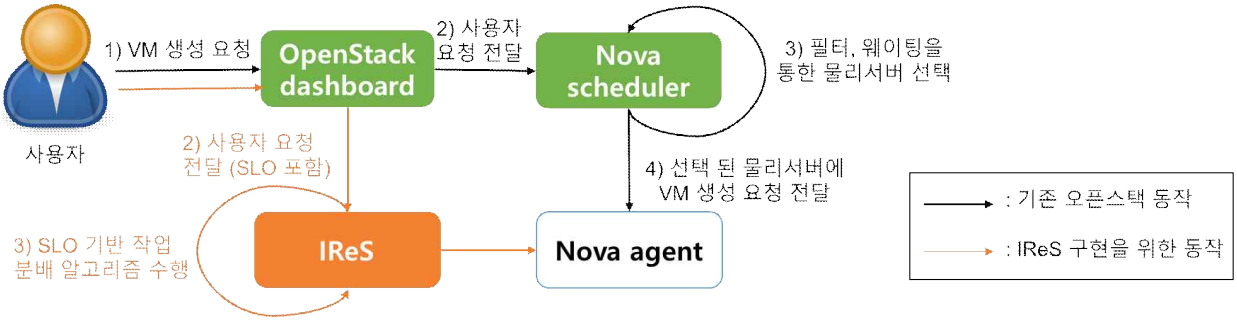


그림 27. 기존 오픈스택 가상머신 생성 동작 순서와 IReS 적용을 위한 동작 비교

6) 보안 SLA를 위한 BaaS 분석 및 보안 파라미터 추출

● Blockchain-as-a-service (BaaS)

- BaaS는 클라우드 환경에서 블록체인을 플랫폼으로 제공하기 위한 서비스임. 그중에서 IBM의 하이퍼레저 패브릭의 경우 사전에 허가받고 신원이 있는 사용자만 블록체인에 접근할 수 있는 블록체인임. 알려진 사용자의 노드만 블록체인에 접근하기 때문에 악의적인 사용자에 의한 트랜잭션 변조보다는 분산 환경에서 실패(Fault)에 대한 대비와 효율성에 더 집중하고 있음. 따라서 BaaS의 성능/보안 SLA 보장 연구를 위해 먼저 하이퍼레저 패브릭의 동작 방식을 분석하였음.

● BaaS의 동작 방식

- 하이퍼레저 패브릭은 엔터프라이즈에서 사용하기 위한 블록체인 솔루션 및 어플리케이션 개발 목적의 허가형 블록체인 플랫폼임. 하이퍼레저는 리눅스 재단의 주도하에 설립되고 다양한 기술운영위원회가 관리하는 오픈소스 프로젝트임. 패브릭은 모듈식 아키텍처 설계 방식을 채택하여 뱅킹, 금융, 보험, 인적 자원 관리, 유통 등 광범위한 산업에 적용이 가능함. 패브릭은 특정 도메인 언어가 아닌 Java, Go 및 Node.js와 같은 범용 프로그래밍 언어로 작성된 스마트 컨트랙트를 지원하는 최초의 분산 원장 플랫폼임. 패브릭은 기존 퍼블릭 블록체인과 달리 신원이 확인된 사용자만이 네트워크에 참여할 수 있는 허가형 블록체인으로, 참가자간 신뢰 기반의 거버넌스 모델로 사용될 수 있음. 하이퍼레저 패브릭 구조는 다음 그림 28과 같음.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

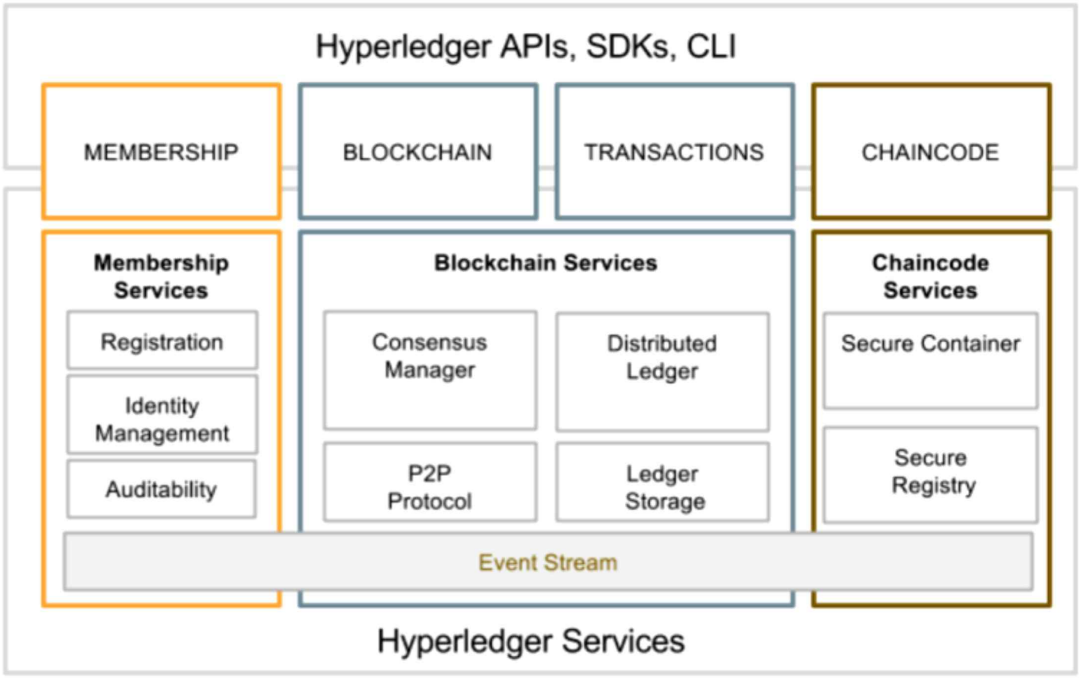


그림 28. 하이퍼레저의 구조

- 멤버십 서비스: 멤버십 서비스는 참여 조직, 피어 노드 및 클라이언트의 신원을 관리함. 네트워크 관련 신원 등록, 출입 통제 및 승인 권한을 관리함.

- 블록체인: 합의 관리자, 분산 원장, P2P프로토콜로 구성됨. 합의 관리자는 트랜잭션 순서의 정확성에 관한 합의를 달성하고자 함. 정책의 유효성이 검증된 트랜잭션을 배치로 처리하여 블록을 생성하고 이를 다른 노드들에 배포하는 역할을 함. 분산 원장은 트랜잭션의 처리 상태와 원장 객체를 관리하고 저장함. P2P 프로토콜은 단일 연결을 통한 양방향 스트리밍, 흐름 제어 등 분산 환경에서 필요한 노드 제어에 관한 규약을 제공함.

- 트랜잭션: 패브릭 네트워크는 트랜잭션들을 배치로 묶어서 하나의 블록을 생성하는 방식이며, 블록 생성 제한 시간이나 하나의 블록에 담을 수 있는 최대 트랜잭션 수를 초과할 경우 새로운 블록을 생성함.

- 체인코드: 체인코드는 유효성을 검사하는 노드에서 실행되는 분산 트랜잭션 프로그램임. 체인코드에는 보안 런타임 환경, 체인코드 레지스트리를 관리함.

- 하이퍼레저 패브릭 네트워크의 트랜잭션 처리 방식 분석

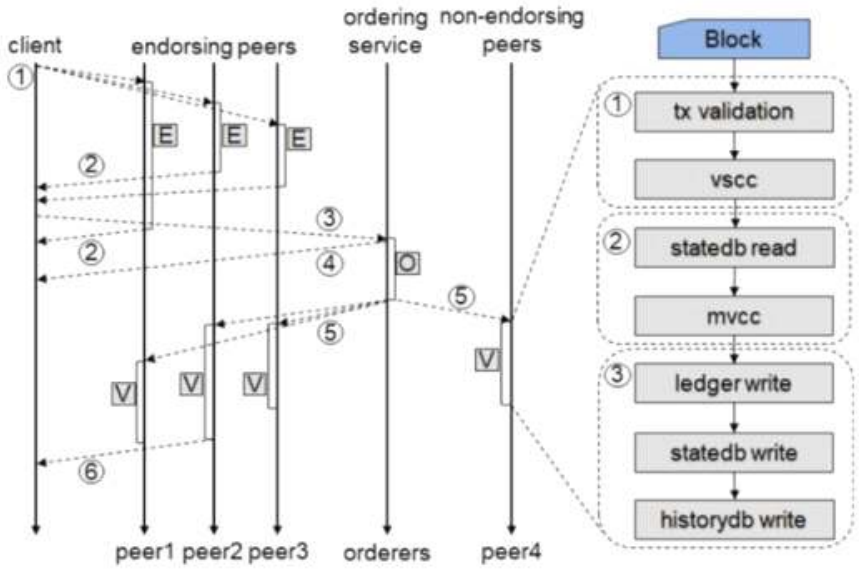


그림 29. 하이퍼레저 패브릭 네트워크의 거래 처리 방식

①클라이언트가 패브릭 네트워크에 트랜잭션 요청을 보내면, 보증 피어들은 검증 시스템 체인코드를 실행하여 트랜잭션 요청의 보증 정책에 관한 유효성을 검증함.

②클라이언트의 트랜잭션 요청에 관한 보증 피어들의 검증 시스템 체인코드 출력 결과가 모두 일치한다면 보증 정책을 위반하지 않은 것으로 판단하고 클라이언트에게 트랜잭션의 보증을 반환함.

③클라이언트는 검증받은 트랜잭션을 패브릭 네트워크의 합의 알고리즘 메커니즘에 따라 트랜잭션의 순서를 결정하고 블록을 생성하는 오더링 서비스 노드에게 제출함.

④오더링 서비스 노드는 해당 트랜잭션이 블록에 포함될 순서를 결정하고, 순서가 결정되면 해당 트랜잭션이 블록 생성에 포함되었다는 것을 클라이언트에게 알림. 오더링 서비스 노드는 블록 생성 제한 시간이 만료되었거나 블록에 포함될 트랜잭션 수가 블록 크기 제한에 도달하면 새로운 블록을 생성함.

⑤오더링 서비스 노드가 블록 생성을 완료하면, 블록에 관한 검증을 위하여 네트워크의 피어 노드에게 전파함. 블록을 전파 받은 피어 노드는 해당 블록에 포함된 모든 트랜잭션을 검증하고 이상이 없을 시, 다중 버전 동시성 제어를 적용 하여 이중 지불을 방지하며 장부와 상태 데이터베이스에 해당 블록을 등록함.

⑥피어 노드의 블록 검증이 완료되고 장부와 상태 데이터베이스에 블록 업데이트를 완료하면, 한 피어 노드가 클라이언트에게 요청한 트랜잭션이 정상적으로 등록되었다는 것을 알림.

- 하이퍼레저 패브릭 예제 분석 (first network, high throughput)

- 하이퍼레저 패브릭 예제는 사전 컴파일된 도커 이미지로 공개되어 있기 때문에 소스 파일을 컴파일하거나 환경설정을 별도로 하지 않아도 동작 확인이 가능. 도커는 가상 기술을 사용한 소프트웨어로 컨테이너형 소프트웨어 실행환경을 구축할 수 있음. 실행되는 OS와 미들웨어, 소프트웨어가 설정된 템플릿에 해당하는 것이 ‘도커 이미지’, 도커 이미지를 바탕으로 실제로 소

프트웨어가 실행되는 것을 ‘도커 컨테이너’라고 함. 하이퍼레저 패브릭의 다양한 예제들은 도커 이미지를 조합해 도커 컨테이너를 기동하는 방식으로 블록체인 네트워크를 구축함. 대표적인 예시들은 다음과 같음.

○ first network 예제

- 목적: 하이퍼레저 패브릭에서 제공하는 예제 중 가장 기본적인 예제로, 하이퍼레저 패브릭이 어떻게 구성되어 있고 네트워크의 흐름을 파악할 수 있음.

- 구조: first network 예제는 그림 30과 같은 실행 구조를 가지고 있음. 기본이 되는 도커 이미지로부터 클라이언트용 cli 컨테이너와 오더러용 컨테이너가 각각 1개씩 실행되고 2개의 조직(1번 조직, 2번 조직)에 2개씩(1번 피어, 2번 피어) 총 4개의 피어 컨테이너가 실행됨.

- 동작 방식: first network가 실행되면 cli에서 각 조직의 1번 피어에 체인코드를 인스톨함. 이는 체인코드를 소스코드 형태로 각 피어 블록체인에 저장하는 것임. 그 후 2번 조직의 0번 피어에 체인코드를 인스턴스화하여 체인코드를 초기화시키고 변수 A에 100, 변수 B에 200을 저장함 (A=100, B=200). cli에서 1번 조직의 0번 피어에게 변수 A 값을 확인하는 쿼리 트랜잭션을 보내면 각 피어들이 가지고 있는 블록체인은 동일한 값을 가지고 있으므로 100이라는 값을 반환함. 다음으로 cli에서 변수 A에서 변수 B로 10이라는 값을 이동시키는 인보크 트랜잭션을 보냄 (A=90, B=210). cli에서 2번 조직의 1번 피어로 체인코드를 인스톨하고 아래의 그림과 같이 변수 A 값을 확인하는 쿼리 트랜잭션을 보내면 90이라는 값을 반환함.

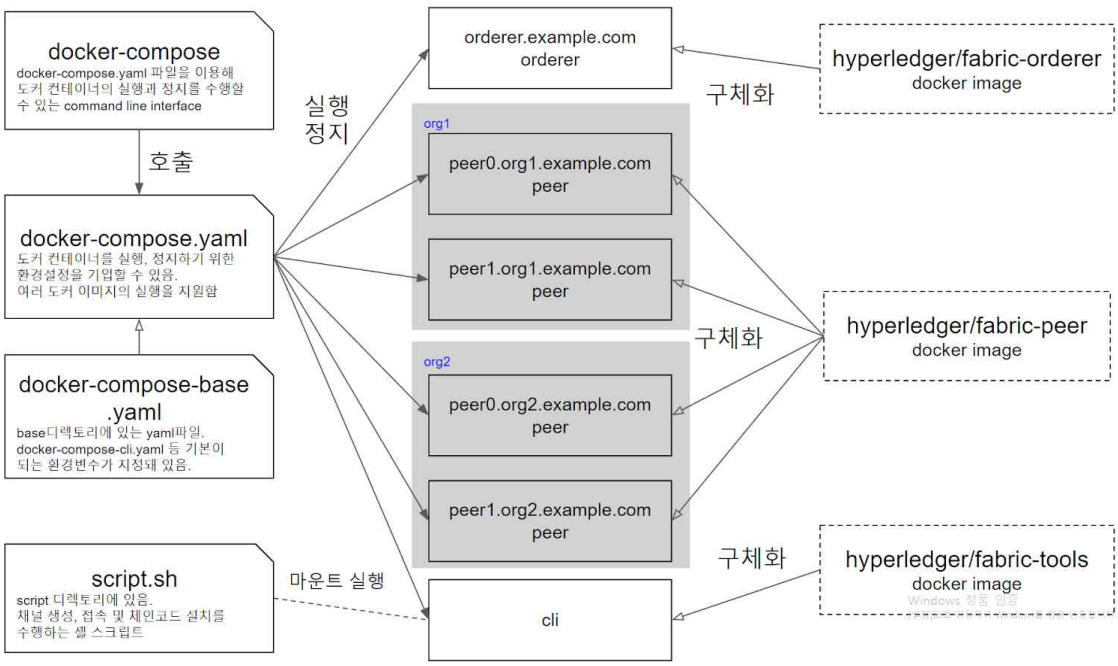


그림 30. first network 실행 구조

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

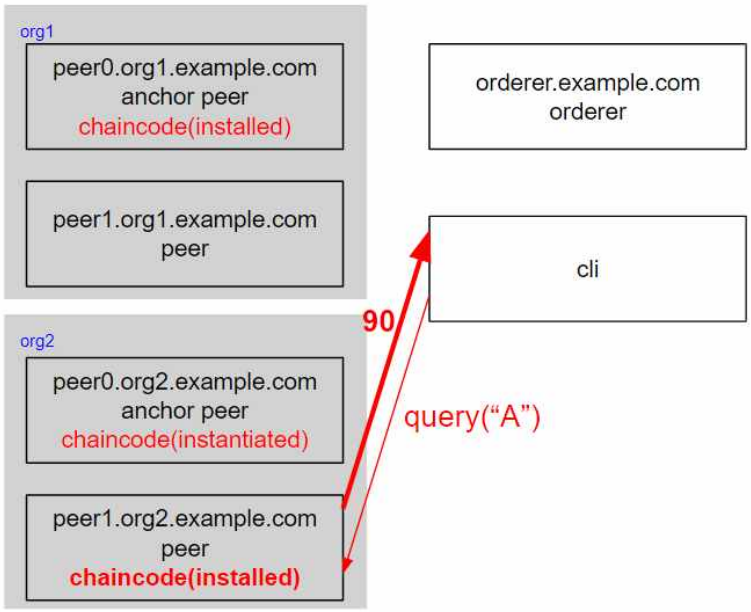


그림 31. first network 동작 방식

○ high-throughput 예제

- 목적: first network 예제에서 수정을 거치면 high-throughput 예제를 실행할 수 있음. high-throughput 예제는 수천 개의 초당 트랜잭션(TPS)을 다룰 때 체인코드 데이터 모델을 적절하게 디자인하는 방법을 이해하기 위한 네트워크로 BaaS 워크로드 분석에 있어 적합한 네트워크로 판단됨.

- 구조: first network 예제와 동일한 구조임.

- 동작 방식: first network와 동일한 구조의 네트워크가 생성되면 하이퍼레저 패브릭에서 제공한 스크립트들을 차례대로 실행시킴. 채널을 생성하여 각 피어 컨테이너들을 채널에 참가시키고 체인코드를 인스톨한 후 인스턴스화시킴. 체인코드를 인스톨하면서 설정된 페이로드 변수에 일정한 값을 증가시키거나 감소시키게 할 수 있는 인보크 트랜잭션을 수천 개 실행시킴.

● 자원 사용량 분석

- 실험 목적: 하이퍼레저 패브릭 워크로드 특성 분석을 위해 high-throughput 예제를 실행하여 자원 사용량을 분석함.

- 실험 환경 및 방법: Intel Xeon E5-2650v4, 48Core@2.2GHz (하이퍼스레딩) CPU, 128GB Memory, 256GB SSD가 장착된 Ubuntu 16.04서버에서 진행함. docker stats, docker logs, perf top 등의 테스트 도구를 활용했음.

- 실험 결과

  1) CPU 사용량: cli (1436.2%)와 peer0.org1.example.com (59.2%)가 가장 높은 CPU 사용량을 보임. 특히 cli 컨테이너를 통해 각 피어에게 트랜잭션을 실행하기 때문에 CPU 자원 사용량이 많은 것으로 보임.

  2) 메모리 사용량: cli 컨테이너를 통해 트랜잭션을 각 피어에게 받고 오더러에게 보내줘야

하는 과정을 거치기 때문에 cli 컨테이너의 메모리 자원 사용량이 가장 많은 것으로 보임.

3) 성능: 초당 82.34 트랜잭션 처리 (82.34 tps)

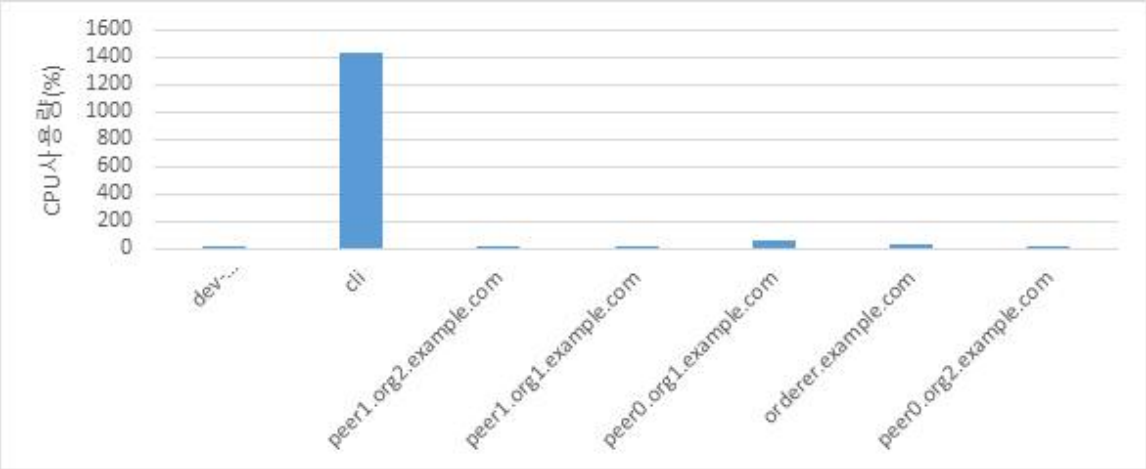


그림 32. 컨테이너별 CPU 사용량

표 5. 컨테이너별 메모리 사용량

컨테이너 항목	메모리사용량 (MiB)
dev-peer0.org1.example.com-bigdatacc-1.0	12.08
cli	682.74
peer1.org2.example.com	103.06
peer1.org1.example.com	90.17
peer0.org1.example.com	218.99
orderer.example.com	154.58
peer0.org2.example.com	90.17

● 보안 SLA를 위한 파라미터 추출

- 하이퍼레저 패브릭 네트워크의 트랜잭션 처리 성능 측정

- 실험목적 : BaaS 환경에서 보안 SLA를 보장하기 위한 파라미터 추출을 위해 트랜잭션-Transfer와 Query의 요청 수와 같은 환경 변수 변화에 따른 성능 변화를 관찰하기 위함 (작업 처리량-transactions per second, 지연시간-latency)

- 실험 환경 : 실험은 Intel Xeon E5-2650v4, 48Core@2.2GHz (하이퍼스레딩) CPU, 128GB Memory, 256GB SSD가 장착된 Ubuntu 16.04 서버에서 진행함. 실험에 사용한 패브릭과 캘리퍼는 1.4.1버전, 0.1.0버전을 실험에 사용함.

- 실험 방법

1) 패브릭 네트워크의 크기: 패브릭 네트워크는 그림 34과 같이 1개의 채널에 2개의 조직으로 구성함. 조직 1에는 2개의 피어 노드, 조직 2에는 3개의 피어 노드를 설정하고 3개의 오더링 서비스 노드, 4개의 카프카 브로커, 3개의 주키퍼 그리고 2개의 인증서 발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

급 노드로 구성함. 총 17개의 노드를 구성하여 실험함.

2) 블록 설정: 패브릭 네트워크에서 생성되는 블록의 최대 크기는 2MB로 설정하였으며, 배치 타임아웃은 2초, 하나의 블록에 담을 수 있는 최대 트랜잭션 수는 600개로 설정함.

3) 트랜잭션 요청 속도 (Send rate): 하이퍼레저 캘리퍼를 활용, 초당 트랜잭션 요청 수를 100TPS에서 2,000TPS로 선형적으로 증가시키며 이에 따른 성능 변화 관찰.

4) 체인코드: 실험에서는 단순한 뱅킹 시스템의 계좌 생성, 송금, 예금, 계좌 조회 트랜잭션을 처리하는 체인코드를 사용하였음. 뱅킹 체인코드를 사용하여 클라이언트 계좌의 변동을 처리하는 트랜잭션(Transfer)과 값의 변동이 없는 계좌 조회 쿼리(Query)에 관한 성능 측정 실험을 진행함.

5) 데이터베이스: 패브릭의 장부와 상태 데이터베이스는 구글의 LevelDB 사용.

- 실험 결과

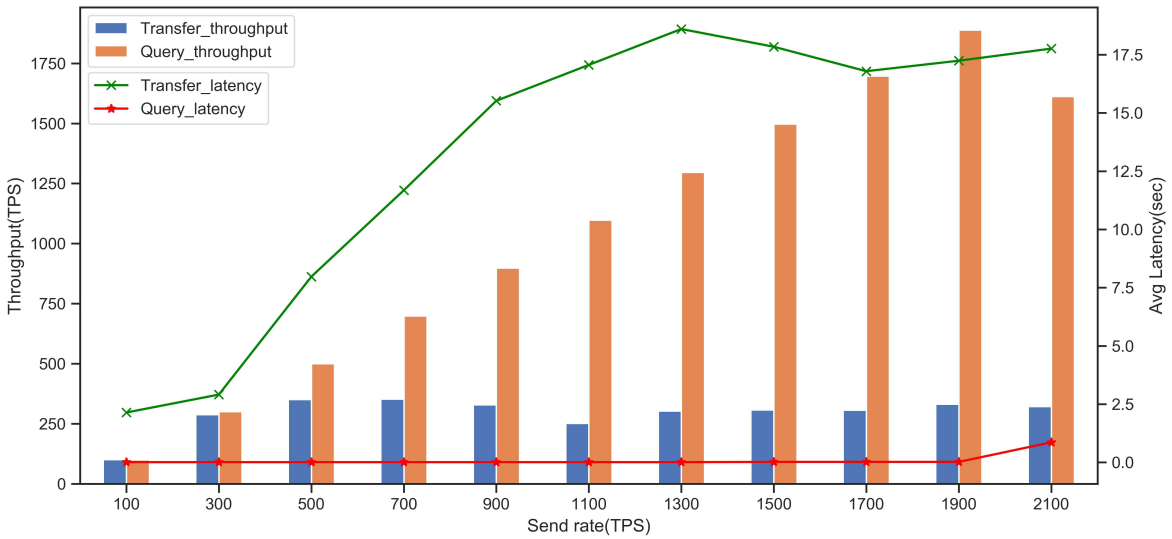


그림 33. 트랜잭션 요청 수 증가에 따른 Transfer, Query의 성능 변화

1) 패브릭의 트랜잭션 처리 성능: Send rate가 증가함에 따라 처리 성능 또한 증가하여, Send rate가 500TPS일 때, 최대 처리 성능인 353.9TPS 달성함. 하지만 Send rate가 500TPS이상이 되는 경우 오히려 처리 성능은 감소. Send rate가 100, 300TPS인 경우 Latency는 평균 2.5s 정도지만, Send rate가 500TPS이상이 되면 Latency 가 급격하게 증가함.

2) 패브릭의 쿼리 처리 성능: Query 트랜잭션 요청이 1900TPS일 때, 처리 성능은 1888.6TPS이며, Latency는 0.02s 임. 쿼리 요청이 1900TPS를 초과하면 처리 성능은 감소하며, Latency가 급격하게 증가함.

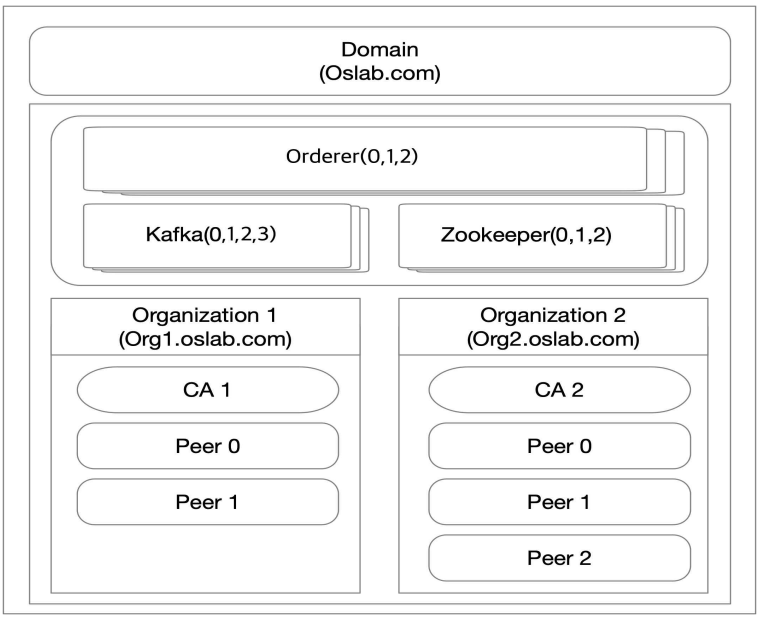


그림 34. 패브릭 네트워크 노드 구성도

- 위의 두 실험 결과를 통해 하이퍼레저 패브릭의 트랜잭션 및 쿼리 처리 성능 (throughput, latency)은 트랜잭션 요청 속도인 Send rate에 영향을 받는 것을 확인하였고, 이를 보안 파라미터로 활용하여 BaaS에서의 보안 SLA 보장 연구를 차년도에 이어서 진행하였음.

● 보안 SLA 보장을 위한 BaaS 감사 도구 설계

- 분산 환경에서의 BaaS 지원을 위한 컨테이너 오케스트레이션 공개SW 적용: 본 연구에서는 클라우드 환경에서 동작하는 BaaS 프레임워크의 보안 SLA 감사를 목표로 하고 있음. 기존의 공개 SW BaaS 프레임워크인 하이퍼레저 패브릭에서는 사용자의 트랜잭션 처리에 대한 단순한 성능 계측 기능만을 제공하고 트랜잭션 처리 성능 (TPS) 이나 지연 시간 (latency)와 같은 보안 SLA에 대한 감사는 제공하지 않음. 본 연구에서 제안하는 BaaS 감사 도구는 그림 35와 같이 컨테이너 오케스트레이션 기능을 제공하는 공개 SW 프로젝트인 쿠버네티스를 기반으로 하고 있음. 쿠버네티스는 구글에서 개발한 공개 SW 플랫폼으로 컨테이너 가상화의 높은 성능, 경량성 등의 장점을 활용한 분산 환경 구축이 가능하여 많은 산업 분야 및 연구 결과물로 활용되고 있음. 본 연구에서는 공개 SW R&D 측면에서의 경쟁력을 강화하기 위해 기존의 서버 가상화 기술을 기반으로 하는 오픈스택뿐만 아니라, 컨테이너 가상화로 연구 플랫폼을 확장하여 성능 및 보안 SLA 연구를 수행하고 있음.

- 쿠버네티스 기반 BaaS 보안 SLA 감사: 쿠버네티스는 분산 환경에서 컨테이너의 생성 및 운용을 위한 kubectl 이라는 CLI 인터페이스를 제공하며, 본 연구에서는 쿠버네티스를 통한 BaaS 서비스 운용을 위해 사용자가 별도의 BaaS 수행을 위한 설정 파일을 제출하면 (①) 이를 kubectl 형태로 변환하여 실제 서버에 구동하는 방식을 사용함 (②). 분산된 서버 (Kubelet nodes)에는 kubelet 이라는 사용자 수준의 에이전트가 동작하며 중앙 컨트롤러 (master)가 이들을 제어함으로써 분산 환경에서의 BaaS 서비스 운용이 가능해짐.

- 보안 SLA 감사를 위한 새로운 컴포넌트 연구: 기존 쿠버네티스와 하이퍼레저 패브릭에서는 보안 SLA 감사 기능을 제공하지 않기 때문에, 본 연구에서는 쿠버네티스 제공하는 구성요소 (kubectl, master, kubelet) 이외에 보안 SLA 감사를 위한 별도의 컴포넌트 (kubestar) 개발을 진행할 예정임. kubestar는 kubelet과 유사하게 분산 서버에서 사용자 수준 에이전트로 동작하며, 하이퍼레저 API와 쿠버네티스 API를 모두 지원하여 하이퍼레저 패브릭과 kubelet으로부터 트랜잭션 처리 정보, 실행 시간 등의 정보를 수집하는 역할을 수행함 (③). 수집한 정보는 보안 SLA 형태로 가공하여 쿠버네티스 대시보드로 전달하는데 (④), 이때 별도의 master와 같은 컨트롤러 없이 독립적으로 동작 가능하여 추가적인 연산 및 커뮤니케이션 오버헤드를 최소화할 수 있음.

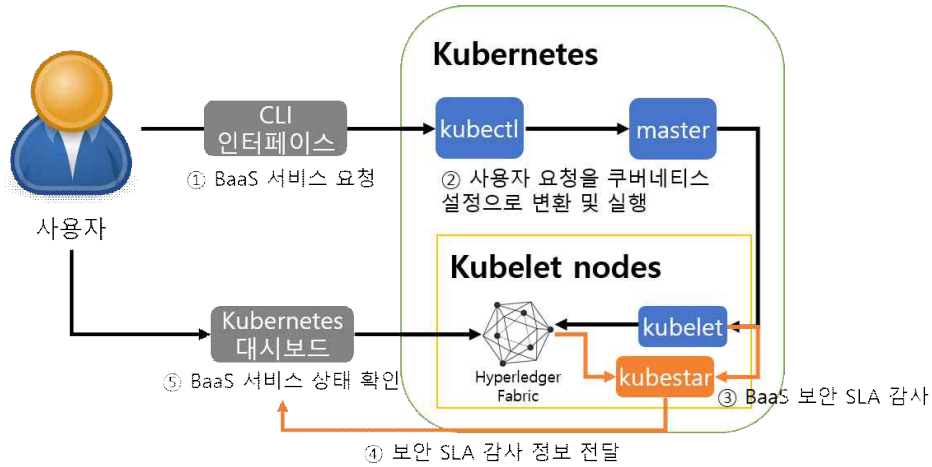


그림 35. 쿠버네티스 기반 보안 SLA 감사 구조

7) Intra BaaS SLO 성능 보장 연구

- BaaS 성능 SLO 분석/모니터링 기법 연구: 초당 트랜잭션 수, 트랜잭션 지연시간

  - 실시간 성능 모니터링을 위한 기존 성능 계측 도구 분석

    ① 실시간 BaaS 성능 모니터링 기법의 필요성: BaaS SLO 보장을 위해서는 BaaS 서비스 수행 시, 실시간으로 서비스 성능을 계측하여 실제 성능이 사용자의 SLO를 만족하는지 판단하는 과정이 필요함. BaaS를 위한 기존 성능 계측 도구인 하이퍼레저 캘리퍼 (이하 '캘리퍼'라 함)는 서비스의 시작과 종료에 따라 전체 동작의 평균 성능 정보를 제공하고 있지만 서비스의 실시간 성능 계측 기능을 제공하지는 않음. BaaS SLO 보장을 위한 실시간 Baas 성능 모니터링 기법 개발을 위해 기존 성능 계측 도구인 캘리퍼의 동작방식을 분석함.

    ② 하이퍼레저 캘리퍼: BaaS 프레임워크를 위한 벤치마크 도구로, 다양한 종류의 BaaS 프레임워크에 대해 심층적으로 이해하기 위해 일관된 기준을 제공함으로 체계적으로 분석하고 성능을 비교할 수 있게 함. 캘리퍼에서 제공하는 벤치마크를 사용하여 BaaS 프레임워크의 성능을 측정할 수 있으며, 측정결과는 보고서로 제공함. 보고서에는 초당 트랜잭션 수 (Transactions Per Second, TPS), 트랜잭션 지연시간, 자원 사용량 등과 같은 다양한 성능 지표를 포함함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

③ 하이퍼레저 캘리퍼의 구조: 캘리퍼는 성능을 측정하고자 하는 BaaS 프레임워크(System Under Test, SUT)에 대한 워크로드를 생성하고 SUT에서 측정되는 지표들을 지속적으로 모니터링하여 수집한 데이터를 기반으로 보고서를 생성함. 그림 36에서 보는 것과 같이 세 부분 (CLI, Core, Adapter)으로 구성됨.

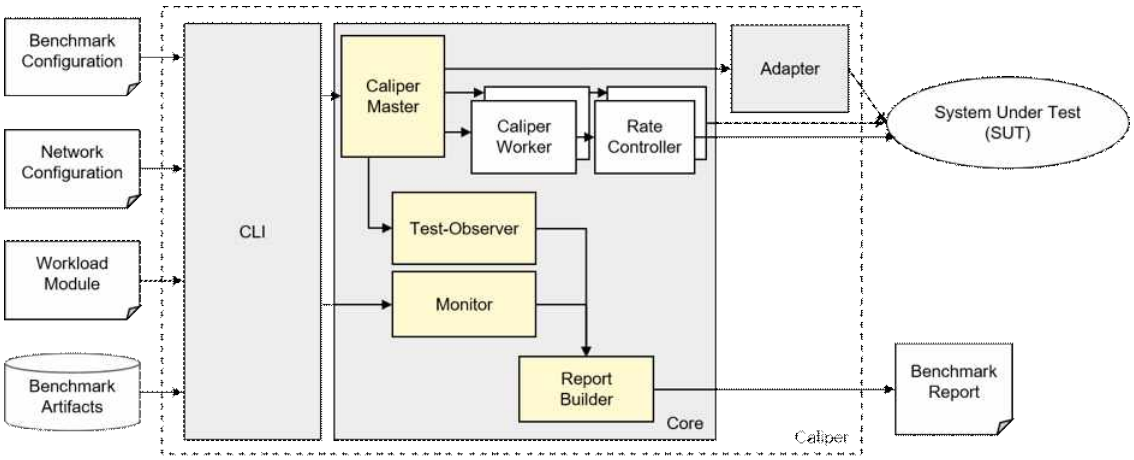


그림 36. 하이퍼레저 캘리퍼의 구조

㉮ CLI: 사용자에게 명령어 인터페이스를 제공하며, 사용자가 설정해놓은 벤치마크 설정 파일 (Benchmark Configuration)과 네트워크 설정 파일 (Network Configuration)에 따라 벤치마크를 실행할 수 있게 함.

㉯ Core: 캘리퍼의 핵심 부분으로 Caliper Master (이하 'Master'라 함), Caliper Worker (이하 'Worker'라 함), Rate Controller로 이루어짐. Master는 벤치마크 전체를 총괄하고 관리하는 역할을 하며 SUT를 모니터링하고 성능측정 결과 보고서(Benchmark Report)를 생성함. Master는 SUT의 성능을 측정하고 수집하는 Test-Observer와 자원 사용량을 모니터링하는 Monitor, 보고서를 생성하는 Report Builder로 구성됨. Worker는 실제로 블록체인 네트워크에서 수행되는 워크로드를 생성하며 Rate Controller는 트랜잭션 전송률을 제어함.

㉰ Adapter: SUT의 종류에 맞춰 캘리퍼와 SUT를 연결함. 하이퍼레저 패브릭, Sawtooth, Burrow, 이더리움, FISCO BCOS 등과 연결할 수 있도록 SUT에 따른 Adapter를 제공하고 있음.

㉱ 이외에도 트랜잭션의 내용을 생성하고 제출할 수 있게 하는 Workload Module과 벤치마크 실행 시 필요한 암호화 관련 파일, 스마트 컨트랙트 소스 코드 등을 지원해주는 Benchmark Artifacts가 있음.

① 하이퍼레저 캘리퍼 동작 방식 분석: 캘리퍼는 scalability를 높이기 위해 단일머신에서의 워크로드 생성뿐만 아니라 여러 대의 머신에서도 워크로드를 생성할 수 있도록 분산 방식을 제공함. 분산 방식의 제공을 위해 캘리퍼의 워크로드 생성과정을 Master process와 Worker process로 구성하고 있음. Master process는 Master에 의해 SUT를 초기화하고 벤치마크 실행을 조정하며 Worker를 관리하고 보고서 생성을 하는 단계임.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

Worker process에서는 여러 개의 Worker가 실행될 수 있으며 각각의 Worker가 독립적으로 실제 워크로드를 생성함. 설정에 따라 단일 머신 방식, 분산 방식의 선택이 가능함. Master process와 Worker process의 세부 동작 방식은 다음과 같음.

㉮ Master process: 캘리퍼에서 기본적으로 제공하는 예제 스크립트인 run.sh를 실행하면 generate.sh가 실행되고 블록체인 네트워크 구성에 대한 정보를 담고 있는 configtx.yaml과 crypto-config.yaml 파일의 설정에 따라 SUT가 생성됨. SUT가 생성될 때 caliper.yaml 파일을 통해 벤치마크 설정, 네트워크 설정, 보고서, Worker와 관련된 설정을 참조함. SUT가 생성되면 해당 SUT의 Adapter가 캘리퍼와 SUT를 바인딩함. Master와 Worker가 실행되면 getType() 함수로 SUT의 종류를 받아오고, init() 함수가 실행되어 SUT를 초기화함. 이후에 installSmartContract() 함수로 트랜잭션 요청을 할 수 있도록 하는 스마트 컨트랙트를 설치하는데, 이때 Master와 SUT 사이에서 Adapter가 해당 함수들을 SUT에 맞게 변환하여 전달함. 워크로드가 생성될 준비가 되면 Master는 벤치마크 실행을 조정하고 Worker들을 관리하며, 벤치마크가 끝나면 스크립트가 종료되고 보고서를 생성함. 벤치마크는 caliper-flow 설정에 따라 앞에서 말한 단계 중 원하는 단계를 선택하여 벤치마크를 구성하는 것이 가능함.

㉯ Worker process: Master가 벤치마크 실행을 조정하면 Worker는 실제 워크로드를 생성함. 워크로드 생성 과정은 먼저 Rate Controller의 활성화를 통해 트랜잭션 전송률이 조정됨. 이후 Worker는 invokeSmartContract(), querySmartContract() 함수를 호출하여 Workload Module을 통해 트랜잭션을 구성해 Adapter에 제출함. 트랜잭션을 제출하면 Adapter는 SUT에 트랜잭션을 전달 (request)함으로 실제 워크로드가 생성됨. Workload Module은 3가지 함수로 구성되어 있는데, 워크로드를 초기화하는 init() 함수, 실행하는 run() 함수, 종료하는 end() 함수임. 워크로드 생성이 진행되는 동안 Worker는 Master에게 진행 상황을 지속적으로 업데이트하여 보냄.

② 성능 지표 모니터링 동작 방식 분석

㉮ Test-Observer: SUT의 TPS, 트랜잭션 지연시간을 수집하며, caliper-core/lib/master/test-observers 디렉터리의 observer-interface.js에 인터페이스가 구현되어 있음. none, local, prometheus 등 3가지 수집 방법의 옵션이 있으며, 벤치마크 설정 파일에서 변경할 수 있음. 옵션에 따라 각각 null-observer.js, local-observer.js, prometheus-observer.js에 구현됨. constructor()의 benchmarkConfig.observer.interval을 통해 벤치마크 설정파일의 interval 값을 가져와 constructor.observerInterval에 설정함. constructor.testData는 설정한 interval마다 SUT의 TPS, 트랜잭션 지연시간을 수집함.

㉯ Monitor: SUT의 자원 사용량을 모니터링하며 caliper-core/lib/master/monitors 디렉터리의 monitor-interface.js에 인터페이스가 구현되어 있음. none, process, docker, prometheus 등 4가지 모니터링 옵션이 있으며, none을 제외하고 옵션에 따라 각각 monitor-process.js, monitor-docker.js, monitor-prometheus.js에 구현됨. constructor()를 통해 설정 파일에서 모니터링하는 대상인 monitorConfig 인자와 모니터링하는 주기인 interval 인자를 받아옴. readContainerStats()에서 호출되는 systeminformation 라이브러리의 dockerContainerStats()을 통해 모니터링하는 컨테이너의 자원 사용량 (CPU, 메모리, 네트워크, 디스크) 을 가져옴. getStatistics()에서 호출되는

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

MonitorUtilities.normalizeStats()와 getResultColumnMap()를 통해 저장된 자원 사용량 값들을 기반으로 최대값과 평균값을 구하고, 보고서에 출력되는 형태로 다시 저장함.

8) 보안 SLA를 위한 BaaS characterization

- 오더링 서비스 노드에 따른 자원 사용량 및 성능 차이 분석

(가) 오더링 서비스 노드 (Ordering service node)

오더링 서비스 노드는 IBM의 프라이빗 블록체인인 하이퍼레저 패브릭에서 합의 알고리즘(Raft와 Kafka)를 수행하는 노드임. 오더링 서비스 노드는 일정량 또는 일정 시간 동안 사용자의 트랜잭션 처리 요청을 수집 및 정렬하고, 이를 새로운 블록으로 생성하여 이를 다른 오더링 서비스 노드에게 전파하는 역할을 수행함. 블록을 전파 받은 오더링 서비스 노드들은 자신이 전파 받은 블록의 검증을 위하여 합의 알고리즘을 수행함. 하이퍼레저 패브릭은 오더링 서비스 노드들의 블록 합의를 위하여 Raft 또는 Kafka 합의 알고리즘을 사용함. 합의 알고리즘 Raft와 Kafka는 동작 방식이 상이하여, 오더링 서비스 노드가 두 개의 합의 알고리즘 중 하나를 선택하여 사용할 때 나타나는 자원 사용량 및 성능에서 차이가 발생함. 그러나, 하이퍼레저 패브릭의 Raft와 Kafka 합의 알고리즘을 수행하는 오더링 서비스 노드들의 자원 사용량과 성능 차이 분석 연구는 미미한 실정임. 이에 보안 SLA를 위한 BaaS characterization 연구를 위해 Raft와 Kafka 합의 알고리즘을 사용하는 오더링 서비스 노드의 동작 방식을 분석하고, 실험을 통하여 자원 사용량과 성능 차이를 분석하였음.

(나) 오더링 서비스 노드의 동작 방식 분석

하이퍼레저 패브릭의 오더링 서비스 노드는 하이퍼레저 패브릭에서 기록하는 블록을 생성하고 전파하는 역할을 수행하며, Raft와 Kafka 합의 알고리즘을 사용함. 오더링 서비스 노드에서 Raft와 Kafka 합의 알고리즘을 사용할 때, 오더링 서비스 노드의 블록 생성 및 전파 방식이 달라짐. 이에 따른 자원 사용량 및 성능 차이를 분석하기 위하여 먼저, 각 합의 알고리즘의 동작 방식을 분석함.

① Kafka 합의 알고리즘

Kafka는 LinkedIn에서 개발한 합의 알고리즘으로, 사용자의 데이터를 분산 시스템에 나누어 저장하고 저장된 데이터의 검증을 위하여 개발되었음. Kafka는 1) producer, 2) consumer, 3) Kafka broker 노드로 이루어진 Kafka 클러스터로 구성되어 있음. Producer는 Kafka 클러스터에 데이터를 전달하여 저장하고, consumer는 producer가 Kafka 클러스터에 저장한 데이터를 불러와서 사용하는 구조임.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

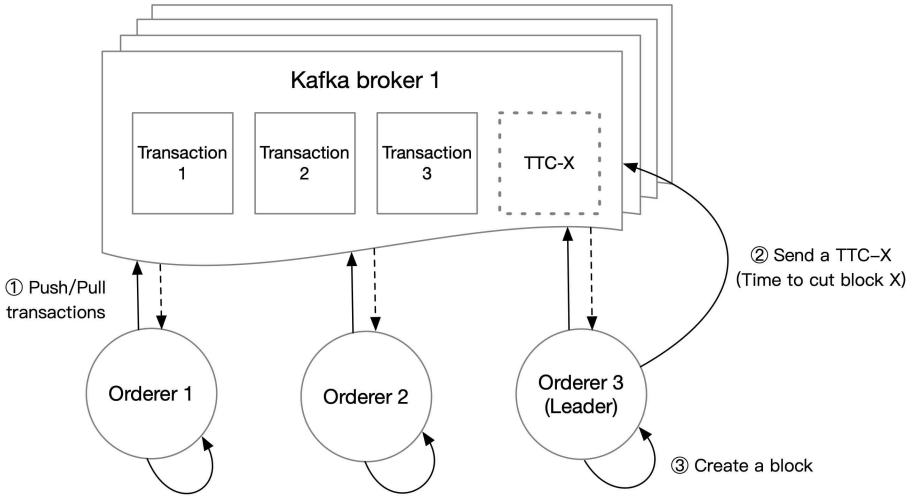


그림 37. 하이퍼레저 패브릭에서의 Kafka 구조

Kafka 합의 알고리즘을 사용하는 하이퍼레저 패브릭은 그림 37와 같이 오더링 서비스 노드가 Kafka의 producer와 consumer 역할을 수행하며 사용자의 트랜잭션 요청을 Kafka 클러스터에 저장 및 불러옴. Kafka 클러스터에 일정량 또는 일정 시간 동안의 트랜잭션이 저장되면, 각 오더링 서비스 노드들은 Kafka 클러스터에 저장되어 있는 트랜잭션을 불러와서 블록을 생성하여 피어 노드에게 전달하는 방식으로 동작함.

② Raft 합의 알고리즘

하이퍼레저 패브릭에서 사용하는 또 다른 합의 알고리즘인 Raft는 리더 노드와 팔로워 노드로 이루어진 클러스터로 구성되어 있음. 리더 노드는 팔로워 노드들의 Heart beat 투표로 선출되며, 리더 노드로 선출된 노드는 팔로워 노드들의 상태를 관리하며 데이터의 전파를 담당함. 리더 노드는 지속적으로 Heart beat를 사용하여 팔로워 노드들의 상태를 확인함. 사용자로부터 데이터의 저장 및 복제 요청이 발생하면 리더 노드는 사용자가 요청한 데이터를 팔로워 노드들에게 전달하는 역할을 수행함.

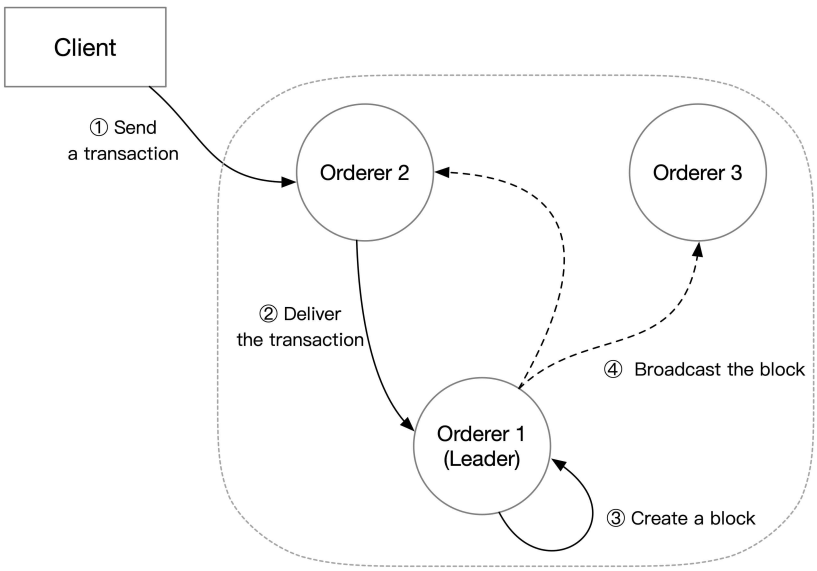


그림 38. 하이퍼레저 패브릭에서의 Raft 구조

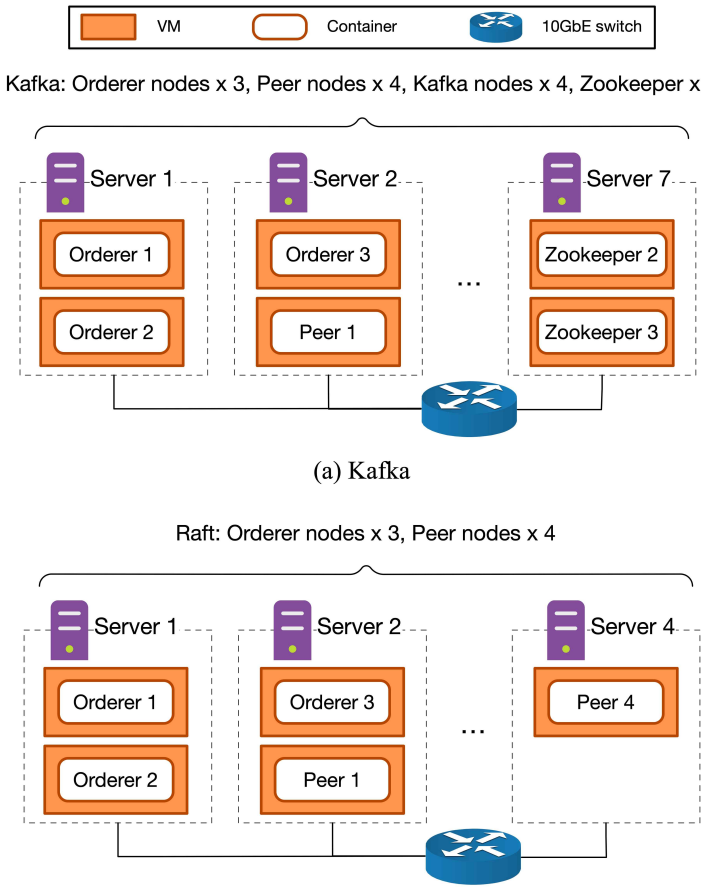
Raft 합의 알고리즘을 사용하는 하이퍼레저 패브릭은 그림 38과 같이 오더링 서비스 노드

가 리더와 팔로워 노드 역할을 수행함. 리더 노드로 생성된 오더링 서비스 노드는 사용자의 트랜잭션 요청을 일정량 또는 일정 시간 동안 저장하고, 블록을 생성하여 팔로워 오더링 서비스 노드들에 복제하는 구조임. 리더 노드는 팔로워 노드로부터 블록 복제를 확인하여 피어 노드에게 전달함.

(다) 오더링 서비스 노드의 자원 사용량 및 성능 차이 분석

① 실험 목적: 하이퍼레저 패브릭의 합의 알고리즘 Raft와 Kafka를 사용하는 오더링 서비스 노드의 자원 사용량 및 성능 차이 분석하기 위함.

② 실험 환경: 오더링 서비스 노드의 자원 사용량 및 성능 차이를 분석하기 위하여 물리머신 하나에서 하이퍼레저 패브릭 컴포넌트가 동작하는 로컬 시스템과 BaaS가 실제 동작하는 클라우드 환경과 유사한 환경인 멀티 시스템에서 하이퍼레저 패브릭 컴포넌트가 동작하는 실험을 진행함. 로컬 시스템 실험은 Intel Xeon E5-2650v4, 24 Core@2.2GHz CPU, 128 GB Memory, 256 GB SSD, Ubuntu 16.04 로 구성된 물리머신에서 실험을 진행함. 멀티 시스템 실험은 아래 그림 39와 같이 Intel Xeon E5-2650 v3 10 Core@2.3GHz, 128 GB Memory, 256 GB SSD, Ubuntu 16.04, 10 GbE 네트워크 스위치로 구성된 7대의 물리머신에서 실험을 진행함.



(a) Kafka

(b) Raft

그림 39. 멀티 시스템 구성

로컬 시스템에서는 하이퍼레저 패브릭 컴포넌트가 컨테이너 환경에서 동작하며, 멀티 시

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

스템에서는 하이퍼레저 패브릭 컴포넌트가 가상머신 안의 컨테이너 환경에서 동작함. 하이퍼레저 패브릭 컴포넌트는 Kafka의 경우, 3개의 오더링 서비스 노드, 4개의 피어 노드, 4개의 Kafka broker 노드, 3개의 Zookeeper 노드로 구성하고, Raft는 3개의 오더링 서비스 노드, 4개의 피어 노드로 구성함.

③ 실험 방법: 블록체인 성능 측정 툴인 하이퍼레저 캘리퍼를 이용하여 Kafka와 Raft로 구성된 오더링 서비스 노드의 자원 사용량과 성능을 측정함. 오더링 서비스 노드에서 생성되는 블록의 크기는 300개의 트랜잭션으로 제한하고, 사용자의 초당 트랜잭션 요청 수를 900 TPS (Transaction per seconds)로 제한하여 실험을 진행함. 실험에 사용한 체인코드는 단순화된 뱅킹 시스템으로, 계좌 생성, 송금, 예금, 계좌 조회의 트랜잭션을 처리하는 smallbank 체인코드를 사용함. 실험은 smallbank 체인코드가 동작할 때 Kafka와 Raft로 구성된 오더링 서비스 노드의 작업 처리량 (TPS)와 지연시간 (second)를 측정하고, 자원 사용량 CPU 사이클(%)와 통신 작업 처리량 (MB/s)를 측정.

④ 실험 결과:

㉮ 오더링 서비스 노드의 성능 작업 처리량과 지연시간

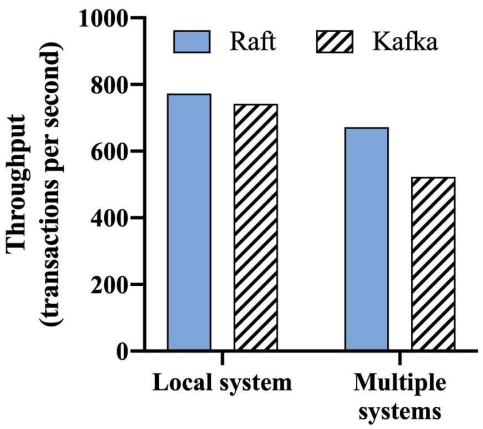


그림 40. 작업 처리량

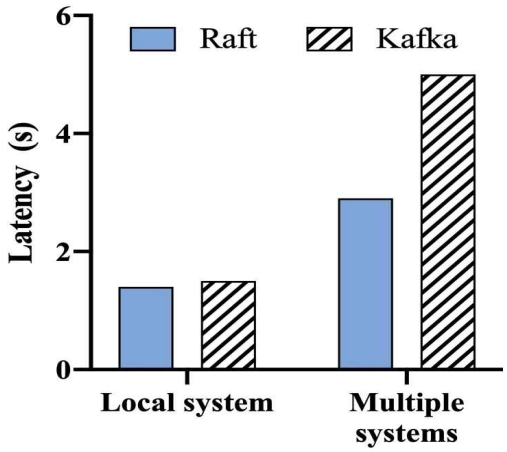


그림 41. 지연시간

그림 40과 41은 각각 로컬 시스템과 멀티 시스템에서 Raft와 Kafka로 구성된 오더링 서비스 노드의 작업 처리량과 지연시간 성능을 나타냄. Raft는 Kafka 보다 로컬 시스템과 멀티 시스템에서 각각 10%, 28% 보다 높은 작업 처리량을 보이고, 지연시간은 6%, 42% 높은 성능을 보임. 이러한 차이가 발생하는 이유는 Kafka는 모든 오더링 서비스 노드가 트랜잭션을 Kafka broker에 저장하고, Kafka broker로부터 트랜잭션을 복제하여 블록을 생성하는 반면, Raft는 리더 오더러 서비스 노드가 트랜잭션을 저장하고 블록을 생성하여 복제시키기 때문에 Raft는 Kafka보다 블록 생성 및 전파과정에서 더 효율적인 모습을 보임.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

㉰ 오더링 서비스 노드의 자원 소모량

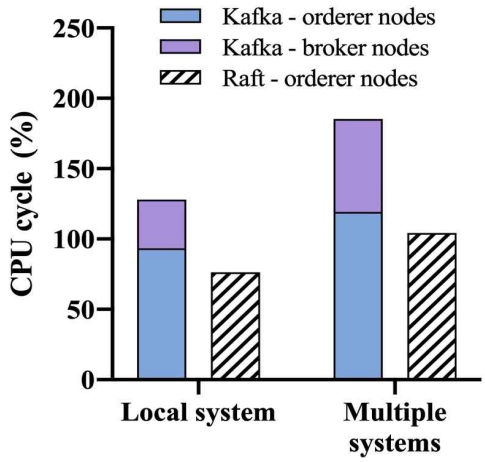


그림 42. CPU 사이클

그림 42는 로컬 시스템과 멀티 시스템에서 Raft와 Kafka 오더링 서비스 노드의 CPU 사이클을 나타냄. Kafka 오더링 서비스 노드는 Raft보다 로컬 시스템과 멀티 시스템에서 각각 67%, 77% 높은 CPU cycles를 보임. 이러한 차이가 발생하는 이유는 Kafka는 블록 생성이 모든 오더링 서비스 노드와 Kafka broker노드에서 이루어지는 반면, Raft는 블록 생성이 리더 노드에서만 발생하기 때문.

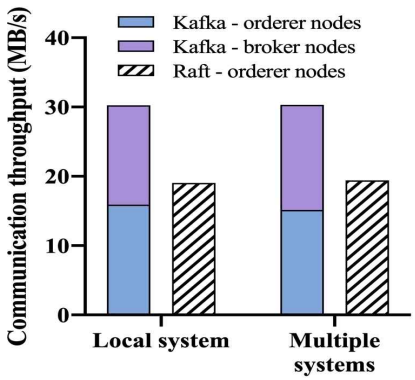


그림 43. 통신 작업 처리량 (transmission, TX)

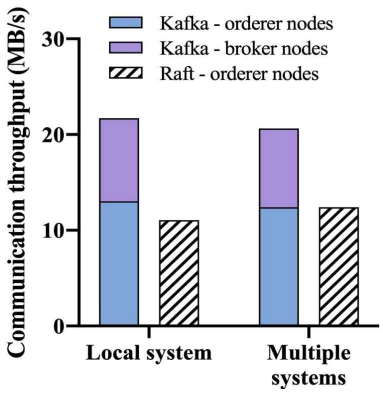


그림 44. 통신 작업 처리량 (reception, RX)

그림 43과 44는 로컬 시스템과 멀티 시스템에서 Raft와 Kafka의 통신 작업 처리량 (TX, RX)을 나타냄. Kafka가 Raft보다 TX 49%, RX 47% 높은 통신 작업 처리량을 보임. 이는 Kafka 오더링 서비스 노드가 블록 생성을 위한 오더링 서비스 노드와 Kafka broker 노드 간의 통신이 많기 때문임. 이에 비해 Raft의 블록 생성은 리더 오더링 서비스 노드에서 팔로워 오더링 서비스 노드로의 블록 복제로 인해 Kafka보다 통신 작업 처리량을 적게 사용함.

실험 결과로 하이퍼레저 패브릭의 Raft 오더링 서비스 노드가 Kafka 오더링 서비스 노드 보다 최대 28%정도 높은 작업 처리량, 지연시간 성능을 보임을 확인할 수 있었고, 자원 소모량은 Kafka가 Raft보다 CPU 사이클 최대 77%, 통신 작업 처리량 최대 49% 더 많이 사용함. 이는 Raft 오더링 서비스 노드가 Kafka 보다 컴퓨팅 자원을 더 효율적으로 사용하며 높은 성능을 보임을 확인하였음.

● BaaS 보안 SLA 파라미터 분석을 위한 보안 SLA 감사 도구 개발 및 테스트

가) 보안 SLA 감사 도구 개발

본 연구 과제에서 목표로 하는 보안 SLA는 BaaS 워크로드에서 시간당 사용자의 요청이 처리되는 성능인 트랜잭션 처리 성능 (transactions per second, tps)으로, tps의 저하가 발생하는 경우 사용자의 요청이 보안된 형태인 블록으로 생성되어 저장이 되는데 걸리는 시간이 지연되어 전반적인 서비스의 보안성이 저하될 수 있음. 이러한 문제를 해결하기 위해 BaaS 워크로드의 보안성에 대한 서비스 수준을 보장하기 위해 tps를 보안 SLA 지표로 설정하여 일정 이상의 보안 수준을 제공하고자 함. 보안 SLA 감사 도구는 BaaS 워크로드가 수행되는 과정에서 실시간으로 보안 SLA를 만족하는지 감사(audit)하고 만약 사용자가 설정한 보안 SLA를 만족하지 않는 경우 이를 보고 (report) 또는 개선하기 위한 제어(control)를 수행함.

6차년도 연구 내용인 “BaaS 성능 SLO 분석 및 모니터링 기법 연구” 를 수행한 결과, 기존의 BaaS 모니터링 기법은 사용자의 요청 처리가 모두 완료된 이후에 트랜잭션 처리 성능을 표시하기 때문에, 요청이 처리되는 과정에서는 성능 수치를 알기가 어려운 단점이 있음. 따라서, 실시간 BaaS 모니터링을 위해 주기적으로 성능 데이터를 수집하는 기술을 개발하였고, 이를 포함한 보안 SLA 감사 도구 전체 구조는 아래 그림과 같음.

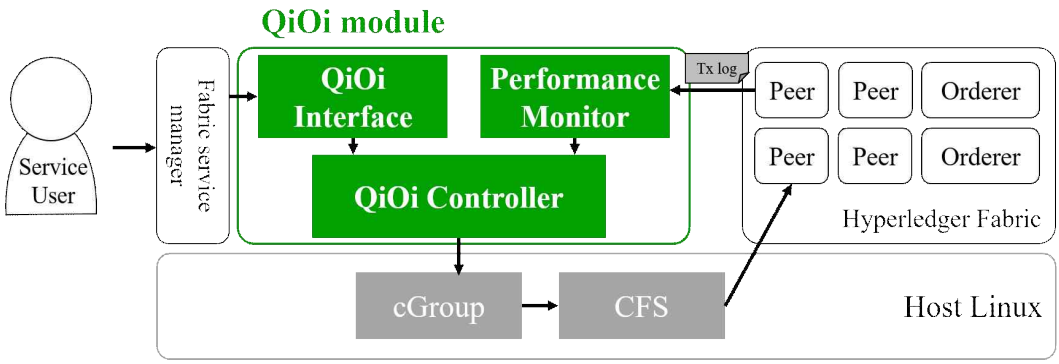


그림 45. 보안 SLA 감사 모듈 (QiOi) 구조

보안 SLA 감사 도구의 기본 동작은 실시간으로 BaaS 워크로드의 성능을 모니터링하여 일정 수준 이상 트랜잭션 처리 성능이 저하하는 경우 이를 CPU 자원 할당을 통해 개선하는 것을 목표로 함. CPU 자원 할당의 의미는 하나의 서버상에 동작하는 여러 컨테이너의 CPU 사용량을 동적으로 제어하는 것을 의미함. CPU 사용량 제어를 위해 본 연구에서는 리눅스에서 제공하는 기능인 cgroups의 CPU bandwidth 기능을 사용함. CPU bandwidth 기능은 사용자가 설정하는 수치에 따라 사용자 프로세스의 CPU 사용량을 제어하며, 100%인 경우 하나의 CPU 코어를 모두 활용하는 것을 의미함. 보안 SLA 감사 도구는 이러한 CPU bandwidth 기능을 기반으로 컨테이너의 CPU 사용량 제어를 통해 BaaS 워크로드의 성능 저하 탐지 및 개선 기능을 수행함. 보안 SLA 감사 도구의 구현은 리눅스 환경에서 사용자 데몬 형태로 이루어졌으며 그림 10과 같이 3개의 컴포넌트로 구성된 QiOi module 로 명명하였으며 세부 컴포넌트의 기능은 다음과 같음.

① QiOi interface: Fabric service manager를 통해 사용자로부터 보안 SLA 정보를 수신받아 이를 QiOi controller에 전달하는 역할을 수행. 보안 SLA 정보는 목표로 하는 트랜잭션 처리 성능과 해당 성능을 달성하는 데 필요한 CPU bandwidth 정보로 구성되어 있음.

② Performance monitor: 실시간으로 BaaS 워크로드의 성능 정보를 수집. 일반적으로 BaaS 워크로드 성능 정보 수집을 위해 사용하는 하이퍼레저 캘리퍼에서는 실시간 성능 정보 수집 기능을 제공하지 않고 있어서, 본 연구 과제에서는 추가적인 정보 수집을 통해 BaaS 워크로드의 실시간 성능 수치를 계산함. 정보 수집을 위해 컨테이너가 저장하는 로그 파일을 활용하며, 해당 파일에 기록된 시간 정보와 처리된 블록에 대한 정보를 기반으로 실시간 성능 수치를 계산하여 QiOi controller에 전달.

③ QiOi controller: QiOi interface에서 전달받은 보안 SLA 정보와 performance monitor로부터 전달받은 실시간 성능 정보를 비교하여 현재 보안 SLA를 만족하는지 주기적으로 확인함. 현재 성능이 보안 SLA를 만족하지 못하는 경우 BaaS 워크로드를 수행하는 컨테이너에 대해 CPU bandwidth 값을 설정하여 트랜잭션 처리 성능을 높이도록 하며, 설정하는 CPU bandwidth 값은 fabric service manager로부터 전달받은 최적 성능을 위한 값을 사용함.

나) 보안 SLA 감사도구 테스트

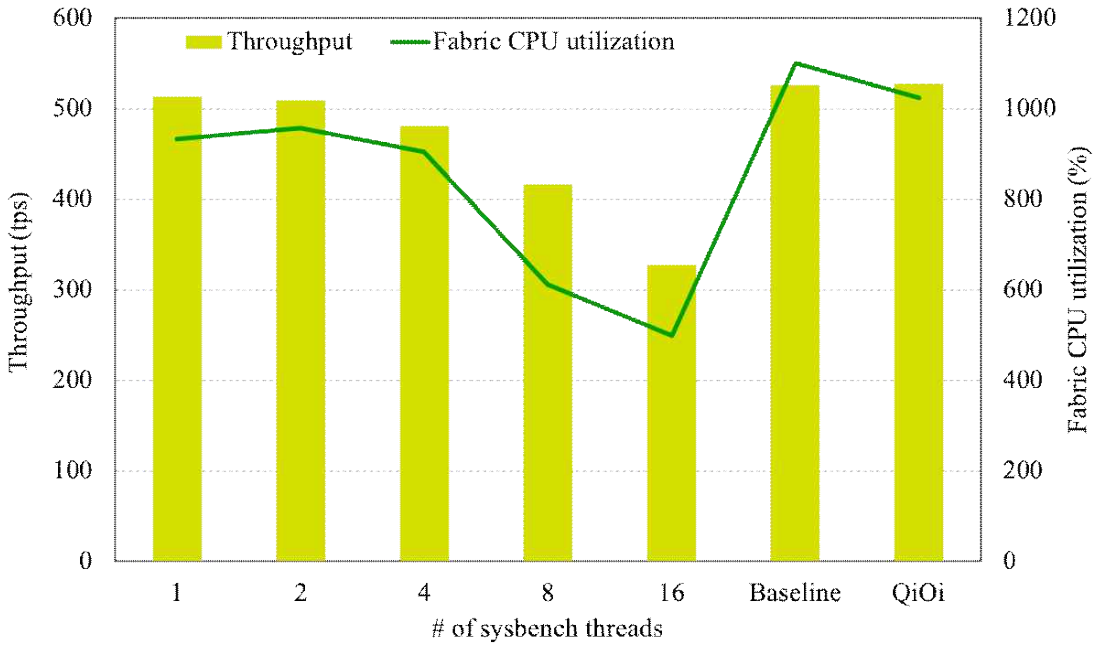


그림 46. QiOi 모듈의 성능 평가 결과

클라우드 환경에서는 가상머신 또는 컨테이너 형태로 다양한 서비스들이 수행되어, 하나의 서버에 여러 작업이 동시에 수행될 수 있음. 특히, CPU나 메모리 같은 컴퓨팅 작업을 과도하게 점유하는 작업과 함께 수행되는 경우 이로 인해 성능 저하가 발생할 수 있음. 그림 46는 BaaS 워크로드가 소수 연산을 연속적으로 수행하는 벤치마크 (i.e., sysbench)와 함께 수행되는 경우 벤치마크의 스레드 숫자 증가에 따라 트랜잭션 처리 성능 (i.e., tps)이 최대 55% 저하되는 결과를 보여줌.

본 연구 과제에서는 이처럼 동일한 호스트 머신에서 수행되는 다른 워크로드에 의한 보안 SLA 저하 문제를 실시간 보안 SLA 모니터링과 CPU bandwidth 제어를 통해 개선할 수 있는 보안 SLA 감사 도구인 QiOi 모듈을 설계 및 개발하였음. QiOi 모듈을 실제 시스템에 적용하여 그림 46과 동일한 환경에서 실행한 결과 종전의 성능 저하를 완전히 개선하였을 뿐만 아니라,

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

CPU 사용량 또한 7% 개선한 결과를 얻을 수 있었음. 이는 개발한 기법으로 인한 오버헤드가 크지 않고 오히려 작업 효율을 높일 수 있음을 보이며, 보안 SLA 모니터링 수준 (최종 목표 100%) 및 보안 SLA 제공으로 인한 CLS 저하 (최종 목표 1%) 정량적 성과 목표치의 달성 가능성을 보임.

9) 멀티 테넌트 기반 BaaS 성능 SLA 보장 연구

- 기존 연구 분석: 성능 간섭에 관한 기존 연구[6]는 컨테이너 기반 가상화 환경에서, 다수의 컨테이너가 자원을 공유할 때 발생하는 성능 간섭을 측정하고 분석한다. 그러나, 해당 연구는 컨테이너가 공유하는 메모리 자원에서 발생하는 성능 간섭을 다루고, 여러 프로세스 간의 CPU 스케줄링에 의해 발생하는 CPU 자원에 대한 성능 간섭은 다루지 않는다. 또 다른 기존 연구[7]에서는 가상 머신 환경에서, 리눅스의 CPU 스케줄러인 CFS로 인한 성능 간섭 문제 지적하지만 컨테이너 기반의 환경에서 측정된 성능 간섭이 아니며 실제 클라우드 환경에서 동작하는 워크로드를 사용한 실험이 수반되지 않았다. 이에 본 연구에서는 컨테이너 기반 가상화 환경에서 CFS에 의한 성능 간섭을 확인하기 위하여 실제 클라우드에서 사용되는 워크로드를 사용하여 성능 간섭을 확인하고 원인을 분석한다.

- 이종 워크로드 구성: 본 연구에서 대상으로 하는 워크로드는 아래 표와 같다. Hyperledger Fabric은 클라우드 서비스를 사용하여 블록체인 어플리케이션을 개발 및 호스팅 할 수 있는 프레임워크이다. Hyperledger Fabric 워크로드는 사용자의 트랜잭션 처리 및 저장을 위한 네트워크 및 디스크의 I/O-intensive 워크로드 특성을 보인다. Memcached는 데이터 베이스의 로드를 줄여 동적 웹 어플리이케이션의 성능을 높이는 분산 메모리 캐싱 시스템으로, 클라우드에서 고성능 웹 서비스를 제공하기 위하여 사용된다. Memcached 워크로드는 웹 서비스에서 사용자의 request 처리를 위한 네트워크 I/O-intensive 워크로드 특성을 보인다. Spark는 대용량 데이터를 처리 및 분석하기 위한 병렬 분산 처리 프레임워크이다. Spark 워크로드는 대용량 데이터를 정렬 및 검증하기 위한 CPU-intensive 워크로드의 특성을 보인다.

표 6. 이종 워크로드 특성 분류

워크로드	Hyperledger Fabric	Memcached	Spark
벤치마크	Hyperledger Caliper's smallbank	Memaslap's multi-set/get	TeraSort
특성	I/O-intensive	I/O-intensive	CPU-intensive
성능 지표	Throughput (TPS)	Throughput (TPS)	Job completion time (sec)
벤치마크 툴	Hyperledger Caliper	Memaslap	Spark-submit
작업 노드	4 peers	4 workers	2 workers
시스템 구성 노드	3 orderers	1 master	1 master, 2 wokers

6) Jeong, JinWon, et al. "Measuring Performance Interference on Memory Resources between Containers in Docker Swarm." Korea Information Processing Society (2020)

7) Donghwa, et at. "Analysis of Performance Interference in a KVM-virtualized Evironment in the Aspect of CPU scheduling." The Korean Institute of Information Scientists and Engineers (2016)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 실험 환경 및 방법: 본 연구에서는 Intel Xeon E5-2650 v3 @ 2.3GHz CPU와 128 GB 메모리를 탑재한 4대의 물리 서버로 분산 환경을 구축하여 실험을 수행한다. 4대의 서버는 10 Gigabit 네트워크 스위치로 연결되어 있음. 표 6에 제시된 워크로드는 각각의 컨테이너에서 동작하며 각 컨테이너에 각 2 CPU core 할당하여 하나의 물리 머신에 배포함. 실험 시나리오는 크게 2가지로 구성함. 먼저, 시나리오 1은 성능 간섭이 발생하지 않는 상황으로 (Alone), 하나의 물리 머신에서 하나의 워크로드가 독립적으로 동작하는 상황을 가정함. 다음으로, 시나리오 2는 하나의 물리 머신에서 세 개의 워크로드가 동시에 동작하는 상황으로 (Together), 특정 워크로드의 과도한 컴퓨팅 자원 점유로 인한 성능 간섭이 발생하는 상황을 가정함. Alone과 Together에서 각 워크로드의 성능과 작업 노드의 CPU 성능을 프로파일링을 수행함. 각 워크로드 성능은 표 6에 제시된 벤치마크 툴을 사용하여 측정하며, 작업 노드의 CPU 성능은 perf를 사용하여 측정함. Alone과 Together에서 측정한 결과를 비교하여 워크로드 특성에 따른 성능 간섭 영향을 분석함.

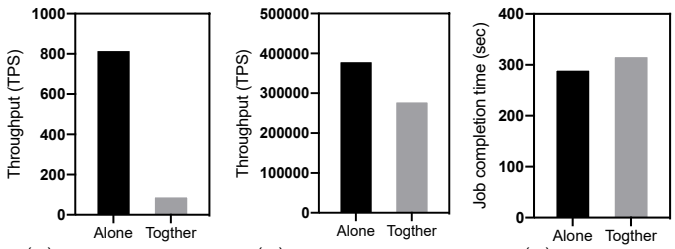


그림 47. 이종워크로드 성능 측정 결과

- 실험 결과: 그림 47은 Alone 과 Together 시나리오에서 측정한 각 워크로드의 성능을 나타냄. Alone 대비 Together에서 Hyperledger Fabric, Memcached, 그리고 Spark 의 성능은 각각 87.8%, 35.4%, 그리고 9% 감소함. CPU -intensive 워크로드인 Spark 워크로드에서 성능 저하가 가장 작게 나타나며, I/O-intensive 워크로드인 Hyperledger Fabric에서 성능 저하가 가장 크게 나타남. 즉, CFS가 각 워크로드에 공평하게 CPU 자원을 배분하지 못하여 성능 간섭이 발생하며, 이 때 I/O-intensive 워크로드 (Hyperledger Fabric)에서 성능 저하가 가장 크게 나타나는 것을 알 수 있음.

표 7. 이종 워크로드의 time slice

Time slice (ms)	Alone	Together
Hyperledger Fabric	0.32	0.23
Memcached	5.01	2.07
Spark	9.80	3.12

표 7는 Alone과 Together에서 측정한 각 워크로드의 time slice를 나타냄. 먼저, Alone에서 Hyperledger Fabric은 가장 짧은 time slice를 보이며, Spark에서 가장 긴 time slice를 보임. 이러한 워크로드들이 동시에 동작하는 Together에서, Hyperledger Fabric, Memcached, 그리고 Spark의 time slice는 Alone 대비 각각 29%, 58%, 그리고 82% 감소함. 이는 CPU 자원을 사용하려는 워크로드의 수가 증가함에 따라 각 워크로드가 CPU를 점유할 수 있는 실행시간이 CFS에 의해 짧게 결정됨을 의미함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

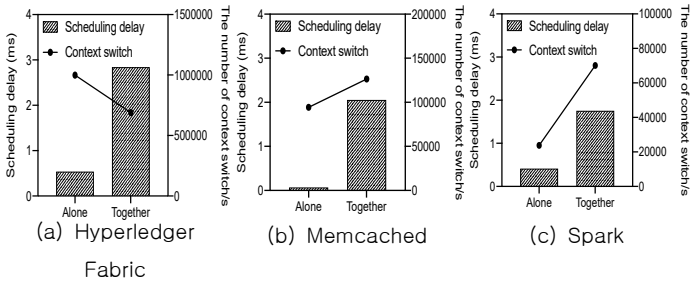


그림 48. 이종 워크로드의 스케줄링 지연과 문맥 교환

그림 48는 Alone과 Together에서 측정한 평균 스케줄링 딜레이와 초당 컨텍스트 스위치 수를 나타냄. 먼저, Together에서 평균 스케줄링 딜레이는 Hyperledger Fabric, Memcached, 그리고 Spark 가 Alone 대비 각각 약 5배, 33배, 그리고 4배 증가함. 이는 각 워크로드가 CPU를 새롭게 할당 받기 위해서 동작 중인 다른 워크로드의 time slice가 모두 소진될 때까지 대기해야 하기 때문임

다음으로, Hyperledger Fabric의 초당 컨텍스트 스위치 수는 31% 감소하는 반면, Memcached와 Spark는 각각 34%, 194% 증가함. Hyperledger Fabric은 time slice가 상대적으로 긴 다른 워크로드가 오랜 기간 CPU를 점유하여 스케줄링 대기 시간이 길어지기 때문에 초당 컨텍스트 스위치 수가 감소함. 반면, Memcached와 Spark는 Alone에 비해 time slice가 짧아져 초당 컨텍스트 스위치 수가 증가함. 실험 결과를 통해, time slice가 짧은 I/O-intensive 워크로드는 time slice가 긴 워크로드와 함께 동작하면, CPU 스케줄링 대기 시간이 길어져, CPU를 점유하는 횟수가 감소하며, 이에 따라 time slice가 짧은 워크로드는 동작에 필요한 CPU를 충분히 점유하지 못하게 되어 time slice가 긴 워크로드에 비해 상대적으로 성능 간섭이 크게 발생함을 알 수 있음.

● 이종 테넌트 간 고립을 지원하는 기법 설계 및 구현

1) 성능 고립을 위한 기법 설계: 본 연구에서는 하이퍼레저 패브릭과 같은 BaaS를 포함한 이종 워크로드가 동시에 동작하는 상황에서 발생하는 성능 간섭을 최소화 하기 위한 성능 고립 기법 설계 및 구현을 목표로 함. 효율적인 성능 고립 기법 설계를 위해 성능 간섭 시 발생하는 현상을 시스템 수준의 프로파일링을 통해 분석함. 그림 49은 하이퍼레저 패브릭이 이종 워크로드와 함께 동작하여 성능 간섭이 발생하는 경우 하이퍼레저 패브릭의 구성요소인 Peer, Orderer, Chaincode의 평균 timeslice, scheduling delay, context switch 횟수를 비교한 결과임. 하이퍼레저 패브릭만이 동작하는 Alone 시나리오에 비해 이종 워크로드와 함께 동작하는 Together 시나리오에서 평균 timeslice는 Peer, Orderer, Chaincode에 각각 22%, 35%, 3%가 증가함. 또한 평균 scheduling delay의 경우 3.2배, 4.4배, 1.9배가 증가하였다. 이를 통해 성능 간섭으로 인한 하이퍼레저 패브릭의 성능 저하의 주요 원인은 급격하게 증가한 scheduling delay 때문으로 볼 수 있음. 특히 그 정도가 Peer, Orderer에 심한 것을 확인하였다. 따라서, 하이퍼레저 패브릭 워크로드의 성능 고립을 위해서는 함께 동작하는 워크로드로 인한 scheduling delay의 증가를 막는 기법이 필요함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

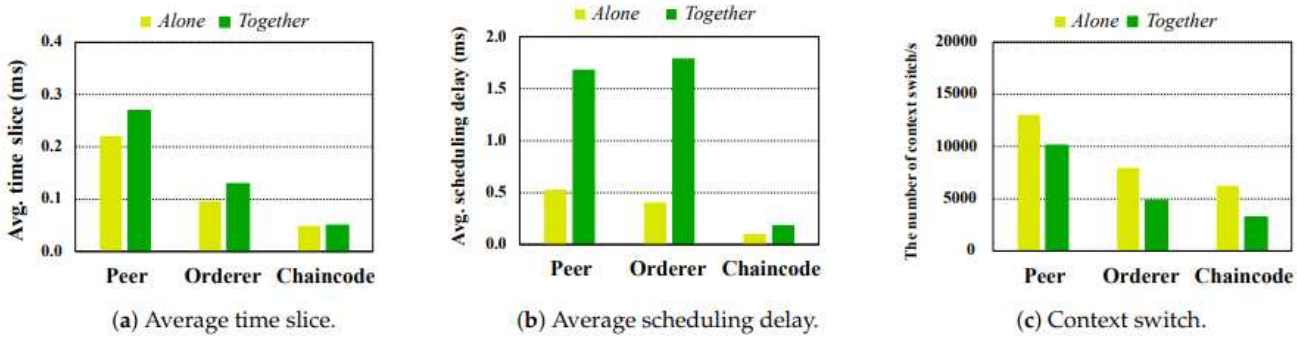


그림 49. 하이퍼레저 패브릭 구성요소 단위 성능 간섭 프로파일링 결과

하이퍼레저 패브릭 워크로드의 scheduling delay의 증가를 막기 위해 본 연구에서는 QiOi 모듈을 설계하였음. QiOi 데몬은 QiOi controller와 Performance monitor로 구성됨. 먼저, Performance monitor는 주기적으로 하이퍼레저 패브릭 워크로드의 성능을 모니터링함. 만약 현재의 성능이 사용자가 Fabric service manager를 통해 입력한 목표 성능에 도달하지 못하는 경우 QiOi controller를 통해 하이퍼레저 패브릭을 구성하는 Peer, Orderer의 scheduling delay를 줄이기 위한 CPU 스케줄링 파라미터를 조절함. QiOi controller에서 조절하는 CPU 스케줄링 파라미터는 CPU share 값으로 CPU 상에 수행되는 프로세스의 가중치를 의미함. 기본적으로 설정되는 값은 1024로, 만약 두 개의 서로 다른 프로세스 P1, P2가 각각 1024, 2048의 share 값을 가지는 경우 P2는 P1에 비해 2배 더 긴 CPU timeslice를 할당받게 됨. 따라서, P2는 P1에 비해 상대적으로 긴 시간 동안 CPU를 점유하게 되고 scheduling delay 또한 짧아지는 효과가 있음. 따라서, QiOi controller는 성능 저하를 겪고 있는 하이퍼레저 패브릭의 Peer, Orderer의 share 값을 상향 조정하여 scheduling delay를 감소시키고 긴 CPU timeslice를 할당하여 성능을 향상함.

10) 네트워크 가상화를 통한 테넌트 간 고립(isolation) 연구

1) 오픈플로우 확장을 통한 네트워크 하이퍼바이저의 주소 가상화 기능 개발

본 연구에서는 제안한 LISP 기반 IP 주소 가상화 기법의 플로우 엔트리 소모 및 기법의 부하 평가를 위하여, 오픈 소스 프로젝트인 OpenVirteX를 확장하여 제안하는 LISP 기반 주소 가상화 기법을 프로토타입 형태로 구현함. 제안하는 기법은 터널링 시 캡슐화, 역 캡슐화를 통해 주소 가상화를 지원함. 하지만 OpenVirteX에 서 사용하고 있는 오픈플로우 버전 1.0에서는 캡슐화를 위한 액션을 따로 제공하지 않음. 따라서 본 연구진 OpenVirteX에 구현된 오픈플로우를 확장하여 캡슐화 및 역 캡슐화를 위한 액션을 추가하는 작 업을 진행함. 또한 본 연구에서는 물리 네트워크 에뮬레이션을 위해 Mininet을 사용하였는데, Mininet 은 토폴로지 구성 시 오픈소스 소프트웨어 스위치인 OpenVSwitch를 활용함. OpenVSwitch는 오픈플 로우 1.0 이상도 지원하지만 OpenVirteX와의 통신 시에는 같은 버전인 1.0을 사용함. 따라서 OpenVSwitch에도 오픈플로우 1.0의 액션을 확장하는 작업을 수행함.

OpenVSwitch는 크게 유저 영역에서 동작하는 데몬과 커널 영역에서 동작하는 커널 모듈로 구성되는데, 데몬의 주요 역할 중 하나는 OpenVSwitch로 전달되는 오픈플로우 패킷을 버전에 상

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

관없는 자료구조로 변환하여 커널 모듈로 전달하는 것임. 커널 모듈은 오픈플로우에 의해 설정된 룰에 따라 실제 패킷을 처리하는 작업을 수행하기 때문에, 실제 캡슐화, 역 캡슐화 기능은 커널 모듈에서 동작하도록 구현함.

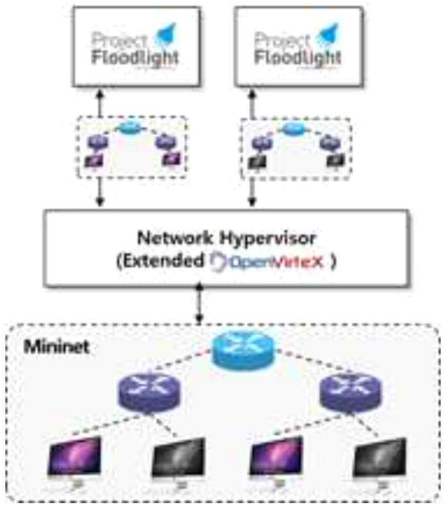


그림 50. 실험 환경 구성

2) 하이퍼바이저 상에서의 주소 가상화 구현: 기존 OpenVirteX 하이퍼바이저는 각 가상화된 SDN 기반 네트워크 (virtualized SDN; vSDN) 컨 트롤러의 오픈플로우 FlowMod 패킷을 변조하여 rewriting 액션을 추가함으로써 주소 가상화를 구현 함. FlowMod 패킷은 오픈플로우 프로토콜에서 스위치의 플로우 테이블을 제어하기 위해 사용되는 패 킷 타입을 의미함. 제안하는 기법은 rewriting 액션 대신 캡슐화/역 캡슐화 액션을 추가하여 주소 가상화를 구현함. 캡슐화 액션에는 L2 주소, L3 주소 및 테넌트 ID가 포함되며, ingress edge 스위치에서 터널링 시 캡슐화를 하도록 명령함. 역 캡슐화 액션은 워드 정렬을 위한 패딩 필드 외에는 별다른 필드를 갖지 않으 며, egress edge 스위치에서 추가했던 패킷 헤더를 제거한 후, 다시 가상 주소로 매칭을 한 번 더 하도록 명령함. 따라서 역 캡슐화를 위한 FlowMod 패킷은 총 두 개가 발생함.

3) 실험을 통한 성능 평가: 제안하는 방식의 장점 제안하는 기법을 평가하기 위해 다음과 같은 실험 환경을 구성하였음. 실험 환경은 두 대의 서버로 구성되는데, 서버 1에는 네트워크 에뮬레이터인 Mininet과 제안하는 주 소 가상화 기능이 구현된 OpenVirteX가 동작하며 서버 2에는 vSDN 컨트롤러인 Floodlight가 동작함. 두 서버는 10Gbps 네트워크로 연결하였음. Mininet은 2.3.0d1 버전을 사용하였으며 OpenVSwitch는 2.5.0 버전을 수정하여 사용하였음. 네트 워크 하이퍼바이저로는 OpenVirteX 0.0.1 버전을 확장하였으며 vSDN 컨트롤러는 Floodlight 1.2 버전이 사용되었음.

* 플로우 엔트리 수 측정 실험

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

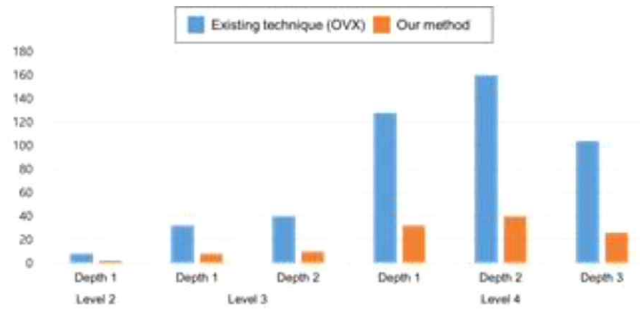


그림 51. 이진 트리에서의 플로우 엔트리 수 측정



그림 52. 액션 처리 지연시간

제안하는 기법이 플로우 엔트리를 적게 소모한다는 것을 보이기 위해 다양한 토폴로지에 대해 가상 네트워크를 생성하고, 플로우 엔트리 수를 측정하는 실험을 진행하였음. 첫 번째로 이진 트리 토폴로지를 구성하고, 트리의 레벨을 달리하여 실험을 반복하였음. 가상 네트워크 토폴로지는 물리 네트워크와 동일하게 구성하였으며, edge 스위치마다 2개의 호스트를 생성한 후 호스트간 pair-wise ping 트래픽을 생성하여 플로우 엔트리 개수를 측정하였음.

그림 51은 트리 토폴로지에서 core 스위치의 플로우 엔트리 수를 측정한 결과를 나타냄. edge 스위치는 기존 기법과 동일한 플로우 엔트리 수를 사용하므로 따로 표기하지 않음. 레벨 2 트리에서는 루트 스위치에 해당하는 Depth 1 스위치의 플로우 엔트리가 8개에서 2개로 줄어든 결과를 보였으며, 레벨 3 트리에서는 Depth 1 스위치의 플로우 엔트리가 32개에서 8개로, Depth 2 스위치 2개에서는 각각 40개에서 10개로 줄어드는 결과를 보임. 마지막으로 레벨 4 트리에 서는 Depth 1에서 128개가 32개로, Depth 2 스위치 2개에서는 160개에서 40개로, Depth 3 스위치 4개에서는 104개가 26개로 줄어드는 결과를 보임. 이진 트리 토폴로지에 포함된 core 스위치의 경우 모든 레벨에서 제안하는 기법이 기존 기법 대비 75%의 플로우 엔트리 소모 절감을 보임.

다음으로 성형 토폴로지와 선형 토폴로지를 구성하여 플로우 엔트리 수를 측정하였음. 성형 토폴로지는 하나의 core 스위치에 5개의 스위치를 연결하였으며, 각각의 스위치에 하나씩의 호스트를 연결함. 실험 결과 core 스위치의 플로우 엔트리가 기존 48개에서 18개로 줄어드는 결과를 보임. 선형 토폴로지는 세 개의 스위치를 일렬로 구성하였으며, 양 끝에 각각 4개의 호스트를 연결하였음. 실험 결과 플로우 엔트리가 32개에서 2개로 줄어드는 효과를 보임. 제안하는 기법은 core 스위치에서 같은 경로의 플로우 엔트리를 하나로 합치는 효과가 있기 때문에 얼마나 많은 플로우가 같은 경로를 공유하는지에 따라 그 효율이 달라질 수 있음. 하지만 메쉬 토폴로지와 같이 두 호스트 사이가 두 홉 이내로 연결된 경우에는 플로우 엔트리 절감 효과가 없음.

* 오버헤드 측정

제안하는 기법으로 인해 추가적으로 발생하는 오버헤드를 측정하는 실험을 진행하였음. 제안하는 기법은 Ingress Edge 스위치에서 캡슐화 액션을 수행하며, Egress Edge 스위치에서는 역캡슐화 액션을 수행한 후 다시 플로우를 매칭함. 이는 기존 기법의 Ingress, Egress 스위치에서 각각 두 번씩 수행하는 set 액션에 대응됨.

그림 52은 OpenVSwitch에서 각각의 액션을 처리하는 시간을 나타냄. 본 연구진이 구현한 캡

슐화 액션은 294.88ns, 역 캡슐화 액션은 557.91ns의 처리 시간이 필요한 것으로 측정되었음. 이는 기존의 패킷에 대해 새로운 프로토콜 헤더를 추가하기 위한 메모리 연산 등의 시간이 발생하기 때문이며 또한, 역 캡슐화에 시간이 더 오래 걸리는 것은 헤더를 제거한 후 패킷 매칭을 한 번 더 하기 때문임. 반면 기존 기법의 set 액션을 두 번 실행하는 시간은 197.33ns 으로 측정되었음. 또한 대조군으로 실험한 VLAN 헤더를 추가하고 제거하는 실험에서는 약 45ns이 소모되었는데 이는 하드웨어가 헤더 처리 작업을 대신하기 때문에 처리 시간이 절감되는 효과가 있음. 패킷 당 처리 시간의 증가와 달리, Iperf를 통한 UDP 트래픽의 전송 성능 측정 결과, 기존 기법과 제안하는 기법 모두 35.3 Mbits/s의 동일한 성능을 나타냄. 즉, 기존 기법에 의해 증가한 패킷 당 처리 지연은 전체 패킷 포워딩에 있어서 무시할 수준임을 확인할 수 있음. 마지막으로 네트워크 하이퍼바이저에서의 주소 가상화 처리시간을 측정함. vSDN 컨트롤러가 생성한 오픈플로우 FlowMod 패킷을 네트워크 하이퍼바이저가 받은 시점부터, FlowMod 패킷을 수정하여 주 소 가상화 명령을 추가한 뒤 스위치로 전달하기까지의 시간을 측정하였음. 제안하는 기법은 기존 기법에 비해 약 0.8ms의 추가 지연 시간이 발생함을 확인함.

라. 이종 멀티 테넌트를 위한 가상 네트워크 모니터링 기법 연구

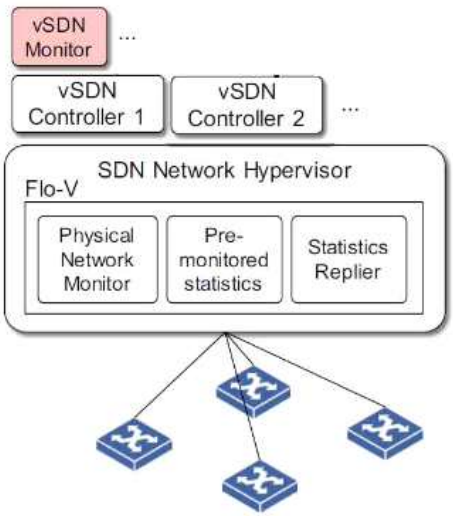


그림 53. Flo-v 구조

1) 가상화된 SDN에서의 네트워크 모니터링 플랫폼 개발 (Flo-v): OVX를 포함한 대부분의 가상화된 SDN은 네트워크 가상화 자체를 제공하는 데 중점을 두었음. 응용 프로그램에서 가상화 기술을 실제로 사용하려면 네트워크 관리 및 최적화에 대한 연구가 필요함. 그 첫 단계로, 네트워크 관리에 대한 전제 조건으로 네트워크 모니터링 연구(Flo-v)를 진행. OVX의 현재 네트워크 모니터링 모델은 정적으로 구성된 주기(기본 폴링 주기인 30초)로 모든 물리적 스위치에서 네트워크 통계를 수집함. 이후 이 방법을 사전 모니터링이라고 부르며, 사전 모니터링의 문제인 네트 워크 하이퍼바이저의 모니터링 정확성 저하 및 상당한 오버 헤드에 대한 평가 비교까지 진행하였음.

2) 기법의 설계: Flo-v는 가상 네트워크의 요청을 고려하여 모니터링 오버헤드를 줄이면서 vSDN 컨트롤러에 정확한 모니터링 정보를 제공하는 것을 목표로 함. 먼저 Flo-v의 전반적인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

디자인 소개 후, 선택 모니터링과 적응 모니터링의 두 가지 핵심 기술을 제시. 선택적 모니터링은 모니터링 할 네트워크 장치를 선택하여 네트워크 하이퍼바이저에서 생성되는 제어 메시지를 줄임. 적응형 모니터링은 모니터링 효율성을 향상시키기 위해 네트워크 하이퍼바이저의 모니터링 주기를 조정함.

그림 53은 Flo-v의 전체 구조를 나타냄. Physical network monitor는 사전 정의 된 간격 동안 네트워크 장치에서 통계를 수집하고 검색된 정보를 Pre-monitored statistics에 저장함. Statistics Replier는 vSDN의 요청에 따라 사전 모니터링 된 데이터를 검색하는데 사용되는 개체이며 각 물리적 스위치의 모니터링 간격을 조정함. 또한 요청에 대한 사전 모니터링 된 데이터가 존재하지 않으면 물리 네트워크에서 통계를 수집하고 vSDN에 대한 통계를 직접 제공함. 기존 네트워크 하이퍼바이저는 컨트롤러의 모니터링 주기와 관계없이 정적으로 구성된 주기에 모든 네트워크 장치에 모니터링 요청을 보냄. 네트워크 하이퍼바이저의 모니터링 주기가 컨트롤러의 모니터 링 주기보다 짧은 경우, 네트워크 하이퍼바이저는 트래픽 모니터링을 요구하는 컨트롤러가 없더라도 모니터링 작업을 수행함. 대조적으로 네트워크 하이퍼바이저의 모니터링 주기가 컨트롤러의 모니터링 주기보다 길면 정확성 문제가 발생함. 이로 인해 컨트롤러가 잘못된 모니터링 정보를 수신할 수 있으며, 이는 ONOS와 같이 플로우 table의 consistency를 유지하는 컨트롤러의 오동작을 초래함. 네트워크 하이퍼바이저에서 불필요한 모니터링을 제거하기 위해 컨트롤러의 모니터링주기에 따라 모니터링 간격을 조정하는 적응형 모니터링을 제안함. 이를 위해 Statistics Replier는 먼저 vSDN 컨트 롤러에서 수신한 각 물리적 스위치의 모니터링 요청 간격을 측정함. 그런 다음 Flo-v는 물리적 스위치의 모니터링 간격을 실제 네트워크 모니터에서 재설정하여 각 스위치의 폴링 간격을 계산함.

3) 실험을 통한 성능 평가: 정확도와 오버헤드 측면에서 Flo-v의 이점을 평가함. 첫째, 네트워크 하이퍼바이저의 모니터링 주기가 변경 될 때 네트워크 모니터링의 정확성을 평가하여 기존 기술의 정확도를 Flo-v의 정확도와 비교함. 또한 네트워크 하이퍼바이저에서의 지연과 네트워크 및 CPU 측면에서의 리소스 사용량을 측정함.

평가를 위해 Mininet을 물리 네트워크로 사용하고 OVX에서 Flo-v를 구현하고 Floodlight SDN 컨트롤 러 및 모니터링 어플리케이션을 사용함. 각 소프트웨어는 별도의 머신에서 실행되고 10 GbE NIC으로 연결됨. 토폴로지는 각각 3개의 호스트가 있는 11개의 물리적 스위치로 총 33개의 호스트로 구성됨. 가상 네트워크는 5개의 스위치 5개의 스위치 (v-sw1 v-sw5)가 선형으로 연결된 형태로 구성하고 상기 그림과 같이 두 개의 호스트(호스트 A와 B)를 Edge 스위치에 연결함. 트래픽은 가상 네트워크의 호스트 A와 B 사이에 ping을 사용하여 생성하였고, vSDN 컨트롤러는 가상 네트워크의 모든 스위치에 대해 매 초 마다 모든 플로우에 대한 모니터링 요청을 보냄. 실험은 사전 모니터링을 하지 않는 경우, 정적 주기 로 사전 모니터링(기존 모니터링 기법)하는 경우, 제안된 Flo-v를 사용하는 경우 세 가지로 진행함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

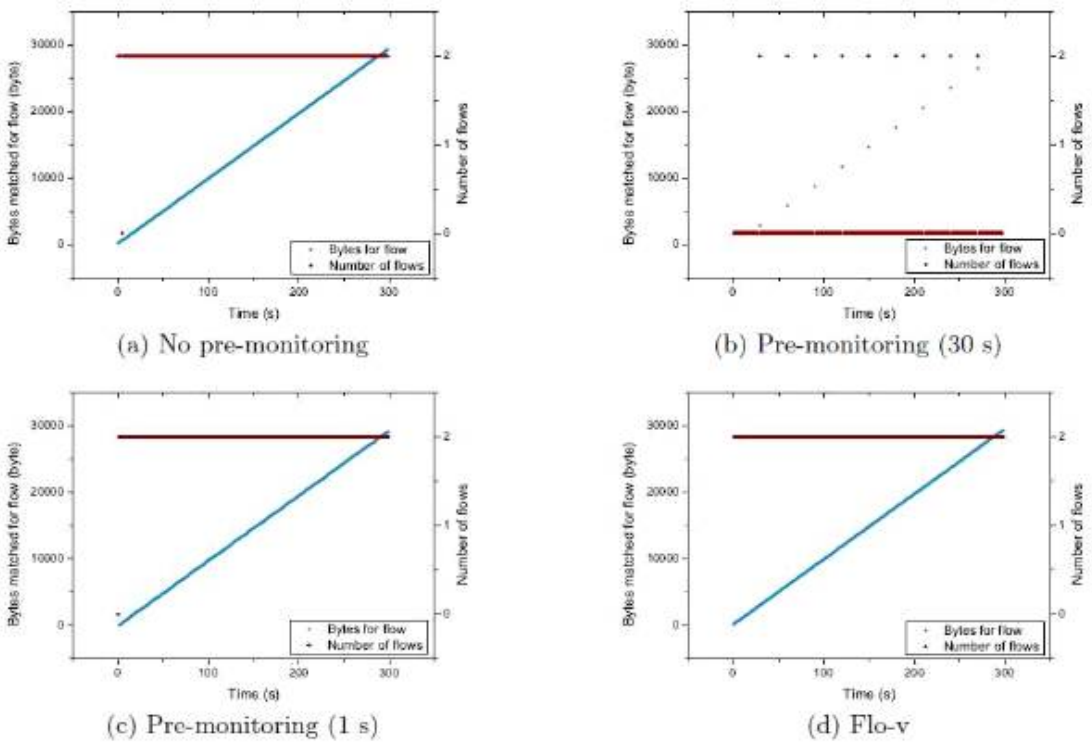


그림 54. vSDN에서 측정된 네트워크 통계 정보

먼저, 정확도의 경우 가상 네트워크에 대한 기존의 네트워크 모니터링 기법은 정확도를 떨어뜨림. 그림 54은 vSDN의 각 모니터링 작업에서 플로우 수 및 플로우 룰에 매칭된 바이트 수를 나타내는데, 그림 54 (a)인 사전 모니터링이 없는 경우는 vSDN 요청에 따라 물리적 스위치에서 직접 통계를 수집하기 때문에 올바른 정보를 제공하는 경우에 해당됨. 반대로 OVX의 기본값인 b의 경우 (30초 주기 사전 모니터링) vSDN에 부정확한 정보가 전달됨을 알 수 있음. 정확한 결과를 제공하기 위해서는 c와 같이 폴링 간격을 1초 이하로 설정해야함. 반면 Flo-v는 그래프 a와 유사하게, 정확한 정보를 제공함. 따라서 오버헤드의 경우 정확한 정보를 제공하는 1초 주기로 모니터링하는 OVX와 Flo-v를 비교함.

다음으로, 모니터링으로 인한 부가적인 오버헤드를 평가하기 위해 네트워크 하이퍼바이저에서의 통계 처리 지연을 측정함. 실험은 이미 핑 트래픽이 흐르고 있는 네트워크를 300초간 모니터링하는 상황에서 수행됨. 그림 55는 3분 동안 vSDN 컨트롤러의 요청과 응답 간의 평균 지연을 나타냄. 여기 에는 네트워크 하이퍼바이저에서 통계를 제공하고 가상화하는 전반적인 처리 지연이 포함됨. 그래프에 서 볼 수 있듯이 사전 모니터링 (1초)은 기본 사례에 비해 지연을 66% 줄임. 제안된 Flo-v는 이 지연을 사전 모니터링 (1초)보다 56 %만큼 감소시킴. 이는 Flo-v가 vSDN 컨트롤러에 따라 모니터링 주기 및 카운터를 리셋하기 때문이며, 사전 모니터링 주기 조절 정책에 의해 최적화될 수 있음.

11) BaaS 성능/보안 SLA 보장 효율성 향상을 위한 최적화 연구

가. 이종 멀티테넌트 네트워크 가상화 및 모니터링 비용 최소화 및 성능 최적화

1) 네트워크 가상화 및 모니터링 비용 최소화

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

① 모니터링 비용 최소화

- **연구동기**: 데이터센터의 핵심 기술인 네트워크 가상화는 물리 네트워크 인프라를 가상화하여 각 테넌트에게 가상 네트워크를 제공하며, 가상 네트워크간 네트워크 트래픽 고립을 제공함. 이 때문에 테넌트가 컴퓨팅 노드(e.g., 가상머신, 컨테이너)에 접근하기 위한 고립된 네트워크를 요구하는 클라우드 데이터센터에 널리 사용됨. 전통적인 네트워크 가상화 방식인 오버레이 네트워킹이 존재하지만, 테넌트가 주어진 가상 네트워크를 별도로 수정할 수 없어 네트워크 최적화를 필요로하는 대다수의 어플리케이션은 심각한 문제를 겪게 된다는 단점이 있음. 이에 대한 해결방안으로 네트워크 제어부분과 패킷전송기능을 분리한 software-defined networking(SDN) 기반 네트워크 가상화(SDN-NV)가 제안되었음. 네트워크 하이퍼바이저(NH)는 물리 네트워크를 가상화하여 가상 스위치, 링크, 포트 등을 제공하며, 각 테넌트는 컨트롤러(SDN controller)를 통해 NH가 제공하는 가상화된 네트워크 자원으로 SDN-NV를 구성 할 수 있음. 또한 네트워크 인프라에서 발생하는 오류를 처리하고 각 테넌트에게 고성능 서비스를 제공하기 위해 가상 네트워크에 대한 모니터링이 필수로 요구됨[8]. 가상 네트워크 모니터링을 위해 테넌트 컨트롤러가 통계 요청 메시지를 보내면, NH는 물리 네트워크(스위치)에 통계 요청 메시지를 전달하고 결과 수신을 기다림. 이 때 테넌트 컨트롤러의 통계 요청과 물리 스위치의 응답 사이의 지연(전송 지연)이 증가할 경우 오래된 네트워크 통계 정보로 인해 테넌트 컨트롤러의 네트워크 최적화에 문제가 발생할 수 있음. 따라서 SDN-NV 모니터링 기법 설계 시 모니터링 비용을 최소화하기 위한 세심한 설계가 필요함.

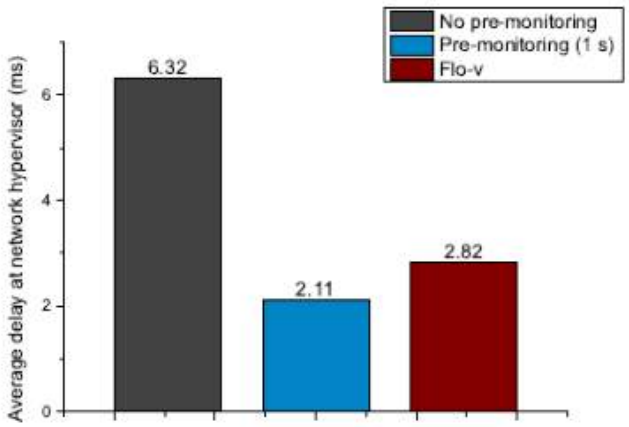


그림 55. 네트워크 하이퍼바이저 지연시간

- **가상 네트워크 모니터링 설계**: SDN-NV에서 가상 네트워크 모니터링 시 발생하는 모니터링 비용은 크게 1) NH와 물리 스위치간의 통계 요청 및 응답에 소요되는 전송 지연과, 2) NH에서 가상 네트워크의 통계를 계산하고 생성하는 시간으로 구분됨. 본 연구에서는 NH와 물리 스위치간의 전송 지연을 최소화하기 위해 1) pStatistics cache와 2) Request interval estimation을 제안함.

  pStatistics cache는 테넌트 컨트롤러의 가상 네트워크의 통계 정보가 요청 되었을 때 물리 네트워크의 통계 정보를 받아오는 것이 아닌, NH에 미리 물리 네트워크의 통계 정보를 캐

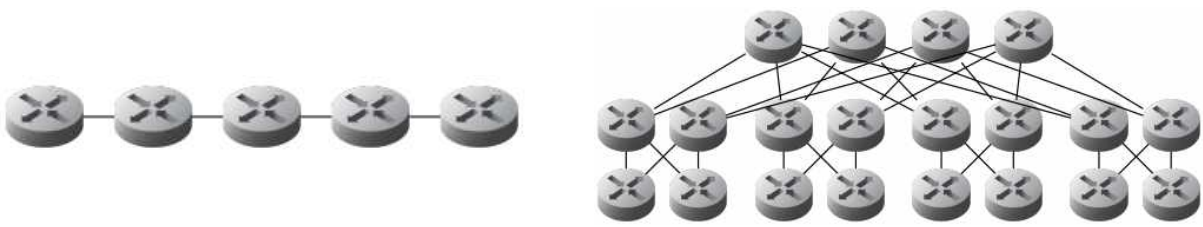
8) Roy, Arjun, et al. "Cloud datacenter sdn monitoring: Experiences and challenges." Proceedings of the Internet Measurement Conference (2018)

싱해여 NH와 물리 스위치간의 전송 지연을 제거하는 기법임. pStatistics cache는 1) pStatistics cache 갱신 시간이 모니터링 간격보다 길거나 2) 테넌트 컨트롤러로부터 가상 네트워크의 통계 정보가 요청되어 이미 사용된 경우 오래된 것으로 간주되어 갱신됨.
Request interval estimation은 적절한 pStatistics cache 갱신 간격을 설정하는 기법임. 모니터링 간격에 비해 pStatistics cache 갱신 간격이 너무 짧을 경우 cache가 hit 되기 전에 갱신되어 CPU 자원과 컨트롤 채널의 대역폭을 낭비하게 됨. 반대로 cache 갱신 간격이 너무 길 경우 pStatistics가 오래된 것으로 간주되어 테넌트 컨트롤러의 가상 네트워크 통계 정보 요청 시 전송 지연을 제거하기 위해 사용될 수 없음. 적절한 pStatistics cache 갱신 간격을 설정하기 위해 각 가상 네트워크의 모니터링이 요청된 간격의 평균과 분산 값을 사용하며, 여러 가상 네트워크의 모니터링 요청 분포 중 가장 작은 모니터링 간격을 가진 분포를 기준으로 pStatistics 갱신 간격을 결정함.

- **실험 환경 및 방법**: 본 연구에서는 총 3개의 물리서버를 사용하며, 물리 네트워크(Mininet), NH(Libera) 및 테넌트 컨트롤러(ONOS)가 각 물리서버에서 독립적으로 실행됨. 따라서 모든 서버 쌍의 RTT를 측정하며, 평균 및 95% tail RTT는 모든 서버 쌍에 대해 각각 0.15 ms 및 0.18 ms임. 실험에 사용한 하드웨어 및 소프트웨어는 아래 표와 같음. 실험을 위해 1) 5개의 스위치로 구성된 linear 토폴로지(그림 56a)와 2) 데이터 센터에서의 효과 분석을 위해 4-ary fat-tree topology 토폴로지(그림 56b)를 구성하여 실험을 진행했음. linear 토폴로지의 경우 3개의 테넌트를 생성하며, 각 테넌트의 TCP 연결수를 2, 4, 6, 8, 10개로 늘려가며 실험함(즉 물리 네트워크에는 총 6, 12, 18, 24, 30개의 TCP 연결이 존재함). Fat-tree 토폴로지의 경우 단일 테넌트에 대해 TCP 연결 수를 2, 4, 8, 16, 32개로 늘려가며 실험함. 통계 전송 지연시간을 측정하기 위해 ONOS 컨트롤러에서 통계 요청 및 응답 시간을 기록하도록 수정했으며, 각 가상 네트워크의 컨트롤러는 5초 간격으로 모든 플로우와 스위치의 포트에 대해 모니터링함.

표 8. 하드웨어 및 소프트웨어 상세

Server (hardware) specifications.	
CPU	Intel Xeon E5-2650 (2.30 GHz)
Memory	64 GB
NIC	Intel 82599ES 10GbE NIC
Software specifications.	
PN	Mininet: 2.3.0d6 with Open vSwitch v2.9.5 OS: Ubuntu 18.04
Network hypervisor	Libera: v0.1 OS: Ubuntu 14.04
Tenant controller	ONOS: v2.0.0 OS: Ubuntu 16.04.6



(a) linear (five switches). (b) fat-tree (4-ary).

그림 56. 실험에 사용한 네트워크 토폴로지

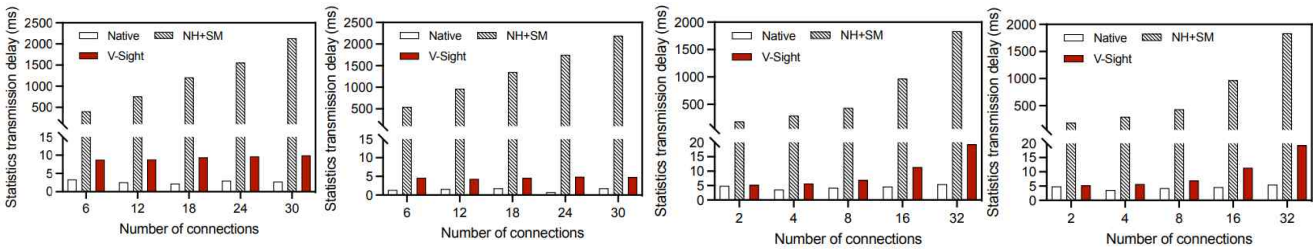
- **비교군**

1) Native: 물리적 스위치가 직접 연결되는 비가상화 SDN으로, NH 없이 ONOS 컨트롤에 연결됨.

2) NH+SM: SDN 기반 네트워크 가상화로, 기존의 NH는 네트워크 모니터링을 지원하지 않으므로, 오픈 소스 NH인 Libera[9]에 간단한 모니터링 기능을 구현했음.

3) V-Sight (본 연구): SDN 기반 네트워크 가상화로, 모니터링 비용 최소화 기법이 적용되었음.

- **실험 결과**: 그림 57는 두 개의 토폴로지에서의 통계 전송 지연의 평균을 그래프로 나타낸 것임. V-Sight를 사용할 경우, linear 토폴로지의 플로우 및 포트 통계 전송 지연 시간의 평균은 각각 9.35 ms와 4.68 ms로 나타났음(그림 57a 및 57b). 또한 V-Sight는 NH+SM에 비해 46배(플로우, 6개의 TCP 연결)에서 454배(포트 통계, 30개의 TCP 연결)의 통계 전송 지연시간을 개선함. Fat-tree 토폴로지의 경우 V-Sight의 플로우 및 포트 통계 전송 지연시간은 각각 9.75 ms, 7.29 ms로 나타났음(그림 57c 및 57d). 또한 V-Sight의 통계 전송 지연시간은 NH+SM에 비해 14배(포트 통계, 2개의 TCP 연결)에서 269배(포트 통계, 32개의 TCP 연결)까지 향상되었음. 특히 NH+SM의 통계 전송 지연시간은 TCP 연결 수에 비례하여 증가한다(최대 2초 이상). 이는 연결 수가 많을수록 vStatistics에 필요한 pStatistics의 수가 증가하며 vStatistics는 해당 pStatistics가 수집된 후에만 테넌트 컨트롤러로 반환되기 때문임. 반면에 V-Sight는 vStatistics는 요청되었을 때 pStatistics cache를 사용할 수 있다면 pStatistics 전송 과정이 불필요하기 때문에 통계 전송 지연시간을 줄일 수 있음.



(a) linear (flow entry). (b) linear (port). (c) fat-tree (flow entry). (d) fat-tree (port).

그림 57. 통계 전송 지연 시간의 평균

② 모니터링 시 가상화 병목(성능) 최소화

- **연구동기**: SDN-NV에서 가상 네트워크 통계의 요청 및 응답은 NH와 물리 네트워크 사이의 컨트롤 채널을 통해 이루어짐. 다수의 테넌트에 의해 가상 네트워크 통계 요청이 발생할 경우 컨트롤 채널에서 병목이 발생할 수 있으며, 이로 인해 다른 트래픽(e.g., 플로우 설치 메시지) 처리에 지연이 발생할 수 있음. 따라서 효율적인 트래픽 채널의 사용을 위해 통계 요청 및 응답 트래픽에 대한 최적화(e.g., 트래픽 병합)가 필수적임. 또한 가상 네트워크 모니터링 최적화를 위해 별도의 쓰레드 실행이 필요하지만, 실험 결과 네트워크 모니터링을 제공하지 않는 기존 NH보다 적은 CPU 사용량을 보임을 확인하고 그 이유를 분석함.

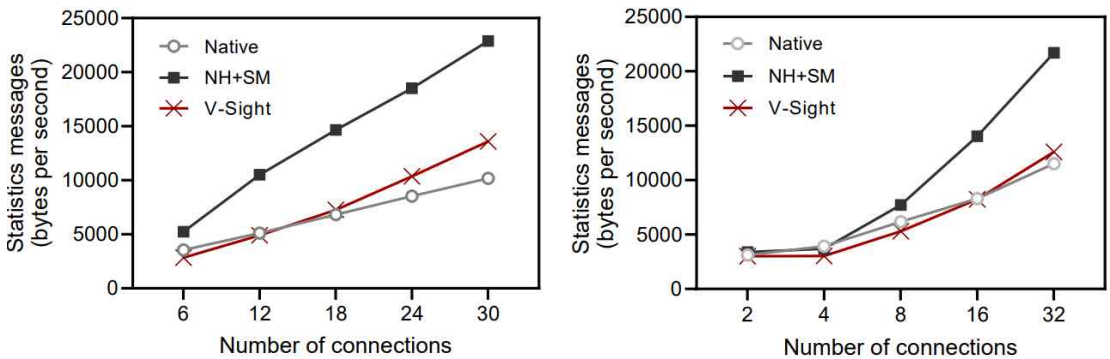
- **컨트롤 채널 트래픽 사용량 최적화 기법**: 본 연구에서 컨트롤 채널 트래픽 사용량 최적화를 위해 제안하는 기법은 1) pCollector filter와 2) pCollector tuner임. pCollector는 테넌트 컨트롤러로부터 가상 네트워크 통계를 요청받았을 때 물리 네트워크의 스위치로부터 통계를 수

9) Yang, Gyeongsik, et al. "Libera for programmable network virtualization." IEEE Communications Magazine 58.4 (2020)

집함. pCollector filter는 통계 정보 요청 트래픽을 병합할 주기(pCollector period)를 결정하고 여러 개의 pCollector를 병합하여 통계 정보 요청 트래픽의 수를 줄이며 불필요한 패킷 헤더를 제거함. pCollector tuner는 병합된 pCollector의 시작 시간을 pCollector period의 95% 만큼 의도적으로 지연시킴으로써 가상 네트워크 통계 정보가 요청된 시간과 pStatistics cache가 갱신된 시간의 차이를 줄여 통계의 정확도를 높임.

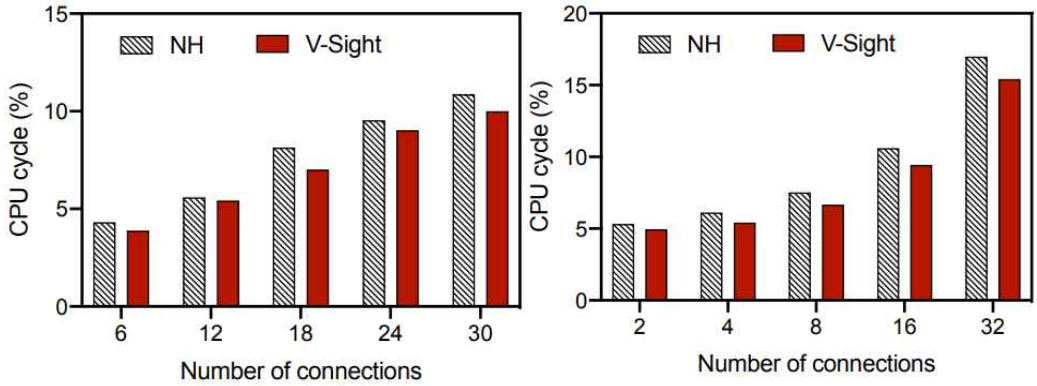
- **실험 환경 및 방법**: 실험 환경 및 방법은 네트워크 가상화 및 모니터링 비용 최소화 연구와 동일함.

- **실험 결과**: 그림 58은 두 토폴로지에서 통계 전송을 위한 컨트롤 채널 트래픽 사용량 그래프임. TCP 연결 수가 많아짐에 따라 모니터링이 필요한 플로우 수도 증가하기 때문에 TCP 연결 수에 비례하여 증가함. V-Sight는 linear 토폴로지(그림 58a)에서 컨트롤 채널 트래픽 사용량을 NH+SM에 비해 평균 1.9배 향상했음. 또한 fat-tree 토폴로지(그림 58b)에서 V-sight의 평균 컨트롤 채널 트래픽 사용량은 NH+SM에 비해 1.44배 개선되었음. CPU 사용량의 경우 V-Sight가 모니터링 최적화를 위해 pCollector를 별도의 쓰레드로 실행시킴에도 불구하고 모니터링을 지원하지 않는 NH에 비해 linear 및 fat-tree 토폴로지에서 각각 평균 0.6%, 0.9% 적은 CPU 사용량을 보였음. 이러한 결과는 V-Sight가 테넌트 컨트롤러의 불필요한 작업을 방지하기 때문에 나타난 결과임. 테넌트 컨트롤러는 네트워크 정책(e.g., 플로우)이 컨트롤러와 네트워크 간에 일관성이 있는지 확인하기 위해 플로우 및 포트 통계를 주기적으로 수집함. 네트워크 모니터링을 지원하지 않는 NH는 이러한 통계 요청에 응답하지 않기 때문에 테넌트 컨트롤러는 일관성이 없는 것으로 판단함. 따라서 NH는 플로우를 제거하거나 재설치하는 등 테넌트 컨트롤러의 재설치 메시지를 반복적으로 처리하게 됨. 반면에 V-Sight는 테넌트 컨트롤러에 적절한 통계를 제공하고 불필요한 플로우 재설치 등을 제거함으로써 모니터링을 지원하지 않는 NH에 비해 CPU 사용량을 개선하였음.



(a) linear. (b) fat-tree.

그림 58. 컨트롤 채널 트래픽 사용량



(a) linear. (b) fat-tree.

그림 59. CPU 사용량

2) 이종 멀티테넌트 컨테이너 성능 최적화

- **동기 및 기존 연구 분석**: 본 연구에서는 이종의 컨테이너 실행환경으로, 최근 센서, fog computing, edge computing 등으로 활발히 연구되는 internet-of-things를 새로운 종류로 정하고, 성능을 최적화함. IIoT(Industrial internet of things)는 통신, 제어, 그리고 동시다발적인 컴퓨팅을 할 수 있는 실행 환경을 필요로 하는데 이러한 환경을 제공할 수 있는 기술이 엣지 컴퓨팅임. IoT 장치의 특성상, 장치가 낼 수 있는 성능에는 한계가 있음. 이 한계를 극복하기 위해 엣지 컴퓨팅은 성능 향상에 도움을 줄 수 있고 배포가 쉬운 컨테이너를 중요 구성요소로 삼음. 다만, 컨테이너로 이루어진 네트워크 환경은 기존의 호스트 네트워크 환경에 추가적인 프로세스를 추가한 형태이기 때문에 호스트 네트워크 환경보다 CPU 사용량이 많아 네트워크 성능이 오히려 떨어짐. 본 연구에서는 호스트와 컨테이너 네트워크 환경에 Ftrace를 이용하여 프로파일링함으로써 네트워크 성능 저하 및 높은 오버헤드의 원인을 크게 세 가지 실험을 통해 분석함.

  첫 번째, 네트워크 성능 측정 툴인 Netperf 벤치마크를 사용하여 64B TCP 패킷을 전송할 때 호스트와 컨테이너 네트워크 환경의 각 레이어에서 발생하는 처리시간을 분석함. 그림 5는 각각 컨테이너와 호스트 네트워크 환경에서 패킷을 처리할 때 프로파일링한 결과를 나타냄. 호스트 환경에 비해 컨테이너 환경에서 단일 TCP 패킷을 처리하는 데 걸리는 시간은 57% 증가함. 이는 컨테이너 환경에는 Linux 브릿지에서의 추가적인 패킷 처리가 필요하기 때문임. 이처럼 추가적인 패킷 프로세스가 총 패킷 처리시간 및 CPU 사용량을 증가시켜 컨테이너 환경의 성능 저하를 발생시킴.

  다음으로, Linux bridge에서 컨테이너 네트워크 환경일 경우 추가로 실행되는 패킷 처리 과정을 조사함. 이전 프로파일링 결과에서 컨테이너 네트워크 스택의 각 레이어에서 실행하는 함수들을 역할에 따라 Netfilter, 헤더 처리, 라우팅, 패킷 유효성 검사 등으로 분류함. 그림6과 같이 Netfilter와 라우팅이 컨테이너의 패킷 처리 과정의 가장 큰 오버헤드를 차지함. 이는 Netfilter로 분류된 함수들인 경우 패킷 처리가 빈번히 일어나고 패킷 처리 시 높은 CPU 오버헤드를 불러오는 spin-lock 오퍼레이션을 필요로 하기 때문임. 또한, 라우팅으로 분류된 함수들인 경우에는 trie 데이터 구조로 구성되어 있어 복잡하고 비용이 많이 듦. 그리고, Netfilter와 라우팅은 동일한 대상을 향해 패킷을 전송하더라도 모든 패킷에 같은 절차를 반복적으로 수행하기 때문에 보다 더 많은 CPU 주기를 소모함.

마지막으로, 널리 쓰이는 IoT 장치인 Raspberry Pi에 Netperf 네트워크 벤치마크를 이용하여 호스트 그리고 컨테이너 각각의 네트워크 두 네트워크 환경에서 패킷 메시지 크기 별 네트워크간의 성능 차이를 분석함. 16KB 메시지의 경우 컨테이너 환경에서 CPU 사용량이 27% 높다. 다만 네트워크 처리량의 경우 호스트와 컨테이너 환경 둘 사이의 성능 차이가 심하지 않음. 이는 컨테이너와 호스트 네트워크 둘 다 100Mbps인 네트워크 링크 용량을 충분히 활용하기 때문임. 반면에 64 B 패킷 메시지를 처리하는 경우 호스트 네트워크 환경보다 컨테이너 환경이 두배 정도 높은 CPU 사용량을 보이고, 네트워크 처리량 또한 컨테이너 네트워크 환경이 11% 낮음. 즉, 컨테이너 환경은 작은 용량의 메시지를 처리할 경우 더 심한 성능 저하를 보이는데 이는 큰 용량의 메시지보다 작은 크기의 메시지일 경우 시스템을 더 빈번하게 호출하여 오버헤드를 증가시키기 때문임. 작은 크기의 메시지 처리에 성능 저하를 보이는 것은 200 B 메시지의 처리가 빈번한 IIoT 네트워크 환경에 치명적인 결함이 될 수 있음.

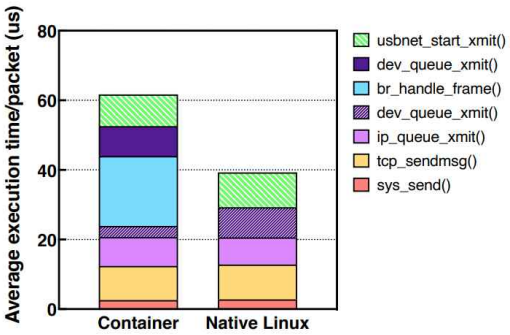


그림 60. 컨테이너 환경에서 네트워크 처리에 대한 프로파일링 결과

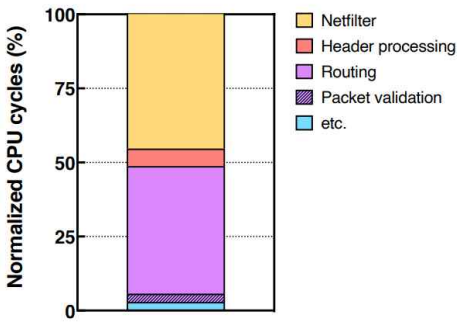


그림 61. Linux bridge에서의 프로파일링 결과

기존 다른 연구들에서도 컨테이너 네트워크 환경에서 발생하는 위와 같은 문제점들을 보완할 수 있는 기술들을 제안함. 다만, 그 기술들은 IoT 장치에 적용하기 힘들다는 점이 있고, CPU 사용률을 감소시키는 것보다 네트워크 성능 증가에만 집중한다는 한계점이 있음.

- **디자인**: 본 연구는 한정적인 자원을 가진 IOT 장치에서 패킷을 처리하는데 사용한 CPU 자원을 줄이고 동시에 네트워크 성능을 향상시키고자 Contable과 Confilter라는 새로운 메커니즘을 도입한 SCON을 제시함.

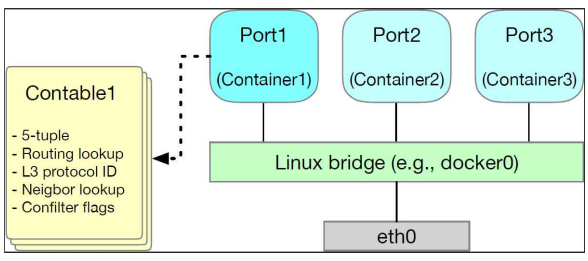


그림 62. SCON에서의 최적화된 Netfilter 프로세스. Contable.

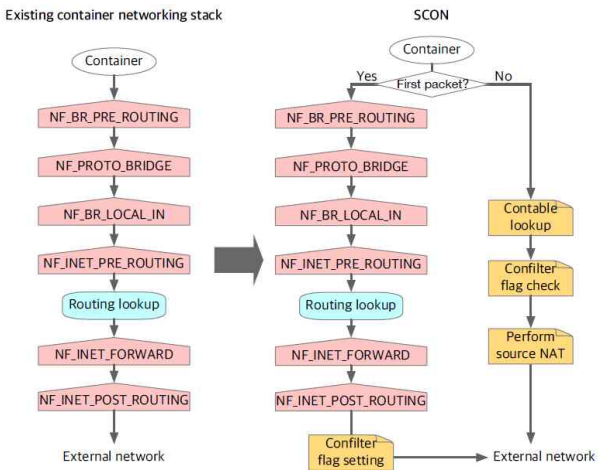


그림 63. SCON에서의 최적화된 Netfilter 프로세스. Confilter.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

Contable는 컨테이너에서 전송한 패킷의 routing 정보를 보관하여 netfilter를 통과했던 패킷이 다시 네트워킹 스택에 관련 기능(e.g. Netfilter, Routing lookup)들을 실행하지 않고 패킷을 전송할 수 있게 함. 그림 62은 Docker Engine을 기반으로 구성한 Contable 구조를 보여줌. Contable는 패킷의 라우팅 정보를 유지하기 위해 5-tuple, Routing lookup, Neighbor routing, L3 프로토콜 ID 정보 그리고 Confilter flag 정보를 튜플로 저장함. 5-tuple은 패킷의 출발지 및 목적지 IP 주소, MAC 주소 그리고 해당 Contable를 식별하는 L4 identifier를 나타냄. Routing lookup field는 패킷의 목적지 네트워크 인터페이스 그리고 해당 인터페이스의 MTU 정보를 나타냄. Neighbor routing field는 next-hop의 L2 정보를 나타냄.

Confilter는 기존 컨테이너 네트워크 환경의 Netfilter 구성요소를 재설계하고 동일한 출발지 주소와 목적지 주소를 가진 패킷에 대해서 화이트리스트 개념을 적용하여 Netfilter 프로세스의 오버헤드를 줄임. 그림 63은 기존 컨테이너 네트워크 환경과 SCON 네트워크 환경의 Netfilter 프로세스를 비교해서 보여줌. 기존 컨테이너 네트워크 환경은 패킷이 호스트 Netfilter 규칙 이외에 컨테이너 네트워크 스택에서 추가적인 Netfilter 규칙을 적용하게 됨. 이처럼 반복적인 Netfilter 처리 과정은 단일 패킷당 높은 처리 오버헤드를 초래함. 이를 해결하기 위해서 Confilter는 같은 flow의 패킷에 대해서 첫번째 패킷만 기존 패킷 처리 과정을 거치고, Netfilter를 통과한 패킷은 해당 flow의 Confilter flag를 1로 설정함. 이어서 들어오는 패킷은 Confilter flag를 통해 이미 인증된 패킷임을 확인하고 Contable에 저장된 정보를 통해 패킷전송에 필요한 정보를 얻어서 Netfilter와 Routing lookup 처리없이 패킷을 전송함. 이를 통해 Netfilter의 오버헤드를 줄이고 네트워크 품질에 향상을 기대할 수 있음.

- **실험 환경**: 본 연구에서는 Contable과 Confilter의 성능 측정을 위해 Linux 커널 4.15.0 버전에서 SCON을 실행하고 CPU 사용량과 네트워크 성능을 측정함. 그림 56과 57와 같이 SCON 환경에서의 프로파일링 결과를 측정하고 기존의 컨테이너 네트워크 환경과의 성능을 비교함. 이때 Raspberry Pi3 Model B 보드, 1.2GHz에서 작동 중인 ARmv7 64-bit quad-core CPU, 그리고 100 메가비트 이더넷 칩을 사용했으며 Raspberry Pi 보드는 클라이언트 장치에 유선으로 직접 연결하였음. 장치는 하이퍼 스레딩이 켜지지 않은 상태인 64GB 메모리의 2650v2@2.6 GHz octa-core 프로세서, 그리고 Ubuntu 16.04와 Linux kernel 5.3.0버전의 256 GB SSD를 이용한다. 추가로 Raspberry Pi board에서 컨테이너 환경을 만들 때 Docker 19.03 버전을 이용하여 실험을 진행함.

- **실험 방법 및 결과**:

  실험은 크게 세 가지로 구성함. 먼저, SCON을 이용한 네트워크 환경과 기존 컨테이너 네트워크 환경에서의 성능 차이를 비교하기 위해 하나의 컨테이너 안에 Netperf 벤치마크를 이용하여 64 B TCP 패킷을 장치에 60초 동안 전송하는 실험을 함. 이때 pidstat과 vnstat을 이용하여 CPU 사용량과 네트워크 성능을 측정함. 그림 64는 측정한 CPU 사용량과 네트워크 throughput을 보여줌. SCON을 이용하는 경우 CPU 사용량이 15% 개선되었고 네트워크 성능 또한 7% 향상되었음. 이 결과를 통해 SCON 디자인이 효과적으로 사용량을 줄이고 컨테이너의 네트워크 성능을 향상시킬 수 있다는 것을 증명함.

  더 구체적으로 평가하기 위해, Ftrace를 사용하여 컨테이너에서 실행되는 Netperf 벤치마크

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

가 64B TCP 패킷을 전송할 때 각 레이어에서 발생하는 처리시간을 수집하고 각 레이어에서 실행하는 함수들을 역할에 따라 분류하여 소모되는 CPU cycle의 양을 측정함. 그림 65은 레이어별 처리시간의 평균치를 나타낸다. 대부분의 패킷 프로파일링 처리시간은 기존 컨테이너 네트워크와 비슷하나 br_handle_frame() 함수의 경우 SCON 환경에서 소모되는 시간이 44.6% 줄어들었음. 패킷당 처리시간은 그림 66에서 볼 수 있듯이 61.8 us (Container)에서 52.9 us (SCON)으로 줄어들었음. 특히, 기존 컨테이너 환경에서 가장 많은 CPU 사용량을 차지했던 Netfilter와 Routing의 경우 SCON에서는 각각 69.6%, 63.2% 줄어들었음. 따라서 SCON의 Contable과 Confilter 디자인이 패킷 처리에서 발생하는 오버헤드를 성공적으로 줄일 수 있다는 것을 의미함.

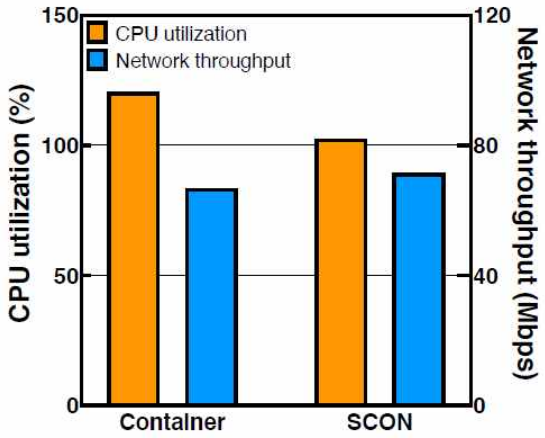


그림 64. native container 환경과 SCON에서의 성능 비교

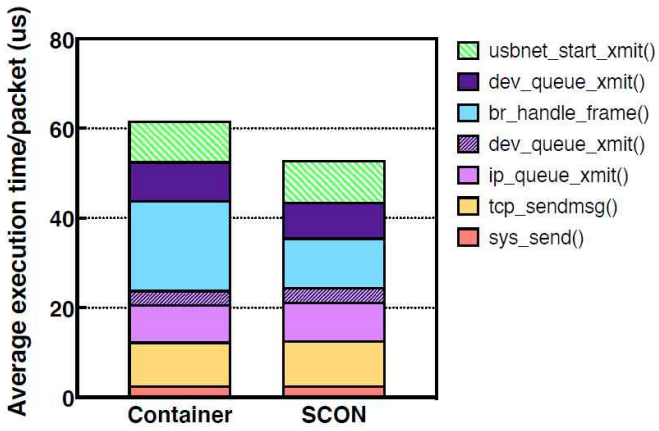


그림 65. 패킷 처리시간

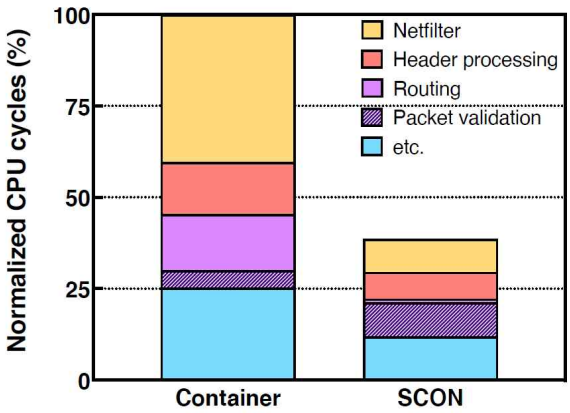


그림 66. CPU 사용량 비교분석

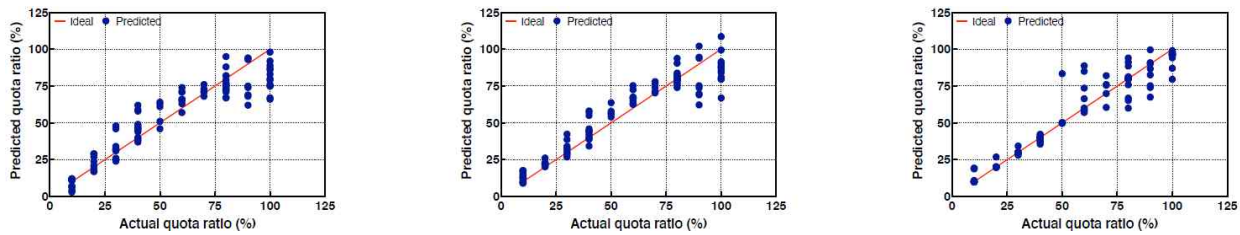
나. 통합 테스트 및 상호 운용 실험 수행 및 최적화

1) 기계학습 기반 통합 제어 기술 최적화

- **연구 동기**: 클라우드 컴퓨팅 환경에서는 네트워크 SLO와 CPU 사용률의 SLO를 충족하는 것이 클라우드 컴퓨팅의 서비스 품질을 결정하는 주요 요소가 됨. 또한, 네트워크 작업은 CPU 자원을 추가 소모하므로 네트워크 SLO를 충족하기 위해 CPU 사용률에 대한 추가적인 제어가 필요함. 이때, 네트워크 처리에 필요한 정확한 CPU 사용량은 여러 가지 요인(i.e., 워크로드, 데이터 크기, 하드웨어 구성)들로 인하여 달라짐. 이러한 CPU 사용량의 변화를 예측 및

제어하기 위해 본 연구에서는 기계학습을 통해 네트워크 SLO와 CPU 사용량을 통합 제어하는 기술을 연구 및 개발함.

- **디자인**: 이종 자원인 네트워크와 CPU를 통합 제어하기 위해 기계학습 기반 모델을 생성함. 네트워크 SLO를 달성하기 위한 CPU 사용량을 기계학습으로 예측하기 위해 워크로드별 학습 데이터셋을 생성하며, 학습을 통해 게스트와 호스트의 필요 CPU 사용량을 각각 예측함. 기계학습을 통한 통합 제어를 위해 두 개의 모듈 1) 통합 제어를 위해 새로운 워크로드의 CPU 사용량 예측을 위한 학습 데이터셋을 생성하는 Collector와 2) 생성된 학습 데이터셋으로 학습을 진행하여 CPU 사용량을 예측 및 CPU 할당을 강제하는 Manager를 생성함. 우선, 네트워크 SLO를 CPU 사용량의 관점에서 파악하기 위해 Collector에서 다양한 상황의 학습 데이터셋을 생성함. Collector는 Test VM을 생성하여 TCP 메시지 사이즈를 다르게 전송하며, 이때 게스트 CPU 및 호스트 CPU의 CPU 할당을 다르게해서 네트워크 SLO를 달성하는지 여부를 바탕으로 학습 데이터셋을 생성함. 이후 Manager는 Collector에서 생성한 학습 데이터셋으로 게스트와 호스트 모두에 대해 각각의 모델(Model-G, Model-H)을 학습함. 학습을 진행할 때, 기계학습 회귀(regression) 알고리즘 선택을 위해 Linear Regression (LR), Support Vector Regression (SVR), Random Forest Regression (RFR)에 대한 실험을 진행함. 실험 결과 그림 67는 실제 CPU 사용량(x축)과 예측 CPU 사용량(y축)을 나타내며, LR(그림 67(a))의 경우 대부분의 예측 사용량이 실제 사용량보다 커서 리소스 효율성이 저하되는 초과할당(over-allocation)을 보임. 또한, 실제 할당량 비율이 75%보다 증가하면 저할당(under-allocation)이 되어 네트워크 SLO를 달성하는데 필요한 충분한 CPU가 할당되지 못해 SLO 위반이 발생하게 됨. SVR(그림 67(b))의 경우 LR과 유사한 결과를 보인다. RFR(그림 67(c))의 경우 실제 할당량 비율이 50% 미만인 경우 LR과 SVR에 비해 초과 할당이 크게 감소함을 보임. 또한, 예측 정확도를 파악하기 위해 정확도를 파악할 때 사용하는 측정값인 평균 제곱 로그 오차(RM-SLE)와 루트 평균 제곱 오차(RMSE)를 계산하여 각 모델의 성능을 측정하였을 때, 표 A와 같이 RMSLE 0.159 및 RMSE 8087.562로 가장 좋은 예측을 제공함. 위 결과를 바탕으로 Model-G, Model-H에서 모두 RFR을 사용함. 이와 같이, 기계학습 모델을 통해 네트워크 SLO를 달성하기 위한 CPU 사용량을 예측하고, Manager에서 각 워크로드별 예측 CPU 사용량을 할당 및 전체 워크플로우를 실행함.



(a) Linear regression (b) Support vector regression (c) Random forest regression

그림 67. 기계학습 회귀 알고리즘 별 실제 CPU 사용량과 예측 CPU 사용량

표 9. 기계학습 회귀 알고리즘 별 RMSLE, RMSE

	RMSLE	RMSE
Linear regression	0.244	11,222.513
Random forest regression	0.159	8,087.562
Support vector regression	0.190	9,355.520

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2) 이종 멀티 테넌트 환경의 CPU-IO 통합 제어 기술을 기반으로, 통합 테스트 및 상호 간섭도 실험

- 실험 환경: CPU-IO 통합 제어 테스트 및 상호 간섭도 실험을 위해 본 실험에서는 두 개의 물리 서버를 사용함. 물리 서버의 구성으로는 각 서버에 Intel Xeon E5 2650 v2@ 2.6과 하이퍼스레딩이 해제된 GHz 10코어 프로세서, 256GB 메모리 및 512GB SSD를 장착함. 또한, 듀얼 포트 Intel 82599 10GbE NIC이 장착되어 10GbE 스위치에 연결되어 통신하고, 두 서버는 모두 Ubuntu 18.04(커널 5.3.18)를 실행하고, 가상화함. 다음으로, Ubuntu 16.04(커널 4.15)를 실행하고, 단일 vCPU를 VM에 할당하는 네트워크 기반 VM(network-bound VM)을 생성함. 또한, 네트워크 처리에서 CPU 효율성을 극대화하기 위해 게스트와 호스트 모두에서 TSO(TCP Segmentation Offload) 및 GRO(Generic Receive Offload)를 사용하도록 설정함. 기존 방법 중 통신 제어와의 비교 실험의 경우 게스트와 하이퍼바이저(호스트) 각각이 분리된 물리적 CPU를 통해 실행되도록 하여 Linux 평등 스케줄링(Linux fair scheduling)에서도 게스트가 충분한 CPU(100%까지)를 사용할 수 있도록 함. 다음으로 vCPU 우선순위 지정 기법과의 비교 실험의 경우 단일 CPU 코어를 계속 사용하고 게스트와 호스트가 같은 물리 CPU를 사용하게 하여 vCPU 우선순위 지정 기법이 VM에 충분한 CPU(90%까지)를 제공할 수 있게 함.

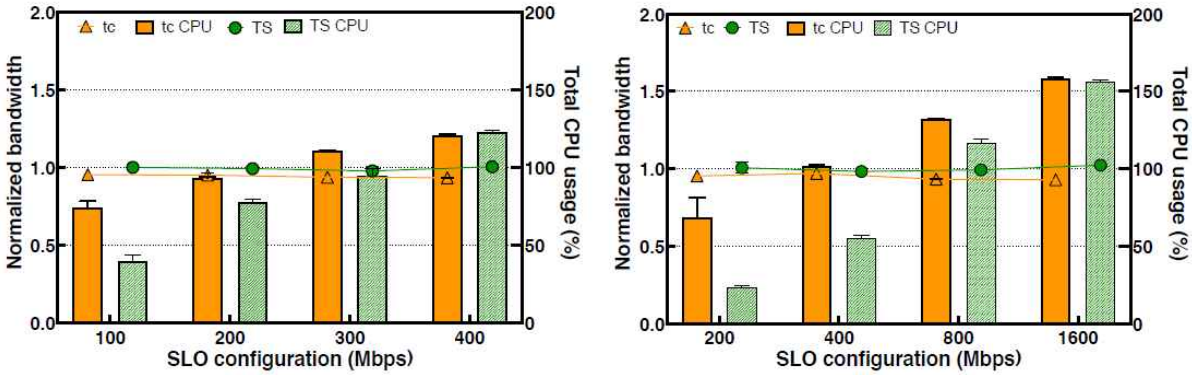
- VM 워크로드 구성: 본 연구에서 실험을 위한 VM 워크로드들을 생성하기 위해 Netperf, 웹 서버, Memcached 세 개의 애플리케이션을 사용함. Netperf 벤치마크는 MapReduce와 같은 대역폭 집약적인 애플리케이션의 셔플 단계를 모델링하기 때문에 네트워크 성능을 측정하는데 널리 사용됨. 웹 서버는 Linux의 기본 웹 서버인 Apache HTTP 서버를 사용하고 ab 벤치마킹 도구[10]를 통해 웹 서버 워크로드를 생성함. 마지막으로 Memcached는 Facebook, YouTube 및 Twitter를 포함한 많은 웹 서비스 제공업체의 기술 스택에 통합된 인기 있는 분산 캐싱 시스템으로 본 연구에서는 Memaslap 도구[11]를 통해 요청을 Memcached 서버로 전송하고, 이때 64B 키와 1KB 값을 가진 Memaslap의 기본 설정을 사용함.

- 실험 방법 및 결과: 본 실험은 크게 네 항목, 1) 본 연구와 통신 제어, vCPU 우선순위 지정 방식과의 비교, 2) CPU 기반 워크로드의 성능 측정, 3) 다른 워크로드를 사용하였을 때의 성능 비교, 4) 사용자 요청 수 증가에 따른 CPU 사용량 예측 지연 시간 비교로 이루어짐. 그림 68와 그림 69에서 TCP 메시지 크기가 64B인 경우와 1024B인 경우에 각각 통신 제어 기법(tc)과 본 연구(TS), vCPU 우선 순위 지정 기법(PR)과 본 연구(TS)를 비교함. TCP 메시지 크기가 64B의 경우 네트워크 SLO를 100Mbps에서 400Mbps로, 1024B의 경우 200Mbps에서 1600Mbps로 증가시켰으며, 통신 제어 기법은 네트워크 SLO에 따라 네트워크 대역폭을 제어함. 그림 68에서 통신 제어 기법은 64B 및 1024B 메시지 모두에 대해 0.95의 정규화된 대역폭을 달성하며 이는 충분한 CPU가 주어졌을 때 정확한 속도 제한(rate-limiting)을 수행할 수 있음을 의미함. 본 연구는 메시지 크기에 관계 없이 평균 1.00의 정규화된 대역폭을 달성하며 속도 제한 정확도를 더욱 향상시킴. 또한, 총 CPU 사용률 측면에서 통신 제어 기법에 비하여 최대 2.9배 향상된 성능을 달성하여 필요한 CPU 사용량을 더 정확하게 예측함.

10) Apache http server. https://httpd.apache.org/docs/.
11) memaslap - Load testing and benchmarking a server. http://docs.libmemcached.org/bin/memaslap.html/.

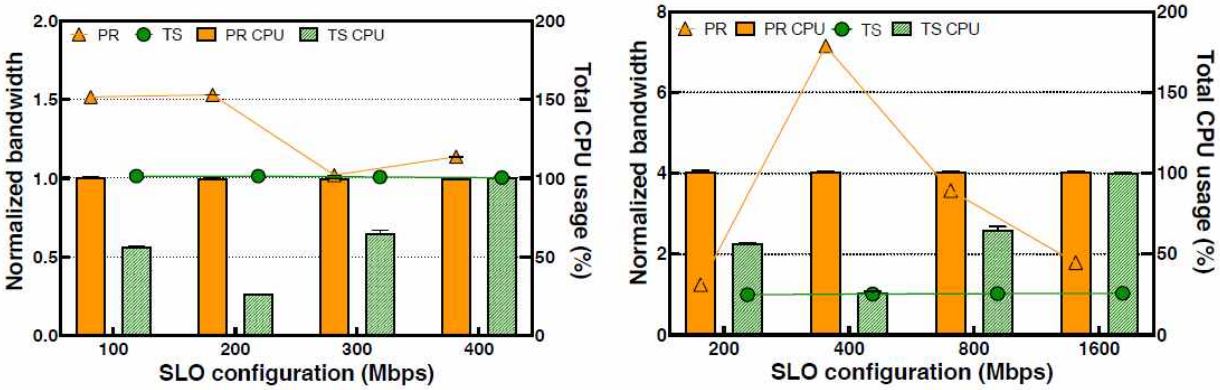
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

64B의 경우 400Mbps, 1024B의 경우 1600Mbps의 SLO에 대해 통신 제어 기법과 본 연구 모두 유사한 총 CPU 사용량을 보여주는데, 이는 VM이 이러한 네트워크 대역폭을 달성하기 위해 CPU 코어를 완전히 활용해야 하기 때문임. 다음으로 그림 C에서 vCPU 우선 순위 지정 방식은 모든 경우에 단일 물리적 코어를 완벽하게 활용하면서 정규화된 대역폭의 변동이 큼. 메시지 크기가 클수록 바이트당 패킷 처리 오버헤드가 감소하여 메시지 크기가 64B(그림 69(a))일 때 정규화된 대역폭은 약 1.53까지 증가하고, 메시지 크기가 1024B(그림 69(b))일 때 약 7.15까지 증가함. 반면 본 연구는 CPU 예측을 기반으로 게스트와 호스트에 적절한 CPU 사용량을 할당하므로 메시지 크기에 관계 없이 다양한 네트워크 SLO를 충족할 수 있으며 평균 1.01의 정규화된 대역폭을 보임. 또한, 본 연구와 달리 vCPU 우선 순위 지정 기법은 Linux CFS를 사용하여 작업 보존 스케줄링을 수행하기 때문에 모든 경우에 100% CPU 사용률을 할당하여 초과 할당하여 리소스 효율성이 저하됨.



(a) 메시지 크기: 64B (b) 메시지 크기: 1024B

그림 68. 본 연구(TS)와 통신 제어 기법(tc)과의 비교

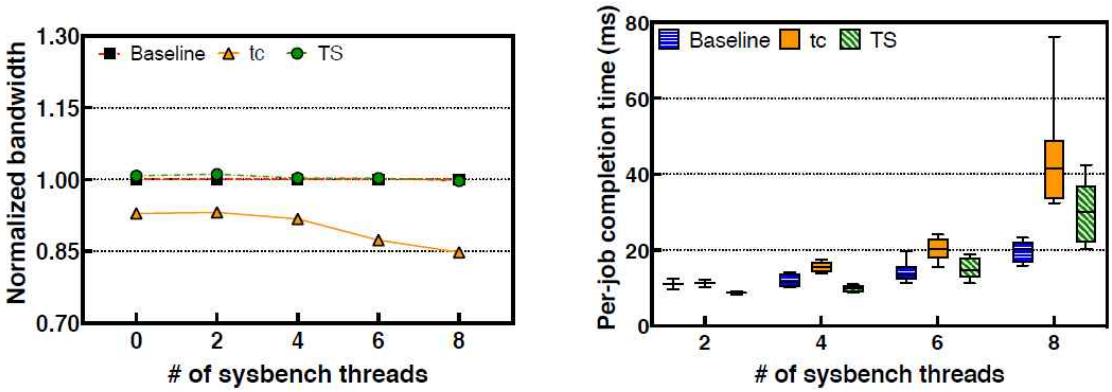


(a) 메시지 크기: 64B (b) 메시지 크기: 1024B

그림 69. 본 연구(TS)와 CPU 우선순위 지정 기법(PR)과의 비교

그림 70에서는 네트워크 기반 VM(network-bound VM)들이 네트워크 기반이 아닌 VM에 어떤 영향을 받는지 확인하기 위해 소수를 계산하는 sysbench CPU 테스트를 실행하는 CPU 기반 VM(CPU-bound VM)을 추가함. 네트워크 기반 VM(network-bound VM)의 경우 네트워크 SLO를 200Mbps로 설정하고 메시지 크기가 64B인 Netperf를 실행함. 또한, 본 연구와 통신 제어 기법을 두 가지 지표 1) 네트워크 기반 VM(network-bound VM)의 네트워크 SLO를 통하여 정규화된 네트워크 대역폭과 2) 최대 8개까지 sysbench 스레드 수가 증가하는 CPU 기반 VM의 작업 완료 시간을 비교함. 그림 70(a)에서 baseline은 정규화된 네트워크 대역폭인 항상 1인 이상적인 상황을 가정함. 그림 70(a)에서 본 연구는 sysbench 스레드 수에 관계 없이 1에 가까운 정규화된 대역폭을 달성하는 반면, 통신 제어 기법은 sysbench 스레드 수가 6개 이상 증가하면 성능

이 저하되어 정규화된 대역폭이 0.85로 감소함. 이는 통신 제어 기법은 CPU 할당을 수행하지 않고 VM의 전송 제어만 처리하지만 본 연구는 네트워크 기반 VM(network-bound VM)에 적합한 CPU 사용량을 결정하고 VM 간의 CPU 사용 경합 문제를 해결하기 때문임. 그림 70(b)는 sysbench 스레드 간의 최대 작업당 완료 시간(PJCT) 분포이며, 여기서 baseline은 CPU 기반 VM이 다른 VM 없이 물리적 시스템에서만 실행될 때 얻어지는 고립 성능(isolated performance)을 의미함. 그림 70(b)에서 sysbench 스레드 수가 증가함에 따라 PJCT의 평균 및 편차가 증가하며, 이는 vCPU가 8개인 CPU 기반 VM과 하이퍼바이저를 포함한 네트워크 기반 VM(network-bound VM)이 10개의 물리적 CPU 코어를 두고 경쟁하기 때문임. 특히 통신 제어 기법에서 네트워크 기반 VM(network-bound VM)이 CPU 코어를 소모하면서 빈번하게 인터럽트 되어 CPU 기반 VM의 성능이 낮아지지만, 본 연구의 경우 CPU의 약 80%를 할당하여 통신 제어 기법 대비 CPU 사용량을 평균 17% 줄이기 때문에 CPU 기반 VM의 성능 저하를 완화하며, 이는 본 연구의 접근 방식이 CPU 사용 경합 시 네트워크 대역폭 요구사항을 충족하면서도 CPU 경합을 완화하는데 효과적임을 보여줌.



(a) 네트워크 기반 VM (b) CPU 기반(CPU-bound) VM

그림 70. sysbench CPU 테스트를 실행하는 CPU 기반 VM(CPU-bound VM)을 추가하여 정규화 대역폭과 작업당 완료시간의 측면에서 통신 제어 기법(tc)과 본 연구(TS) 비교

그림 71은 본 연구가 다른 워크로드에 적용 가능하다는 것을 보여줌. 그림 71은 세 가지 애플리케이션 1) Netperf, 2) 웹 서버, 3) Memcached와 통신 제어 기법에 Netperf를 사용한 결과를 보여줌. Netperf의 경우 본 연구와 통신 제어 기법을 비교하면, 통신 제어 기법은 실험 결과의 80%의 정규화 대역폭이 0.93이지만, 본 연구는 실험 결과 중 2.5%만이 0.92보다 낮은 정규화된 대역폭을 달성함. 따라서 본 연구가 SLO에 훨씬 더 가까운 네트워크 대역폭을 제공하는 것을 보임. 다른 워크로드의 결과로 웹 서버 애플리케이션의 경우 0.95~1.07 사이의 정규화 대역폭을 보이며, Memcached는 0.97~1.04 사이의 정규화 대역폭을 보임.

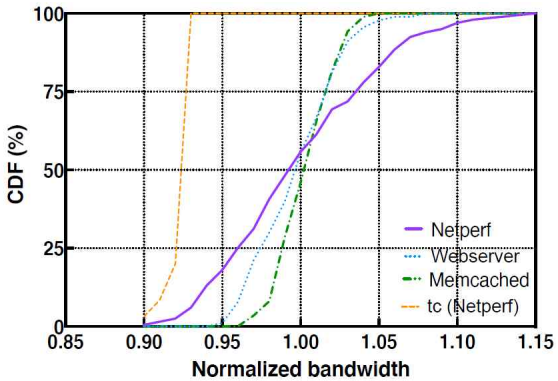


그림 71. 다른 워크로드에 적용 결과 비교

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

본 연구의 디자인은 1) 네트워크 SLO가 포함된 사용자의 요청을 받으면, 2) 네트워크 SLO를 만족하기 위한 CPU 사용량을 예측하고, 3) 게스트와 호스트에 CPU 할당으로 이루어지므로, 만약 사용자 요청이 많을 경우 CPU 사용량 예측에 지연 시간이 생길 수 있음. 따라서 그림 72은 본 연구의 CPU 사용량 예측에 걸리는 시간에 대한 분석을 진행함. 그림 72에서 사용자 요청 수(N)를 100에서 10000으로 늘리면서 예측 지연 시간을 측정함. 사용자 요청 수가 100에서 10000으로 증가함에 따라 평균 대기 시간은 135ms에서 387ms로 증가하였으며 이는 2.9배 증가에 불과함. 또한, 사용자 요청의 수가 1000 미만의 경우 요청의 75%가 300ms 이내에 처리되며, 사용자 요청의 수가 10000일 경우에도 0.8초 이내에 CPU 사용량 예측이 완료됨을 확인하였으며, 이는 1초 안에 많은 동시 요청을 처리할 수 있다는 것을 의미함.

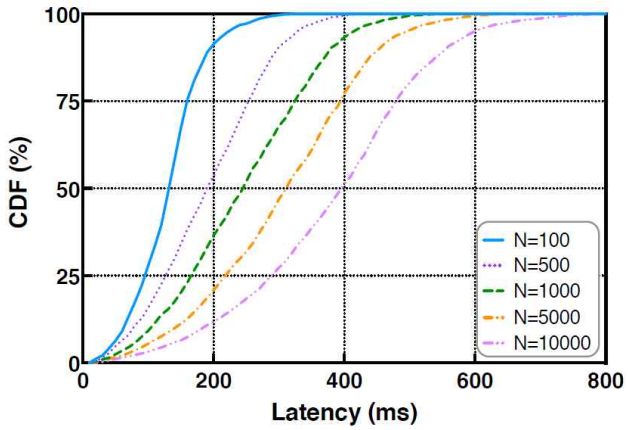


그림 72. CPU 사용량 예측 지연 시간 (N은 사용자 요청 수)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3) 공인인증을 통해 2단계 연구결과의 객관적 평가 추진:

공인인증 평가를 통해 연구과제를 통해 연구된 연구결과물의 성능이 목표치를 달성하였음을 확인함.



kolas.kbet.or.kr 발행기관 사전승인 없이 전부 혹은 일부 복사 불가

G48(www.g4b.go.kr)진위확인코드 : HLKGwAxiQb4=

한국건물에너지기술원 Korea Institute of Building Energy Technology

시험확인 진본 2022-12-01 17:47:38 KST 한국시험확인센터

시 험 성 적 서

한국건물에너지기술원 서울 서초구 반포대로30길 81 유진타워 8F (Tel.02-525-1025, Fax.02-555-3152)	성적서번호 : 2022-E-0015 페이지(1)/총(9)	KBeT

1. 신청기관
   - 회 사 명 : 고려대학교 산학협력단
   - 대 표 자 : 조석주
   - 주 소 : 서울특별시 성북구 안암로 145(안암동5가, 고려대학교내.)
2. 시험성적서의 용도 : 정보통신기획평가원 제출용
3. 시험대상 제품명 및 버전 : CPU-네트워크 SLA 보장기법 v1.0
4. 접수일자 : 2022.11.09
5. 시험일자 : 2022.11.22 - 2022.12.01
6. 시험환경
   시험장소 : 현장시험실(주소: 서울특별시 성북구 안암로 145 자연계지역 우정정보관 308호)
   온도 : (23.0 ± 1.0) ℃ 습도 : (43 ± 4) % R.H.
7. 시험방법 : 신청기관 제시 기준 및 방법
8. 시험결과 : 시험결과 참고

확 인	작성자 성 명 : 권오철 (서명)	기술책임자 성 명 : 김재훈 (서명)

- 이 성적서는 신청기관이 제공한 시료 및 시료 명으로만 시험한 결과로서, 제공된 시료 외 전체 제품에 대한 품질을 보증하지는 않습니다.
- 이 성적서는 당 기관의 사전 서면 동의 없이 홍보, 선전, 광고 및 소송용으로 사용될 수 없으며, 용도 이외의 사용을 금합니다.
- 이 성적서는 신청기관이 제시한 시험방법에 의한 시험결과입니다.
- 이 성적서는 KS Q ISO/IEC 17025와 KOLAS 인정과 관련이 없습니다.

2022년 12월 1일

한국건물에너지기술원장 (직인)

KBeT-QIF-04-05(00) 한국건물에너지기술원

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 연구개발과제의 수행 결과 및 목표 달성 정도

1) 연구수행 결과

(1) 정성적 연구개발성과

구분	연구 목표	연구개발 수행 내용	연구 결과
1연차 /1단계	- 차세대 고성능 I/O 장치를 위한 SLA 보장 기법	- 기존 플래시 스토리지를 위한 스케줄링 기법의 문제점 및 관련 연구 동향 분석 - 기존 클라우드 환경에서의 네트워크 QoS 관리 기법 및 관련 연구 동향 분석 - 블록 I/O SLA 보장을 위한 새로운 스케줄러 구조 설계 및 개발 - 네트워크 자원 사용량에 따른 동적 네트워크 자원 할당 기법 개발	- 8년 연구에 대한 요구사항 분석 및 초기 설계 수행 - 국내 특허 2건 출원 - 국내 9건 논문 출판
2연차 /1단계	- SLA 보장 CPU 스케줄러 개발	- 네트워크 I/O 성능 보장을 위한 스케줄러 구조와 메인 아이디어 도출 - 피드백 제어 기반의 스케줄링 모듈 개발 - First-order 모델로 인한 피드백 제어의 한계 극복을 위해 동적 피드백 제어 모델 설계 - 피드백 제어 모델 개선을 통한 성능 변동성 최소화	- SLA 보장 네트워크 스케줄러 소스코드 공개 - JCR 상위 10% 저널 (IEEE Transactions on Parallel and Distributed Systems) 논문 출판 1건 외 SCI/E 저널 논문 출판 3건 - 국외 학술대회 논문 발표 4건 - 국내 12건의 논문 출판 및 국내 학회 우수발표 논문상 수상 - 국내 특허 출원 5건, 국내 특허 등록 1건, 국제 특허 출원 1건
3연차 /1단계	- CPU-I/O 통합 스케줄링 구조 설계	- 오픈스택을 포함한 기존의 단일 및 복합자원 처리 연구 현황 조사 및 분석 - 실험을 통해 단일 및 복합 작업의 자원 사용량에 따른 성능 측정 - 작업특성 분석 결과를 기반으로 CPU-블록 I/O 성능 보장이 가능한 스케줄링 알고리즘 설계 및 개발	- CPU-IO 통합 스케줄링 프레임워크 (GoS) 소스코드 공개 - 국외 학술대회 논문 발표 4건 - 국내 9건의 논문 출판 및 국내 학회 우수발표 논문상 수상 - 국내 특허 출원 2건, 국내 특허 등록 4건, 국제 특허 출원 2건
4연차 /1단계	- CPU, 네트워크, 스토리지 통합 스케줄러 개발/오픈스택 적용	- CPU-I/O 통합 스케줄러 구조 확장 및 세부 동작 방식 정의 - 통합 자원 스케줄링 구현 - 오픈스택 컴포넌트와 통합 자원 스케줄러의 연동	- 오픈스택 기반 통합 스케줄러 구현물 오픈소스 공개 - JCR 상위 15% 저널 (Sensors) 논문 출판 1건 외 SCI/E 저널 논문 출판 5건 - 국외 학술대회 논문 발표 3건 - 국내 9건의 논문 출판

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

			- 국내 특허 출원 2건, 국내 특허 등록 2건, 국제 특허 출원 2건, 국제 특허 등록 1건
5연차 /1단계	- 클라우드 레벨에서 성능 SLA 보장을 위한 자원 관리 기법	- 관련 연구 조사 및 분석 - SLO를 고려한 클라우드 레벨 스케줄링 알고리즘 설계 - 기존 오픈스택 환경에서의 작업 분배 방식 분석 - 오픈스택 환경에 SLO 기반 작업 분배 알고리즘 구현	- 클라우드 레벨 자원 관리 기법 오픈소스 공개 - JCR 상위 10% 저널 논문 출판 및 revision 진행 (IEEE Systems Journal, IEEE Communications Magazine) - 국외 학술대회 논문 발표 4건 - 국내 10건의 논문 발표 및 학술대회 우수발표 논문상 수상, 학부생 경진대회 최우수 논문상 수상 - 국내 특허 출원/등록 각 2건 달성 및 국제 특허 출원/등록 각 1건 달성
	- 보안 SLA를 위한 BaaS 분석 및 보안 파라미터 추출	- 공개SW BaaS 플랫폼 환경 구축 및 BaaS 예제 동작 방식 분석 - 공개SW BaaS 예제 성능 평가 및 자원 사용량 분석 - 공개SW BaaS 파라미터 추출을 위한 변수 설정 및 성능 평가 - 성능 측정 결과를 기반으로 한 보안 SLA 보장 BaaS 감사 도구 설계	
6연차 /1단계	- Intra BaaS SLO 성능 보장 연구	- 실시간 성능 모니터링을 위한 기존 성능 계측 도구 분석 - 오더링 서비스 노드에 따른 자원 사용량 및 성능 차이 분석	- 클라우드 기반 블록체인 성능보장 기법 오픈소스 소프트웨어 공개 - JCR 상위 10% 저널 논문 revision 진행 (IEEE Communications Magazine) 외 SCI/E 저널 논문 출판 1건 - 한국 정보과학회 최우수 학술대회(IEEE INFOCOM) 제출 1건 외 국외 학술대회 논문 발표 3건 - 국내 학술대회 논문 10건 - 국내 특허 출원 3건, 국내 특허 등록 2건 달성 - 국제 특허 출원/등록 각 1건 달성 - 공개 SW 등록 4건 및 공개 SW 커밋 수 5063회 달성 (목표치: 20)
	- 보안 SLA를 위한 BaaS characterization	- BaaS 보안 SLA 파라미터 분석을 위한 보안 SLA 감사 도구 개발 및 테스트	
7연차 /1단계	- 이종 멀티 테넌트 기반 성능 SLA 보장 연구	- 클라우드 레벨에서의 성능 SLA 보장 연구 동향 조사 - BaaS를 포함한 이종 워크로드 성능 고립을 위한 기법 개발	- 네트워크 가상화 솔루션 글로벌 수준 오픈소스 소프트웨어 산출 - JCR 상위 10% 저널 논문 (IEEE Transactions on Cloud Computing) 출판 1건 외 SCI/E 저널 논문 출판 1건 - 국외 학술대회 논문 발표 4건 - 국내 13건의 논문 출판
	- 이종 멀티 테넌트 네트워크의 가상화를	- 기존 네트워크 가상화 연구 동향 조사 및 가상 네트워크 연구를 위한 요구사항 분석 - 클라우드 레벨 가상 네트워크	

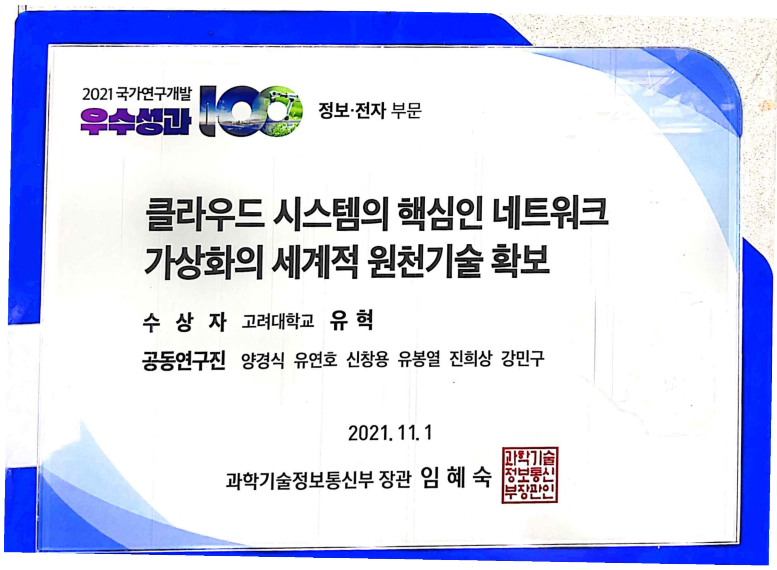
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

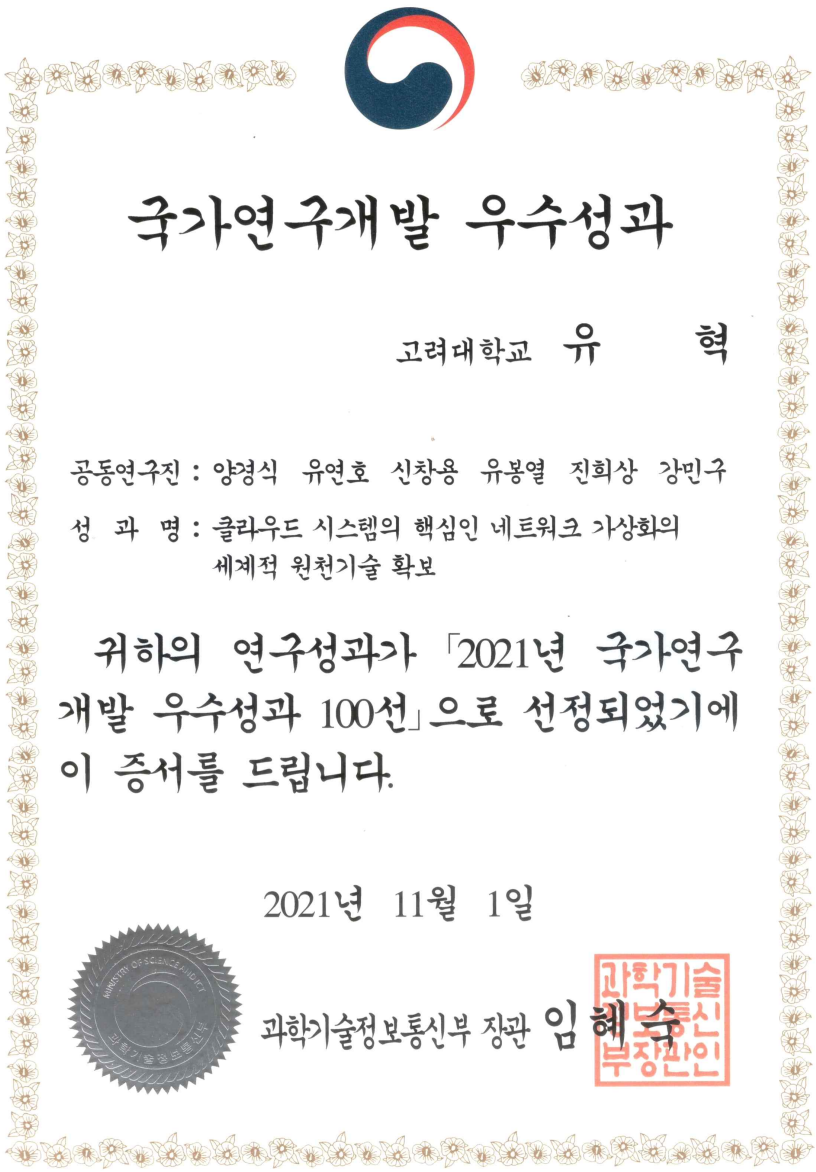
	위한 최적화 연구	최적화를 위한 기법 설계	- 국내 특허 출원 1건, 국내 특허 등록 4건
8연차 /1단계	- BaaS 성능/보안 SLA 보장 효율성 향상을 위한 최적화 연구	- 통합 테스트 및 상호 운용 실험 수행 및 최적화	- 인공지능 기반 클라우드 최적화 및 네트워크 가상화 오픈소스 소프트웨어 공개 - JCR 상위 10% 저널 (IEEE Transactions on Cloud Computing) 논문 출판 2건 외 SCI/E 저널 논문 출판 1건 - 한국 연구재단 우수 국제학술대회 기준 우수 국외 학술대회 논문 발표 2건 - 국내 8건의 논문 출판 - 국내 특허 출원 4건, 국제 특허 출원 3건(2023년 상반기 중 출원), 국제 특허 등록 2건

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(2) 정량적 연구개발성과

○ **[요약]** 본 연구과제는 2021년 과학기술정보통신부 선정 국가연구개발 우수성과 100선, 그 중에서도 정보·전자 분야 중 **IT 분야 과제로 유일하게 "최우수" 과제로 선정**됨





국가연구개발 우수성과

고려대학교 유 혁

공동연구진 : 양경식 유연호 신창용 유봉열 진희상 강민구

성 과 명 : 클라우드 시스템의 핵심인 네트워크 가상화의 세계적 원천기술 확보

귀하의 연구성과가 「2021년 국가연구개발 우수성과 100선」으로 선정되었기에 이 증서를 드립니다.

2021년 11월 1일

과학기술정보통신부 장관 임 혜 숙

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ **SCIE 논문 및 SCIE인정 국제최우수학술대회**

1) SCIE논문 19건[1] 출판

- **JCR 상위 10% 논문 총 8건 (42% 차지)**
- 상위 1%, 4% 최상급 논문 포함

2) SCIE인정 국제최우수학술대회 **9편**: (한국정보과학회 기준) 연구목표 300% 달성

- 한국연구재단 BK사업 리스트 기준 9편 (IF 4급 2편, 3급 1편, 2급 1편, 1급 5편) 발표
- 한국정보과학회 리스트 기준 4편 (최우수학술대회 3편, 우수 학술대회 1편) 발표
- 2년연속 Google Scholar 기준 분야 내 H5 index 1위 학술대회 발표 달성
- Acceptance rate 10.7% 등 최고수준 학술대회 논문 발표 포함

○ **국제학술대회 논문:** 연구목표 110% 달성

- CoNEXT 등 최우수학술대회 부대워크샵 등 최상위 국제학술대회 논문 21편 발표
- ACM 및 IEEE 등 저명 단체로부터 Best paper award 등 2회 수상

○ **기타저널 및 국내학술대회 논문:** 연구목표 102% 달성

- 한국정보과학회, 한국통신학회, 한국정보보호학회 등 다양한 분야의 학술지 및 학술대회 참여
- 학생연구원 논문발표를 통해 **우수논문상 11회 이상 수상**

○ **기술이전/산학협력 우수성**

- 본래 과제에서 목표되지 않았으나, LG전자 및 미국에 $80,000 수준 기술이전 달성
- 본 연구과제를 통해 도출된 원천기술을 기반으로, 다양한 응용연구를 도출하고 기술의 파급을 도모하여 8개년 간 총 686,289,000원 수준의 산학연구 연계 수행
- 본 연구과제의 기술 우수성을 인정받아, Microsoft Research로부터 지속적인 기술 협력을 요청받고 격주 화요일 11시마다 공동연구 회의 및 기술교류를 진행중

○ **공개 SW 우수성**

- 스타수, 커미트수, Fork 수 모두 목표치 대비 초과 달성
  : 연구목표 대비 Commit 459%, Fork 695%, Star 380%, Pull 74건(목표치 0건) 달성
- 연구한 기술 중 네트워크 성능 보장/가상화 기술은 그 우수성을 입증받아 미국 유관 분야 오픈소스 소프트웨어 및 표준을 주도하는 Open Networking Foundation과 협의 하에, OpenVirteX라는 국제적인 소프트웨어의 소유권과 커뮤니티 운영권한을 이양받아 국제적인 원천기술을 보유 및 운영중임
- 상기 소프트웨어 및 본 연구진의 오픈소스SW 커뮤니티는 월 2,000회 이상 조회되는 수준으로 성장하였음

○ **특허 우수성**

- 국제특허 출원 9건[2], 등록 4건 달성
- 국내특허 출원 19건, 등록 16건 달성
- 국제특허 등록 3건이 현재 목표 대비 미달이나, 심의가 진행중인 기 출원건들을 고려할 때 2023년 및 2024년까지 완전 달성이 가능할 것으로 판단됨

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ **인력양성 우수성**

- 정년트랙 전임교수 3명 배출(성균관대, 경북대, 전북대 등)
- 최우수 연구인력 양성(대학원 과정 중 SCIE급 논문 주저 10편 이상 저술)
- Microsoft Research Asia 인턴 2명 선발 및 한국인 최초 Student TechFest 수상
- 우수한 연구실적을 통해 박사과정 1명 BK 4단계 BK21 미래인재 펠로우십 선정
- 석사과정 전원 최선호 직장 취업 달성
- 석사과정 중 ETRI/국방과학연구소/KIST 등 정부출연연구소 연구원 6명 배출
- 석사과정 연구원 ACM/IEEE 학술대회 Top paper award 2회 및 국내학술대회 우수논문상 11회 수상

1) 2023년 2월 accepted SCIE 학술지 논문 1건 포함

2) 2023년 상반기 중 출원 완료 확정 3건 포함 (2022년 12월에 심의 및 출원이 승인되었으나, 고려대학교 산학협력단 일정으로 인해 명세서 작성 등 절차가 지연중)

< 정량적 연구개발성과표 >

(단위 : 건, 천원)

성과지표명 \ 연도				1단계 (2015~2018)		2단계 (2019~2020)		3단계 (2021-2022)		단계 총 합계		가중치 (%)
				목표	실적	목표	실적	목표	실적	목표	실적	
전담기관 등록·기탁 지표[1]	특허 실적	출원	국내	8	9	4	5	5	5	17	19	
			국제	4	4	2	2	3	3	9	9	
			합계	12	13	6	7	8	8	26	28	
		등록	국내	8	8	4	4	4	4	16	16	
			국제	3	0	1	2	3	2	7	4	
			합계	11	8	5	6	7	6	23	20	
		질적 성과	SMART 값(평균)	-	-	-	-	-	-	-	0	
	논문 실적	논문	SCIE	9	10	6	3	6	6	21	19	
			비SCIE	-	-	-	-	-	-		0	
			소계(a)	9	10	6	3	6	6	21	19	
		학술대회 발표[3]	최우수 학술대회	-	-	-	1	-	2	-	3	
			우수 학술대회	-	-	1	-	-	1	1	1	
			소계(b)	-	-	1	1	-	2	1	3	
		기타 저널, 컨퍼런스 발표논문	국내	35	40	19	15	20	21	74	76	
			국제	9	11	5	7	5	3	19	21	
			소계(c)	44	51	24	22	25	24	93	97	
		질적 성과	JCR 상위 10% 논문수	2	3	3	2	1	3	6	8	
		합계(a+b+c)		53	61	31	26	31	32	115	119	
	표준화	국내	기고서제출	-	-	-	-	-	-	-	-	
			기고서채택	-	-	-	-	-	-	-	-	
			표준안채택	-	-	-	-	-	-	-	-	
		국제	기고서제출	-	-	-	-	-	-	-	-	
			기고서채택	-	-	-	-	-	-	-	-	
			표준안채택	-	-	-	-	-	-	-	-	
	SW 등록			4	4	2	4	2	4	8	12	
	기술요약정보(기술문서)			15	15	8	8	8	8	31	31	
연구개발과제 특성 반영 지표[2]	시제품 제작			-	-	-	-	-	-	-	-	
	성과홍보			-	-	-	7	-	14	-	21	
	기술이전			-	-	-	-	-	-	-	-	
	기술료			-	-	-	-	-	2건 (83,100)	-	2건 (83,100)	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	산업체 연구비 수주		-	428,780	-	77,000	-	180,509	-	686,289	
	영상제작		-	-	-	-	-	-	-	-	
	SW스타랩 배출인력[4]	학사학위	6	3	3	2	3	3	12	8	
		석사학위	8	13	3	4	3	5	14	22	
		박사학위	1	3	1	2	2	0	4	5	

* 1」전담기관 등록·기탁 지표: 논문[에스시아이 Expanded(SCIE), 비SCIE, 평균Impact Factor(IF)], 특허, 보고서원문, 연구시설·장비, 기술요약정보, 저작권(소프트웨어, 서적 등), 생명자원(생명정보, 생물자원), 표준화(국내, 국제), 화합물, 신품종 등을 말하며, 논문, 학술발표, 특허의 경우 목표 대비 실적은 기재하지 않아도 됩니다.
* 2」연구개발과제 특성 반영 지표: 기술실시(이전), 기술료, 사업화(투자실적, 제품화, 매출액, 수출액, 고용창출, 고용효과, 투자유치), 비용 절감, 기술(제품)인증, 시제품 제작 및 인증, 신기술지정, 무역수지개선, 경제적 파급효과, 산업지원(기술지도), 교육지도, 인력양성(전문 연구인력, 산업연구인력, 졸업자수, 취업, 연수프로그램 등), 법령 반영, 정책활용, 설계 기준 반영, 타 연구개발사업에의 활용, 기술무역, 홍보(전시), 국제화 협력, 포상 및 수상, 기타 연구개발 활용 중 선택하여 기재합니다
(연구개발과제 특성별로 고유한 성과지표를 추가할 수 있습니다).
* 3」최우수학술대회, 우수학술대회는 한국정보과학회가 2020년 발간한 <소프트웨어분야 2020년 우수학술대회 개편목록>에 따름
* 4」배출인력은 해당학위 졸업자를 의미하며, 2022년 배출인력은 2022년 참여연구원으로 6개월 이상 과제를 수행(2022년 신규과제는 3개월 이상)하고 2022.8월 졸업 또는 2023.2월 졸업예정인 자를 말함. 예를 들어 2015년 시작 과제의 경우 1단계(2015~2018) 배출인력은 2015.8월 졸업자부터 2019.2월 졸업자까지를 말함

< 연구개발성과 성능지표 >

평가 항목 (주요성능[1])			단위	전체 항목에서 차지하는 비중[2] (%)	세계 최고		연구개발 전 국내 성능 수준	연구개발 목표치			목표설정 근거	달성도	평가 방법[3]
					보유국/보유기관	성능수준		1단계 (2015-2018)	2단계 (2019-2020)	3단계 (2021-2022)			
1	보안 SLA	보안 SLA 제공으로 인한 CLS저하율	%	20	-	-	-	50	20	1	세계최고 및 전단계 대비 개선된 성능목표	달성	자체평가 및 일부 공인인증
2		보안 SLA 모니터링 수준	%	10	미국	35	-	-	10	100		달성	
3	복합성능 SLA 보장 (성능 SLA)	CPU-스토리지 CLS	%	20		96	75	95	97	99.9		달성	
4		CPU-네트워크 CLS	%	20		96	38	95	97	99.9		달성	
5		CPU-스토리지, 네트워크 복합 CLS	%	30	-	-	56	90	92	95.9		달성	

* 1」 정밀도, 인장강도, 내충격성, 작동전압, 응답시간 등 기술적 성능판단기준이 되는 것을 의미합니다.
* 2」 비중은 각 구성성능 사양의 최종목표에 대한 상대적 중요도를 말하며 합계는 100%이어야 합니다.
* 3」 평가방법은 “자체평가”, “공인시험성적(확인서)”, “수요기업 평가” 등으로 구분하여 기재

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(3) 세부 정량적 연구개발성과(해당되는 항목만 선택하여 작성하되, 증빙자료를 별도 첨부해야 합니다)

[과학적 성과]

□ 논문(국내외 전문 학술지) 게재

① 논문 총괄현황

국내		국외		SCIE 비율 (전체 SCIE 논문 수/ 전체 논문수)	SCI 피인용도				SCI Impact Factor(IF)	
SCIE	비SCIE	SCIE	비SCIE		1회 이상 피인용된 논문수	전체 논문수 대비 피인용 비율(%)	피인용된 논문의 피인용 건수의 합(건)	SCIE 1편당 평균 피인용 건수(건)	IF 합계	SCI 1편당 평균IF
-	9	19	0	67.8%	18	64.3	182	9.6	69.659	3.7

* SCI급은 SCI 및 SCIE 포함, LNCS는 SCI에서 제외
* SCIE급 논문 중 SW스타랩 ACK가 있는 논문 대상으로 함
* Impact Factor는 Thompson 발표자료 적용

② 논문 세부내역

번호	논문명	학술지명	주저자명	호	국명	발행기관	SCIE 여부 (SCIE/비SCIE)	게재일	기여율
1	Storage SLA Guarantee with Novel SSD I/O Scheduler in Virtualized Data Centers	IEEE Transactions on Parallel and Distributed Systems	Hyunchan Park	27		IEEE	SCIE	2016. 08.01	
2	VADI: GPU Virtualization for an Automotive Platform	IEEE Transactions on Industrial Informatics	Chiyoung Lee	12		IEEE	SCIE	2016. 02.01	
3	ANCS: Achieving QoS through Dynamic Allocation of Network Resources in Virtualized Clouds	Scientific Programming	Cheol-Ho Hong	0		hindawi	SCIE	2016. 02.01	
4	Eliminating bandwidth estimation from adaptive video streaming in wireless networks	Signal Processing: Image Communication	Jaehyun Hwang	47		sciencedirect	SCIE	2016. 06.28	
5	The Impact of Container Virtualization on Network Performance of IoT Devices	Mobile Information Systems	Kyungwoon Lee			hindawi	SCIE	2018. 04.01	
6	O(1) 크래시 복구 수행시간을 갖는 FTL의 설계와 구현	정보과학회논문지: 컴퓨팅의 실제	박준영	21	KR	한국정보과학회	비SCIE	2015. 10.01	
7	PathSavanna: Xen 기반 가상 라우터에서의 GPGPU를 이용한 실제적인 패킷 라우팅	정보과학회논문지: 시스템 및 이론	박근영	43	KR	한국정보과학회	비SCIE	2016. 01.01	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

8	KVM 기반 가상화 환경에서 I/O 성능 간섭 해결을 위한 CPU-Network 통합 스케줄링 구조 제안	정보과학회논문지: 컴퓨팅의 실제	강동화	22	KR	한국정보과학회	비SCIE	2016.05.01	
9	가상화 환경에서 스토리지 성능 향상을 위한 호스트 캐시 마이그레이션 엔진 설계 및 구현	정보과학회 논문지: 컴퓨팅의 실제	박준영	22	KR	한국정보과학회	비SCIE	2016.06.01	
10	가상화 된 SDN에서 효과적인 자원 활용을 위한 LISP 기반 IP 주소 가상화 기법	정보과학회 논문지	고영근	43	KR	한국정보과학회	비SCIE	2016.02.01	
11	가상화 환경에서 네트워크 I/O 성능 간섭 해결을 위한 피드백 제어 CPU 스케줄링 기법 분석	정보과학회 논문지: 컴퓨팅의 실제	고현석	23	KR	한국정보과학회	비SCIE	2017.09.01	
12	KVM 기반 가상화 환경에서 CPU 스케줄링 관점으로 본 Network IO 성능간섭 현상 분석	한국정보과학회 동계학술발표회	강동화	22권 9호	KR	한국정보과학회	비SCIE	2016.09.01	
13	CRIM: Conditional Remapping to Improve the Reliability of Solid-State Drives with Minimizing Lifetime Loss	Scientific Programming	김영필	2018	US	hindawi	SCIE	2018.09.20	
14	Enhancing the Isolation and Performance of Control Planes for Fog Computing	Sensors	이경운	2018	US	MDPI	SCIE	2018.09.28	
15	커널 레벨의 메모리 최적화를 통한 가상 스위치 포워딩 성능 향상 기법	정보과학회 논문지	최흥식	45	KR	한국정보과학회	비SICE	2018.06.20	
16	Kafe: Can OS Kernels Forward Packets Fast Enough for Software Routers?	IEEE/ACM Transactions on Networking	Cheol-Ho Hong		US	IEEE	SCIE	2018.11.20	
17	LiteVisor: A Network Hypervisor to Support Flow Aggregation and Seamless Network Reconfiguration for VM Migration in Virtualized Software-defined Networks	IEEE Access	Gyeongsik Yang	99	US	IEEE	SCIE	2018.11.20	
18	qCon: QoS-Aware Network Resource Management for Fog Computing	Sensors	Cheol-Ho Hong	18	US	MDPI	SCIE	2018.10.13	
19	Dynamic network scheduling for virtual routers	IEEE Systems Journal	이경운	Early access		IEEE	SCIE	2019.09.24	50
20	Libera for Programmable Network	IEEE Communica	양경식	58(4)		IEEE	SCIE	2020.04.20	50

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	Virtualization	tions Magazine							
21	Proactive Congestion Notification for Distributed Deep Learning	Sensors	양경식	0		MDPI	SCIE	2020. 12.29	50
22	QiOi: Performance Isolation for Hyperledger Fabric	Applied Sciences-basel	김정수	0	CH	MDPI	SCIE	2021. 04.25	50
23	Accurate and Efficient Monitoring for Virtualized SDN in Clouds	IEEE Transactions on Cloud Computing	양경식	11		IEEE	SCIE	2021. 06.16	40
24	Autothrottle: Satisfying Network Performance Requirements for Containers	IEEE Transactions on Cloud Computing	이경운	Early access		IEEE	SCIE	2022. 07.04	20
25	Resource Analysis of Blockchain Consensus Algorithms in Hyperledger Fabric	IEEE Access	양경식	10		IEEE	SCIE	2022. 07.14	45
26	TeaVisor: Network Hypervisor for Bandwidth Isolation in SDN-NV	IEEE Transactions on Cloud Computing	유연호	Early access		IEEE	SCIE	2022. 12.01	45
27	ARM 캐시 일관성 인터페이스를 이용한 안드로이드 OS의 스크린 잠금 기능 부채널 공격	정보보호학회논문지 (한국정보보호학회)	이경운	32	KR	한국정보보호학회	비SCIE	2022. 04.30	35
28	Network SLO-aware Container Scheduling in Kubernetes	The Journal of Supercomputing	이경운	Early access		Springer	SCIE	2023. 02. 15.	25

*** SW스타랩 ACK가 있는 논문만 실적으로 기재**

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

③ 대표논문(학술지) : 5개 이내

NO 1			
주저자	Chiyoung Lee	사업년도	2016
공동저자	Sewon Kim, Chuck Yoo	게재일자	2016.02.01
논문명	VADI: GPU Virtualization for an Automotive Platform		
게재 학술지명	IEEE Transactions on Industrial Informatics	Vol(No.)	12
Journal Imapact factor	6.764	기여율	50%
논문 내용 요약	Modern vehicles1 are evolving with more electronic components than ever before. One notable example is Graphical Processing Unit (GPU), which is a key component to implement a digital cluster. To implement the digital cluster that displays all the meters (such as speed and fuel gauge) together with infotainment services (such as navigator and browser), the GPU needs to be virtualized; however, the GPU virtualization for the digital cluster has not been addressed yet. This paper presents Virtualized Automotive DIsplay system (VADI) to virtualize a GPU and its attached display device. VADI manages two execution domains: one for the automotive control software, and the other for the In-Vehicle Infotainment (IVI) software. Through the GPU virtualization, VADI provides GPU rendering to both execution domains, and it simultaneously displays their images on a digital cluster. In addition, VADI isolates GPU from the IVI software in order to protect it from potential failures of the IVI software. We implement VADI with Vivante GC2000 GPU and perform experiments to ensure requirements of ISO safety standards. The results show that VADI guarantees 30 fps that is the minimum frame rate for digital cluster mandated by ISO safety standards even with the failure of the IVI software. It also achieves 60 fps in a synthetic workload.		
논문의 우수성	"VADI: GPU Virtualization for an Automotive Platform"는 자동차 산업에서 사용할 수 있는 GPU 가상화 솔루션을 제안하는 연구 논문으로, JCR 상위 1%에 해당하는 최상위 학술지 논문임. 다음과 같은 우수성을 지님. - 자원 활용 문제 해결: 이 논문은 여러 애플리케이션이 동시에 실행되어야 하는 현대 자동차 시스템에서 자원 활용 문제를 해결함. 제안된 솔루션은 GPU의 가상화를 가능하게 하여 여러 애플리케이션 간의 자원 공유를 효율적으로 수행함. - 개선된 보안: 이 논문은 가상화가 애플리케이션을 격리시켜 상호 간의 데이터 접근을 방지하여 보안을 개선하며,. 이는 보안이 중요한 자동차 산업에서 특히 중요한 문제임 - 기존 플랫폼과의 호환성: 제안된 가상화 솔루션은 기존의 자동차 플랫폼과 호환될 수 있도록 설계되어 하드웨어나 소프트웨어에 대한 큰 변경이 필요하지 않음 - 성능: 이 논문은 제안된 솔루션이 낮은 오버헤드와 함께 좋은 성능을 제공함을 입증하며, 가상화된 GPU는 거의 원래 GPU만큼 잘 작동하면서 여러 애플리케이션을 동시에 실행할 수 있음		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

NO 2			
주저자	양경식	사업년도	2020
공동저자	유봉열, 진희상, 유혁	게재일자	2020.04.20
논문명	Libera for Programmable Network Virtualization		
게재 학술지명	IEEE Communications Magazine	Vol(No.)	58(4)
Journal Imapact factor	4.94	기여율	50%
논문 내용 요약	Current network virtualization allows tenants to have their own virtual networks. However, new demands to “program” virtual networks at a finer granularity have arisen as tenants want the abili- ty to provision and control switches and links in their virtual networks. This study proposes a new concept called the p-NIaaS model. The p-NIaaS model enables tenants to program their own packet processing logic and monitor network sta- tus from any virtual network infrastructure, which is not possible with current network virtualization. This article presents the Libera network hypervi- sor that implements the p-NIaaS model. Libera overcomes the shortcomings of existing network hypervisors such as scalability, VM migration sup- port, and flexibility. The evaluation shows that the Libera hypervisor is highly scalable and effectively supports VM migration. We also present the over- heads of Libera. Libera incurs up to 11 percent overhead in comparison with a non-virtualized network, which we believe is promising in the first prototype of the p-NIaaS model.		
논문의 우수성	본 논문은 네트워크 가상화에 대한 새로운 접근 방법을 제안하며, 네트워크 운영자를 위한 프로그래밍 가능성과 유연성을 제공함. JCR 분야 4%에 해당하는 최상위 학술지 논문이며, 이 논문의 주요 우수성은 다음과 같음 - "프로그래밍 가능 가상화"라는 개념을 소개하며, 네트워크 가상화와 네트워크 프로그래밀리티의 이점을 결합함. 이 접근 방식은 네트워크 운영자가 사용자 정의 네트워크 서비스 및 정책을 쉽게 생성하고 배포할 수 있도록 하는, 가상화의 새로운 패러다임에 해당함. - "Libera"라는 새로운 네트워크 가상화 아키텍처를 제안하며, 네트워크의 제어 평면과 데이터 평면을 분리합니다. 이 아키텍처는 가상 네트워크 관리의 유연성과 확장성을 제공하여, 클라우드 시스템 성능 제어의 새로운 개념에 해당함. - Libera의 제어 평면 설계와 구현을 설명하며, P4 및 OpenFlow 등의 프로그래밍 언어를 활용하여 사용자 정의 네트워크 정책을 생성합니다. 이 접근 방식은 네트워크 운영자가 특정 요구에 맞게 맞춤형 정책을 정의할 수 있도록 함. - Libera의 성능과 확장성을 보여주는 실험 결과를 제시하여, Libera가 많은 수의 가상 네트워크와 정책을 효율적으로 처리할 수 있어 대규모 생산 네트워크에 적합하다는 것을 실제로 입증함.		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

NO 3			
주저자	Cheol-Ho Hong	사업년도	2018
공동저자	Kyungwoon Lee, Jaehyun Hwang, Hyunchan Park, Chuck Yoo	게재일자	2018.11.20
논문명	Kafe: Can OS Kernels Forward Packets Fast Enough for Software Routers?		
게재 학술지명	IEEE/ACM Transactions on Networking	Vol(No.)	26
Journal Imapact factor	3.597	기여율	100%
논문 내용 요약	Abstract- It is widely believed that software routers based on commodity operating systems cannot deliver high-speed packet processing, and a number of alternative approaches (including user-space network stacks) have been proposed. This paper revisits the inefficiency of kernel-level packet processing inside modern OS-based software routers and explores whether a redesign of kernel network stacks can improve the incompetence. We present a case contrary to the belief through a redesign: Kafe-a kernel-based advanced forwarding engine that can process packets as fast as user-space network stacks. The Kafe neither adds any new API nor depends on proprietary hardware features, but the Kafe outperforms Linux by seven times and RouteBricks by three times. The current implementation of the Kafe can forward 64-byte IPv4 packets at 28.2 Gbps using eight cores running at 2.6 GHz. Our evaluation results show that the Kafe achieves similar packet forwarding performance to Intel DPDK while consuming much less CPU and memory resources.		
논문의 우수성	본 논문은 패킷 전달을 위해 운영 체제 커널을 사용하여 상용 하드웨어에서 실행되는 소프트웨어 라우터의 성능을 개선함. JCR 상위 10% 최상위 학술지에 출판되었으며, 논문의 우수성은 다음과 같음. - 상용 하드웨어 및 운영 체제 커널이 소프트웨어 라우터에 대해 높은 패킷 전달 성능을 달성할 수 있음을 입증함. 이 발견은 고성능 패킷 포워딩에 전용 하드웨어가 필요하다는 일반적인 통념을 반박하는 것으로, 시스템 및 소프트웨어 연구에 도전적인 결과를 제시하는 것임. - 패킷 처리 오버헤드를 줄이고 성능을 향상시키기 위해 커널 바이패스 기술을 사용하는 Kafe라는 새로운 소프트웨어 라우터 디자인을 소개함. Kafe 라우터 설계는 상용 하드웨어에서 최대 속도 전달 속도를 달성함. - 실제 트래픽을 사용하여 Kafe를 구현하고 평가함. 실험 결과는 Kafe가 다른 고성능 소프트웨어 라우터와 비슷하거나 더 나은 패킷 포워딩 속도를 달성할 수 있음을 보여줌. - 향후 소프트웨어 라우터 솔루션의 개발을 안내하는 데 도움이 될 수 있는 고속 패킷 포워딩을 위한 소프트웨어 라우터의 설계 및 최적화에 대한 연구 방향을 제시함.		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

NO 4			
주저자	유연호	사업년도	2022
공동저자	이정환, 신창용, 김호석, 양경식, 유혁	게재일자	2022.12.01
논문명	TeaVisor: Network Hypervisor for Bandwidth Isolation in SDN-NV		
게재 학술지명	IEEE Transactions on Cloud Computing	Vol(No.)	Early access
Journal Imapact factor	5.938	기여율	45%
논문 내용 요약	We introduce TeaVisor that provides bandwidth isolation guarantee for network virtualization (NV) based on software-defined networking (SDN). SDN-based NV (SDN-NV) offers many benefits to clouds, such as topology and address virtualization while allowing flexible resource provisioning, control, and monitoring on virtual networks. In SDN-NV, however, routing is done by tenants independently; thus, existing studies have difficulties in bandwidth isolation guarantee due to the overloaded link problem. Bandwidth isolation guarantee is essential for providing stable and reliable throughput on network services in SDN-NV. Without bandwidth isolation guarantee, tenants suffer degraded service qualities and significant loss in revenue. To address this problem, we design and implement TeaVisor in three components: path virtualization, bandwidth reservation, and path establishment. Through extensive experiments, TeaVisor shows that bandwidth isolation is guaranteed with near-zero errors, which is three orders of magnitude better than existing studies. In addition, TeaVisor guarantees the minimum and maximum bandwidth at the same time. We also present an overhead analysis of TeaVisor in control traffic and memory consumption.		
논문의 우수성	본 논문은 SDN 기반 클라우드 가상화 시스템에서 네트워크 대역폭 성능의 격리 및 보장을 위한 기술을 다루고 있으며, IEEE Transaction 중 JCR 상위 10% 최상위 학술지에 출판되었음. 논문의 우수성은 다음과 같음: - 논문에서는 SDN-NV에서 발생하는 과부하 문제로 인해 대역폭 격리 보장이 어려운 문제를 최초로 발견 및 구체적인 실험을 통해 원인을 분석하였음. 이를 해결하기 위해 TeaVisor를 제안함. - TeaVisor는 경로 가상화, 대역폭 예약, 경로 설정의 3가지 구성 요소로 구성되어 있으며, 실험 결과 대역폭 격리가 거의 오차 없이 보장되는 것을 입증함. - 기존의 연구에 비해 세 배 이상 더 나은 오차율을 보이며, 최소 및 최대 대역폭을 동시에 보장함. 또한, TeaVisor의 제어 트래픽 및 메모리 소비에 대한 오버헤드 분석 결과도 제시하여, 기존 기술 대비 추가되는 오버헤드가 거의 미미한 수준임을 입증하였음.		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

NO 5			
주저자	이경운	사업년도	2022
공동저자	이관훈, 박현찬, 황재현, 유혁	게재일자	2022. 07. 04.
논문명	Autothrottle: Satisfying Network Performance Requirements for Containers		
게재 학술지명	IEEE Transactions on Cloud Computing	Vol(No.)	Early access
Journal Imapact factor	5.938	기여율	20%
논문 내용 요약	This paper investigates how to satisfy network performance requirements that are crucial in achieving the service level objectives (SLOs) in clouds. Traditional techniques for network performance management have a limited ability to satisfy the network SLOs. Our in-depth analysis reveals that the fundamental reason comes from decoupling of the CPU scheduler and the network traffic controller as the current CPU scheduler is not aware of such network requirements but only provides a fair-share amount of CPU to all containers. Thus, the container cannot perform the amount of network processing as needed to satisfy its SLO when the CPU allocation is insufficient. In this paper, we propose Autothrottle that dynamically adjusts the CPU allocation for the containers to satisfy their network SLOs. The key element of Autothrottle is a throttle algorithm that autonomously determines the amount of CPU for each container needed to satisfy the requirement. We implement Autothrottle in the Linux kernel and evaluate it with massive real-world workloads such as Apache Kafka. Our evaluation results show that Autothrottle successfully satisfies the given network SLO only with a 2% gap while the existing scheme achieves 20% less than the SLO. We further observe that Autothrottle also reduces the CPU overhead in network processing by 19%, improving the network throughput by 27% compared to the existing scheme.		
논문의 우수성	본 논문은 컨테이너에서의 네트워크 및 CPU 성능 제어와 관련된 기술을 다루고 있음. EEE Transaction 중 JCR 상위 10% 최상위 학술지에 출판되었음. 논문의 우수성은 다음과 같음: - 논문에서는 컨테이너 기반의 분산 시스템에서 발생하는 네트워크 성능 문제를 제기하고, 이를 해결하기 위해 Autothrottle을 제안함 - Autothrottle은 컨테이너의 네트워크 사용량을 모니터링하고, 필요에 따라 자동으로 네트워크 처리량을 조절하여 성능을 최적화함 - 실험 결과, Autothrottle을 사용하면 대역폭 부족 현상이 감소하고 네트워크 지연 시간이 줄어들어 전반적인 성능이 향상됨 - 또한, Autothrottle은 다른 컨테이너 관리자 시스템과의 호환성이 높고, 자동화된 성능 조절 기능을 제공하여 운영자의 작업 부담을 줄임		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 국내 및 국제 학술회의 발표

① 발표 세부내역

번호	회의 명칭	발표자	발표 일시	장소	국명
1	International Symposium on Embedded Technology (ISET)	Sang Hun Lee	20160526	대전 호텔인터시티	KR
2	International Symposium on Embedded Technology (ISET)	Dong Hwa Kang	20160526	대전 호텔인터시티	KR
3	CFI '16: Proceedings of the 11th International Conference on Future Internet Technologies	Gyeongsik Yang	20160615	중국 난징	CN
4	ACM CoNEXT Workshop on Cloud-Assisted Networking	Bong-yeol Yu	20161212	Irvine California USA	US
5	International Conference on Big data, IoT, and Cloud Computing	Youngpil Kim	20170822	제주 라마다호텔	KR
6	International Conference on Big data, IoT, and Cloud Computing	Jea-Min Lim	20170822	제주 라마다호텔	KR
7	ICTC	Kyungwoon Lee	20171018	제주 라마다호텔	KR
8	CNSM	Wontae Jeong	20171126	일본 도쿄	JP
9	NetSoft '18	Bong-yeol Yu	20180625	Montreal, Canada	CA
10	IEEE CLOUD '18	Gyeongsik Yang	20180702	SAN FRANCISCO, CA, USA	US
11	IEEE Blockchain '18	Jaemin Lim	20180730	Halifax, Canada	CA
12	한국정보과학회 동계학술발표회	양경식	20151201	강원도 휘닉스파크	KR
13	한국정보과학회 동계학술발표회	이상헌	20151201	강원도 휘닉스파크	KR
14	한국정보과학회 동계학술발표회	임재민	20151201	강원도 휘닉스파크	KR
15	한국정보과학회 컴퓨팅의 실제 논문지	박준영	20151201	강원도 휘닉스파크	KR
16	한국정보과학회 동계학술발표회	김형태	20151201	강원도 휘닉스파크	KR
17	한국정보과학회 동계학술발표회	고영근	20151201	강원도 휘닉스파크	KR
18	한국컴퓨터종합학술대회(KCC)	유소망	20160601	제주 ICC&부영호텔	KR
19	한국컴퓨터종합학술대회(KCC)	임재민	20160601	제주 ICC&부영호텔	KR
20	한국컴퓨터종합학술대회(KCC)	김형태	20160601	제주 ICC&부영호텔	KR
21	한국통신학회 하계종합학술발표회	고영근	20160601	제주국제컨벤션센터	KR
22	한국컴퓨터종합학술대회(KCC)	고현석	20161201	제주 ICC&부영호텔	KR

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

23	한국컴퓨터종합학술대회(KCC)	김봉규	20161201	제주 ICC&부영호텔	KR
24	한국컴퓨터종합학술대회(KCC)	이시오	20161201	제주 ICC&부영호텔	KR
25	한국컴퓨터종합학술대회(KCC)	유소망	20161201	제주 ICC&부영호텔	KR
26	한국정보보호학회 동계학술발표회	임재민	20161203	연세대학교	KR
27	한국통신학회 동계종합학술발표회	정원태	20170101	강원도 하이원리조트	KR
28	한국통신학회 동계종합학술발표회	최흥식	20170101	강원도 하이원리조트	KR
29	한국컴퓨터종합학술대회	양경식	20170601	제주 라마다호텔	KR
30	한국컴퓨터종합학술대회	최흥식	20170601	제주 라마다호텔	KR
31	한국컴퓨터종합학술대회	정원태	20170601	제주 라마다호텔	KR
32	한국컴퓨터종합학술대회	이시오	20170601	제주 라마다호텔	KR
33	한국컴퓨터종합학술대회	임재민	20170601	제주 라마다호텔	KR
34	한국컴퓨터종합학술대회	이경운	20171201	제주 라마다호텔	KR
35	한국컴퓨터종합학술대회	이경운	20180620	제주국제컨벤션센터	KR
36	한국컴퓨터종합학술대회	정윤진	20180620	제주국제컨벤션센터	KR
37	한국컴퓨터종합학술대회	임재민	20180620	제주국제컨벤션센터	KR
38	한국컴퓨터종합학술대회	유봉열	20180620	제주국제컨벤션센터	KR
39	한국소프트웨어종합학술대회	임재민	20181219	평창 휘닉스파크	KR
40	한국소프트웨어종합학술대회	양경식	20181219	평창 휘닉스파크	KR
41	한국소프트웨어종합학술대회	유봉열	20181219	평창 휘닉스파크	KR
42	한국소프트웨어종합학술대회	진희상	20181219	평창 휘닉스파크	KR
43	한국소프트웨어종합학술대회	강민구	20181219	평창 휘닉스파크	KR
44	2019 International Conference on Information Networking (ICOIN)	유봉열	20190109	Renaissance Kuala Lumpur Hotel	MY
45	2019 International Conference on Information Networking (ICOIN)	진희상	20190109	Renaissance Kuala Lumpur Hotel	MY
46	2019 IEEE 12th International Conference on Cloud Computing (CLOUD)	진희상	20190708	MILAN, ITALY	IT
47	The 15th International Conference on emerging Networking EXperiments and Technologies	진희상	20191209	Orlando Florida	US

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

48	2019 한국컴퓨터종합학술대회	김수완	20190626	제주국제컨벤션센터	KR
49	2019 한국컴퓨터종합학술대회	강민구	20190626	제주국제컨벤션센터	KR
50	2019 한국컴퓨터종합학술대회	김동은	20190628	제주국제컨벤션센터	KR
51	2019 한국컴퓨터종합학술대회	김한주	20190628	제주국제컨벤션센터	KR
52	2019 한국컴퓨터종합학술대회	Anumeha	20190626	제주국제컨벤션센터	KR
53	IEEE International Conference on Computer Communications 2020 (INFOCOM 2020)	양경식	20200804	Online	CA
54	2020 IEEE 13th International Conference on Cloud Computing (CLOUD 2020)	강민구	20201018	Online	CN
55	2020 IEEE 13th International Conference on Cloud Computing (CLOUD 2020)	유연호	20201018	Online	CN
56	2nd conference on Blockchain Research and Applications for Innovative Networks and Services (BRAINS 2020)	김정수	20200928	Paris, France	FR
57	2020 한국컴퓨터종합학술대회 (KCC 2020)	임재민	20200702	Online	KR
58	2020 한국컴퓨터종합학술대회 (KCC 2020)	김문교	20200702	Online	KR
59	2020 한국컴퓨터종합학술대회 (KCC 2020)	김효경	20200702	Online	KR
60	2020 한국컴퓨터종합학술대회 (KCC 2020)	최원미	20200702	Online	KR
61	2020 한국컴퓨터종합학술대회 (KCC 2020)	김정수	20200702	Online	KR
62	2020 한국소프트웨어종합학술대회 (KSC2020)	김은숙	20201221	Online	KR
63	2020 한국소프트웨어종합학술대회 (KSC2020)	이준호	20201221	Online	KR
64	2020 한국소프트웨어종합학술대회 (KSC2020)	이지유	20201221	Online	KR
65	2020 한국소프트웨어종합학술대회 (KSC2020)	김용래	20201221	Online	KR
66	2020 한국소프트웨어종합학술대회 (KSC2020)	유연호	20201221	Online	KR
67	INFOCOM 2021	양경식	20210510	Online	CA

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

68	MASCOTS 2021	양경식	20211103	Online	US
69	ICOIN2021	김은숙	20210114	제주 라마다호텔 &온라인	KR
70	Euro-Par 2021: Parallel Processing Workshops	김은숙	20210830	Online	PT
71	2021년도 한국통신학회 하계종합학술발표회 논문집	구인회	20210616	제주 라마다호텔	KR
72	2021년도 한국통신학회 하계종합학술발표회 논문집	강전진	20210616	제주 라마다호텔	KR
73	한국정보과학회 2021 한국컴퓨터종합학술 대회 논문집	최원미	20210623	ICC 제주&씨에스호텔 &온라인	KR
74	한국정보과학회 2021 한국컴퓨터종합학술 대회 논문집	이재윤	20210623	ICC 제주&씨에스호텔 &온라인	KR
75	한국정보과학회 2021 한국컴퓨터종합학술 대회 논문집	이관훈	20210623	ICC 제주&씨에스호텔 &온라인	KR
76	한국정보과학회 2021 한국컴퓨터종합학술 대회 논문집	이효원	20210623	ICC 제주&씨에스호텔 &온라인	KR
77	한국정보과학회 2021 한국소프트웨어학술 대회 논문집	이상엽, 김은숙	20211220	ICC 제주&씨에스호텔 &온라인	KR
78	한국정보과학회 2021 한국소프트웨어학술 대회 논문집	오혜빈, 최원미	20211220	ICC 제주&씨에스호텔 &온라인	KR
79	한국정보과학회 2021 한국소프트웨어학술 대회 논문집	최예찬, 신창용	20211220	ICC 제주&씨에스호텔 &온라인	KR
80	한국정보과학회 2021 한국소프트웨어학술 대회 논문집	김호석, 유연호	20211220	ICC 제주&씨에스호텔 &온라인	KR
81	한국정보과학회 2021 한국소프트웨어학술 대회 논문집	이정환	20211220	CC 제주&씨에스호텔 &온라인	KR
82	한국정보과학회 2021 한국소프트웨어학술 대회 논문집	이효원	20211220	CC 제주&씨에스호텔 &온라인	KR
83	한국정보과학회 2021 한국소프트웨어학술 대회 논문집	이효원	20211220	CC 제주&씨에스호텔 &온라인	KR
84	2022 ACM SIGMETRICS conference	양경식	20220506	Mumbai, India	IN
85	IEEE CLOUD 2022 (International Conference on Cloud Computing)	신창용	20220710	BARCELONA, SPAIN	ES
86	한국정보과학회 2022 한국컴퓨터종합학술 대회 논문집	김호석	20220601	제주 라마다호텔	KR
87	한국정보과학회 2022 한국컴퓨터종합학술 대회 논문집	이정환	20220601	제주 라마다호텔	KR
88	한국정보과학회 2022 한국컴퓨터종합학술 대회 논문집	이진형	20220601	제주 라마다호텔	KR
89	한국통신학회 2022 한국통신학회추계종 합학술발표회논문집	고영훈	20221101	경주 라한셀렉트	KR

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

90	한국통신학회 2022 한국통신학회추계종합학술발표회논문집	이준석	20221101	경주 라한셀렉트	KR
91	한국통신학회 2022 한국통신학회추계종합학술발표회논문집	고의진	20221101	경주 라한셀렉트	KR
92	한국통신학회 2022 한국통신학회추계종합학술발표회논문집	신창용	20221101	경주 라한셀렉트	KR

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

② 대표논문(학술회의) : 5개 이내

NO 1			
주저자	양경식	사업년도	2020
공동저자	진희상, 강민구,문기준, 유혁	발표일자	2020.08.04
논문명	Network Monitoring in SDN Virtualization	기여율	50%
발표 컨퍼런스명	2020 IEEE International Conference on Computer Communications (IEEE INFOCOM 2020)	개최기간	2020.07.06.~09
개최장소	ONLINE		
컨퍼런스의 중요성 등	IEEE International Conference on Computer Communications (IEEE INFOCOM)은 컴퓨터 네트워크 및 시스템 분야에서 가장 권위있는 학회임. 이 학회는 매년 컴퓨터 네트워킹 분야의 연구자, 전문가, 실무자 등이 모여 최신 연구 결과, 혁신, 동향 등을 발표하고 논의하는 자리를 제공함. 특히 본 학술대회는 **한국정보과학회 우수학술대회 중 최우수학술대회**로 지정되어 있고, 한국연구재단 BK사업 SCIE인정 학술대회 리스트에서 **최상위 등급인 IF4 등급**으로 지정됨. 뿐만 아니라, Google Scholar 기준으로는 H5 index 기준으로 **분야 내 학술대회 중 1위**에 위치되어 있는 최고 학술대회임. 해당연차 논문의 수락율(acceptance rate)은 19.8%임.		
논문 내용 요약	This paper proposes V-Sight, a network monitoring framework for software-defined networking (SDN)-based virtual networks. Network virtualization with SDN (SDN-NV) makes it possible to realize programmable virtual networks; so, the technology can be beneficial to cloud services for tenants. How- ever, to the best of our knowledge, although network monitoring is a vital prerequisite for managing and optimizing virtual networks, it has not been investigated in the context of SDN-NV. Thus, virtual networks suffer from non-isolated statistics between virtual networks, high monitoring delays, and excessive control channel consumption for gathering statistics, which critically hinders the benefits of SDN-NV. To solve these problems, V-Sight presents three key mechanisms: 1) statistics virtualization for isolated statistics, 2) transmission disaggregation for reduced transmission delay, and 3) pCollector aggregation for efficient control channel consumption. V-Sight is implemented on top of OpenVirteX, and the evaluation results demonstrate that V- Sight successfully reduces monitoring delay and control channel consumption up to 454 times.		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

NO 2			
주저자	양경식	사업년도	2021
공동저자	유연호, 강민구, 진희상, 유혁	발표일자	2021.07.26
논문명	Bandwidth Isolation Guarantee for SDN Virtual Networks	기여율	50%
발표 컨퍼런스명	2021 IEEE International Conference on Computer Communications (IEEE INFOCOM 2021)	개최기간	2021.05.10.~13
개최장소	online		
컨퍼런스의 중요성 등	IEEE International Conference on Computer Communications (IEEE INFOCOM)은 컴퓨터 네트워크 및 시스템 분야에서 가장 권위있는 학회임. 이 학회는 매년 컴퓨터 네트워킹 분야의 연구자, 전문가, 실무자 등이 모여 최신 연구 결과, 혁신, 동향 등을 발표하고 논의하는 자리를 제공함. 특히 본 학술대회는 **한국정보과학회 우수학술대회 중 최우수학술대회**로 지정되어 있고, 한국연구재단 BK사업 SCIE인정 학술대회 리스트에서 **최상위 등급인 IF4 등급**으로 지정됨. 뿐만 아니라, Google Scholar 기준으로는 H5 index 기준으로 **분야 내 학술대회 중 1위**에 위치되어 있는 최고 학술대회임. 해당연차 논문의 수락율(acceptance rate)은 19.9%임.		
논문 내용 요약	We introduce TeaVisor, which provides bandwidth isolation guarantee for software-defined networking (SDN)-based network virtualization (NV). SDN-NV provides topology and address virtualization while allowing flexible resource provision- ing, control, and monitoring of virtual networks. However, to the best of our knowledge, the bandwidth isolation guarantee, which is essential for providing stable and reliable throughput on network services, is missing in SDN-NV. Without bandwidth isolation guarantee, tenants suffer degraded service quality and significant revenue loss. In fact, we find that the existing studies on bandwidth isolation guarantees are insufficient for SDN-NV. With SDN-NV, routing is performed by tenants, and existing studies have not addressed the overloaded link problem. To solve this problem, TeaVisor designs three components: path virtualization, bandwidth reservation, and path establishment, which utilize multipath routing. With these, TeaVisor achieves the bandwidth isolation guarantee while preserving the routing of the tenants. In addition, TeaVisor guarantees the minimum and maximum amounts of bandwidth simultaneously. We fully implement TeaVisor, and the comprehensive evaluation results show that near-zero error rates on achieving the bandwidth isolation guarantee. We also present an overhead analysis of control traffic and memory consumption.		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

NO 3			
주저자	양경식	사업년도	2022
공동저자	신창용, 이정환, 유연호, 유혁	발표일자	2022.05.06
논문명	Prediction of the Resource Consumption of Distributed Deep Learning Systems	기여율	45%
발표 컨퍼런스명	2022 ACM SIGMETRICS conference	개최기간	2022.06.06.~10
개최장소	Mumbai, India		
컨퍼런스의 중요성 등	ACM SIGMETRICS (ACM Special Interest Group on Measurement and Evaluation) 학회는 컴퓨터 시스템 성능 분석 및 평가 분야에서 가장 권위 있는 학회임. 이 학회는 성능 분석, 측정, 모델링, 통계 분석 등 다양한 연구 주제를 다룸. ACM SIGMETRICS의 중요성은 컴퓨터 시스템의 성능을 평가하고 최적화하는 기술과 방법을 연구하는 것에 있음. 이 학회는 다양한 분야에서의 성능 분석과 평가에 대한 새로운 기술과 방법을 발표하고, 이를 활용해 새로운 컴퓨터 시스템을 설계하고 최적화하는 연구를 지원함. 특히 본 학술대회는 **한국정보과학회 우수학술대회 중 최우수학술대회**로 지정되어 있고, 한국연구재단 BK사업 SCIE인정 학술대회 리스트에서 **IF3 등급**으로 지정됨. 동 리스트들에서 시스템 성능 모델링 및 프로파일링 학술대회로서는 최우수 및 등급 상 최상위에 위치한 최고 학술대회임. 해당연차 논문의 수락율(acceptance rate)은 **10.7%**임.		
논문 내용 요약	The prediction of the resource consumption for the distributed training of deep learning models is of paramount importance, as it can inform a priori users how long their training would take and also enable users to manage the cost of training. Yet, no such prediction is available for users because the resource consumption itself varies significantly according to “settings” such as GPU types and also by “workloads” like deep learning models. Previous studies have aimed to derive or model such a prediction, but they fall short of accommodating the various combinations of settings and workloads together. This study presents Driple that designs graph neural networks to predict the resource consumption of diverse workloads. Driple also designs transfer learning to extend the graph neural networks to adapt to differences in settings. The evaluation results show that Driple can effectively predict a wide range of workloads and settings. At the same time, Driple can efficiently reduce the time required to tailor the prediction for different settings by up to 7.3×.		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

NO 4			
주저자	양경식	사업년도	2021
공동저자	신창용, 유연호, 유혁	발표일자	2021.11.22
논문명	A Case for SDN-based Network Virtualization	기여율	40%
발표 컨퍼런스명	IEEE MASCOTS (Modeling, Analysis, and Simulation On Computer and Telecommunication Systems) 2021	개최기간	2021.11.03.~05
개최장소	ONLINE		
컨퍼런스의 중요성 등	IEEE MASCOTS (Modeling, Analysis, and Simulation On Computer and Telecommunication Systems)는 전 세계적으로 유명한 컴퓨터 시스템과 통신 분야의 국제 학술 대회 중 하나임. 이 학회는 모델링, 분석 및 시뮬레이션 기술을 활용하여 컴퓨터 및 통신 시스템의 성능과 안전성을 평가하는 것을 목표로 함. 이러한 목표를 위해, MASCOTS는 컴퓨터 시스템 및 통신 분야의 전문가들이 모이는 국제적인 포럼으로써, 새로운 기술 및 연구 결과를 발표하고 공유하며, 연구자 및 산업 현장 전문가들 사이의 협력을 촉진하는 역할을 함. 특히 본 학술대회는 **한국정보과학회 우수학술대회 중 우수학술대회**로 지정되어 있고, 한국연구재단 BK사업 SCIE인정 학술대회 리스트에서 **IF2 등급**으로 지정됨.		
논문 내용 요약	Network virtualization (NV) becomes an essential technology in cloud computing that isolates network flows for tenants. However, because existing NV technologies like overlay do not enable tenants to directly program (i.e., provision, control, and monitor) network resources, software- defined networking (SDN)- based NV (SDN- NV) has been proposed. Despite its great benefits, SDN- NV has been believed to bring considerable overheads due to the network hypervisor (NH). However, to date, there is no definite performance evaluation that proves the overheads of SDN-NV. To this end, this paper comprehensively investigates the performance and overheads of SDN-NV. Our experiment results reveal that SDN-NV provides the data plane performance comparable to or even better (up to 10.5× better TCP throughput) than the existing NV technologies. Also, the re- sults on NH show that its overheads remain mostly constant, even when the number of switches, virtual networks, or network flows increases. In short, our evaluation indicates that the overhead of SDN-NV should not deter its practical use in datacenters. Index Terms—Network virtualization, Performance evaluation, Cloud computing, Software- defined networking		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

NO 5			
주저자	Gyeongsik Yang	사업년도	2018
공동저자	Bong-yeol Yu, Wontae Jeong, Chuck Yoo	발표일자	2018.07.02
논문명	FlowVirt: Flow Rule Virtualization for Dynamic Scalability of Programmable Network Virtualization	기여율	
발표 컨퍼런스명	IEEE CLOUD (International Conference on Cloud Computing) 2018	개최기간	2018.07.02.~07
개최장소	SAN FRANCISCO, CA, USA		
컨퍼런스의 중요성 등	IEEE CLOUD (International Conference on Cloud Computing)는 클라우드 컴퓨팅 분야에서 가장 권위있는 국제 학술 대회 중 하나임. 이 학회는 클라우드 컴퓨팅 분야의 연구자와 산업 현장 전문가들이 모여 최신 기술과 연구 결과를 발표하고 공유하는 대표적인 venue임. 이를 통해 클라우드 컴퓨팅 분야의 전문성과 혁신성을 증진시키며, 기술 발전과 산업 발전을 이끄는 역할을 수행함. 특히, IEEE CLOUD 학회는 클라우드 컴퓨팅 분야에서 다양한 주제를 다루며, 멀티 클라우드, 대규모 데이터 처리, 보안 및 개인 정보 보호, 분산 컴퓨팅 등의 주제에 대해 깊이 있는 논의가 이루어지고 있어, 이 분야의 발전과 발전 방향을 제시하는 데 중요한 역할을 함. 특히 한국연구재단 BK사업 SCIE인정 학술대회 리스트에서 **IF1 등급**으로 지정됨. 해당연차 논문의 수락율(acceptance rate)은 **18.97%**임.		
논문 내용 요약	We propose a new concept called “flow rule virtu- alization” (FlowVirt) for programmable network virtualization (P-NV). In P-NV, network hypervisor is a key component in that it plays a role in creating and managing virtual networks. This paper first reports a critical limitation of network hypervisor - scalability problem, which results in the high consumption of the switch memory, control channel, and CPU cycles: 3.9, 4.7, and 1.7 times higher than host-based net- work virtualization, respectively. This scalability problem arises because all the flow rules from the virtual network controllers are directly installed into switches. To resolve the scalability problem, FlowVirt introduces a flow rule abstraction: virtual and physical flow rules. By separating virtual and physical flow rules, the abstraction virtualizes flow rules so that FlowVirt can merge virtual flow rules to a smaller number of physical flow rules to be installed in switches. The evaluation results show the enhanced scalability of FlowVirt. The number of flow rules to be installed in switches decreases by up to 10 times compared to the previous P-NV. The control channel bandwidth and CPU cycles are also reduced by up to 14 and 3 times, respectively.		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 기술 요약 정보

연도	기술명	요약 내용	기술 완성도	등록 번호	활용 여부	미활용사유	연구개발기관 외 활용여부	허용방식

□ 보고서 원문

연도	보고서 구분	발간일	등록 번호

□ 생명자원(생물자원, 생명정보)/화합물

번호	생명자원(생물자원, 생명정보)/화합물 명	등록/기탁 번호	등록/기탁 기관	발생 연도

[기술적 성과]

□ 지식재산권(특허, 실용신안, 의장, 디자인, 상표, 규격, 신품종, 프로그램)

번호	지식재산권 등 명칭 (건별 각각 기재)	국명	출원				등록			기여율	활용 여부
			출원인	출원일	출원 번호	등록 번호	등록인	등록일	등록 번호		
1	HIGH-SPEED PACKET PROCESSING SYSTEM AND CONTROL METHOD THEREOF	US					유혁, 홍철호, 이경운	20190122	10,178,330	50	유상 기술실시 계약체결
2	Flow rule virtualization apparatus and method thereof in programmable network virtualization	US					유혁,양경식,유봉열	20200616	US 10,686,700	50	기술이전 준비중
3	METHOD FOR CONTROLLING PERFORMANCE IN VIRTUALIZED ENVIRONMENT AND INFORMATION PROCESSING DEVICE FOR THE SAME	US					이경운	20220809	11,429,420	50	기술이전 준비중
4	NETWORK MONITORING APPARATUS AND METHOD THEREOF IN PROGRAMMABLE NETWORK VIRTUALIZATION	US					양경식	20230124	11,563,655	45	기술이전 준비중
5	METHOD FOR RECEIVING ADAPTIVE STREAMING SERVICE	XI	Jung-Hwan Lee, Jea-Min Lim, Jae Hyun Hwang, Nakjung Choi,Ch	20160501	PCT/KR2016/003924					50	기술이전 준비중

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

			uck Yoo								
6	HIGH-SPEED PACKET PROCESSING SYSTEM AND CONTROL METHOD THEREOF	US	유혁, 홍철호, 이경운	20170501	15/595,150					50	기술이전 준비중
7	METHOD FOR RECEIVING ADAPTIVE STREAMING SERVICE	US	이정환, 임재민, 황재현, 최낙중, 유혁	20171001	15/566,095					50	기술이전 준비중
8	APPARATUS AND METHOD FOR MEASURING SECURITY STRENGTH OF LOCK PATTERN APPLIED TO MULTI-CORE BASED TERMINAL DEVICE	US	유혁, 김영필, 유시환, 이경운	20181114	16/190,928					50	기술이전 준비중
9	METHOD FOR CONTROLLING PERFORMANCE IN VIRTUALIZED ENVIRONMENT AND INFORMATION PROCESSING DEVICE FOR THE SAME	US	유혁, 이경운, 김수완	20191011	16/599,376					50	기술이전 준비중
10	NETWORK MONITORING APPARATUS AND METHOD THEREOF IN PROGRAMMABLE NETWORK VIRTUALIZATION	US	양경식, 강민구, 유혁	20201221	US 11,563,655					50	기술이전 준비중
11	APPRATUS AND METHOD FOR CONTROL CHANNEL ISOLATION BASED ON TIME-SERIES PREDICTION OF CONTROL TRAFFIC IN PROGRAMMABLE NETWORK VIRTUALIZATION	US	유연호, 양경식, 신창용, 이정환, 유혁	2023년 상반기 중 출원				45	기술이전 준비중		
12	APPARATUS AND METHOD FOR PROFILING-BASED JOB ORDERING FOR DISTRIBUTED DEEP LEARNING	US	신창용, 양경식, 유연호, 이정환, 유혁	2023년 상반기 중 출원				45	기술이전 준비중		
13	DEVICE AND METHOD FOR PREDICTING TRAINING TIME AND	US	양경식, 신창용, 유연호,	2023년 상반기 중 출원				45	기술이전 준비중		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	RESOURCE CONSUMPTION THEREOF IN DISTRIBUTED DEEP LEARING SYSTEM		이정환, 유혁								
14	적응형 스트리밍 서비스를 수신하는 방법	KR					임재민	20160801	10-1654898	50	기술이전 준비중
15	DDoS 공격 탐지 장치 및 방법	KR					오석영	20170101	10-1701310	50	기술이전 준비중
16	가상화 환경에서 네트워크 성능 차등화를 위한 동적 자원 할당 장치 및 방법	KR					이경운	20170501	10-1743028	50	기술이전 준비중
17	가상환경의 스토리지 성능 향상을 위한 클라우드 스토리지 시스템 및 그 관리 방법	KR					박준영	20170501	10-1740962	50	기술이전 준비중
18	SDN 기반 네트워크 가상화 플랫폼에서의 IP 주소 가상화 방법 및 시스템	KR					고영근	20171001	10-2016-0129691	50	기술이전 준비중
19	CPU 네트워크 통합 스케줄링 시스템 및 그 제어방법	KR					유혁, 강동화, 박현찬, 이경운, 고현석	20180101	10-1816718	50	기술이전 준비중
20	고속 패킷 처리 시스템 및 그 제어방법	KR					유혁, 이경운, 홍철호	20180401	10-1854377	50	유상 기술실시 계약체결
21	가상 머신의 CPU 및 블록 I/O 작업에 성능 보장을 위한 자원 할당 시스템 및 방법	KR					유혁, 고현석, 이우진, 이경운	20181127	10-1924467	50	기술이전 준비중
22	SDN 기반 경로 지연 시간 측정 방법	KR					유혁, 양경식, 김성문	20190320	10-1962346	50	기술이전 준비중
23	프로그래밍 가능한 네트워크 가상화에서의 플로우 룰 가상화 장치 및 그 방법	KR					유혁, 양경식, 김성문	20190520	10-1982230	50	기술이전 준비중
24	멀티코어 기반의 단말 장치에 적용되는 잠금 패턴의 보안 강도를 측정하는 장치 및 방법	KR					유혁,김영필,유시환,이경운	20200214	10-2079868	50	기술이전 준비중

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

25	프로그래밍 가능한 네트워크 가상화에서의 대역폭을 고려한 링크 장애 우회 장치 및 방법	KR					유혁,진희상,양경식	20201026	10-2172232	50	기술이전 준비중
26	가상화 환경에서의 성능 제어 방법 및 이를 위한 정보 처리 장치	KR					유혁,이경운,김수완	20210108	10-2203204	50	기술이전 준비중
27	프로그래머블 스위치에서의 패킷 스케줄링을 이용한 분산 딥러닝 방법	KR					강민구,양경식,유혁	20210825	10-2296319	90	기술이전 준비중
28	프로그래밍 가능한 네트워크 가상화에서의 모니터링 방법 및 시스템	KR					강민구,양경식,유혁	20211101	10-2322454	45	기술이전 준비중
29	네트워크 하이퍼바이저 및 이를 이용한 컨트롤 채널 대역폭 할당 방법	KR					유혁,양경식,유연호	20211101	10-2322451	90	기술이전 준비중
30	가상화 환경에서 네트워크 성능 차등화를 위한 동적 자원 할당 기법	KR	유혁, 이경운	20160114	10-2016-0004794					50	기술이전 준비중
31	가상환경의 스토리지 성능 향상을 위한 클라우드 스토리지 시스템 및 그 관리 방법	KR	유혁, 박준영, 박현찬, 고현석	20160324	10-2016-0035398					50	기술이전 준비중
32	CPU 네트워크 통합 스케줄링 시스템 및 그 제어방법	KR	유혁, 강동화, 박현찬, 이경운, 고현석	20160901	10-2016-0129691					50	기술이전 준비중
33	SDN 기반 네트워크 가상화 플랫폼에서의 IP 주소 가상화 방법 및 시스템	KR	유혁, 고영근, 양경식, 유봉열	20161001	10-2016-0121321					50	기술이전 준비중
34	고속 패킷 처리 시스템 및 그 제어방법	KR	유혁, 홍철호, 이경운	20161101	10-2016-0149422					50	기술이전 준비중
35	SDN 기반 경로 지연 시간 측정 방법	KR	유혁, 양경식, 김성문	20170601	10-2017-0074806					50	기술이전 준비중
36	가상 머신의 CPU 및 블록 I/O 작업에 성능 보장을 위한 자원 할당 시스템 및 방법	KR	유혁, 고현석, 이우진, 이경운	20170901	10-2017-0111836					50	기술이전 준비중
37	프로그래밍 가능한 네트워크 가상화에서의	KR	유혁, 양경식,	20180706	10-2018-0078870					50	기술이전

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	플로우 룰 가상화 장치 및 그 방법		유봉열		2						준비중
38	멀티코어 기반의 단말 장치에 적용되는 잠금 패턴의 보안 강도를 측정하는 장치 및 방법	KR	유혁, 김영필, 유시환, 이경운	20170826	10-2018-0087371					50	기술이전 준비중
39	가상화 환경에서의 성능 제어 방법 및 이를 위한 정보 처리 장치	KR	유혁, 이경운, 김수완	20190508	10-2019-0053729					50	기술이전 준비중
40	프로그래밍 가능한 네트워크 가상화에서의 대역폭을 고려한 장애우회 장치 및 그 방법	KR	유혁, 진희상, 양경식	20191001	10-2019-0121383					50	기술이전 준비중
41	프로그래머블 스위치에서의 패킷 스케줄링을 이용한 분산 딥러닝 방법	KR	유혁, 양경식, 강민구	20200429	10-2020-0052711					50	기술이전 준비중
42	프로그래밍 가능한 네트워크 가상화에서의 모니터링 방법 및 시스템	KR	유혁, 양경식, 강민구	20200428	10-2020-0051783					50	기술이전 준비중
43	네트워크 하이퍼바이저 및 이를 이용한 컨트롤 채널 대역폭 할당 방법	KR	유혁, 양경식, 유연호	20200519	10-2020-0059484					50	기술이전 준비중
44	컨테이너에 대한 리소스 간 의존성 스케줄링 방법 및 이를 수행하는 네트워크 시스템	KR	유혁,김은숙,이경운	20210901	10-2021-0116349					100	기술이전 준비중
45	전송 제어 프로토콜 연결 처리 장치 및 방법	KR	이준호, 양경식, 유연호, 유혁	20220218	10-2022-0021310					45	기술이전 준비중
46	프로그래밍 가능한 네트워크 가상화에서의 제어 트래픽 시계열 예측 기반의 제어 채널 고립 방법 및 시스템	KR	유연호, 양경식, 신창용, 이정환, 유혁	20221118	10-2022-0155709					50	기술이전 준비중
47	프로파일링 기반 분산 딥러닝 작업 실행 순서 조정 방법 및 장치	KR	신창용, 양경식, 유연호, 이정환, 유혁	20221117	10-2022-0154609					50	기술이전 준비중
48	분산 딥러닝 학습 시간 및 자원 소모량 예측 방법 및 시스템	KR	양경식, 신창용, 유연호, 이정환, 유혁	20221108	10-2022-0147865					50	기술이전 준비중

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

ㅇ 지식재산권 활용 유형

※ 활용의 경우 현재 활용 유형에 √ 표시, 미활용의 경우 향후 활용 예정 유형에 √ 표시합니다(최대 3개 중복선택 가능).

번호	제품화	방어	전용실시	통상실시	무상실시	매매/양도	상호실시	담보대출	투자	기타
1				V						
2			V	V		V				
3			V	V		V				
4			V	V		V				
5			V	V		V				
6			V	V		V				
7			V	V		V				
8			V	V		V				
9			V	V		V				
10			V	V		V				
11			V	V		V				
12			V	V		V				
13			V	V		V				
14			V	V		V				
15			V	V		V				
16			V	V		V				
17			V	V		V				
18			V	V		V				
19			V	V		V				
20				V						
21			V	V		V				
22			V	V		V				
23			V	V		V				
24			V	V		V				
25			V	V		V				
26			V	V		V				
27			V	V		V				
28			V	V		V				
29			V	V		V				
30			V	V		V				
31			V	V		V				
32			V	V		V				
33			V	V		V				
34			V	V		V				
35			V	V		V				
36			V	V		V				
37			V	V		V				
38			V	V		V				
39			V	V		V				
40			V	V		V				
41			V	V		V				
42			V	V		V				
43			V	V		V				
44			V	V		V				
45			V	V		V				
46			V	V		V				
47			V	V		V				
48			V	V		V				

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 저작권(소프트웨어, 서적 등)

번호	저작권명	창작일	저작자명	등록일	등록 번호	저작권자명	기여율
해당없음							

□ 신기술 지정

번호	명칭	출원일	고시일	보호 기간	지정 번호
해당없음					

□ 기술 및 제품 인증

번호	인증 분야	인증 기관	인증 내용		인증 획득일	국가명
			인증명	인증 번호		
해당없음						

□ 표준화

ㅇ 국제표준 및 사실표준

번호	표준화 기구명	표준화 분과명[1]	표준 종류[2]	표준 단계[3]		제안(기고)한 표준[4]		반영(채택)된 표준[5]		제안자	승인(채택) 일자
				구분	단계명	표준번호	표준명	표준번호	표준명		
해당없음											

* 1」 해당 표준을 진행한 표준화기구 분과명 및 분과번호를 하위 조직까지 기재합니다.
* 2」 제정 또는 개정 중 해당하는 사항을 기재합니다.
* 3」 국제표준 진행 단계 중 해당하는 사항을 기재합니다.
(예) ITU-T의 경우, 신규 작업항목 제안 채택(NWI), 표준문서개발(Draft Recommendation), 국제표준승인(Recommendation)
* 4」 제안(기고)한 표준문서가 제안/개발/승인 문서에 반영(채택)되었을 경우, 제안(Contribution)한 표준문서 정보를 기재합니다.
* 5」 제안(기고)한 표준문서가 제안/개발/승인 문서에 반영(채택)되었을 경우, 반영(TD/Recommendation)된 표준문서 정보를 기재합니다.

ㅇ 국가표준 및 단체표준

번호	표준화 기구명[1]	표준종류[2]	표준명	표준번호	제안자	승인(채택) 일자
해당없음						

* 1」 제정 또는 개정한 표준화기구를 국립전파연구원(RRA), 국가기술표준원(KATS), 한국정보통신기술협회(TTA) 등에서 해당하는 사항으로 기재합니다.
* 2」 제정 또는 개정 중 해당하는 사항을 기재합니다.

ㅇ 국제표준 정책기고

번호	표준화 기구명	표준화 정책위원회명[1]	문서종류[2]	문서반영 단계구분[3]	결의안/기고명	문서번호	제안자	승인(채택) 일자
해당 없음								

* 1」 정책위원회에 해당하는 ITU PP, ITU-T WTSA, ITU-T TSAG, ITU-R RA/WRC, ITU-R RAG/WP5D, ITU-D WTDC, ITU-D TDAG, APT, MC/APG/AWG/WTSA/ASTAP, JTC 1 총회 및 산하 자문그룹, ISO 총회, ISO TMB, IEC 총회, IEC SMB, 3GPP PCC/OP/TSGs, W3C 자문위원회 등에서 해당하는 사항으로 기재합니다.
* 2」 제정 또는 개정 중 해당하는 사항을 기재합니다.
* 3」 결의안, 국가선도기술 제안, 정책위원회 기고 반영의 진행 단계 중 해당하는 사항을 기재합니다.

ㅇ 오픈소스 연계 표준개발

번호	구분	표준문서 및 오픈소스 정보								과제검증 시험 결과서 번호[4]
해당 없음	표준	표준화 기구명	표준화 분과명[1]	표준 종류[2]	표준단계		반영(채택)된 표준[3]		승인(채택) 일자	
					구분	단계명	표준번호	표준명		
		3GPP	*SG5/WP1/Q2*	*제정*	*제안*	*NWI*				
	오픈 소스	프로젝트명			반영 소스명				승인(Commit) 일자	
		O-RAN								

* 1」 해당 표준을 진행한 표준화기구 분과명 및 분과번호를 하위 조직까지 기재합니다.
* 2」 제정 또는 개정 중 해당하는 사항을 기재합니다.
* 3」 제안(기고)한 표준문서가 제안/개발/승인 문서에 반영(채택)되었을 경우, 반영(TD/Recommendation)된 표준문서 정보를 기재합니다.
* 4」 표준문서와 오픈소스의 정합에 대하여 TTA에서 발급한 시험 결과서 번호를 기재합니다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

ㅇ 표준특허

번호	표준특허 구분[1]	표준							특허				
		표준화 기구	표준 단계[2]		표준명	표준번호	승인/개발 일자	IPR 선언 일자	특허 단계 구분[3]	특허명	특허번호	국명	출원/등록 일자
			구분	단계명									
해당 없음									*등록*				

* 1」 국내/국제 표준특허 승인 또는 후보 중 해당하는 사항을 기재합니다.
* 2」 국내/국제표준 진행 단계 중 해당하는 사항을 기재합니다.
* 3」 특허 출원, 등록 중 해당하는 사항을 기재합니다.

ㅇ 표준 국제협력

번호	의장단 수임						위원회/그룹 신설			국제회의 국내유치		
	표준화 기구명	표준화 분과명[1]	의장단 직위[2]	신규/계속 여부	수임자	수임기간	표준화 기구명	표준화 분과명[1]	신설일자	표준화 기구명	표준화 회의명	회의일자
해당 없음												

* 1」 해당 표준을 진행한 표준화기구 분과명 및 분과번호를 하위 조직까지 기재합니다.
* 2」 의장단 수임 직위를 의장, 부의장, 라포처, 컨비너, 에디터, 프로젝트리더 중에서 해당하는 사항으로 기재합니다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[경제적 성과]

□ 시제품 제작

번호	시제품명	출시/제작일	제작 업체명	설치 장소	이용 분야	사업화 소요 기간	인증기관 (해당 시)	인증일 (해당 시)
해당없음								

□ 기술 실시(이전)

번호	기술 이전 유형	기술 실시 계약명	기술 실시 대상 기관	기술 실시 발생일	기술료 (해당 연도 발생액)	누적 징수 현황
1	통상실시	Secure Container를 위한 경량 VM 구성 방안 탐색	LG전자	20220315		
2	통상실시	High-speed packet processing system and control method thereof	AST (Allied Security Trust I)	20221101	83100458	

* 내부 자금, 신용 대출, 담보 대출, 투자 유치, 기타 등

□ 산업체 연구비 수주

번호	산업체/ 기관명	협업주제	연구시작일
1	LG전자	Secure Container를 위한 경량 VM 구성 방안 탐색	20220315

□ 사업화 투자실적

번호	추가 연구개발 투자	설비 투자	기타 투자	합계	투자 자금 성격*
해당없음					

□ 사업화 현황

번호	사업화 방식[1」]	사업화 형태[2」]	지역[3」]	사업화명	내용	업체명	매출액		매출 발생 연도	기술 수명
							국내 (천원)	국외 (달러)		
해당없음										

* 1」 기술이전 또는 자기실시
* 2」 신제품 개발, 기존 제품 개선, 신공정 개발, 기존 공정 개선 등
* 3」 국내 또는 국외

□ 매출 실적(누적)

사업화명	발생 연도	매출액		합계	산정 방법
		국내(천원)	국외(달러)		
해당없음					
합계					

□ 사업화 계획 및 무역 수지 개선 효과

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<table>
<tr><td colspan="2">성과</td><td colspan="3">해당없음</td></tr>
<tr><td rowspan="8">사업화 계획</td><td colspan="2">사업화 소요기간(년)</td><td colspan="3"></td></tr>
<tr><td colspan="2">소요예산(천원)</td><td colspan="3"></td></tr>
<tr><td rowspan="2">예상 매출규모(천원)</td><td>현재까지</td><td>3년 후</td><td>5년 후</td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td rowspan="3">시장 점유율</td><td>단위(%)</td><td>현재까지</td><td>3년 후</td><td>5년 후</td></tr>
<tr><td>국내</td><td></td><td></td><td></td></tr>
<tr><td>국외</td><td></td><td></td><td></td></tr>
<tr><td colspan="2">향후 관련기술, 제품을 응용한 타 모델, 제품 개발계획</td><td colspan="3"></td></tr>
<tr><td rowspan="3">무역 수지 개선 효과(천원)</td><td rowspan="2">수입대체(내수)</td><td>현재</td><td>3년 후</td><td>5년 후</td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td>수출</td><td></td><td></td><td></td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 고용 창출

순번	사업화명	사업화 업체	고용창출 인원(명)		합계
			yyyy년	yyyy년	
해당없음					
합계					

□ 고용 효과 해당없음

구분			고용 효과(명)
고용 효과	개발 전	연구인력	
		생산인력	
	개발 후	연구인력	
		생산인력	

□ 비용 절감(누적)

순번	사업화명	발생연도	산정 방법	비용 절감액(천원)
해당없음				
합계				

□ 경제적 파급 효과

(단위: 천원/년)

구분	사업화명	수입 대체	수출 증대	매출 증대	생산성 향상	고용 창출 (인력 양성 수)	기타
해당 연도							
기대 목표							

□ 산업 지원(기술지도)

순번	내용	기간	참석 대상	장소	인원
해당없음					

□ 기술 무역

(단위: 천원)

번호	계약 연월	계약 기술명	계약 업체명	계약업체 국가	기 징수액	총 계약액	해당 연도 징수액	향후 예정액	수출/ 수입
해당없음									

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[사회적 성과]

□ 법령 반영

번호	구분 (법률/시행령)	활용 구분 (제정/개정)	명 칭	해당 조항	시행일	관리 부처	제정/개정 내용
해당없음							

□ 정책활용 내용

번호	구분 (제안/채택)	정책명	관련 기관 (담당 부서)	활용 연도	채택 내용
해당없음					

□ 설계 기준/설명서(시방서)/지침/안내서에 반영

번호	구 분 (설계 기준/설명서/지침/안내서)	활용 구분 (신규/개선)	설계 기준/설명서/ 지침/안내서 명칭	반영일	반영 내용
해당없음					

□ 전문 연구 인력 양성

번호	분류	기준 연도	현황										
			학위별				성별		지역별				
			박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
1	안종현	2015			V		V		V				
2	황태경	2015	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V			V	V				
3	김세원	2015	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
			V				V		V				
4	이치영	2015	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
			V				V		V				
5	오석영	2015	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
6	임재민	2015	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
7	박준영	2015	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
8	박현찬	2016	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
			V				V					V	
9	이상헌	2016	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V			V			
10	강동화	2016	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
11	고영근	2016	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V				V		
12	유봉열	2016	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V		V		V				
13	고현석	2017	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
14	정원태	2017	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
15	이시오	2017	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
16	이우진	2017	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V		V		V				
17	최흥식	2017	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
18	조유현	2016	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V		V		V				
19	유소망	2016	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V				V	V				
20	유봉열	2019	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
21	양경식	2019	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
			V				V		V				
22	진희상	2019	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V			V			

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

23	김동은	2019	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V		V			V			
24	ANUMEHA	2020	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V				V	V				
25	김수완	2020	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
26	김문교	2020	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V		V		V				
27	이경운	2020	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
			V					V			V		
28	김정수	2021	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V				V		V			
29	이관훈	2021	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
30	유연호	2021	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
31	이준호	2021	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V			V		V				
32	김은숙	2021	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
				V				V	V				
33	구인회	2021	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V		V			V			
34	이상엽	2022	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V		V		V				
35	김호석	2022	박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타
					V		V		V				

① 참여인력(수혜인력) 수

사업연도	연 구 책임자		참여학생						연구 전담 교수		post-doc		산업체		기타 (전임연구원 등)		계	
			학사 과정		석사 과정		박사 과정											
	남	여	남	여	남	여	남	여	남	여	남	여	남	여	남	여	남	여
2015	1	0	0	0	9	1	1	1	0	0	3	0	0	0	0	0	14	2
2016	1	0	0	0	10	1	0	1	0	0	2	0	0	0	0	0	13	2
2017	1	0	0	0	6	0	2	1	0	0	1	0	0	0	0	0	10	1
2018	1	0	2	1	3	1	2	1	0	0	1	0	0	0	0	0	9	3
2019	1	0	3	0	7	1	2	1	0	0	1	0	0	0	0	0	14	2
2020	1	0	8	1	5	4	0	0	0	0	1	1	0	0	0	0	15	6
2021	1	0	2	0	5	4	1	0	0	0	1	1	0	0	0	0	10	5
2022	1	0	3	0	1	1	4	2	0	0	0	0	0	0	0	1	9	4

② 학위별 배출인력 수

사업연도	학사학위		석사학위		박사학위		계	
	목표	실적	목표	실적	목표	실적	목표	실적
2015	1	2	2	3	1	2	4	7
2016	2	1	2	5	0	1	4	7
2017	1	0	2	5	1	0	4	5
2018	2	0	2	0	0	0	4	0
2019	2	1	1	2	1	1	4	4
2020	1	1	1	2	0	1	3	4
2021	2	1	1	5	1	0	4	6
2022	1	2	2	0	0	0	3	2

③ 배출인력 진로현황

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

사업연도	산업체			석사진학 (B)	박사진학 (C)	연구소 (D)	Post-Doc (E)	정부기관 (F)	학계 (G)	기타취업 (H)	창업 (I)	군입대 (J)	미취업 (K)	배출인원 합계 L:A~K	취업대상 M: L-B-C-E-J	취업률 (M-K/M)	전공분야 취업인원 (N)	전공취업률 (N/(M-K))
	대기업	중소·중견기업	소계 (A)															
2015	4	0	4	1	1	0	0	1	0	0	0	0	0	7	5	100	5	100
2016	2	0	2	2	0	2	0	0	1	0	0	0	0	7	5	100	5	100
2017	3	2	5	0	0	0	0	0	0	0	0	0	0	5	5	100	5	100
2018	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2019	1	1	2	1	0	0	0	0	1	0	0	0	0	4	3	100	3	100
2020	2	1	3	0	0	0	1	0	0	0	0	0	0	4	3	100	3	100
2021	0	0	0	1	3	2	0	0	0	0	0	0	0	6	2	100	2	100
2022	0	1	1	1	0	0	0	0	0	0	0	0	0	2	1	100	1	100
합계	12	5	17	6	4	4	1	1	2	0	0	0	0	35	24	100	24	100

④ 인력양성 추진실적의 우수성

○ 본 연구과제를 통해 35명의 우수한 인력을 양성 및 배출하였음.
그 중 취업대상 인원 24명 모두 전공분야로 취업하여, 전공 취업률 100%를 달성함.

○ 신진연구인력 및 박사과정 배출인력

- 성균관대(황재현), 전북대(박현찬), 경북대(이경운) **정년트랙 전임교수 3명**이 배출되었고, 3명 모두 클라우드 분야 연구자로 임용되어 본 연구과제와 연계된 후학 양성에 매진하고 있음
- 박사과정 및 박사후과정 1명(양경식)는 과정 중 분야 내 최고 학술대회인 IEEE INFOCOM, ACM SIGMETRICS 등 한국정보과학회 및 연구재단 기준 SCIE인정 학술대회에 주저자로 10편 이상의 논문을 발표하였음
- 박사과정 1명(유연호) 우수한 연구실적을 통해 박사과정 1명 BK 4단계 BK21 미래인재 펠로우십 선정됨
- 박사과정 2명은 2018년(양경식), 2022년(유연호)에 Microsoft Research Asia 연구인턴으로 선발되어 국제 공동연구를 수행하고, 우수한 성과를 도출하였음. 특히, 한국인 최초로 MSRA Student TechFest에서 입상하고 우수인턴상을 수상함.

○ 석사/학사과정 배출인력

- 전원 코스콤, 카카오, 네이버, 삼성전자, KT, LG전자 등 최선호 직장으로 취업 달성
- 특히 석사과정 중 ETRI 2명, 국방과학연구소 3명, KIST 1명 등 **정부출연연구소 연구원 6명을 배출**하여, 연구인력 양성을 성공적으로 수행하고 있음을 입증하였음
- 석사과정 배출인력 중 2명(유봉열 석사, 진희상 석사)은 과정 중 IEEE 및 ACM 국제 학술대회에 논문을 발표하고 Best paper award를 수상하는 등 우수한 연구인력으로 입증받음
- 석사/학사과정 중 한국정보과학회 등 다양한 학술대회에 논문을 발표하고, 11건 이상 우수논문상을 수상함

□ 산업 기술 인력 양성

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	프로그램명	프로그램 내용	교육 기관	교육 개최 횟수	총 교육 시간	총 교육 인원
해당없음						

□ 다른 국가연구개발사업에의 활용

번호	중앙행정기관명	사업명	연구개발과제명	연구책임자	연구개발비
해당없음					

□ 국제화 협력성과

번호	구분 (유치/파견)	기간	국가	학위	전공	내용
해당없음						

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 홍보 실적

번호	홍보 유형	매체명	제목	홍보일
1	언론보도	이엠디뉴스	네트워크 가상화 세계적 원천 기술 확보, 시연 진행	2020
2	언론보도	대학저널	고려대 유혁 교수팀, 네트워크 가상화 세계적 원천 기술 확보, 시연 진행	2020
3	언론보도	한국대학신문	고려대 유혁 교수팀, 네트워크 가상화 세계적 원천 기술 확보, 시연 진행	2020
4	언론보도	에너지경제신문	고려대 유혁 교수팀, 네트워크 가상화 세계적 원천 기술 확보, 시연 진행	2020
5	언론보도	베리타스알파	고려대 유혁 교수팀, 네트워크 가상화 세계적 원천 기술 확보	2020
6	언론보도	한국강사신문	고려대학교 컴퓨터학과 유혁 교수팀, '프로그래밍 가능한' 네트워크 가상화 기술 개발	2020
7	기술워크샵	한국정보과학회	제8회 가상화 워크샵	2020
8	언론보도	스마트경제	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
9	언론보도	메트로신문	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
10	언론보도	교수신문	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
11	언론보도	핀포인트뉴스	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
12	언론보도	뉴시스	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
13	언론보도	전자신문	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
14	언론보도	한국대학신문	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
15	언론보도	이데일리	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
16	언론보도	뉴스1	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
17	언론보도	인공지능신문	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
18	언론보도	이슈메이커	'네트워크 가상화' 세계적 원천기술 확보	2021
19	언론보도	내외통신	2021년 국가연구개발 우수성과 100선 선정 홍보	2021
20	언론보도	전자신문	[미래 여는 ICT R&D 우수성과]<중>IITP 지원으로 네트워크 가상화 기술, 5G 적합 광트랜시버 개발	2022

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 포상 및 수상 실적

번호	종류	포상명	포상 내용	포상 대상	포상일	포상 기관
1	국가R&D 포상	2021 국가연구우수성과 정보전자분야 최우수	본 연구과제의 질적 우수성을 인정받아 수만 여 국책과제 중 정보분야 최우수 1개 과제로 선정	유혁	2021	과학기술정보통신부
2	수상	2021 SW R&D 우수과제 장관상	본 연구과제의 우수 수행 및 성과로 인해 IITP 과제 중 장관상 선정 및 표창	유혁	2021	과학기술정보통신부
3	수상	4단계 BK21 미래인재 펠로우십	과제 참여인력인 유연호 박사과정 우수 연구수행으로 BK 미래인재 펠로우십 선정	유연호	2022	교육부/고려대학교
4	수상	Most Feasibility Award	Microsoft Student TechFest에서 연구인턴 중 한국인 최초 입상	양경식	2018	Microsoft Research Asia
5	수상	The Excellence Award	Microsoft Research Asia 최우수 연구인턴으로 선정되어 수상	양경식	2018	Microsoft Research Asia
6	수상	Best paper award	CoNEXT 워크샵 우수논문상	유봉열	2016	ACM
7	수상	Top paper award	ICOIN 학술대회 우수논문상	진희상	2019	IEEE
8	수상	우수논문상	우수 대학원생 논문 포상	이경운	2019	고려대학교
9	수상	정보과학회 학부생논문경진대회 최우수상	KCC 2019 우수논문상	김동은	2019	한국정보과학회
10	수상	정보과학회 우수발표논문상	KCC 2019 우수논문상	김수완	2019	한국정보과학회
11	수상	KSC2019 학부생/주니어논문경진대회 우수논문상	우수논문상	신창용	2019	한국정보과학회
12	수상	KSC2019 우수발표논문상	우수논문상	Anumeha	2019	한국정보과학회
13	수상	KCC2020 학부생/주니어논문경진대회 장려상	우수논문상	김문교	2020	한국정보과학회
14	수상	KCC2020 학부생/주니어논문경진대회 장려상	우수논문상	김효경	2020	한국정보과학회
15	수상	KCC2020 학부생/주니어논문경진대회 장려상	우수논문상	임재민	2020	한국정보과학회
16	수상	KCC2020 학부생/주니어논	우수논문상	최원미	2020	한국정보과학회

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

		문경진대회 장려상				
17	수상	2021 한국소프트웨어종합학술대회(KSC2021) 특별공로상	본 연구실의 연구성과를 인정받아 포상	유혁	2021	한국정보과학회
18	수상	KSC2021 학부생/주니어논문경진대회 최우수상	우수논문상	최예찬	2022	한국정보과학회
19	수상	KSC2021 학부생/주니어논문경진대회 장려상	우수논문상	김호석	2022	한국정보과학회

[인프라 성과]

□ 연구시설・장비

구축기관	연구시설/연구장비명	규격(모델명)	개발여부(○/×)	연구시설・장비 종합정보시스템* 등록여부	연구시설・장비 종합정보시스템* 등록번호	구축일자(YY.MM.DD)	구축비용(천원)	비고(설치 장소)
	해당없음							

* 「과학기술기본법 시행령」 제42조제4항제2호에 따른 연구시설・장비 종합정보시스템을 의미합니다.

<국가연구개발 시설·장비의 관리 등에 관한 표준지침>
제20조(자산등재) ① **연구기관의 장**은 도입을 완료한 연구시설장비에 대하여 관계법령에 따라 **기관자산으로 등재**하고 **자산등록 관리번호를 부여**하여야 한다.
② 연구기관의 장은 도입 완료 후 자산등록 관리번호가 포함된 **자산비표를 발급**하여 **부착**하고 **연구시설장비 이력카드를 관리**하여야 한다.
제21조(정보등록) ① 연구기관의 장은 공동관리규정 제25조제3항 및 제5항에 따라 **3천만원 이상 연구시설장비**와 3천만원 미만이라도 **공동활용이 가능한 연구시설장비**를 제20조에 준하여 **자산등재 완료 후 30일 이내**에 **ZEUS에 등록**하여야 한다. 이때 등록대상이 되는 연구시설장비의 기준은 별표3의1에서 정한 바에 따른다.
② 연구기관의 장은 연구시설장비 도입 완료 후 ZEUS에서 발급하는 국가연구시설장비정보등록증의 **발급여부를 점검**하여야 한다.
③ 연구기관의 장은 ZEUS에 등록된 연구시설장비의 **전자인식표** 등을 지원기관으로부터 발급받아 해당 연구시설장비에 **부착**하여 관리하여야 한다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[공개SW 성과]

□ 공개SW성과

사업연도	지표1)	목표	결과2)		내용(자체평가 포함)
			실적	정성적 평가	
2015	해당없음	해당없음	해당없음	해당없음	2015년 1차년도는 공개sw 관련 지표 설정 및 목표 제안이 없어, 별도의 추산이 없었고 2차년도부터 구체적인 목표수립 및 지표 수집이 시작되었음.
2016	SW결과물의 공개 여부	1	1	상	공개된SW 결과물이 목표치를 충족하였으며, 공개 접근 및 코드의 유지가 가능한 공개SW 사이트를 통해 배포 되었음
	기술 문서 배포 여부	4	4	상	공개된 SW의 활용과 이해를 돕고, 재생산 및 개선을 위해 필요한 기술 문서가 목표치에 맞게 작성/공개되었음.
	공개SW 참여 활동 여부	2	2	상	공개SW 및 커뮤니티 활성화에 관련된 외부 워크샵, 행사 등에 참석하였음
	공개 워크샵 개최 여부	1	1	상	과제의 핵심 기술 개발과 품질 향상 및 피드백이 가능한 수준의 인력을 바탕으로 연구 기술 워크샵을 개최하였음
2017	SW 결과물의 공개 여부	1	1	상	당해연도의 연구수행 결과인 GoS를 공개 SW 배포 사이트인 소스포지(http://sourceforge.net) 를 통해 공개, 연구 내용을 기반으로 오픈스택 기반 클라우드 구축 (http://stella.korea.ac.kr/horizon)
	기술 문서 배포 여부	4	4	상	당해연도의 기술 문서 4건을 본 과제의 연구진이 운영하는 공개 사이트인 스텔라 홈페이지(stella.korea.ac.kr)를 통해 공개함.
	공개 SW 참여 활동 여부	행사 참여 2건, 관련 프로젝트 등록 1건	행사 참여 2건, 관련 프로젝트 등록 1건	상	세계 최대 오픈소스 클라우드 컨퍼런스인 오픈스택 서밋 참가 2건(보스턴, 시드니), 국내 오픈소스 클라우드 컨퍼런스인 오픈스택 데이 서울 참가 1건, Stella를 위한 오픈스택 관련 프로젝트를 런치패드(https://launchpad.net/) 등록하여 오픈스택 공식 프로젝트에 기여할 수 있는 초석 마련하고 이와 관련하여 오픈스택 서밋 시드니 발표 신청, 클라우드 성능 향상 기능 제안 및 리눅스 커널 코드 기여.
	공개 워크샵 개최 여부	1	1	상	스타랩을 민간에 홍보하고, 연구성과를 알리기 위해 SWRC 포럼을 준비 및 개최함
2018	SW 결과물의 공개 여부	1	4	상	1. Stella-Scheduler 2. Stella-OpenStack 3. Stella-Horizon 4. Network QoS module for Docker
	기술 문서 배포 여부	4	4	상	1. KU-OS-TR-2018-01 2. KU-OS-TR-2018-02 3. KU-OS-TR-2018-03 4. KU-OS-TR-2018-04
	공개 SW 참여 활동 여부	행사 참여: 연 2건 관련프로젝트 등록: 1건	행사 참여 및 발표 1건, 발표 예정 1건, 관련프로젝트 등록 완료 및 유지중	상	행사 1. OpenInfraDays 2018 발표 진행(2018-06-28) 행사 2. 오픈스택 국내 커뮤니티 정기 세미나 참석 및 발표 진행예정 (예정일: 12월 7일) 관련프로젝트 1. KU.Stella (https://launchpad.net/ku.stella)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

사업연도	지표1)	목표	결과2)		내용(자체평가 포함)
			실적	정성적 평가	
2019	저장소	1	7	상	연구 개발물 또는 이와 관련된 프로젝트의 공개 저장소를 매년 1개 이상 지속적으로 운영
	기술 문서 배포	4	4	상	사용자 매뉴얼, 설치 메뉴얼 연구 동향 같은 프로젝트 관련 문서를 스텔라 홈페이지를 통해 배포
	공개 SW 참여활동	1	1	상	클라우드와 관련된 공개 SW 커뮤니티의 행사 참석해서 개발자들과 교류
	스타수	20	60	상	과제 관련 저장소에 대해 총 50개 이상의 스타 확보
	커미트 수	50	424	상	Github 등 저장소 커밋 횟수를 포함한 오픈 소스 데이터베이스에 기여한 커밋 횟수
	Fork 수	10	53	상	Github 등 과제와 관련 있는 저장소 fork 수
	홍보	1	1	상	공개 SW 관련 커뮤니티 행사에서 발표 진행 등
2020	저장소	1	5	상	매년 새로운 SW 개발 및 Github을 통한 경과물 공개를 통해 공개SW 커뮤니티 활성화 노력
	기술문서배포	4	4	상	사용자 매뉴얼, 연구 동향 조사 및 연구 과제 수행에 따른 결과물 및 관련 문서를 과제 홈페이지를 통해 배포
	공개SW참여활동	1	1	상	클라우드와 관련된 공개 SW 행사에 참여하여 커뮤니티 개발자들과 연구 내용 공유 및 최신 개발 동향 조사
	스타수	20	179	상	논문을 통한 연구 결과물의 홍보를 통해 공개SW 저장소에 대한 관심 유도
	커미트수	50	5063	상	최근 활용도 및 관심이 높아지고 있는 컨테이너 기반의 클라우드 플랫폼인 쿠버네티스로의 이전을 통해 활발한 커뮤니티 참여 유도 및 공개SW 활성화
	Fork수	10	163	상	논문을 통한 연구 결과물의 홍보를 통해 공개SW 저장소에 대한 관심 유도
	홍보	1	1	상	본 연구 과제를 알리고 클라우드 관련 기술 동향 파악 및 전문가와의 교류를 위해 워크샵 개최
2021	저장소	1	1	상	끊임없는 기술 개발에 매진하여 추가 공개SW 기반의 기술 개발 및 저장소 관리
	기술 문서 배포	4	4	상	블록체인 및 클라우드 분야의 최신 동향 기술 연구 및 조사 내용 기술
	공개 SW 참여활동	1	2	상	한국정보과학회 학술대회 부속 워크샵 개최를 통해 연구 개발물의 성과 홍보 및 연구 교류 활동
	스타수	20	26	상	기존 목표치를 상회하는 스타수 달성을 통해 연구 개발물의 공개 SW를 통한 기술 공유에 매진
	커미트 수	50	99	상	활발한 공개 SW 개발 활동을 통해 연구 개발물의 질적 완성도 향상 노력
	Fork 수	10	17	상	학술대회를 통한 연구 개발물의 홍보를 통해 공개 SW의 외부 공유 활성화
	홍보	1	5	상	외부 매체를 통해 연구 개발 결과물의 우수성 홍보

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

사업연도	지표1)	목표	결과2)		내용(자체평가 포함)
			실적	정성적 평가	
2022	저장소	1	1	상	8년간 연구를 지속/발전하여 인공지능 기반 클라우드 SLA 제어 및 보장 기법으로 발전하고 결과물을 오픈소스SW로 공개
	기술 문서 배포	4	4	상	클라우드 성능 보장, 경량 컨테이너, GPU 성능 보장 등 최신 기술 동향을 포함한 기술문서 4건 산출
	공개 SW 참여활동	1	1	상	연구논문의 artifact evaluation에 참여하여, 국제 수준의 공개SW 품질을 검증받고 외국 연구자들과 공개SW 개발을 함께 수행함
	스타수	20	84	상	국제 학술대회 및 다양한 워크샵 참여 등을 통해 목표치를상회하는 스타수 달성
	커미트 수	50	71	상	연구개발 중 코드구현 형상관리를 꾸준히 수행하여, 목표수를 초과하여 커미트 수 달성
	Fork 수	10	88	상	월 1,000회 수준 조회되는 공개sw 커뮤니티 사이트를 포함하여, 8개년 간 지속된 노력에 힘입어 SW 포크수 목표 대비 8배 이상 초과 달성
	홍보	1	1	상	고려대학교 내 산학 연계 워크샵을 2회 추가 개최하여 공개SW의 활성화를 도모함

□ 공개SW R&D활성화 성과

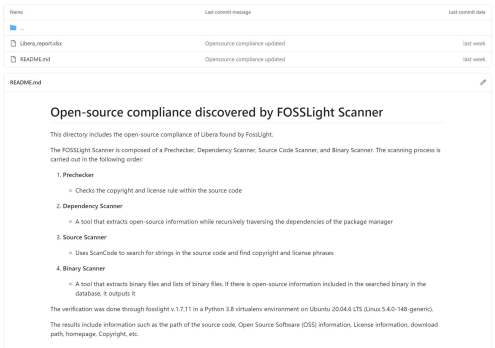
- 본 연구진은 8개년 간의 SW스타랩 과제를 수행하는 동안, 산출되는 다수의 연구성과를 오픈소스 SW의 형태로 공개하였음.
- 특히 연구논문의 경우, 대다수의 결과를 Github 레파지토리 등으로 공개하고, 개발 과정의 형상관리를 Github 커밋과 push/pull 등의 기능을 적극적으로 활용하였음.
- 더 나아가, 공개SW를 활성화하기 위한 산학워크샵을 매년 1회 수행하였고, 일부 논문은 공개된 코드를 artifact submission 및 evaluation에 제출하여 코드를 국제적으로 평가받고 공개할 수 있도록 검증받았음.
- 그 결과, 최종 연차인 8차년도에는 스타수, 커미트수, Fork 수 모두 목표치 대비 초과 달성하였음.
- 특히 코드를 개인 계정으로 복제하여, 추가 개발을 진행하는 Fork 수의 경우 목표 대비 8.8배 초과하였고, 이는 개발한 결과물이 다양한 연구자들의 후속 연구로 이어져 활성화되고 있음을 간접적으로 드러냄.
- 특히, 연구한 기술 중 네트워크 성능 보장 및 가상화 기술의 경우 그 우수성을 입증받아 미국 유관 분야 오픈소스 소프트웨어 및 표준을 주도하는 Open Networking Foundation과 협의하에, OpenVirteX라는 국제적인 소프트웨어의 소유권과 커뮤니티 운영권한을 이양받아 국제적인 원천기술을 보유 및 운영중임.
- 상기 소프트웨어 및 본 연구진의 오픈소스SW 커뮤니티는 월 2,000회 이상 조회되는 수준으로 성장하였으며, 이러한 활성화 성과를 인정받아 **2021년 국가R&D 우수성과 중 최우수 성과로 전자/정보 분야의 IT과제로 유일하게 선정**되었음.

□ 공개SW R&D전략의 적절성 및 활성화 노력

- 본 과제의 공개SW R&D 주요 결과물은 크게 두 가지로, 1단계의 성능 SLA 보장 노드 수준 스케줄러, 2단계의 성능 및 보안 SLA 보장 클라우드 시스템임. 특히 2단계는, 최신 보안 기능인 블록체인 서비스 (BaaS)를 대상으로 보안 SLA를 연구하였음.
- 1단계 결과물의 활성화를 위해, 국내 오픈스택 커뮤니티의 주관행사(https://www.openinfradays.kr/)에서 결과물의 기술적 내용과 더불어 공개SW의 설치 및 참여

방법 등을 포함 정보 공개를 하고, 스텔라 프로젝트 페이지, 유투브(youtube), 공개 워크샵에서 홍보 및 마이크로소프트, ETH 등 국외 저명 연구소 및 삼성, KT 등 기업 내 기술세미나 등을 진행했음.

- 2단계 결과물의 활성화를 위해서, 1단계의 활성화 방안들을 기본으로 수행하고, 확장된 연구 주제인 블록체인 서비스의 보안과 관련된 공개SW 커뮤니티 교류(e.g. https://www.hyperledger.org/), 블록체인 및 클라우드 국외 컨퍼런스 발표(e.g. IEEE Blockchain), 블록체인 클라우드 서비스 보안 관련 세미나 개최 등을 추가로 진행하였음.
- 더불어, 2단계에서는 공개SW를 더욱 적극적으로 활성화하기 위해 정보과학회 최우수국제학술대회 발표를 더욱 적극적으로 추진하여 목표치로 산정하지 않았던 최우수 학술대회 3건 및 우수학술대회 1건을 추가로 산출하였고, 공개SW를 홍보하였음
- 또한 매년 가상화 워크샵을 개최하여 삼성전자, SK텔레콤, LG전자, SK하이닉스 등 개발기술의 유관기업에 연구결과물을 공개 및 전파할 수 있도록 노력하였음
- 마지막으로 공개SW R&D 결과물의 실용적인 활용을 위해 Digital healthcare와 같은 클라우드 기반의 실제 서비스 플랫폼에 적용하기 위한 후속연구를 추진 및 수행중임
- 추가로 공개SW를 정리하여 공개하기 위해, FossLight 도구를 활용하여 공개 SW에 대해 소스코드, 바이너리, 디펜던시 분석 결과와 저작권/라이선스 분석하였음. 분석한 결과들은 모두 공개SW 레파지토리에 업로드하였으며, 업로드된 결과의 예시는 아래 그림과 같음.





[그 밖의 성과]

- 초기 계획하지 않았으나, 연구 성과물의 우수성을 인정받아 국내 LG 전자 및 미국 기업에 $80,000불 규모의 기술이전을 달성함

- 연구 성과 및 연구 기술의 우수성을 인정받아 Google로부터 Cloud 연구비용 $57,200을 지원받음

(4) 계획하지 않은 성과 및 관련 분야 기여사항

해당없음 (상기 그 밖의 성과와 동일함)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2) 목표 달성 수준

추 진 목 표	달 성 내 용	달성도(%)
○ 이종 멀티 테넌트 환경에서 보안 SLA 보장	- 클라우드 레벨 보안 SLA 제공으로 인한 CLS 저하율 1% 미만 달성	100
	- 클라우드 레벨 보안 SLA 모니터링 신뢰성 100% 달성	100
○ 이종 멀티 테넌트 환경에서 복합 성능 SLA 보장	- 클라우드 환경에서 CPU, 네트워크, 스토리지 성능에 대한 SLA 만족도 최대 99.9% 달성	100



kolas.kbet.or.kr 발행기관 사전동의 없이 전부 혹은 일부 복사 불가

G4B(www.g4b.go.kr)진위확인코드 : HLKGwAxiQb4=

한국건물에너지기술원 Korea Institute of Building Energy Technology

시험확인 사진본 2022-12-01 17:47:38 KBET 경북시험확인센터

시 험 성 적 서

한국건물에너지기술원 서울 서초구 반포대로30길 81 웅진타워 8F (Tel:02-525-1025, Fax:02-555-3152)	성적서번호 : 2022-E-0015 페이지(1)/총(9)	KBeT

1. 신청기관
   - 회 사 명 : 고려대학교 산학협력단
   - 대 표 자 : 조석주
   - 주 소 : 서울특별시 성북구 안암로 145(안암동5가, 고려대학교내,)
2. 시험성적서의 용도 : 정보통신기획평가원 제출용
3. 시험대상 제품명 및 버전 : CPU-네트워크 SLA 보장기법 v1.0
4. 접수일자 : 2022.11.09
5. 시험일자 : 2022.11.22 - 2022.12.01
6. 시험환경
   시험장소 : 현장시험실(주소: 서울특별시 성북구 안암로 145 자연계지역 우정정보관 308호)
   온도 : (23.0 ± 1.0) ℃ 습도 : (43 ± 4) % R.H.
7. 시험방법 : 신청기관 제시 기준 및 방법
8. 시험결과 : 시험결과 참고

확 인	작성자 성 명 : 권오철	기술책임자 성 명 : 김재훈

- 이 성적서는 신청기관이 제공한 시료 및 시료 명으로만 시험한 결과로서, 제공된 시료 외 전체 제품에 대한 품질을 보증하지는 않습니다.
- 이 성적서는 당 기관의 사전 서면 동의 없이 홍보, 선전, 광고 및 소송용으로 사용될 수 없으며, 용도 이외의 사용을 금합니다.
- 이 성적서는 신청기관이 제시한 시험방법에 의한 시험결과입니다.
- 이 성적서는 KS Q ISO/IEC 17025와 KOLAS 인정과 관련이 없습니다.

2022년 12월 1일

한국건물에너지기술원장

KBeT-QIF-04-05(00) 한국건물에너지기술원

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

평가의 착안점 및 기준

○ 개발기술의 평가 방법 및 평가 항목

< 연구개발성과 성능지표 >

평가 항목 (주요성능[1])			단위	전체 항목에서 차지하는 비중[2] (%)	세계 최고		연구개발 전 국내 성능수준	연구개발 목표치	달성도	평가 방법[3]
					보유국/보유기관	성능수준	성능수준	최종		
1	보안 SLA	보안 SLA 제공으로 인한 CLS저하율	%	20	-	-	-	1	달성	공인인증 및 자체평가
2		보안 SLA 모니터링 수준	%	10	미국	35*	-	100		
3	복합성능 SLA 보장 (성능 SLA)	CPU-스토리지 CLS	%	20		96*	75	99.9		
4		CPU-네트워크 CLS	%	20		96*	38	99.9		
5		CPU-스토리지, 네트워크 복합 CLS	%	30	-	-	56	95.9		

[보안 SLA 모니터링 수준] Moustafa, Saadeldin, et al. “Slam: SLA monitoring framework for federated cloud services.“ 2015 IEEE/ACM 8th International Conference on Utility and Cloud Computing (UCC). IEEE, 2015.

[CPU-스토리지 CLS] Li, Youhuizi, et al. “PINE: Optimizing performance isolation in container environments.“ IEEE Access 7 (2019): 30410-30422.

[CPU-네트워크 CLS] Khalid, Junaid, et al. “Iron: Isolating Network-based CPU in Container Environments.“ 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18). 2018.

① 기존 클라우드 서비스 (구글 클라우드, Amazon, Microsoft)에서 지원하는 SLA 지표

- SLA (Service level agreement) - 클라우드 사용자와 서비스 제공자 간 명시된 요구 성능 (service level objectives, SLO) 및 제공 성능에 대한 합의 사항.
- Availability - 99.95% Monthly Uptime (월별 가동시간)
- 성능 SLA - CPU, 스토리지, 네트워크 각각의 최대 용량
  - CPU: 가상머신이 구동 가능한 가상 CPU의 성능
  - 스토리지: 가상머신의 최대 가용 저장공간 (e.g. $0.17/1GB, 임시 저장소 240GB)
  - 네트워크: 가상머신의 최대 가용 네트워크 전송량 (e.g. 450 ~ 10000Mbps)

② 본 연구과제에서 추가로 제공하는 SLA 지표

- CLS (Customer level satisfaction) - SLA에 명시된 SLO에 대한 제공 성능의 만족도 비율. 기존 SLA는 달성/위반으로 평가하므로, 그 달성 수준을 구체적으로 보이기가 어려워 CLS를 추가적인 지표로 산정함.

③ 공인시험평가 및 자체 평가 및 기존 기술과의 비교

- 정량적 평가 항목의 공인시험평가의 경우 공인시험 성적서를 제출함.
- 공인시험 성적서와 자체 평가 보고서에는 정량적 수치 지표의 평가환경에 대한 설명과 달성 근거를 포함하도록 하며 기존 기법과의 성능 비교 수치를 제시함.
- 보안 성능 SLA 및 복합 성능 SLA 보장 항목의 경우 최종 목표치 및 본 기술을 통한 성능 달성 정도 및 CLS 달성 정도에 관한 내용을 포함하도록 함.

④ 항목별 평가방법

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1) 보안 SLA

- 보안 SLA 제공으로 인한 CLS 저하: CPU 등 일반적인 컴퓨팅 자원을 집중적으로 사용하는 워크로드가 구동되는 상황에서, 보안 SLA 제공을 위한 추가적인 서비스 (가령, Hyperledger Fabric)를 구동한다. 이 때, 보안 서비스로 인한 성능 및 보안 품질(CLS)의 저하 유무 (가령, TPS 저하, latency 증가)를 확인하고 저하된 경우 해당 수치를 측정한다. 즉, 최초 요구 성능 대비 보안 서비스의 유무에 따른 성능 수치를 기준으로 CLS를 계산하고, 보안 서비스 구동으로 인해 저하된 CLS 도출하여 저하량의 비율을 계산한다.
- 보안 SLA 모니터링 수준: 보안 SLA의 평가는 주기적인 모니터링을 통해 이루어져야 하며, 이는 보안 기능의 성능 보장을 위해 선행되어야 할 주요 기술임. 모니터링은 단발성이 아닌 주기적인 연산이 되므로, 되도록 적은 오버헤드를 가져야 하며, 모니터링 연산은 최대한 실패 없이 실행되어야 함. 보안 SLA 모니터링 수준은 이 두 가지를 평가하는 항목이며, 보안 SLA의 CLS를 모니터링 오버헤드와 신뢰도로 구성함. 모니터링 오버헤드는 낮을수록 좋고, 신뢰도는 높을수록 좋으므로, 모니터링 수준의 평가는 (모니터링 신뢰도) / (모니터링 오버헤드) 비율의 퍼센티지로 도출함. 실험의 경우, 보안 서비스에 필요한 블록체인 내의 transaction의 throughput과 latency를 가장 낮은 주기로 측정하는 방식, 즉 모니터링 신뢰도가 1일 때 발생하는 모니터링 오버헤드의 비율로서 산출한다.

2) 복합 성능 SLA

- 다수 물리머신이 10GbE 로 연결된 small cloud 환경에서, 컴퓨팅 머신(컨테이너 또는 가상머신)을 생성하고 연구기술들을 적용하여, 복합성능 SLA의 개선치를 측정한다
- 이 때, 사용자는 자신이 사용할 총 3개의 컴퓨팅 머신을 생성하고 (pod 1~3), 각 pod에서 각기 다른 벤치마크를 구동한다. 특히, 각 pod이 완전히 다른 종류의 자원을 intensive하게 사용하는 상황, 즉 pod1은 CPU(sysbench), pod2는 storage (fio), pod3은 network(netperf) 자원을 집중 사용한다. 이 때, 3개의 pod이 동시 구동되는 상황에서도 목표하는 복합 성능을 모두 달성하는 지 판단한다.
- K8s를 기반으로, pod 내 컨테이너는 자원 부족 시 scale up이 가능하나, 추가 물리 자원이 부여되지 않고 제한된 환경에서도 성능이 목표를 달성하는 지 측정하기 위해 각 pod은 하나의 컨테이너만 사용하도록 제한한다.
- 정량적 목표항목에 따라 서로 다른 pod를 동시에 실행한다.
o CPU-스토리지 CLS – pod1, pod2
o CPU-네트워크 CLS – pod1, pod3
o CPU-스토리지-네트워크 복합 CLS – pod1, pod2, pod3
- 각 자원에 대한 요구 성능 만족도는 다음과 같이 산출한다.
o CPU: 가상머신의 CPU 요구 성능 (1 CPU=100% 기준) 대비 실제 사용량 백분율 (e.g. 200% CPU를 요청, 실제 180% 사용 → 90% 만족도 달성)
o 스토리지: 가상머신 디스크 처리 대역폭 (MB/s) 요구 성능 대비 실제 대역폭 백분율 (e.g. 100MB/s 요청, 실제 75MB/s 처리 → 75% 만족도)
o 네트워크: 가상머신의 네트워크 대역폭 (Mbps) 요구 성능 대비 실제 성능

4. 목표 미달 또는 미흡한 사항에 대한 원인분석

1) 목표 미달 또는 미흡한 연구결과에 대한 원인(사유) 자체분석 내용

해당 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2) 미달 또는 미흡한 결과물에 대한 보완 활동

해당 없음

3) 연구개발 과정의 성실성

본 연구를 통해 연구진은 **2021년 국가R&D 우수성과, 그 중에서도 정보 전자 분야에서 IT 연구로서는 유일하게 최우수 연구로 선정**되었음.
이는 연구개발의 성실성을 보여주는 대표적인 성과로 판단함.

5. 연구개발성과의 관련 분야에 대한 기여 정도

한국은 클라우드 인프라 기술에 있어 세계 최선도국인 미국에 비해 23% 이상의 기술력 격차가 있음. 이는 미국, 중국, 일본, 유럽과 대비하여 최하위 수준의 기술력을 의미함.
본 연구진은 2015년부터 8년 간, 클라우드의 성능 보장이라는 중요한 주제를 집중적으로 연구할 수 있도록 지원받았고, 지원 기간 동안의 기여 정도를 요약하면 아래와 같음.

- **학문적 측면:** SCIE 상위 10% 이내 논문 8건, BK사업 기준 SCIE 인정 최우수 학술대회 논문 9건 (IF 4급 2건, 3급 1건, 2급 1건, 1급 5건)을 발표하였음.
  특히 대부분의 연구 성과가 오픈소스SW로 산출되었고, 오픈소스로 공개한 성과는 미국 Open Networking Foundation에서 그 기술력을 인정받아 국제적으로 저명한 소프트웨어로 인정받고, 월 평균 1,000회 이상 조회되는 커뮤니티로 발전하였음.

- **산업적 측면:** 개발 기술은 국내 LG전자 및 미국 기업에 $80,000 수준의 기술이전을 달성하였음. 또한, Microsoft, 삼성전자, LG전자 등 다양한 산업체와의 공동연구를 달성할 수 있었음.

- **인력양성 측면:** 최우수 연구인력 35명을 배출하였음. 구체적으로 박사과정 2명의 Microsoft Research 인턴쉽 수행, 연구과제 참여인력의 성균관대학교, 전북대학교, 경북대학교 정년트랙 교수임용, 석사과정 참여인력의 ETRI, ADD, KIST 등 정출연 정규채용 등으로 입증되는 우수한 기술인재를 양성하였음.

위와 같은 측면으로 볼 때, 본 연구진은 8년 간의 집중 지원을 통해 상대적으로 뒤쳐진 클라우드 인프라 분야에서 학문, 산업, 인력양성 측면에서 국내 경쟁력을 향상시키기 위한 노력을 꾸준히 매진하였음.
이러한 성과를 인정받아 2021년에는 국가 R&D 우수성과 중 전자/정보 분야에서 IT과제 중 유일하게 최우수 과제로 선정되었음.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

6. 연구개발성과의 관리 및 활용 계획

본 연구과제의 연구개발성과는 모두 연구논문, 특허, 오픈소스SW의 순으로 순차적으로 공개 및 관리되고 있음.
8개 년의 과제를 진행하는 동안, 산출되고 있는 연구성과가 존재하여 과제 종료 후에도 실적 산출을 지속하여 마무리하고자 함.
더 나아가, 기 개발한 클라우드의 SLA 보장 기술은 인공지능, 디지털 헬스케어, 자율 주행 등 클라우드를 기반으로 하는 다양한 연구에 응용 가능함. 이에 아래의 연구분야를 포함하여, 다양한 연구분야로의 활용을 도모하고자 함. 향후 연구로의 활용은, 본 연구진의 후속 연구 연계, 산업현장에의 연구 성과 홍보를 통한 활용 촉진 등 다양한 전략을 활용하고자 함.

향후 후속 연계 예시: 클라우드, 하이퍼렛져 패브릭을 활용하여 블록체인 기반 Privacy ECG data 수집 및 저장 연구
본 연구개발 성과인 공개SW R&D 결과물들을 적극 활용하여, Digital healthcare와 같은 클라우드 기반의 실제 서비스 플랫폼에 적용하기 위한 연구를 추진 중임. 실시간 BaaS 워크로드의 성능보장 연구를 활용하여, ECG 데이터에 대한 트랜잭션을 모니터링하고, 트랜잭션 처리 성능이 저하하는 경우 추가 CPU 자원 할당을 통해 개선할 수 있으며, 이를 활용하여 트랜잭션의 규모가 큰 디지털 헬스케어 에코시스템에도 활용될 수 있음을 확인고자 함. 또한, 디지털헬스 케어 플랫폼에 적용하여 환자들의 실시간 ECG 데이터 수집 및 저장을 보장할 수 있는 기술을 개발하고자 함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

< 연구개발성과 활용계획표 >

구분(정량 및 정성적 성과 항목)		연구개발 종료 후 5년 이내
국외논문	SCIE	연 1건
	비SCIE	
	계	연 1건
국내논문	SCIE	
	비SCIE	
	계	
특허출원	국내	연 1건
	국외	
	계	연 1건
특허등록	국내	
	국외	
	계	
인력양성	학사	
	석사	
	박사	
	계	
사업화	상품출시	
	기술이전	
	공정개발	
제품개발	시제품개발	
국제 표준화	기고서 제출	
	기고서 채택	
	표준안 채택	
국내 표준화	기고서 제출	
	기고서 채택	
	표준안 채택	
성과홍보		
포상 및 수상실적		
정성적 성과 주요 내용		

1) 향후 5년간 성과활용·확산 활용방안 및 계획(활동계획)

o (후속연구) 연구개발 기술 산출물 논문화 지속
- 본 연구는 8차년도까지 지속적으로 연구결과물을 개발해온 바, 아직 논문까지 산출되지 않은 연구결과물이 존재함
- 이를 종료 이후 +3년까지 SCIE 상위 10% 학술지 또는 최우수학술대회 위주로 논문화하여, 연구성과의 확산을 도모할 예정임

o (후속연구) 딥러닝 시스템에 대한 클라우드 기술 확장
- 최근 Microsoft Azure, AWS 등의 공공 클라우드는 GPU를 이용한 인공지능 학습을 위한 클라우드로 그 서비스를 확장 중임
- GPU 클라우드는 아직 네트워크 가상화, 성능 차등화, 제어가능성(programmability) 등이 연구되지 않았음
- 이에 보유기술을 기반으로 빠르게 인공지능 학습 속도 개선을 위한 클라우드 기술을 연구할 예정임

o (성과확산) 오픈소스 소프트웨어 공개를 통한 원천기술 확산 도모
- 본 연구개발물은 ONF와 협력하여 오픈소스 소프트웨어의 형태로 공개 추진중.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 특히 해당 분야의 필수적인 원천기술을 본 연구진이 보유하고 있는 바, 지속적으로 개발물을 관리하고 확산될 수 있도록 관리할 예정

<연도별 주요 활동계획(요약표)>

구분	종료+1 (2023년)	종료+2 (2024년)	종료+3 (2025년)	종료+4 (2026년)	종료+5 (2027년)
연구개발 기술 논문 산출	1건	1건	1건		
AI 클라우드 후속연구	과제 제안	연구 추진	연구 추진	연구 추진	
오픈소스SW 성과산출	기개발물 공개를 위한 오픈소스SW 홈페이지 구성	출판되는 논문의 소스코드를 지속적으로 공개			

2) 주요 성과활용 (기대)성과목표

(1) 정성적 목표

o 본 연구결과를 기반으로 한 후속연구 및 과제 추진

o 성과활용을 위한 연1회 산학연 공동 워크샵 개최

- 본 연구진은 연구과제를 수행하면서, 삼성전자, 네이버, sk텔레콤, KT 등 다수의 유관기업이 참여하는 가상화 워크샵을 연 1회 개최하고 있음
- 이를 연구 종료 이후에도 지속적으로 추진하여 연구 결과의 활용 도모

(2) 정량적 목표

<연도별 성과활용 (기대)성과목표(요약표)>

구분	종료+1 (2023년)	종료+2 (2024년)	종료+3 (2025년)	종료+4 (2026년)	종료+5 (2027년)
논문 제출	2건	1건	1건		
특허 출원	1건	1건		1건	
가상화 워크샵	1회	1회	1회	1회	1회
유관 연구과제	1건				

3) 성과활용·관리 추진체계

o 본 과제의 행정처리를 관할한 고려대학교 융합소프트웨어연구소가 매 분기마다, 본 연구과제와 연관된 성과 산출 여부를 참여 인력을 대상으로 확인하고 형상관리 도구를 이용하여 지속적으로 추적함

o 특히 산학연 체계를 통한 성과활용을 위해 매년 1회 가상화 워크샵 시행함

o 본 연구결과가 공개된 https://openvirtex.com을 통해 과제 종료 이후 산출되는 결과물을 지속적으로 갱신하고 관리함

o 4-2-(1)에서 제시한 후속연구를 연구과제의 형태로 추진하여, 본 연구의 성과가 지속적으로 발전될 수 있도록 함

4) 기타

o 기타사항 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

7. 연구개발성과에 대한 후속연구 및 추가 개발 계획

개발한 클라우드의 SLA 보장 기술은 인공지능 네트워크 가상화 등 클라우드를 기반으로 하는 다양한 연구에 응용 가능함. 이에 아래의 연구분야를 포함하여, 다양한 연구분야로의 활용을 도모하고자 함.

1. 네트워크 가상화 성능 보장 연구

1) Control plane

네트워크 가상화를 통한 테넌트 간 고립연구를 수행하면서, SDN 컨트롤러와 스위치 간 control traffic 간섭 문제를 확인하였음. Control traffic은 가상 네트워크 제어의 핵심 요소로서, 테넌트의 컨트롤러가 의도하는 제어동작을 하기위해서 요구하는 latency를 달성해야함. 기존 연구를 바탕으로, 테넌트 간 고립되지 않았던 컨트롤 채널의 문제를 확인하고, 해당 문제를 해결하기 위한 디자인을 설계하고자 함. 이를 해결하기 위해 기존의 연구들은 컨트롤 트래픽에 대한 수식 예측 모델을 설계하였는데, 컨트롤러의 종류, 컨트롤 애플리케이션, 토폴로지 종류등 환경이 변화할 때마다 새로운 모델이 필요하다는 단점이 있음. 본 연구진은 이 문제를 심도 있게 분석하고, 기존 연구의 한계를 극복하기 위해, 머신러닝을 활용하여 예측모델을 설계하고, 이를 활용하여 테넌트 간 제어 채널 고립 연구를 진행할 예정임. 본 연구에 대해서도 오픈소스 공개, 논문 publish를 목표로 진행할 예정임.

2) Data plane

지난 연구에서, 네트워크 가상화 환경에서 테넌트의 대역폭 성능 보장을 달성하기 위한 연구를 진행하였음. 본 연구는 네트워크 하이퍼바이저 기반으로, 가상 네트워크의 패킷 라우팅 경로에 대해 다른 추가 경로를 물리 네트워크에서 계산하여, 테넌트 간 간섭된 대역폭만큼을 다른 경로에서 보장하였음. 해당 연구를 진행하면서, 두 개 이상의 다른 경로로 스위치가 패킷을 포워딩할 때, 스위치 자체의 트래픽 분배 성능이 전체 네트워크 성능에 영향을 미친다는 것을 추가로 확인하였음. 후속 연구로, 스위치의 정확한 트래픽 분배에 대한 연구 및 개발을 진행하여 기존 연구와의 시너지를 기대할 수 있음.

2. AI system 연구

기존 GPU 클라우드에서 GPU 활용률이 매우 저조하여, 많은 자원들이 낭비되고 있으며, 이를 해결하기 위해 동시에 2개 이상의 작업을 실행하기 위한 연구들이 제안되고있음. 그러나 기존 연구들은 동시에 실행되는 작업의 GPU 사용 특성이 고려되지 않기 때문에 작업 완료 시간(JCT)이 크게 감소하지 않거나, 오히려 증가할 수 있음. 따라서, 이 문제를 해결하기 위해 본 연구진은 동시 실행되는 작업의 GPU 사용 특성을 고려하여, 동시에 실행시킬 작업들을 효과적으로 배치하는 초기 연구를 진행하였음. 후속 연구로, 이기종 GPU, 딥러닝 모델, 데이터셋에 따른 워크로드별 GPU 사용 특성을 분석하고, 동시 실행되는 워크로드 간의 JCT 증가시간을 예측할 수 있는 딥러닝 모델을 개발 및 추진중임. 학습된 모델의 예측된 값을 기반으로 GPU에 동시에 실행할 작업을 배치하는 연구를 하고자 함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

8. 연구개발비 사용실적

1) 연구개발비 사용명세서

(단위 : 원, %)

항목 \ 구분		연구개발비 현황					사용금액 ⑥	사용잔액 (⑤-⑥)	사용률 (⑥/⑤×100)	불용금액 (해당 시)
		전단계 이월액 (해당 시) ①	당초 연구개발비 ②	변경 연구개발비 ③	발생이자 중 연구개발비 산입금 ④	소계 ⑤ (①+②+③+④)				
가. 직접비	현금	12,342,743	228,000,000	0	190,822	240,533,565	240,246,181	287,384	99.9	
	현물		60,000,000			60,000,000	60,000,000	0	100	
	소계	12,342,743	288,000,000	0	190,822					
1) 인건비	현금	8,595,543	805,000	24,474,703		33,875,246	33,875,246	0	100	
	현물		60,000,000				60,000,000	0	100	
2) 학생인건비	현금		116,160,000	7,684,343		123,844,343	123,844,343	0	100	
3) 연구시설・장비비	현금		0	2,587,200		2,587,200	2,587,200	0	100	
	현물									
4) 연구재료비	현금	313,532	23,420,000	-19,088,726	11,468	4,656,274	4,438,890	217,384	95.3	
	현물									
5) 위탁연구개발비[1」]	현금									
6) 국제공동연구개발비	현금									
7) 연구개발부담비	현금									
8) 연구활동비	현금	3,433,668	64,615,000	-15,657,520	179,354	52,570,502	52,500,502	70,000	99.8	
	현물									
9) 연구수당	현금		23,000,000			23,000,000	23,000,000	0	100	
나. 간접비	현금		57,000,000			57,000,000	57,000,000	0	100	
	소계									
합계	현금	12,342,743	285,000,000	0	190,822	297,533,565	297,246,181	287,384	99.9	
	현물		60,000,000			60,000,000	60,000,000	0	100	
	합계	12,342,743	345,000,000	0	190,822	357,533,565	357,246,181	287,384	99.9	

* 1」 위탁연구개발비 현황 및 사용금액은 주관연구개발기관만 기재합니다.

2) 발생이자 사용명세서

(단위 : 원)

구분	발생금액 (A)	사용용도	사용금액 (B)	사용잔액 (A-B)
연구개발기간 중 발생이자	287,384	연구활동비/재료비	287,384	0
연구개발기간 후 발생이자				
합계	287,384	연구활동비/재료비	287,384	0

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3) 반납액 명세서

(단위 : 원, %)

연구개발비 반납액				발생이자 반납액		총 반납액 (C+E)
구분	현금	현물	소계			
사용잔액	287,384		287,384	연구개발기간 중	0	
불인정 사용금액	0		0	연구개발기간 후	0	
소계(A)	287,384		287,384	소계(D)	0	
정부지분율(B)	100		100	정부지분율(B)	0	
정부지분액(C=A×B)	287,384		287,384	정부지분액(E=D×B)	0	287,384

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

붙임 2. 공개 SW 요약서 (공개SW과제만 작성)

<table>
<tr><td colspan="2">공개 SW명</td><td colspan="5">Stella Cloud－SLA and SLO guarantee</td></tr>
<tr><td colspan="7">분류</td></tr>
<tr><td colspan="2">기술분류</td><td colspan="5">SW · AI/클라우드 컴퓨팅/차세대 클라우드 컴퓨팅</td></tr>
<tr><td colspan="2">최종 업데이트일</td><td colspan="5">2023. 02. 15.</td></tr>
<tr><td colspan="7">공개 SW 정보</td></tr>
<tr><td colspan="2">개요
(공개 SW요약)</td><td colspan="5">Next-generation cloud infra-software toward the guarantee of performance and security SLA (Service Level Agreement)</td></tr>
<tr><td colspan="2">저장소별 주소</td><td colspan="5">https://github.com/KUoslab
https://github.com/os-libera</td></tr>
<tr><td colspan="2">구글 등 검색여부</td><td colspan="5">Y</td></tr>
<tr><td rowspan="2">버전</td><td>안정화 버전</td><td>버전명</td><td>v1</td><td>날짜</td><td colspan="2">2020. 11. 19.</td></tr>
<tr><td>최종 버전</td><td>버전명</td><td>v1.3</td><td>날짜</td><td colspan="2">2023. 02. 09.</td></tr>
<tr><td colspan="2">라이선스</td><td colspan="5">Apache-2.0 또는 MIT license</td></tr>
<tr><td rowspan="6">공개SW
실적</td><td>Commit</td><td colspan="2">목표 : 200건(누적)</td><td colspan="3">실적 : 91940건(누적)</td></tr>
<tr><td>Fork</td><td colspan="2">목표 : 40건(누적)</td><td colspan="3">실적 : 278건(누적)</td></tr>
<tr><td>Star</td><td colspan="2">목표 : 80건(누적)</td><td colspan="3">실적 : 304건(누적)</td></tr>
<tr><td>Issue(closed)</td><td colspan="2">목표 : -건(누적)</td><td colspan="3">실적 : 5건(누적)</td></tr>
<tr><td>Pull request
(closed)</td><td colspan="2">목표 : -건(누적)</td><td colspan="3">실적 : 74건(누적)</td></tr>
<tr><td>실적 관리자</td><td colspan="5">소속: 고려대학교,이름: 최원미, 연락처: 02-3290-3639
, E-Mail: ymcui@os.korea.ac.kr</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

주 의

1. 이 보고서는 과학기술정보통신부에서 시행한 정보통신·방송연구개발사업의 SW컴퓨팅산업 원천 기술개발과제 최종보고서이다.

2. 이 연구개발내용을 대외적으로 발표할 때에는 반드시 과학기술정보통신부(정보통신기획평가원)에서 시행한 정보통신·방송연구개발사업의 결과임을 밝혀야 한다.

3. 국가과학기술 기밀 유지에 필요한 내용은 대외적으로 발표 또는 공개하여서는 안 된다.